药材烘干过程热质耦合与非线性扩散模型研究

摘要

针对药材烘干中的热湿传递问题，本文从传热传质机理出发，建立药材内部温度场与水分场的耦合模型，研究烘干过程中温度和水分的时空变化规律，在此基础上确定烘干终止时间以及相关工艺。

对于问题一，首先将模型合理简化为一维轴对称径向问题. 基于 **Fourier 导热定律**和 **Fick 扩散定律**，分别建立药材内部温度场和含水率场的偏微分方程，结合中心对称条件和表面传热传质条件，建立**一维热质热-湿传递模型**。采用**有限体积法**和**后向欧拉进行离散**，求解得到预热阶段的 0 至 1800s 内不同径向位置的温度和湿度，具体结果存于 result1.xlsx 中。

对于问题二，在问题一的模型基础上，将物性参数替换成题目附录中给出的经验公式，建立**非线性热湿耦合模型**。引入 **Kirchhoff 变换**以处理扩散系数非线性问题，并结合**有限体积法，后向欧拉格式，Picard 迭代**实现**耦合求解**。求解结果存于 result2.xlsx 中。结果表明药材温度在短时间内趋于稳定，而水分浓度仍持续下降，水分迁移能力逐渐减弱，解释了烘干后期耗时较长的原因。

对于问题三，首先以药材内部最大水分浓度建立烘干终止判据。考虑到连续空间无法进行数值求解，因此将空间离散为有限个时间点，得到**离散化的烘干终止判据**，建立**非稳态热湿耦合烘干终止模型**。采用有限体积法与 **Picard 迭代**逐时间层进行求解温度场与含水率场，并在每一步检测最大含水率. 针对离散时间步造成的终点定位误差，对跨越临界值的相邻时间层进行线性插值局部精化。结果表明，药材内部最湿位置控制烘干终点，计算得到的满足要求的烘干时间 **57.2107h**，详细结果存于 result3.xlsx 中。

对于问题四，在问题三模型的基础上，进一步考虑药材烘干过程中的尺寸收缩，引入**材料坐标**将随时间变化的计算区域映射到固定区域，建立**考虑收缩效应的非线性水分扩散模型**。考虑到药材内部热响应较快，将内部温度近似为环境温度，采用有限体积法，后向欧拉格式和 Picard 迭代进行数值求解。以药材内部最大水分浓度达到临界值 0.15 kg/kg 作为烘干终止判据，计算得到满足要求的烘干时间为 **50.7458h**，详细结果存于 result4.xlsx 中。

综上，本文建立了药材烘干过程的**热质传递与水分扩散模型**，并通过数值计算得到药材内部温度、含水率及烘干时间的变化规律。研究结果能够描述药材内部不同位置的干燥程度，并确定满足含水率要求的烘干终点，为实际生产中的烘干时间控制提供依据。

关键字：中药材热风干燥；热质耦合；非线性偏微分方程；柱坐标有限体积法；移动边界

一、 问题重述

1.1 问题背景

热风烘干作为常见的中药材干燥方式，包含预热平衡和恒温干燥两个过程。在烘干过程中，温度，湿度等工艺参数选取决定了生产效率，成品品质和生产能耗。然而传统的试验优化方法存在着成本高，试验周期长等问题。因此，需要通过机理分析和数值仿真得到药材内部的温度和水分的变化规律，为烘干过程的分析和调控提供理论依据。

1.2 问题要求

问题 1：对于给定尺寸的圆柱形中药材，需要根据其初始温度，初始水分浓度以及烘房温度和水分浓度随时间的变化建立预热平衡阶段药材温度和水分浓度变化规律的数学模型，计算指定时刻，距药材中心不同位置处的温度和水分浓度。

问题 2：相较于问题一，需要考虑预热平衡阶段以及恒温干燥阶段两个阶段，结合给出的经验公式，建立整个烘干过程的药材温度和水分变化规律的数学模型，计算指定时刻，距药材中心不同位置处的温度和水分浓度。

问题 3：需要在给定的烘干要求下，利用所建立的干燥模型，计算药材烘干所需要的时间并给出烘干过程中不同位置处的水分浓度变化。

问题 4：在之前模型的基础上，需要考虑在烘干过程中的药材尺寸变化，建立考虑收缩效应的干燥模型，确定药材的烘干时长。

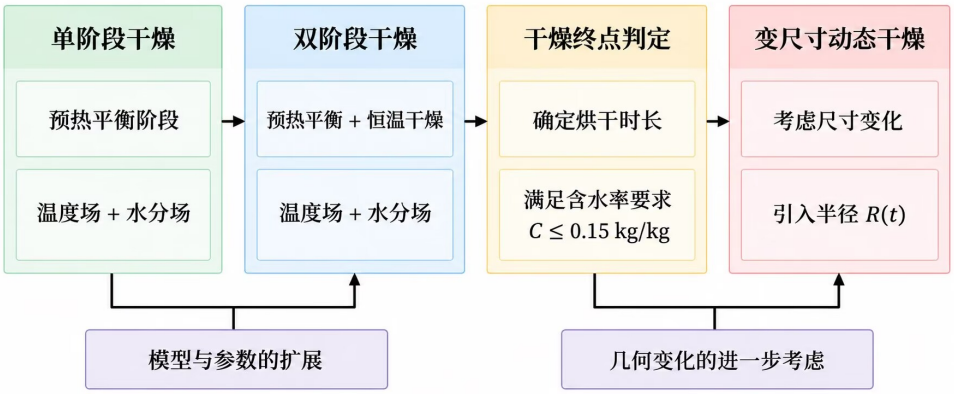


图 **1**　各问题关系图

二、 问题分析

2.1 问题一分析

对于问题一，本问属于带对流边界的非稳态热湿耦合传递问题。药材长径比为 6.25，可以忽略轴向以及端面效应，简化为圆柱坐标下的一维径向模型。数值求解采用有限体积法进行空间离散，后向欧拉公式进行时间推进，得到预热干燥过程中的温度场和含水率场。

2.2 问题二分析

对于问题二，相较于问题一研究单一阶段的干燥问题，问题二要求考虑正式干燥过程中外部热风条件的变化以及温度场和含水率场之间的相互作用。由于药材内部温度和含水率均随时间和径向位置不断变化，需要建立非稳态热湿耦合传递模型，结合单一干燥阶段的初始条件与边界条件进行数值求解，从而刻画在整个阶段过程中的温度场和含水率场。

2.3 问题三分析

对于问题三，要求确定药材各处水分浓度均低于 0.15kg/kg 时所需的烘干时间，可沿用第二问建立的模型，只需延长计算时域。需要引入烘干终止判据，将药材内部水分均低于临界值转化为药材内部最大水分浓度的首次达标，在非稳态热湿耦合模型的基础上求解烘干结束时刻。

2.4 问题四分析

对于问题四，相较于问题三固定尺寸的干燥过程，问题四进一步考虑药材在烘干过程中因水分流失产生的尺寸收缩。由于药材半径随时间不断减小，内部水分扩散区域也随之变化，需要处理随时间变化的计算区域。水分扩散与温度场存在耦合，但药材内部温度响应较快，因此可对温度场进行适当简化。在此基础上，建立考虑尺寸收缩的非线性水分扩散模型，并求解满足含水率要求的烘干时间。

三、 模型假设

为简化问题，本文做出以下假设：

- 假设 1：初始时刻药材内部各点温度和含水率均相同。
- 假设 2：对流换热系数，对流传质系数在整个过程中均为常数。
- 假设 3：不考虑水分蒸发潜热对于热量存储的影响。

- 假设 4：题目给定的对流传质系数可以反映空气与药材界面的吸附平衡。
- 假设 5：水分在药材表面不产生积累。

四、 符号说明

表 **1**　符号说明

符号	说明	单位
W_0	初始含水率	kg/kg
T_0	初始温度	°C
$T(r,t)$	药材温度场	°C
$T_{air}(t)$	热风温度	°C
$C_{air}(t)$	热风水分浓度	kg/kg
$C(r,t)$	干基含水率场	kg/kg
k	热传导系数	W/(m·K)
h	对流换热系数	W/(m^2·K)
h_m	对流传质系数	m/s
D	水分扩散系数	m^2/s

五、 问题一的模型的建立和求解

5.1 预热阶段的一维径向热质热—湿传递模型的建立

5.1.1 模型简化与合理性分析

为了判断药材内部温度梯度是否可以忽略以及预热阶段热量在径向方向上的扩散程度，引入 Biot 数与 Fourier 数进行无量纲分析。Biot 数不可忽略，说明药材内部存在明显温度梯度，因此保留径向温度场。Fourier 为后续数值结果提供量级上的合理性检验。

由于圆柱长度与半径之比为 6.25，圆柱长度明显大于其径向尺度，除靠近两端的局部区域外，两端引起的轴向传热与传质对主体区域温度场和含水率场的影响较弱。因此在研究药材内部热量与水分迁移规律时，可以忽略端面效应以及轴向梯度，认为温度和含水率主要沿径向变化，为后续建立一维模型提供基础。

5.1.2 一维径向导热方程的建立

取药材横截面的对称轴为坐标原点，径向坐标 r 指向表面。热量延径向传递时，垂直于热流方向的换热面是半径为 r，长度为 L 的圆柱侧面，将其展开为长 $2\pi r$，宽 L 的

矩形，可得到 $A(r) = 2\pi rL$。因此，圆柱体径向换热面积与半径成正比。根据傅里叶定律，半径 r 处径向向外的传导率为：

$$Q(r) = -kA(r)\frac{\partial T}{\partial r} = -2\pi rLk\frac{\partial T}{\partial r} \tag{1}$$

取半径为 r，厚度为 $\mathrm{d}r$ 的圆环微元，则其体积为：

$$\mathrm{d}V = \pi\left[(r+\mathrm{d}r)^2 - r^2\right]L \approx 2\pi rL\,\mathrm{d}r \tag{2}$$

对所取圆环微元做能量守恒分析，由于微元储热量等于内表面流入热量减去外表面流出热量，可以推导出：

$$Q(r) - Q(r+\mathrm{d}r) = \rho c_p\,\mathrm{d}V\frac{\partial T}{\partial t} \tag{3}$$

将式 (2)，代入式 (3)，两边除以 $2\pi rl\mathrm{d}r$，并令 $\mathrm{d}r \to 0$，则可以计算出一维径向导热方程：

$$\rho c_p\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rk\frac{\partial T}{\partial r}\right) \quad 0 < r < R, \quad t > 0 \tag{4}$$

5.1.3 一维径向扩散方程的建立

由于在烘干过程中，表层含水率会降低，导致扩散能力减弱，因此将扩散系数 $D(C)$ 保留在散度算子内，从而避免表层干燥能力被低估。类似于一维径向导热方程的建立，依据水分守恒和菲克第二扩散定律，可以得到药材内部的含水率满足：

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rD(C)\frac{\partial C}{\partial r}\right] \qquad 0 < r < R, \quad t > 0 \tag{5}$$

其中，水分扩散系数由题目所给的经验关系式做确定：

$$D(C) = 7 \times 10^{-9}e^{-\frac{0.89}{C}} \tag{6}$$

5.1.4 扩散方程的初始条件

由于在模型假设时确定了药材内部的温度和含水率分布近似均匀，同时由于圆柱形药材关于中心轴对称，中心轴处不存在径向热流以及水分通量，因此温度场与含水率场在 $r = 0$ 处满足零梯度边界条件，由此可以得到：

$$T(r,0) = 28\,^{\circ}\mathrm{C}, \qquad C(r,0) = 2.55\,\mathrm{kg/kg}, \qquad \left.\frac{\partial T}{\partial r}\right|_{r=0} = 0, \qquad \left.\frac{\partial C}{\partial r}\right|_{r=0} = 0 \tag{7}$$

5.1.5 扩散方程的边界条件

药材表面处与流动热风直接接触，内部传导至表面处的热量与水分均以对流方式与热风环境交换。依据热量守恒以及水分守恒，在药材表面处的内部导热通量等于外部对

流换热通量，内部水分扩散通量等于外部对流传质通量。因此可以得到一维径向扩散方程的边界条件为：

$$\begin{cases} -k(C)\left.\dfrac{\partial T}{\partial r}\right|_{r=R} = h\left[T(R,t) - T_{\text{air}}(t)\right] & h = 25\ \text{W}/(\text{m}^2\cdot\text{K}) \\ -D(C)\left.\dfrac{\partial C}{\partial r}\right|_{r=R} = h_m\left[C(R,t) - C_{\text{air}}(t)\right] & h_m = 8\times10^{-7}\ \text{m/s} \end{cases} \tag{8}$$

5.2 模型求解

5.2.1 有限体积法进行空间离散

考虑到热量与水分的传递满足局部守恒关系，并且圆柱径向控制面面积随半径变化，因此求解过程中采用有限体积法进行空间离散，保证离散方程在每个控制体上的能量守恒和水分守恒。将径向计算区域 [0,R] 均匀划分为 $N = 20$ 个控制体，网络步长为 $\Delta r = 1\times10^{-3}\text{m}$。离散节点为 $r_i = i\Delta r(i = 0, 1, \cdots, N)$，共得到 21 个节点，内部节点对应各控制体中心。几何划分与离散约定如下图所示：

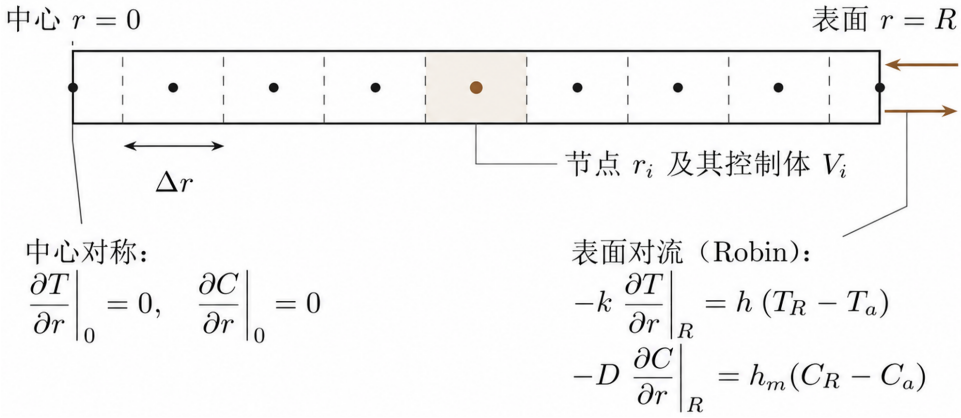


图 **2**　圆柱几何与径向离散示意图

5.2.2 后向欧拉格式进行时间离散

为避免由于显式时间推进在较大时间步长下可能出现的数值振荡或发散问题，因此本文采用后向欧拉格式进行时间离散。该格式将扩散通量统一取在新时间层 t^{n+1} 上，使得每一步求解得到的未知量满足新时间层上的控制体的能量与水分守恒关系，减少不稳定性。对于第 i 个控制体，其含水率在一个时间步长 Δt 内的变化率可以近似为：

$$\int_{V_i}\frac{\partial C}{\partial t}\text{d}V \approx V_i\frac{C_i^{n+1} - C_i^n}{\Delta t} \tag{9}$$

其中，V_i 为第 i 个控制体体积，由此，将连续时间域上的含水率变化转化为相邻时间层 n 与 n+1 之间的离散递推关系，保持良好的数值稳定性。

5.2.3 时间离散方程组装与线性系统求解

完成时间项离散后，还需要将前述有限体积空间离散结果与后向欧拉格式进行耦合。对于第 i 个控制体，其含水率变化由左右两个控制面上的净扩散通量共同决定。计 $A_{i-\frac{1}{2}}$ 和 $A_{i+\frac{1}{2}}$ 分别为控制体左右界面 $r_{i-\frac{1}{2}}$ 和 $r_{i+\frac{1}{2}}$ 处的传质面积。因此，在新时间层上，左右控制面扩散通量可以表示为：

$$A_{i-\frac{1}{2}}D_{i-\frac{1}{2}}\left.\frac{\partial C}{\partial r}\right|_{i-\frac{1}{2}}^{n+1} \qquad A_{i+\frac{1}{2}}D_{i+\frac{1}{2}}\left.\frac{\partial C}{\partial r}\right|_{i+\frac{1}{2}}^{n+1} \tag{10}$$

进一步，将控制面处的含水率梯度用相邻节点的差商近似：

$$\left.\frac{\partial C}{\partial r}\right|i+\frac{1}{2}^{n+1} \approx \frac{Ci+1^{n+1}-C_i^{n+1}}{\Delta r}, \qquad \left.\frac{\partial C}{\partial r}\right|i-\frac{1}{2}^{n+1} \approx \frac{C_i^{n+1}-Ci-1^{n+1}}{\Delta r} \tag{11}$$

将其代入上式，可以得到第 (i) 个内部控制体的有限体积离散方程：

$$\frac{V_i}{\Delta t}\left(C_i^{n+1}-C_i^n\right)=A_{i+\frac{1}{2}}D_{i+\frac{1}{2}}\frac{C_{i+1}^{n+1}-C_i^{n+1}}{\Delta r}-A_{i-\frac{1}{2}}D_{i-\frac{1}{2}}\frac{C_i^{n+1}-C_{i-1}^{n+1}}{\Delta r} \tag{12}$$

为处理扩散系数差异较大的情况，较好反映相邻区域内串联扩散阻力的影响，因此对于相邻节点之间的界面扩散系数采用调和平均 $\frac{2D_iD_{i+1}}{D_i+D_{i+1}}$ 进行处理。温度方程采用相同的有限体积离散思想，结合表面对流换热边界条件进行处理，离散后，可以推导出：

$$\begin{cases} a_i=-\dfrac{A_{i-\frac{1}{2}}D_{i-\frac{1}{2}}}{\Delta r} \\ b_i=\dfrac{V_i}{\Delta t}+\dfrac{A_{i-\frac{1}{2}}D_{i-\frac{1}{2}}}{\Delta r}+\dfrac{A_{i+\frac{1}{2}}D_{i+\frac{1}{2}}}{\Delta r} \\ c_i=-\dfrac{A_{i+\frac{1}{2}}D_{i+\frac{1}{2}}}{\Delta r} \\ d_i=\dfrac{V_i}{\Delta t}C_i^n \end{cases} \tag{13}$$

将各内部节点的离散方程依次排列，结合中心对称边界条件与表面对流边界条件，得到关于新时间层的三对角线性方程组：

$$\begin{bmatrix} b_0 & c_0 & & & 0 \\ a_1 & b_1 & c_1 & & \\ & a_2 & b_2 & c_2 & \\ & & \ddots & \ddots & \ddots \\ 0 & & & a_N & b_N \end{bmatrix}\begin{bmatrix} C_0^{n+1} \\ C_1^{n+1} \\ C_2^{n+1} \\ \vdots \\ C_N^{n+1} \end{bmatrix}=\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ \vdots \\ d_N \end{bmatrix}. \tag{14}$$

其中，首行由中心对称边界条件确定，末行由表面对流传质边界条件确定。

5.2.4 托马斯算法求解方程组

由于每个节点仅与自身和相邻两个节点发生耦合，因此系数矩阵具有三对角结构。本文采用托马斯算法对该方程组进行求解，从而得到下一时间层各节点的含水率分布。具体求解流程如图 3所示。

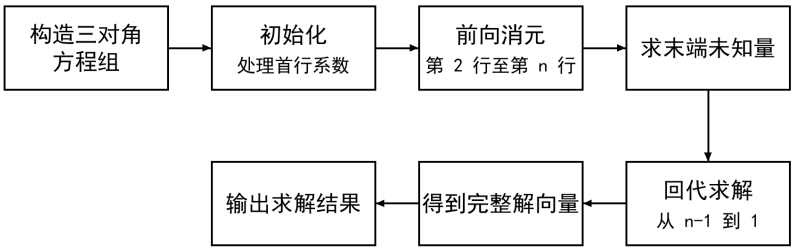


图 **3**　问题一求解过程示意图

Step1：确定三对角线性方程组的下对角，主对角，上对角系数以及右端项，并初始化首行系数

Step2：从第 2 行至第 n 行依次进行航向消元，更新主对角系数和右端项，将方程组化为上三角形式

Step3：由最后一行求得末端未知量，在从第 n-1 行至第 1 行逐行回代，得到全部未知量，完成方程组求解

5.3 结果与分析

5.3.1 求解结果

求解得到的 1800 s 内每隔 1 s、到药材中心距离每隔 0.1 cm 的完整结果保存到文件 result1.xlsx 中，部分求解结果如下表所示：

表 **2**　预热平衡阶段药材温度 $T(r,t)$

t/s	$r=0$	$r=0.5\,\mathrm{cm}$	$r=1.0\,\mathrm{cm}$	$r=1.5\,\mathrm{cm}$	$r=2.0\,\mathrm{cm}$
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 **3**　预热平衡阶段药材水分浓度 $C(r,t)$

t/s	$r=0$	$r=0.5\,\mathrm{cm}$	$r=1.0\,\mathrm{cm}$	$r=1.5\,\mathrm{cm}$	$r=2.0\,\mathrm{cm}$
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

5.3.2 结果分析

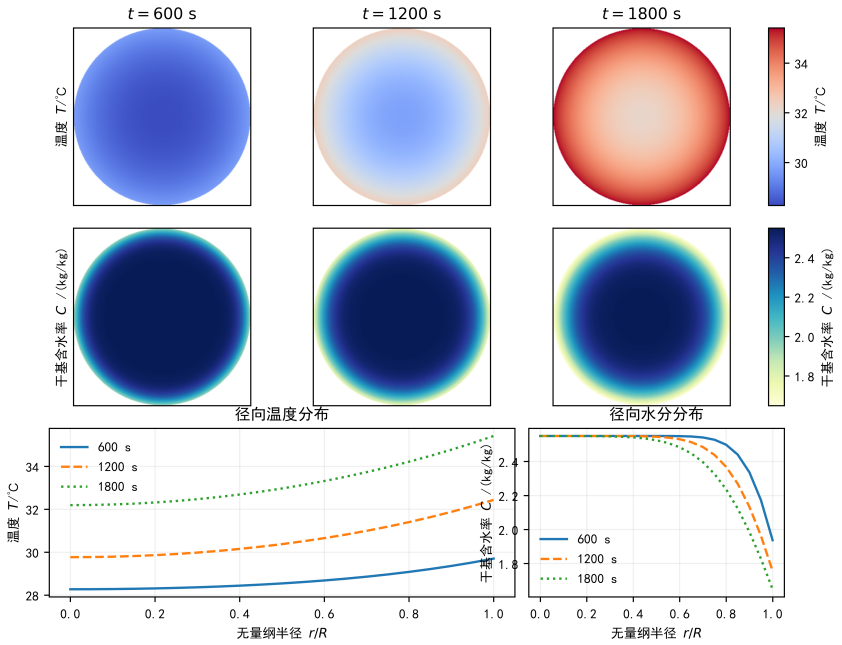


图 **4**　温度场与水分场时空演化结果

对 600s，1200s 和 1800s 的温度场和含水率场进行模拟。从图中可以发现，温度变化能够较快向内部传播，而水分变化主要集中在近表层区域，说明了热扩散响应快于水分扩散响应。因此，在实际烘干过程中，需要在预热阶段给予内部充分升温时间，避免仅仅依据表面含水率判断预热是否完成。

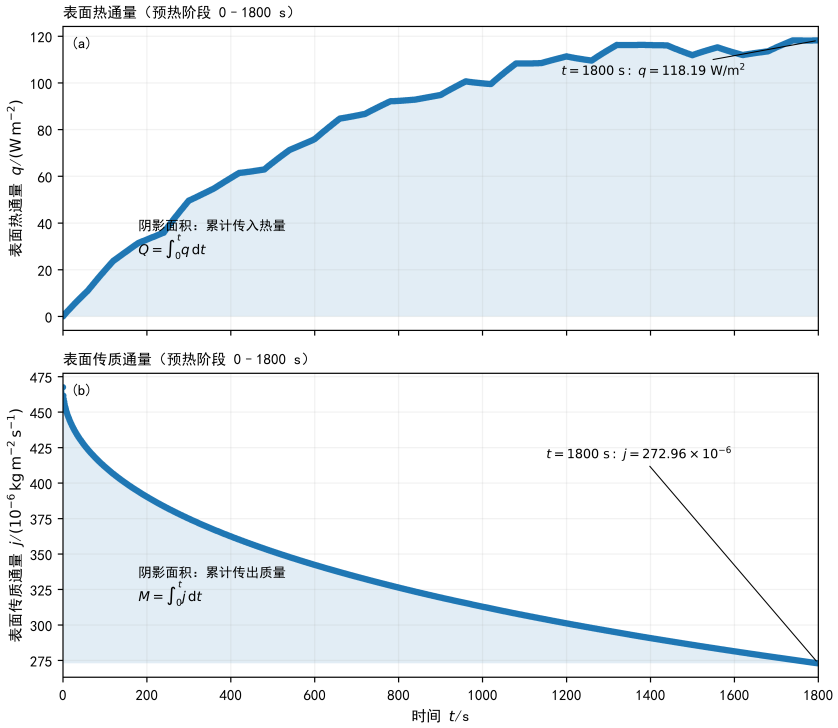


图 **5**　表面通量与传质通量演化

为进一步解释上述温度场与含水率场的演化差异，因此考察药材表面的换热通量与传质通量随时间的变化。结果表明，持续的热量输入促进了温度场较快的建立，而持续衰减的传质驱动力使水分迁移速度相对缓慢，解释了温湿场差异的原因。

六、 问题二的模型的建立和求解

6.1 非线性热湿耦合模型建立

6.1.1 温度场控制方程的建立

将问题一中建立的一维径向导热方程中的药材密度，比热容，导热系数替换成题目附录中的经验关系式，可以得到温度场控制方程：

$$(650+128C)\left(1450+2736\frac{C}{C+1}\right)\frac{\partial T}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\left[r\left(0.21+0.38\frac{C}{C+1}\right)\frac{\partial T}{\partial r}\right](0<r<R) \tag{15}$$

6.1.2 含水率场控制方程的建立

将问题一中建立的一维径向扩散方程中的扩散系数替换成题目附录中的经验关系式，可以得到含水率场控制方程：

$$\frac{\partial C}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\left[2.4\times10^{-3}r\exp\left(-\frac{0.45}{C}\right)\exp\left(-\frac{3850}{T+273.15}\right)\frac{\partial C}{\partial r}\right]\qquad 0<r<R \tag{16}$$

6.1.3 含水率场控制方程的约束条件

控制方程的初始条件与问题一中相同，但边界条件需要代入题目附件中的经验公式，可以得到第二问的边界条件为：

$$\begin{cases} -k(C)\left.\dfrac{\partial T}{\partial r}\right|_{r=R} = h\left[T(R,t) - T_{\text{air}}(t)\right] \qquad h = 25\ \text{W}/(\text{m}^2\cdot\text{K}), \\ -D(C,T)\left.\dfrac{\partial C}{\partial r}\right|_{r=R} = h_m\left[C(R,t) - C_{\text{air}}(t)\right] \qquad h_m = 8\times10^{-7}\ \text{m/s} \end{cases} \tag{17}$$

6.2 模型求解

6.2.1 模型的有限体积离散

由于控制方程中的 ρ，c_p，k，D 均随状态变量变化，难以获得解析解，而有限体积法可以自然考虑圆柱坐标下控制体面积随时间变化的几何特性，并且方便处理温度，含水率相关的物理参数，因此采用有限体积法进行数值解的求解。

依据问题一模型求解过程中的几何划分与离散约定，对于内部节点 (i)，在控制体 $\left[r_i - \frac{\Delta r}{2}, r_i + \frac{\Delta r}{2}\right]$ 上对含水率场控制方程进行积分，根据高斯散度思想，将控制体内部的扩散项转化为两侧界面的净通量，得到：$r_{i-\frac{1}{2}} D_{i-\frac{1}{2}} \frac{C_i - C_{i-1}}{\Delta r}$，其中 $h_m = 8\times10^{-7}\ \text{m/s}$。温度方程采用相同的控制体思想，只需要将扩散系数替换成导热系数，同时在时间积累项中引入 ρc_p。

6.2.2 后向欧拉格式进行时间离散

完成空间离散后，还需要在时间方向对温度场和含水率场进行离散。为了提高数值计算的稳定性，本文采用后向欧拉格式，将时间导数在新时间层上近似。以含水率场为例，其时间导数为：

$$\left.\frac{\partial C}{\partial t}\right|_i^{n+1} \approx \frac{C_i^{n+1} - C_i^n}{\Delta t} \tag{18}$$

6.2.3 变扩散系数的界面处理

得到数值解的关键在于界面扩散系数的确定，由经验公式可知，当含水率由初始状态逐渐降低时，$D(C,T)$ 可以跨过多个数量级。若直接采用节点扩散系数的算术平均，容易使低含水率区域的扩散通量产生较大的离散误差。因此，为了提高非线性扩散过程的计算精度，引入了 Kirchhoff 变换，定义 $\int_0^C e^{-0.45/u}\,\mathrm{d}u$。于是相邻节点之间的等效扩散系数取为：

$$2.4\times10^{-3} e^{-3850/\overline{T}_{i+\frac{1}{2},K}} \frac{\Phi(C_{i+1}) - \Phi(C_i)}{C_{i+1} - C_i} \tag{19}$$

相比之下，导热系数的变化幅度较小，因此界面导热系数采用的相邻节点算数平均可满足计算精度要求。

6.2.4 时间与空间耦合求解

将上述时间离散与前文有限体积空间离散相结合，可以将第 i 个内部控制体的离散方程整理为：

$$a_i C_{i-1}^{n+1} + b_i C_i^{n+1} + c_i C_{i+1}^{n+1} = d_i, \qquad i = 1, 2, \ldots, N-1 \tag{20}$$

温度方程采用相同的时间离散方式，只需要根据能量守恒方程重新构造相应系数。因此，对于温度场和含水率场，均可以统一写成：

$$a_i \varphi_{i-1}^{n+1} + b_i \varphi_i^{n+1} + c_i \varphi_{i+1}^{n+1} = d_i \tag{21}$$

其中，φ 分别取 T 和 C，对应温度场和含水率场。将各个内部节点的离散方程依次排列，结合中心轴处的对称边界条件以及药材表面的对流换热，对流传质边界条件，可以得到关于新时间层未知量的三对角线性方程组：

$$\begin{bmatrix} b_0 & c_0 & & & 0 \\ a_1 & b_1 & c_1 & & \\ & a_2 & b_2 & c_2 & \\ & & \ddots & \ddots & \ddots \\ 0 & & & a_N & b_N \end{bmatrix} \begin{bmatrix} \varphi_0^{n+1} \\ \varphi_1^{n+1} \\ \varphi_2^{n+1} \\ \vdots \\ \varphi_N^{n+1} \end{bmatrix} = \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ \vdots \\ d_N \end{bmatrix} \tag{22}$$

6.2.5 Picard 迭代求解方程组

由于本物理模型中的物理性质均会随着温度或者含水率发生变化，因此方程组是非线性的。为了处理非线性，在每一个时间步内采用 Picard 迭代进行逐次修正。具体求解过程如图 12所示：

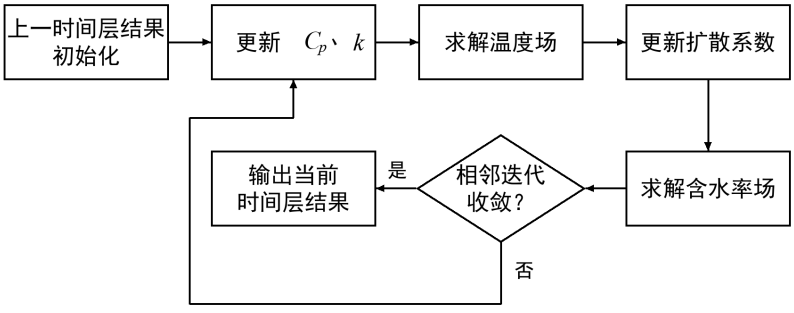


图 **6**　问题二求解过程示意图

Step1：首先以上一时间层结果作为当前时间层的初始估计，根据含水率更新 C_p，k，构造并求解温度场离散方程

Step2：利用更新后的温度场与含水率重新计算扩散系数，求解含水率场，得到新的温度和含水率

Step3：再次得到物理参数，重复上述过程，直到相邻两次迭代结果满足收敛条件

6.3 结果与分析

6.3.1 求解结果

求解得到的 3h 每 0.5h，各个指定位置的温度与含水率数值列于表 4与表 5，其他结果存于表 result2.xlsx。

表 **4**　**3** 小时内药材的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	32.1909	32.3837	32.9674	33.9620	35.4146
1.0	40.3819	40.5539	41.0609	41.8790	42.9987
1.5	45.8464	45.9345	46.1928	46.6051	47.1407
2.0	48.4499	48.4878	48.5982	48.7735	49.0034
2.5	49.4667	49.4790	49.5135	49.5654	49.6605
3.0	49.8495	49.8554	49.8746	49.9101	49.9664

表 **5**　**3** 小时内药材的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	2.5499	2.5487	2.5249	2.3267	1.6486
1.0	2.5249	2.4940	2.3577	2.0234	1.4705
1.5	2.3854	2.3252	2.1343	1.8020	1.3471
2.0	2.1706	2.1083	1.9235	1.6258	1.2307
2.5	1.9566	1.9005	1.7359	1.4718	1.1163
3.0	1.7662	1.7165	1.5701	1.3331	1.0078

6.3.2 结果分析

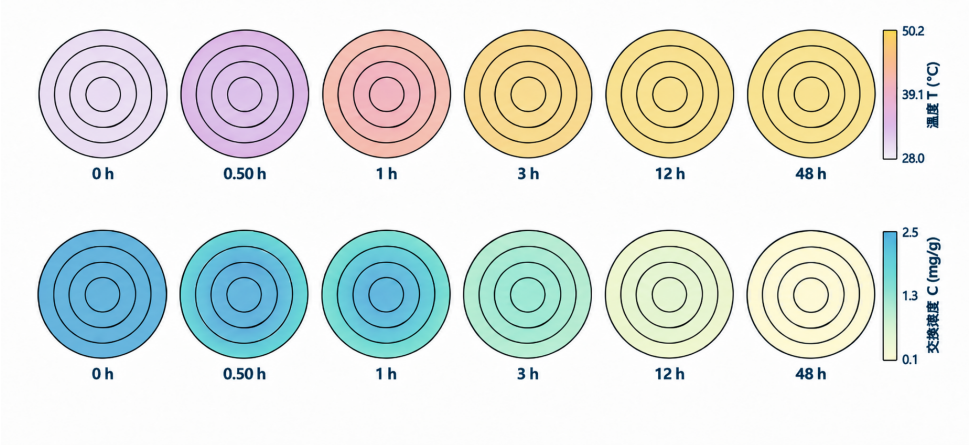


图 **7**　温度场与含水率场的截面演化

选取 0，0.5，1，3，12，48 典型时刻进行温度场与含水率场的模拟，可以发现药材内部传热过程较快，而水分变化明显更慢，存在明显的横向梯度。由此可见，药材干燥过程中传热速度明显快于传质速度。因此，在实际烘干过程中，中后期工艺优化需要更多关注内部水分迁移，并非单纯延长加热时间。

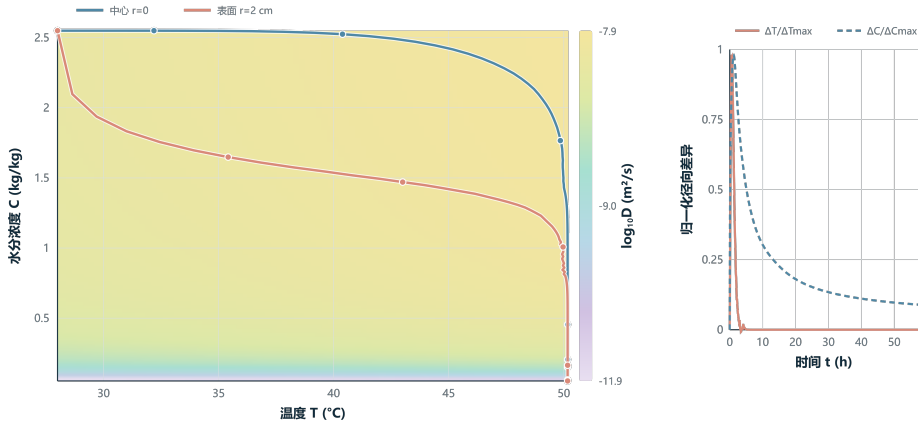


图 **8**　温湿耦合与扩散机理

从图中可见，干燥初期温度迅速升高，表面水分下降更快，随后温度趋于稳定，但水分仍持续下降。同时，水分扩散系数随温度和含水状态动态变化，随着含水率降低总体呈现减小趋势，使得水分迁移在后阶段较为缓慢，解释了热量能够较快传入药材内部，而水分排出受到内部扩散能力的持续限制的原因。

七、 问题三的模型的建立和求解

7.1 非稳态热湿耦合烘干终止模型建立

7.1.1 烘干终止判据

由于干燥过程中不同径向位置的失水速率存在差异，只要药材内部存在某一点的水分浓度高于临界值，就不能认为烘干过程已经完成。因此，本文采用药材内部最大水分浓度作为烘干状态的判定指标。定义时刻 t 的最大水分浓度为：

$$C_{\max}(t) = \max_{0\le r\le R} C(r,t) \tag{23}$$

由于 $C_{\max(t)}$ 表示当前时刻药材内部最湿位置的水分状态，因此，当且仅当 $C_{\max(t)}$ 小于临界值 C_{crit} 时，才能保证整个径向区域均满足 $C(r,t)$ 小于临界值。因此，将烘干所需时间定义为最大水分浓度首次下降至临界值以下的时刻，即：

$$t_{\text{dry}} = \inf\{t > 0 : C_{\max}(t) < C_{\text{crit}}\} \tag{24}$$

7.1.2 终止判据的离散化

由于在后续数值计算中，无法直接对连续空间中的全部位置进行求解，因此将连续的径向空间离散为有限个空间点，近似表示连续的含水率场。于是，在第 n 个时间层上，连续形式的最大水分浓度可由各离散节点的水分浓度近似表示为：

$$C_{\max}^{n} = \max_{i} C_i^n \tag{25}$$

当相邻两个时间层满足 $C_{\max}^{n} \ge C_{\text{crit}}, C_{\max}^{n+1} < C_{\text{crit}}$，则说明烘干临界状态发生在 t_n 与 t_{n+1} 之间，即烘干终止时刻满足：$t_n < t_{\text{dry}} \le t_{n+1}$。因此，连续空间的烘干终止判据可以转换为离散节点的最大水分浓度的阈值判定。当上述条件首次成立时，则可以认为数值解已经跨越烘干临界状态，从而进一步确定烘干终止时刻。

7.2 模型求解

问题三与问题二具有相同的热湿传递机理，因此沿用问题二建立的热湿耦合模型以及数值求解方法，仅将计算时间域向后延伸，并且在时间推进过程中引入烘干终止判据，具体求解过程如下：

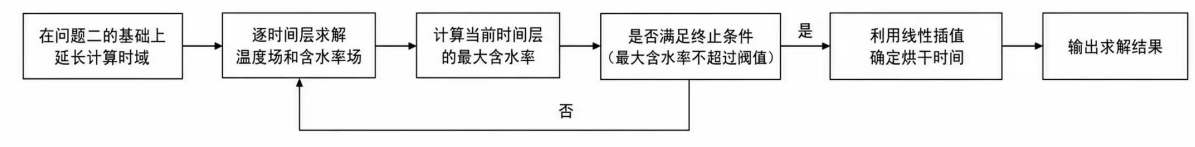


图 **9**　问题三求解过程示意图

Step1：热湿耦合模型的时间推进

沿用问题二的有限体积离散与 Picard 迭代方法，逐时间层求解温度场和含水率场。

Step2：烘干终止时间检测

每获得一个时间层的数值解，就调用离散终止判据进行检测。若当前状态未满足烘干要求，则继续推进下一时间层，若检测的最大含水率首次跨越临界值，则终止全时域计算，并记录临界状态前后的两个时间层 $((t_n, C_{\max}^n), (t_{n+1}, C_{\max}^{n+1}))$，将烘干终点缩小到相邻两个时间层中间。

Step3：临界时刻局部精化

由于离散时间推进只能将烘干终止时刻定位在宽度为 Δt 的区间内，为进一步提高终点精度，因此在临界区域内对最大含水率随时间的变化进行局部近似。根据临界区域两端的数值结果，得到

$$t_{\text{dry}} = t_n + \frac{C_{\max}^n - 0.15}{C_{\max}^n - C_{\max}^{n+1}} \Delta t \tag{26}$$

由此得到药材最大水分浓度首次达到临界值的时刻，并将其作为最终烘干时刻。

7.3 结果与分析

7.3.1 求解结果

求解得到的药材烘干所需要的时间为 57.2107h。部分结果如下表所示，完整结果存于 result3.xlsx 中

表 **6**　药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
6	1.0156	0.9868	0.9005	0.7546	0.5333
12	0.4548	0.4419	0.4020	0.3290	0.1643
18	0.2980	0.2907	0.2680	0.2248	0.0845
24	0.2372	0.2322	0.2162	0.1852	0.0663
30	0.2053	0.2014	0.1888	0.1638	0.0596
36	0.1855	0.1821	0.1715	0.1501	0.0564
42	0.1717	0.1688	0.1594	0.1405	0.0546
48	0.1615	0.1589	0.1504	0.1332	0.0535
54	0.1536	0.1512	0.1434	0.1275	0.0528
57.2107	0.1500	0.1477	0.1402	0.1249	0.0525

7.3.2 结果分析

得到烘干终点后，为了进一步分析结果对应的水分迁移特征，将水分浓度的时空变化进行可视化。从图中得知，随着干燥过程的不断进行，低含水率区首先出现在药材表面，并逐渐向中心扩展。因此，在实际烘干过程中，不能仅根据表面含水率判断烘干是否完成。

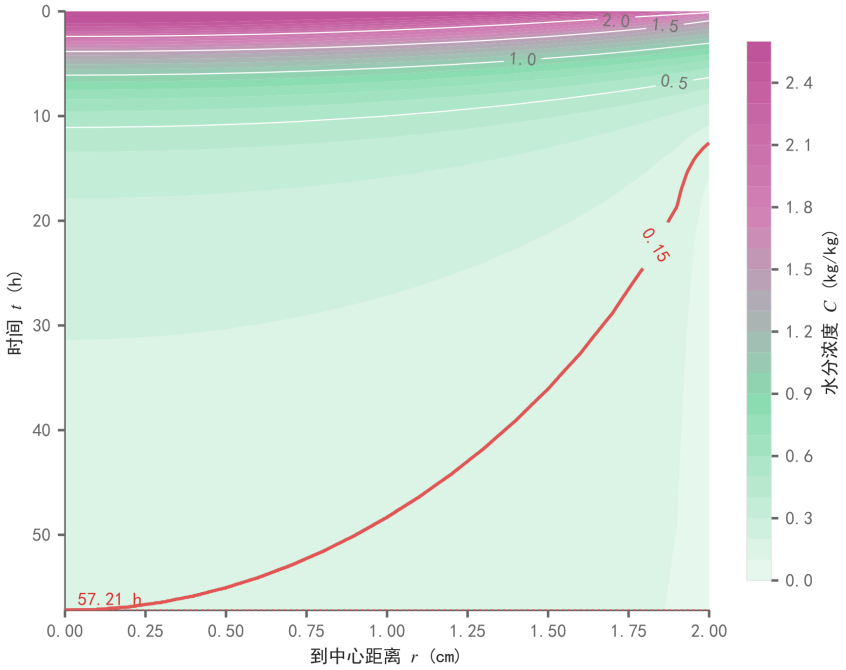


图 **10**　水分分布的时空演化

为了进一步解释结果对应的整体脱水规律，反映整个烘干过程的脱水效率以及变化趋势，绘制了干燥水率-累计脱水率曲线。结果表明，前期水分排出较快，大部分水分能够在较短时间内脱除；随着干燥程度提高，后续水分去除速度明显降低。与时空分布的结果结合，后期虽然新增脱水量已经较小，但主要对应内部残余水分的进一步迁移与排出，因此仍需要较长时间才能满足最终含水率要求。

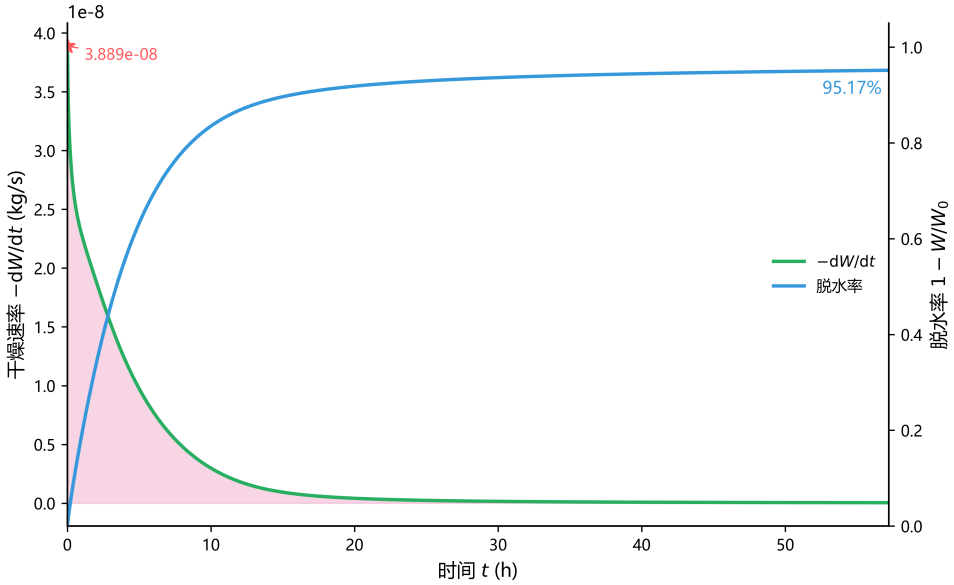


图 **11**　干燥水率-累计脱水率曲线

八、 问题四的模型的建立和求解

8.1 建立考虑收缩效应的非线性水分扩散模型

8.1.1 移动边界下的材料坐标变换

问题四在问题三的模型基础上，进一步考虑药材在烘干过程中因为水分流失而产生的尺寸收缩。假设药材在烘干过程中仅发生均匀径向收缩，各径向位置的收缩比例相同，并始终保持轴对称圆柱形状。随着干燥进行，药材半径不断减小，其内部水分扩散区域也随时间缩小。若仍采用原物理坐标 r 描述水分扩散过程，则计算区域 $0 \le r \le R(t)$ 会随药材半径缩小而变化，从而形成移动边界问题。为便于后续数值计算，将变化的计算区域映射到固定区间 $[0,1]$，从而引入无量纲材料坐标

$$\xi = \frac{r}{R(t)}, \qquad 0 \le \xi \le 1 \tag{27}$$

其中 ξ 表示药材内部质点的位置标签，同一质点在收缩过程中保持 ξ 不变，而其实际物理位置随 $R(t)$ 的变化而变化。

8.1.2 温度场简化及合理性分析

由于水分扩散系数 $D(C,T)$ 同时受含水率和温度影响，水分扩散与热传导存在耦合，理论上需要同时求解温度场与水分场, 但考虑到问题四的主要研究对象为药材内部水分迁移及其收缩效应，需要判断温度场是否可以简化。为了比较药材内部热扩散与整

个干燥过程的时间尺度，引出热扩散特征时间：

$$\tau_T \approx \frac{R_0^2 \rho c_p}{k} \tag{28}$$

计算得 $\tau_T \approx 2.06 \times 10^3\,\mathrm{s}$，远小于整个干燥过程的时间尺度，表明药材内部温度能够较快响应环境温度变化。因此，将温度场简化为环境温度

$$T(\xi, t) = T_a(t) \tag{29}$$

进一步与完整热质耦合模型进行对比，所得干燥时间分别为 $50.7458\,\mathrm{h}$ 和 $50.8241\,\mathrm{h}$，相对差异仅为 0.155%，因此上述温度场简化是合理的。

8.1.3 收缩水分扩散控制方程

在材料坐标 ξ 下，药材的收缩区域被映射至固定区间 $[0, 1]$。结合水分质量守恒与菲克第二扩散定律，可得材料坐标下的水分扩散控制方程为

$$\frac{\partial C}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi D(C, T) \frac{\partial C}{\partial \xi} \right) \qquad 0 < \xi < 1, \quad t > 0 \tag{30}$$

其中，C 为局部含水率，$D(C, T)$ 为水分扩散系数，$R(t)$ 为药材实际半径。式(30)中 $R(t)$ 随时间变化，使收缩效应直接进入水分扩散过程。由于采用材料坐标描述质点运动，均匀径向收缩产生的运动项已被坐标变换吸收，无需在方程中额外引入收缩速度项。

8.1.4 初始条件与传质边界条件

问题四的初始条件与前述模型相同，中心处满足对称无通量条件，表面采用传质边界条件。在材料坐标 ξ 下，可表示为

$$\begin{cases} C(\xi, 0) = C_0 \\ \left. \dfrac{\partial C}{\partial \xi} \right|_{\xi=0} = 0 \\ -\dfrac{D(C_s, T_s)}{R(t)} \left. \dfrac{\partial C}{\partial \xi} \right|_{\xi=1} = h_m \left[C_s - C_a(t) \right] \qquad C_s = C(1, t) \end{cases} \tag{31}$$

8.2 模型求解

问题四采用基于材料坐标的收缩水分扩散求解算法。有限体积离散、向后 Euler 时间推进及 Picard 迭代沿用前述模型，具体求解过程如下。

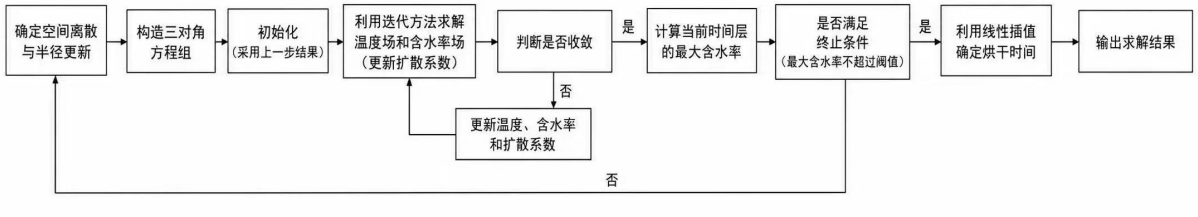


图 **12**　问题四求解过程示意图

Step 1：空间离散与半径更新

将材料坐标 $\xi \in [0,1]$ 划分为 N 个网格，并按时间步推进。第 $n+1$ 个时间层取药材实际半径为

$$R_{n+1} = R(t_{n+1}) \tag{32}$$

将 R_{n+1} 代入收缩水分扩散方程，构造当前时间层的离散方程。

Step 2：非线性迭代求解

由于扩散系数 $D(C,T)$ 随含水率变化，每个时间步采用 Picard 迭代更新 D，并求解对应的三对角线性方程组。当满足

$$\max_i \left| C_i^{(k+1)} - C_i^{(k)} \right| < 10^{-12} \tag{33}$$

时，接受当前时间层的计算结果。

Step 3：烘干终止判定

每个时间层计算药材内部最大含水率

$$C_{\max}^{n} = \max_i C_i^{n} \tag{34}$$

当首次满足

$$C_{\max}^{n+1} \le 0.15 \tag{35}$$

时，认为达到烘干终点。

Step 4：烘干时间计算

设烘干终点位于 t_n 与 t_{n+1} 之间，采用线性插值得到烘干时间

$$t_{\text{dry}} = t_n + \frac{C_{\max}^{n} - 0.15}{C_{\max}^{n} - C_{\max}^{n+1}} \left(t_{n+1} - t_n \right) \tag{36}$$

最终采用 $N = 400$，$0 \le t \le 14400\,\text{s}$ 时 $\Delta t = 1.25\,\text{s}$，其后取 $\Delta t = 5\,\text{s}$ 进行计算。

8.3 求解结果与分析

8.3.1 求解结果

求解得到的药材烘干所需要的时间为 50.7458h。部分结果如下表所示，完整结果存于 result4.xlsx 中。

表 **7**　问题四药材烘干过程的含水率

时间/h	位置/cm			
	0	0.5	1.0	表面
12	0.7268	0.5992	0.3361	0.0617
24	0.3007	0.2738	0.1866	0.0572
36	0.1917	0.1787	0.1311	0.0558
42	0.1704	0.1596	0.1195	0.0539
48	0.1555	0.1462	0.1112	0.0528
50.7458	0.1500	0.1413	0.1081	0.0525

8.3.2 结果分析

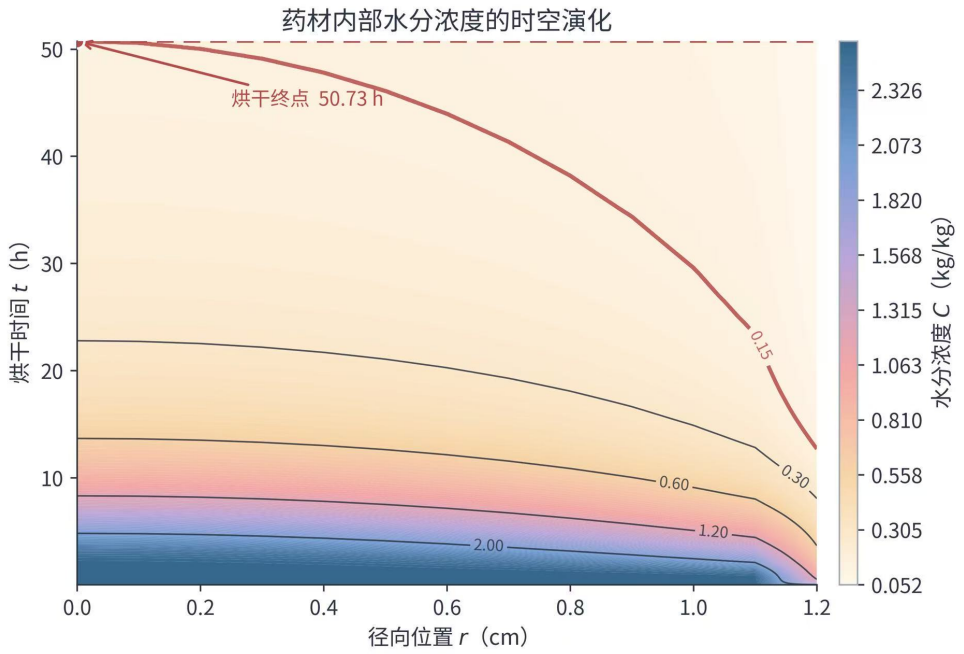


图 **13**　水分浓度的时空演化

为直观反映水分浓度在时间和空间上的演化规律，绘制水分浓度时空分布图。从图中可见，低水分区域逐渐由表面向中心扩散，而中心水分浓度下降较慢，说明烘干后期

主要受内部水分扩散限制。

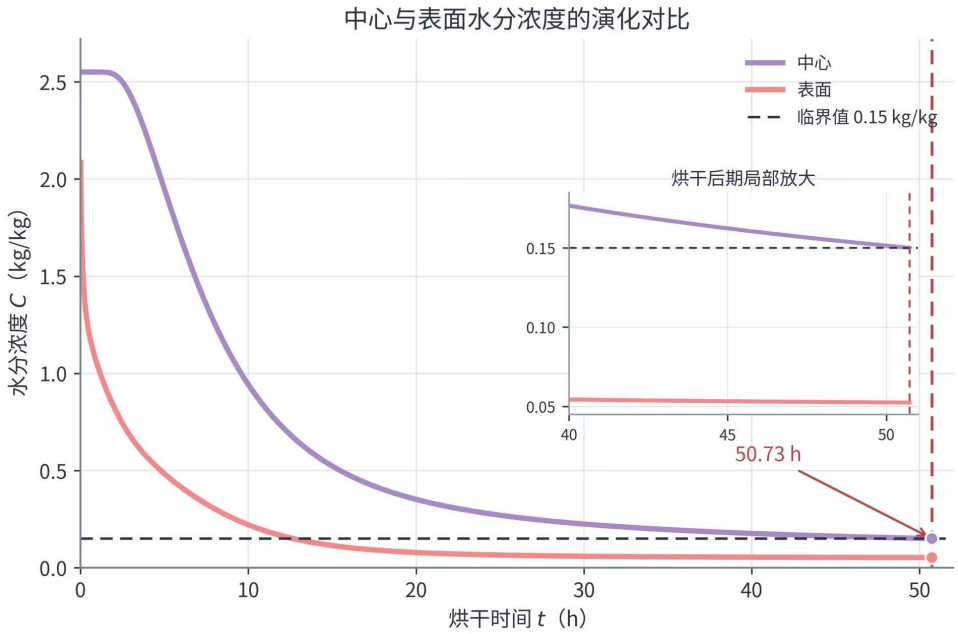


图 **14**　中心与表面水分浓度的演化对比

在此基础上，为了进一步确定烘干时间的关键位置，绘制中心与表面水分浓度演化对比图. 从图中可见，表面水分浓度较早低于临界值，而中心始终是水分浓度最高的位置，表明中心是烘干时间的的控制位置.。

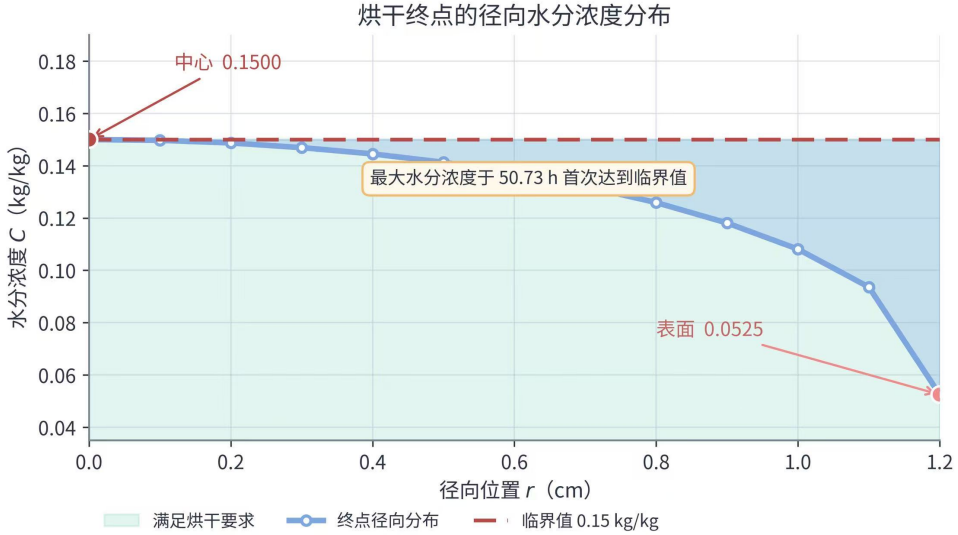


图 **15**　烘干终点的径向水分浓度分布

最后，为验证该时刻药材各处是否达到要求，绘制烘干终点的径向水分浓度分布图. 从图中可见，水分浓度由中心向表面逐渐降低，中心为 0.1500kg/kg，表面为 0.0525kg/kg，

其余位置均未超过临界值。因此，结合上文得到的控制位置以及临界时刻，可以确定得到的烘干时间 50.73h 的正确性。

九、 模型的分析与检验

9.1 误差分析

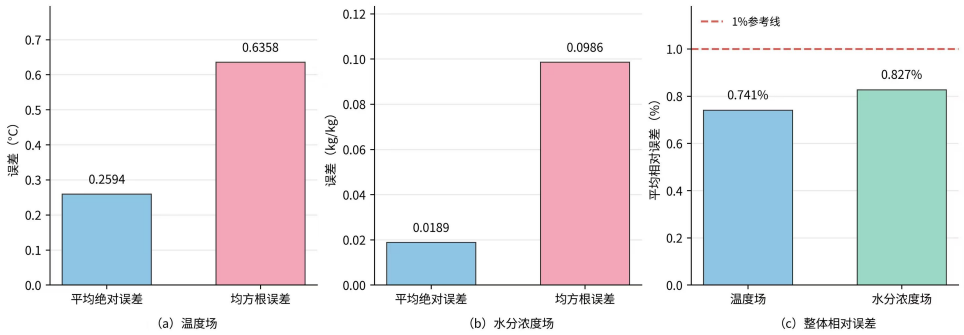


图 **16**　一维与二维模型误差对比

将文章建立得简化一维模型与二维模型进行比较，对温度场和水分浓度场的误差指标进行了对比。结果表明两场在两个模型下的误差小于百分之一，整体误差较小。因此，一维模型能够较准确的描述药材主体区域的温湿变化，具有简化合理性。

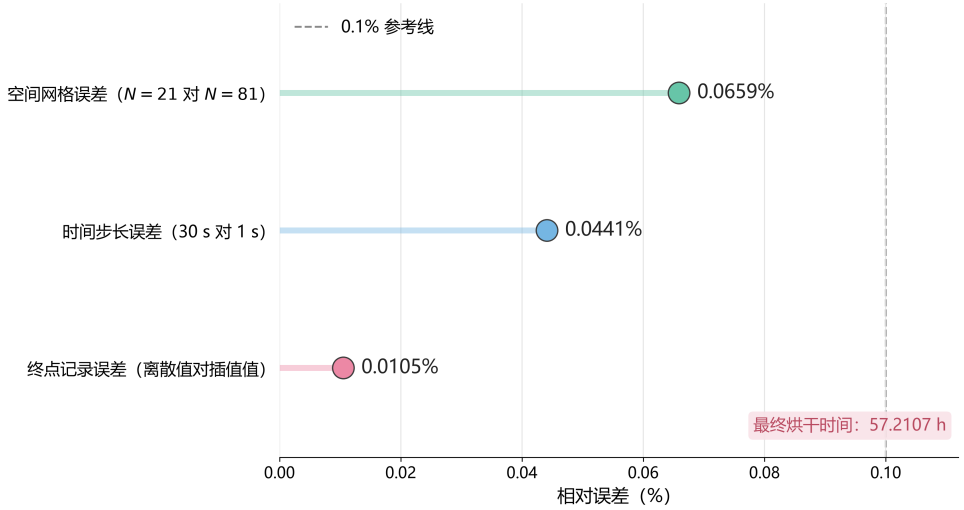


图 **17**　数值离散对误差影响

为检验数值离散对烘干时间的影响，分别计算空间网格，时间步长以及终点记录产生的相对误差，具体结果如图所示。结果可以发现，三类误差均低于百分之 0.1，其中空间网格误差相对较大，但仍然处于较低水平，说明计算结果对网格划分，时间步长和记录间隔均不敏感. 因此，本文得到的烘干时长具有较好的数值稳定性和可靠性。

十、 模型的评价

10.1 模型的优点

- 模型从热量和水分守恒出发建立控制方程，并在问题四中引入材料坐标处理药材收缩，使移动边界能够纳入水分扩散模型。
- 采用有限体积法进行空间离散，较好保持离散过程中的能量与物质守恒，提高数值解的求解精度。
- 以全域最大含水率作为烘干终止判据，保证最难干燥位置难以满足，使得求解的烘干时间更加可靠。
- 模型能够根据不同问题的实际要求逐步扩展，从固定区域的热质传递到非线性耦合，再到考虑尺寸收缩的水分扩散，模型结构具有较好的连续性。

10.2 模型的缺点

- 将药材简化为圆柱体，忽略了实际形状和内部结构差异，使得局部温度和水分迁移存在偏差，影响烘干时间的预测精度。
- 模型未考虑蒸发过程中的相变潜热，使得能量消耗估计偏低，导致温度场升温速度较快，影响烘干时间的判定。

10.3 模型的推广

通过调整材料的物理性质和水分扩散参数，可以推广至食品，木材等介质的干燥过程，实现不同材料的烘干时间的预测. 模型还可以预测材料内部的温度场和含水率场，从而减少局部不足或者过度烘干现象，为制定工业烘干策略提供参考。

十一、 AI 工具使用声明

11.1 AI 工具使用概述

本团队在 2026 年全国大学生数学建模竞赛 A 题的研究与论文撰写过程中，使用 AI 工具辅助开展题意梳理、建模与算法思路讨论、数学公式推导讨论、文献检索、程序编写与调试、图表生成、结果检查与解释、论文结构和文字表达，以及本报告的整理。具体的交互过程见支撑材料的 AI 使用报告。

使用过程中，AI 提供候选方法、程序框架或表述建议，团队结合题目要求讨论其可行性并进行取舍。部分 AI 提出的数值方法建议、程序代码、绘图框架和论文表达方案进入了最终成果。团队对采用内容开展了代码审查与修改、程序运行测试、题目数据核对、公式校验、文献原文阅读及结果数值合理性检查。

11.2 AI 工具基本信息

表 **8**　**AI** 工具基本信息

序号	工具或平台	后端模型或显示名称	使用日期
1	Codex	GPT-6 Astra	2026 年 9 月 10 日至 13 日

11.3 人工决策与责任说明

团队对 AI 提供的候选方法、程序代码、文献信息及文字建议进行了讨论与取舍，并作出最终采用决定。对于采用的程序，团队开展了审查、修改与运行测试；对于相关计算内容，开展了题目数据核对、公式校验和数值合理性检查；对于引用文献，人工确认其存在并阅读原文。

本报告披露 AI 在建模与算法讨论、程序实现、文献检索、图表生成、结果检查及论文表达等环节的实际参与情况。AI 输出不作为无需审查即可成立的依据。团队对最终提交的论文、程序、数据、图表、结论及本报告的真实性、原创性与准确性承担责任。

参考文献

[1] RUDOBASHTA S P, KARTASHOV E M, ZUEVA G A. Heat and mass transfer during continuous infrared drying of a cylindrical body[J/OL]. Journal of Engineering Physics and Thermophysics, 2026. DOI: 10.1007/s10891-026-03392-3.

[2] HERNANDEZ RAMIREZ J F, MENDEZ VASQUEZ R, PACHECO PEREZ Y Y. Effect of moisture content on mass transfer coefficient during non-isothermal drying and devel-

opment of a new thin-layer model[J/OL]. Chemical Engineering Communications, 2026, 213(6):983-997. DOI: 10.1080/00986445.2025.2598588.

[3] XANTHOPOULOS G, LENTZOU D, PAPADAKIS S. Moving-boundary finite element modeling and inverse identification of moisture-dependent diffusivity in convective drying of apricot hemispheres: Uncertainty quantification[J/OL]. Drying Technology, 2026, 44 (12):1804-1820. DOI: 10.1080/07373937.2026.2686407.

[4] ZHU Y, WANG P, SUN D, et al. Multiphase porous media model with thermo-hydro and mechanical bidirectional coupling for food convective drying[J/OL]. International Journal of Heat and Mass Transfer, 2021, 175:121356. DOI: 10.1016/j.ijheatmasstransfer.2021. 121356.

[5] NING Z, KHIR R, NIEDERHOLZER F, et al. Modeling of heat and moisture transfer for during convective drying of in-hull almonds considering non-isotropic shrinkage and variable moisture diffusivity[J/OL]. International Journal of Heat and Mass Transfer, 2026, 258:128286. DOI: 10.1016/j.ijheatmasstransfer.2025.128286.

[6] PATANKAR S V. Numerical heat transfer and fluid flow[M]. New York: Hemisphere Publishing Corporation, 1980.

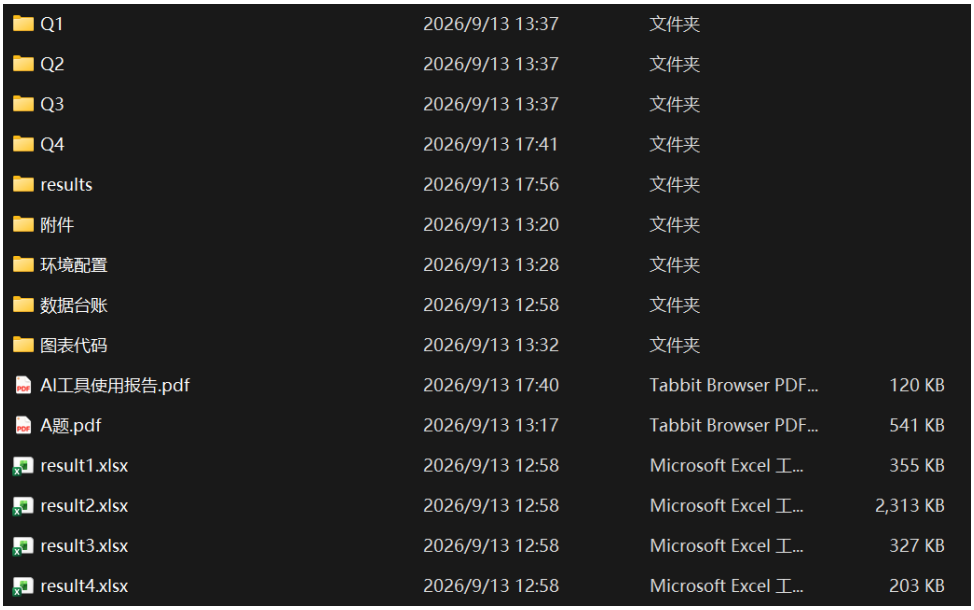
[7] CARSLAW H S, JAEGER J C. Conduction of heat in solids[M]. 2nd ed. Oxford: Oxford University Press, 1959.

[8] KOH C H, BROUWERS H J H. A simple two-phase approach to predict moisture-dependent thermal conductivity for porous materials[J/OL]. International Journal of Heat and Mass Transfer, 2024, 234:126138. DOI: 10.1016/j.ijheatmasstransfer.2024.126138.

[9] NABIPOOR HASSANKIADEH M, SPITERI R J, ZEYNALI R, et al. Modeling of multiphase heat and mass transport and drying behavior of potash particles in a packed bed: Experiments and simulation[J/OL]. Chemical Engineering Research and Design, 2023, 200:419-430. DOI: 10.1016/j.cherd.2023.10.057.

[10] NANDI T N, YEO D. A solution verification study for urans simulations of flow over a 5:1 rectangular cylinder using grid convergence index and least squares procedures[J/OL]. Journal of Verification, Validation and Uncertainty Quantification, 2023, 8(4):041001. DOI: 10.1115/1.4063818.

附录 A　文件列表



Q1	2026/9/13 13:37	文件夹	
Q2	2026/9/13 13:37	文件夹	
Q3	2026/9/13 13:37	文件夹	
Q4	2026/9/13 17:41	文件夹	
results	2026/9/13 17:56	文件夹	
附件	2026/9/13 13:20	文件夹	
环境配置	2026/9/13 13:28	文件夹	
数据台账	2026/9/13 12:58	文件夹	
图表代码	2026/9/13 13:32	文件夹	
AI工具使用报告.pdf	2026/9/13 17:40	Tabbit Browser PDF...	120 KB
A题.pdf	2026/9/13 13:17	Tabbit Browser PDF...	541 KB
result1.xlsx	2026/9/13 12:58	Microsoft Excel 工...	355 KB
result2.xlsx	2026/9/13 12:58	Microsoft Excel 工...	2,313 KB
result3.xlsx	2026/9/13 12:58	Microsoft Excel 工...	327 KB
result4.xlsx	2026/9/13 12:58	Microsoft Excel 工...	203 KB

附录 B　代码

q1.py

```python
from __future__ import annotations
import json
import math
import sys
import time
from pathlib import Path
import numpy as np
from scipy.linalg import solve_banded
from scipy.sparse import coo_matrix
from scipy.sparse.linalg import splu

_PROJ = Path(__file__).resolve()
ROOT = _PROJ.parents[1]

from solver_q1 import (
    solve_q1, build_grid, D_of_C, alpha, H, HM, K, RHO, CP, R,
    R_CM,
    T_INIT, C_INIT, interp_driver, surface_robin_terms, RHO_D,
```

```
    rho_v_sat, rho_v_inf,
)
from independent_check_q1_temp import (
    load_oven_temperature, duhamel_temperature,
)

T_REF_CENTER = 33.575344826514
T_REF_SURF = 36.785560351561
T_END = 1800.0
DR_CM = 0.01
DT_IN = 1.0 / 64

def _mass_step_buggy(C_old, r, dr, dt, t_new, d_bug_iters=30):
    N = len(r) - 1
    C = C_old.copy()
    rp = r + dr / 2.0
    rm = r - dr / 2.0
    geo_in = np.zeros(N + 1)
    geo_out = np.zeros(N + 1)
    geo_in[1:N] = dt * rm[1:N] / (r[1:N] * dr * dr)
    geo_out[1:N] = dt * rp[1:N] / (r[1:N] * dr * dr)
    c0_factor = 4.0 * dt / (dr * dr)
    C_air_new = interp_driver(t_new, "linear")[1]
    for _ in range(d_bug_iters):
        C_it = C.copy()
        Dv = D_of_C(np.maximum(C_it, 1e-12))
        dm = 0.5 * (Dv[:-1] + Dv[1:])
        d = np.zeros(N + 1)
        l = np.zeros(N + 1)
        u = np.zeros(N)
        d[1:N] = 1.0 + geo_in[1:N] * dm[0:N - 1] + geo_out[1:N
   ] * dm[1:N]
        l[1:N] = -geo_in[1:N] * dm[0:N - 1]
        u[1:N] = -geo_out[1:N] * dm[1:N]
        a0 = c0_factor * dm[0]
```

```
        d[0], u[0] = 1.0 + a0, -a0
        m, a_in, a_out = surface_robin_terms(dr, dm[N - 1], HM
    )
        d[N] = m / dt + a_in + a_out
        l[N] = -a_in
        rhs = C_it.copy()
        rhs[N] = (m / dt) * C_it[N] + a_out * C_air_new
        ab = np.zeros((3, N + 1))
        ab[0, 1:] = u
        ab[1, :] = d
        ab[2, :-1] = l[1:]
        C = solve_banded((1, 1), ab, rhs)
    return C


def verify_picard_old_layer():
    r, dr, N = build_grid(DR_CM)
    dt = DT_IN
    few = solve_q1(case="A", dr_cm=DR_CM, dt=dt, t_end=50.0,
    dt_out=50.0, picard_max=2)
    many = solve_q1(case="A", dr_cm=DR_CM, dt=dt, t_end=50.0,
    dt_out=50.0, picard_max=50)
    dmax = float(np.max(np.abs(few["C_out"][-1] - many["C_out"
    ][-1])))
    print(f"t=50 s 场：max|C(2 次迭代) - C（50 次迭代）| = {
    dmax:.3e} kg/kg")
    print(f"生产运行 Picard 统计：{many['picard']}")

    prod = solve_q1(case="A", dr_cm=DR_CM, dt=DT_IN, t_end=
    T_END, dt_out=T_END)
    C_surf_correct = float(prod["scalars"]["CN"][-1])
    C_bug = np.full(N + 1, C_INIT)
    dt_bug = 1.0
    n_steps = int(round(T_END / dt_bug))
    for k in range(1, n_steps + 1):
```

```
        C_bug = _mass_step_buggy(C_bug, r, dr, dt_bug, k * dt_bug, d_bug_iters=5)
    C_surf_bug = float(C_bug[-1])
    print(f"反例复现：表面 C（1800 s）正确 = {C_surf_correct:.6f}；"
          f"把上一迭代解当上一时间层 = {C_surf_bug:.6f} "
          f"（偏差 {100*(C_surf_bug-C_surf_correct)/C_surf_correct:+.2f}%）")
    return {"max_abs_diff_2_vs_50_iters": dmax,
            "pass": bool(dmax < 1e-9),
            "criterion": "2 次 vs 50 次迭代的场差 ≤1e-9 kg/kg",
            "picard_prod": prod["picard"],
            "counterexample_surface_C_correct": C_surf_correct,
            "counterexample_surface_C_buggy": C_surf_bug,
            "counterexample_rel_pct": 100 * (C_surf_bug - C_surf_correct) / C_surf_correct}


def axial_2d_links(dr_cm=0.05, dz_cm=0.5, L_CM=25.0):
    Nr = int(round(R_CM / dr_cm))
    dr = R / Nr
    Lh = L_CM / 2.0 / 100.0
    Nz = int(round(Lh / (dz_cm / 100.0)))
    dz = Lh / Nz
    r = np.arange(Nr + 1) * dr
    rp = np.minimum(r + dr / 2.0, R)
    rm = np.maximum(r - dr / 2.0, 0.0)
    r2d = rp ** 2 - rm ** 2
    dzj = np.full(Nz + 1, dz)
    dzj[0] = dz / 2.0
    dzj[Nz] = dz / 2.0
    nr, nz = Nr + 1, Nz + 1
    n = nr * nz
```

```
110
111     def idx(i, j):
112         return i * nz + j
113
114     vol = np.empty(n)
115     bcoef_geom = np.zeros(n)
116     lp, lq, lAd = [], [], []
117     for i in range(nr):
118         for j in range(nz):
119             p = idx(i, j)
120             vol[p] = r2d[i] * dzj[j]
121             if i > 0:
122                 lp.append(p); lq.append(idx(i - 1, j)); lAd.
    append(2.0 * rm[i] * dzj[j] / dr)
123             if i < Nr:
124                 lp.append(p); lq.append(idx(i + 1, j)); lAd.
    append(2.0 * rp[i] * dzj[j] / dr)
125             else:
126                 bcoef_geom[p] += 2.0 * R * dzj[j]
127             if j > 0:
128                 lp.append(p); lq.append(idx(i, j - 1)); lAd.
    append(r2d[i] / dz)
129             if j < Nz:
130                 lp.append(p); lq.append(idx(i, j + 1)); lAd.
    append(r2d[i] / dz)
131             if j == 0:
132                 bcoef_geom[p] += r2d[i]
133     return dict(r=r, z=np.arange(Nz + 1) * dz, dr=dr, dz=dz,
    Nr=Nr, Nz=Nz, nr=nr, nz=nz,
134                 n=n, idx=idx, vol=vol, bcoef_geom=bcoef_geom,
135                 lp=np.array(lp), lq=np.array(lq), lAd=np.array
    (lAd))
136
137
138 def assemble_axial_2d(G, links, bcoef, dt):
```

```
    Gl = 0.5 * (G[links["lp"]] + G[links["lq"]]) * links["lAd"]
    sumG = np.bincount(links["lp"], weights=Gl, minlength=links["n"])
    dod = 1.0 + (sumG + bcoef) * dt / links["vol"]
    off = -Gl * dt / links["vol"][links["lp"]]
    rows = np.concatenate([links["lp"], np.arange(links["n"])])
    cols = np.concatenate([links["lq"], np.arange(links["n"])])
    data = np.concatenate([off, dod])
    return coo_matrix((data, (rows, cols)), shape=(links["n"], links["n"])).tocsc()


def solve_axial_2d(dr_cm=0.05, dz_cm=0.5, dt=1.0 / 8, t_end=T_END, dt_out=1.0,
                   n_picard=2, L_CM=25.0, verbose=False):
    links = axial_2d_links(dr_cm, dz_cm, L_CM)
    n, vol, bcoef_geom = links["n"], links["vol"], links["bcoef_geom"]
    al = alpha()
    Ah = assemble_axial_2d(np.full(n, al), links, bcoef_geom * H / (RHO * CP), dt)
    lu_h = splu(Ah)
    bh_geom = bcoef_geom * (H / (RHO * CP)) * dt / vol

    T = np.full(n, T_INIT)
    C = np.full(n, C_INIT)
    n_steps = int(round(t_end / dt))
    gap = int(round(dt_out / dt))
    T_last = None
    C_last = None
    for step in range(1, n_steps + 1):
        t_new = step * dt
```

```
        T_air, C_air = interp_driver(t_new, "linear")
        T = lu_h.solve(T + bh_geom * T_air)
        C_old = C.copy()
        C_rhs = C_old + bcoef_geom * HM * dt / vol * C_air
        for _ in range(n_picard):
            Dv = D_of_C(np.maximum(C, 1e-12))
            Am = assemble_axial_2d(Dv, links, bcoef_geom * HM, dt)
            C = splu(Am).solve(C_rhs)
        if step % gap == 0:
            T_last = T.copy()
            C_last = C.copy()
    return dict(T=T_last.reshape(links["nr"], links["nz"]),
                C=C_last.reshape(links["nr"], links["nz"]), geom=links)


def conservation_pair(A, B):
    sc, scB = A["scalars"], B["scalars"]
    consA = float(abs((sc["mass"][0] - sc["mass"][-1]) - sc["cum_flux"][-1])
                  / (sc["mass"][0] - sc["mass"][-1]))
    consB = float(abs((scB["mass"][0] - scB["mass"][-1]) - scB["cum_flux"][-1])
                  / abs(scB["mass"][0] - scB["mass"][-1]))
    consB_abs = float(abs((scB["mass"][0] - scB["mass"][-1]) - scB["cum_flux"][-1]))
    consB_scaled = consB_abs / scB["mass"][0]
    print(f"P3  守恒校验：口径 A 相对误差 {consA:.3e}（判据 ≤1e-8） "
          f"{'PASS' if consA <= 1e-8 else 'FAIL'}")
    print(f"        口径 B 相对失水量误差 {consB:.3e}（失水量仅 {scB['mass'][0]-scB['mass'][-1]:.3e}，"
          f"该比值被机器舍入主导）")
    print(f"        口径 B 绝对闭合误差 {consB_abs:.3e}，相对
```

```
    初始总量 {consB_scaled:.3e}  "
194             f"{'PASS' if consB_scaled <= 1e-8 else 'FAIL'}")
195     return {"caseA_rel_err": consA,
196             "caseB_rel_err_vs_loss": consB,
197             "caseB_abs_err": consB_abs,
198             "caseB_rel_err_vs_total_mass": consB_scaled,
199             "criterion": "相对误差 ≤1e-8",
200             "pass": bool(consA <= 1e-8 and consB_scaled <= 1e
    -8)}
201
202
203 def physical_checks(A, B, sc):
204     print("S1–S10 物理自检")
205     S = {}
206     dT0 = float(np.max(np.abs(A["T_out"][0] - T_INIT)))
207     dC0 = float(np.max(np.abs(A["C_out"][0] - C_INIT)))
208     S["S1_t0_equals_init"] = {"max_dev_T": dT0, "max_dev_C":
    dC0,
209                               "pass": bool(dT0 <= 1e-12 and
    dC0 <= 1e-12)}
210     dT = np.diff(sc["T0"])
211     S["S2_center_T_nondecreasing"] = {"min_step_delta": float(
    dT.min()),
212                                       "n_negative_steps": int
    ((dT < 0).sum()),
213                                       "pass": bool(dT.min() >=
     0.0)}
214     diff3 = sc["TN"] - sc["T0"]
215     S["S3_surface_T_ge_center_T"] = {"min_surface_minus_center
    ": float(diff3.min()),
216                                      "at_1800s": float(diff3
    [-1]),
217                                      "pass": bool(diff3.min()
    >= -1e-12)}
218     T_air_series = np.array([interp_driver(t, "linear")[0] for
```

```
 t in A["t_out"]])
 exc = float(np.max(A["T_out"] - T_air_series[:, None]))
 S["S4_below_oven_T"] = {"max_excess_C": exc, "criterion": 1e-9,
                         "pass": bool(exc <= 1e-9)}
 dm = np.diff(sc["mass"])
 S["S5_mass_strictly_decreasing"] = {"max_step_delta": float(dm.max()),
                                     "n_positive_steps": int((dm > 0).sum()),
                                     "mass_ratio_1800": float(sc["mass"][-1] / sc["mass"][0]),
                                     "pass": bool(dm.max() < 0.0)}
 dC_prof = np.diff(A["C_out"], axis=1)
 S["S6_C_profile_radially_nonincreasing"] = {
     "max_increase": float(dC_prof.max()), "pass": bool(dC_prof.max() <= 1e-12)}
 dT_prof = np.diff(A["T_out"], axis=1)
 S["S7_T_profile_radially_nondecreasing"] = {
     "min_increase": float(dT_prof.min()), "pass": bool(dT_prof.min() >= -1e-12)}
 dev8 = float(abs(A["C_out"][int(T_END)][0] - C_INIT))
 S["S8_center_moisture_static"] = {"abs_dev_1800": dev8, "criterion": 1e-3,
                                   "pass": bool(dev8 <= 1e-3)}
 t_air, T_oven = load_oven_temperature()
 i_out = np.arange(0, A["N"] + 1, 10)
 T_an = duhamel_temperature([T_END], i_out * DR_CM / 100.0, t_air, T_oven,
                            n_roots=320)[0]
 e9 = np.abs(A["T_out"][int(T_END)][i_out] - T_an)
 S["S9_vs_analytic"] = {"max_abs_err_C": float(e9.max()), "at_center": float(e9[0]),
```

```
                            "at_surface": float(e9[-1]), "criterion": 5e-5,
                            "pass": bool(e9.max() < 5e-5)}
    CsB = float(B["C_out"][int(T_END)][-1])
    devB = float(np.max(np.abs(B["C_out"][int(T_END)] - C_INIT)))
    S["S10_caseB_no_drying"] = {"surface_C_1800": CsB, "max_field_dev": devB,
                                "scoping_surface_ref": 2.549885,
                                "pass": bool(abs(CsB - 2.549885) < 1e-3 and devB < 1e-3)}
    for k, v in S.items():
        print(f"  {k:<40} {'PASS' if v['pass'] else 'FAIL'}  "
              + ", ".join(f"{kk}={vv:.6g}" for kk, vv in v.items()
                          if kk not in ("pass",) and isinstance(vv, (int, float))))
    return S, T_air_series


def case_AB_compare(A, B):
    print("对比验证：口径 A vs 口径 B")
    qA = HM * (A["C_out"][1:, -1]
               - np.array([interp_driver(t, "linear")[1] for t in A["t_out"][1:]]))
    TsB = B["T_out"][1:, -1]
    w_all = np.array([interp_driver(t, "linear")[1] for t in B["t_out"][1:]])
    Ta_all = np.array([interp_driver(t, "linear")[0] for t in B["t_out"][1:]])
    qB = HM * (rho_v_sat(TsB) - rho_v_inf(w_all, Ta_all)) / RHO_D
    sc, scB = A["scalars"], B["scalars"]
    cmpAB = {
```

```
        "surface_C_1800s": {"A": float(A["C_out"][int(T_END)][-1]),
                            "B": float(B["C_out"][int(T_END)][-1])},
        "center_C_1800s": {"A": float(A["C_out"][int(T_END)][0]),
                           "B": float(B["C_out"][int(T_END)][0])},
        "mass_ratio_1800s": {"A": float(sc["mass"][-1] / sc["mass"][0]),
                             "B": float(scB["mass"][-1] / scB["mass"][0])},
        "flux_kg_m2_s_1800s": {"A": float(qA[-1] * RHO_D), "B": float(qB[-1] * RHO_D)},
        "flux_ratio_A_over_B": float(abs(qA[-1] / qB[-1])),
        "rho_d_kg_m3": RHO_D,
    }
    print(f"  表面 C(1800 s): A {cmpAB['surface_C_1800s']['A']:.6f} vs "
          f"B {cmpAB['surface_C_1800s']['B']:.9f}")
    print(f"  总水量比: A {cmpAB['mass_ratio_1800s']['A']:.6f} vs "
          f"B {cmpAB['mass_ratio_1800s']['B']:.9f}")
    print(f"  通量比 A/B = {cmpAB['flux_ratio_A_over_B']:.4e}")
    return cmpAB, qA, qB


def plane_2d_compare(A, T_air_series, quick):
    print("对比验证: 1D vs 2D 含端面")
    if quick:
        print("quick: 跳过 2D")
        return {"skipped": True}
    t0 = time.time()
    DT_2D = 1.0 / 4
```

```
    o2 = solve_axial_2d(dr_cm=0.05, dz_cm=0.5, dt=DT_2D, t_end=T_END, n_picard=2)
    Tm = o2["T"]
    C2 = o2["C"]
    g2 = o2["geom"]
    k_mid = g2["Nz"]
    o1 = solve_q1(case="A", dr_cm=0.05, dt=DT_2D, t_end=T_END, dt_out=T_END)
    T_1d_c = float(o1["T_out"][-1][0])
    C_1d_c = float(o1["C_out"][-1][0])
    d1 = float(abs(Tm[0, k_mid] - T_1d_c))
    d2 = float(Tm[0, 0] - T_1d_c)
    d3 = float(abs(C2[0, k_mid] - C_1d_c))
    zrow = Tm[0, :]
    d4 = float((zrow.max() - zrow.min()) / (T_air_series[-1] - T_INIT))
    cmp2d = {"dr_cm": 0.05, "dz_cm": 0.5, "Nz": g2["Nz"], "dt_s": DT_2D,
             "T_mid_center_1800": float(Tm[0, k_mid]),
             "T_end_center_1800": float(Tm[0, 0]),
             "T_1d_center_1800_same_mesh_and_dt": T_1d_c,
             "T_1d_center_1800_production": float(A["T_out"][int(T_END)][0]),
             "delta1_mid_plane_C": d1, "delta2_end_face_C": d2,
             "delta3_mid_plane_C_moisture": d3,
             "delta4_z_nonuniformity": d4,
             "criteria": {"delta1_le_0.5C": bool(d1 <= 0.5),
                          "delta3_le_5e-3": bool(d3 <= 5e-3)},
             "wall_s": time.time() - t0}
    print(f"  Δ1 中截面偏差 = {d1:.4f} ℃  {'PASS' if d1 <= 0.5 else 'FAIL'}")
    print(f"  Δ2 端面中心偏差 = {d2:.4f} ℃")
    print(f"  Δ3 中截面水分偏差 = {d3:.3e} kg/kg  {'PASS' if d3 <= 5e-3 else 'FAIL'}")
```

```
318     print(f"  Δ4 z 向不均匀度 = {d4:.4f}; 耗时 {cmp2d['wall_s
    ']:.1f} s")
319     return cmp2d
320
321
322 def diffusivity_and_interp_compare(A, N):
323     print("对比验证：常数 D vs D(C)；线性 vs 阶梯插值")
324     D_CONST = 1.0 / 8
325     oD = solve_q1(case="A", dr_cm=DR_CM, dt=D_CONST, t_end=
    T_END, dt_out=T_END,
326                   D_CONST=float(D_of_C(C_INIT)))
327     oS = solve_q1(case="A", dr_cm=DR_CM, dt=D_CONST, t_end=
    T_END, dt_out=T_END,
328                   INTERP="step")
329     oL = solve_q1(case="A", dr_cm=DR_CM, dt=D_CONST, t_end=
    T_END, dt_out=T_END)
330     cmpDC = {"D_const_value": float(D_of_C(C_INIT)),
331              "surface_C_1800_variableD": float(oL["C_out"
    ][-1][-1]),
332              "surface_C_1800_constD": float(oD["C_out"
    ][-1][-1]),
333              "surface_diff": float(oD["C_out"][-1][-1] - oL["
    C_out"][-1][-1]),
334              "center_diff": float(oD["C_out"][-1][0] - oL["
    C_out"][-1][0]),
335              "r1cm_diff": float(oD["C_out"][-1][N // 2] - oL["
    C_out"][-1][N // 2]),
336              "mass_ratio_variableD": float(oL["scalars"]["mass
    "][-1] / oL["scalars"]["mass"][0]),
337              "mass_ratio_constD": float(oD["scalars"]["mass"
    ][-1] / oD["scalars"]["mass"][0])}
338     cmpINT = {"linear_surface_C_1800": float(oL["C_out"
    ][-1][-1]),
339               "step_surface_C_1800": float(oS["C_out"
    ][-1][-1]),
```

```
            "surface_diff": float(oS["C_out"][-1][-1] - oL["C_out"][-1][-1]),
            "linear_center_C_1800": float(oL["C_out"][-1][0]),
            "step_center_C_1800": float(oS["C_out"][-1][0]),
            "center_diff": float(oS["C_out"][-1][0] - oL["C_out"][-1][0]),
            "T_center_diff_1800": float(oS["T_out"][-1][0] - oL["T_out"][-1][0]),
            "T_surface_diff_1800": float(oS["T_out"][-1][-1] - oL["T_out"][-1][-1])}
    print(f"  常数 D：表面 {cmpDC['surface_C_1800_constD']:.6f} vs 变系数 "
          f"{cmpDC['surface_C_1800_variableD']:.6f}（差 {cmpDC['surface_diff']:+.6f}）")
    print(f"  阶梯插值：表面 {cmpINT['step_surface_C_1800']:.6f} vs 线性 "
          f"{cmpINT['linear_surface_C_1800']:.6f}（差 {cmpINT['surface_diff']:+.6f}）")
    return cmpDC, cmpINT


def driving_envelope(i_out, dt_scan):
    print("对比验证：驱动势包络 h_m × {0, 0.001, …, 1000, ∞}")
    env = []
    mults = [0.0, 0.001, 0.01, 0.1, 1.0, 10.0, 100.0, 1000.0]
    for m in mults:
        o = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=T_END, dt_out=T_END, HM=HM * m)
        env.append({"hm_mult": m, "hm_value": HM * m,
                    "C_1800": [float(v) for v in o["C_out"][-1][i_out]],
                    "mass_ratio": float(o["scalars"]["mass"][-1] / o["scalars"]["mass"][0])})
```

```
    o_inf = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=T_END, dt_out=T_END,
                     HM_INF=True)
    env.append({"hm_mult": float("inf"), "hm_value": None,
                "C_1800": [float(v) for v in o_inf["C_out"][-1][i_out]],
                "mass_ratio": float(o_inf["scalars"]["mass"][-1] / o_inf["scalars"]["mass"][0])})
    for e in env:
        print(f"  h_m×{str(e['hm_mult']):<7} C(2cm,1800) = {e['C_1800'][-1]:.6f}"
              f"   C(1cm)={e['C_1800'][10]:.6f}  mass={e['mass_ratio']:.6f}")
    mon = all(env[i]["C_1800"][k] >= env[i + 1]["C_1800"][k] - 1e-12
              for i in range(len(env) - 1) for k in range(len(i_out)))
    return env, {
        "rows": env, "monotone_nonincreasing_in_hm": bool(mon),
        "acceptance": {"hm0_max_dev_from_init": float(np.max(np.abs(
            np.array(env[0]["C_1800"]) - C_INIT))),
            "hm_inf_surface_vs_Cair": float(abs(env[-1]["C_1800"][-1] - 0.03307))}}


def parameter_sensitivity(N, dt_scan):
    print("灵敏度：单参数（h、h_m、k、ρ、c□）+ 多参数联合（h×h_m）")
    scans = {
        "h": ("H", [H * 0.5, H, H * 2.0]),
        "h_m": ("HM", [HM * 0.5, HM, HM * 2.0]),
        "k": ("K", [K * 0.5, K, K * 2.0]),
        "rho": ("RHO", [RHO * 0.5, RHO, RHO * 2.0]),
```

```
        "c0": ("C_INIT", [C_INIT * 0.9, C_INIT, C_INIT * 1.1]),
    }
    base_vals = {"H": H, "HM": HM, "K": K, "RHO": RHO, "C_INIT": C_INIT}
    sens = {}
    for name, (key, vals) in scans.items():
        rows = []
        for v in vals:
            o = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=T_END, dt_out=T_END,
                         **{key: v})
            rows.append({"value": float(v), "mult": float(v / base_vals[key]),
                         "T_center_1800": float(o["T_out"][-1][0]),
                         "T_surface_1800": float(o["T_out"][-1][-1]),
                         "C_center_1800": float(o["C_out"][-1][0]),
                         "C_r1cm_1800": float(o["C_out"][-1][N // 2]),
                         "C_surface_1800": float(o["C_out"][-1][-1]),
                         "mass_ratio": float(o["scalars"]["mass"][-1] / o["scalars"]["mass"][0])})
        sens[name] = {"param_key": key, "rows": rows}
        print(f"  {name:<5} " + " | ".join(
            f"×{row['mult']:.2f}: C_s={row['C_surface_1800']:.4f} T_c={row['T_center_1800']:.4f}"
            for row in rows))

    o_rho = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=T_END, dt_out=T_END,
                     RHO=976.4)
```

```
    sens["alpha_pm19pct"] = {
        "note": " ρ 换成附录3 的 ρ(C=2.55)=976.4  α 降 19.07%，仅影响温度场",
        "T_center_1800": float(o_rho["T_out"][-1][0]),
        "T_surface_1800": float(o_rho["T_out"][-1][-1]),
        "C_surface_1800": float(o_rho["C_out"][-1][-1]),
    }

    joint = []
    for hh in [H * 0.5, H, H * 2.0]:
        for mm in [HM * 0.5, HM, HM * 2.0]:
            o = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=T_END, dt_out=T_END,
                         H=hh, HM=mm)
            joint.append({"h": float(hh), "hm": float(mm),
                          "T_center_1800": float(o["T_out"][-1][0]),
                          "C_surface_1800": float(o["C_out"][-1][-1]),
                          "mass_ratio": float(o["scalars"]["mass"][-1] / o["scalars"]["mass"][0])})
    sens["joint_h_x_hm"] = {"rows": joint}
    return sens, joint


def error_budget(conv):
    return {
        "temperature_time_discretization": {
            "min_combo": conv["table2_dt_sweep_temperature"]["min_combo_meeting_acceptance"],
            "order_p": conv["table2_dt_sweep_temperature"]["p_order_center"]},
        "temperature_space_discretization": {
            "p_order": conv["table1_dr_sweep_temperature"]["p_order_center"], },
```

```
436         "moisture_surface_grid_uncertainty": {
437             "p_order": conv["table4_dr_sweep_moisture_surface"
    ]["p_order"],
438             "richardson": conv["
    table4_dr_sweep_moisture_surface"]["richardson_surface_C"],
439             "uncertainty": conv["
    table4_dr_sweep_moisture_surface"]["
    grid_uncertainty_vs_richardson"]},
440         "model_error_latent_heat": {
441             "note": "能量方程未耦合蒸发潜热  系统性高估表面温
    度；"
442                     "潜热/显热比 ≈5.2~18.5",
443             "ratio_range": [5.2, 18.5]},
444      }
445
446
447 def write_artifacts(res, VER, sens, cmpAB, cmp2d, cmpDC,
    cmpINT):
448      for fn, obj in [("q1_verification.json", VER),
449                      ("q1_sensitivity.json", sens),
450                      ("q1_compare.json", {"caseAB": cmpAB, "
    twoD": cmp2d,
451                                           "D_const": cmpDC, "
    interp": cmpINT,
452                                           "envelope": VER["
    compare_envelope_hm"]})]:
453         (res / fn).write_text(json.dumps(obj, ensure_ascii=
    False, indent=2), encoding="utf-8")
454
455
456 def write_figure_inputs(res, conv, sens, joint, env, cmpAB,
    i_out):
457      figd = res / "fig_data"
458      eb = [{"label": "Δr 空间（温度，Δt=1 s）",
459             "x": [row["dr_cm"] for row in conv["
```

```
    table1_dr_sweep_temperature"]["rows"]],
460             "err_center": [row["err_center"] for row in conv["
    table1_dr_sweep_temperature"]["rows"]],
461             "err_surface": [row["err_surf"] for row in conv["
    table1_dr_sweep_temperature"]["rows"]]},
462            {"label": "Δt 时间（温度，Δr=0.01 cm）",
463             "x": [row["dt"] for row in conv["
    table2_dt_sweep_temperature"]["rows"]],
464             "err_center": [row["err_center"] for row in conv["
    table2_dt_sweep_temperature"]["rows"]],
465             "err_surface": [row["err_surf"] for row in conv["
    table2_dt_sweep_temperature"]["rows"]]},
466            {"label": "Δr 空间（水分表面列，Δt=1/32 s）",
467             "x": [row["dr_cm"] for row in conv["
    table4_dr_sweep_moisture_surface"]["rows"]],
468             "value": [row["C_surf"] for row in conv["
    table4_dr_sweep_moisture_surface"]["rows"]]}]
469     (figd / "error_bars.json").write_text(json.dumps(eb,
    ensure_ascii=False, indent=2),
470                                           encoding="utf-8")
471     with open(figd / "sensitivity.csv", "w", encoding="utf-8-
    sig") as f:
472         f.write("param,mult,value,T_center_1800,T_surface_1800
    ,C_center_1800,"
473                 "C_r1cm_1800,C_surface_1800,mass_ratio\n")
474         for name, blk in sens.items():
475             if name in ("joint_h_x_hm", "alpha_pm19pct"):
476                 continue
477             for row in blk["rows"]:
478                 f.write(",".join([name, f"{row['mult']:.4f}",
    f"{row['value']:.6g}",
479                                   f"{row['T_center_1800']:.9f}
    ", f"{row['T_surface_1800']:.9f}",
480                                   f"{row['C_center_1800']:.9f}
    ", f"{row['C_r1cm_1800']:.9f}",
```

```
                                    f"{row['C_surface_1800']:.9f}", f"{row['mass_ratio']:.9f}"]) + "\n")
        for row in joint:
            f.write(",".join(["h_x_hm_joint", f"{row['h']:.6g}x{row['hm']:.6g}", "",
                              f"{row['T_center_1800']:.9f}", "", "", "",
                              f"{row['C_surface_1800']:.9f}", f"{row['mass_ratio']:.9f}"]) + "\n")
    with open(figd / "envelope_hm.csv", "w", encoding="utf-8-sig") as f:
        f.write("hm_mult,r_cm," + ",".join(f"C_1800_r{r:g}cm" for r in i_out * DR_CM * 100) + "\n")
        for e in env:
            f.write(f"{e['hm_mult']}," + ",".join(f"{v:.9f}" for v in e["C_1800"]) + "\n")
    (figd / "ab_compare.json").write_text(json.dumps(cmpAB, ensure_ascii=False, indent=2),
                                          encoding="utf-8")


def main():
    quick = "--quick" in sys.argv
    t_start = time.time()
    res = ROOT / "results"
    (res / "fig_data").mkdir(parents=True, exist_ok=True)
    VER = {}

    A = solve_q1(case="A", dr_cm=DR_CM, dt=DT_IN, t_end=T_END, dt_out=1.0)
    B = solve_q1(case="B", dr_cm=DR_CM, dt=DT_IN, t_end=T_END, dt_out=1.0)
    sc = A["scalars"]
    N = A["N"]
    i_out = np.arange(0, N + 1, 10)
```

```
    dt_scan = 1.0 / 8

    VER["P0_picard_frozen_old_layer"] = verify_picard_old_layer()
    VER["P3_conservation"] = conservation_pair(A, B)
    S, T_air_series = physical_checks(A, B, sc)
    VER["S_checks"] = S

    cmpAB, qA, qB = case_AB_compare(A, B)
    VER["compare_2D"] = plane_2d_compare(A, T_air_series, quick)

    cmpDC, cmpINT = diffusivity_and_interp_compare(A, N)
    VER["compare_D_const"] = cmpDC
    VER["compare_interp"] = cmpINT

    env, env_block = driving_envelope(i_out, dt_scan)
    VER["compare_envelope_hm"] = env_block

    sens, joint = parameter_sensitivity(N, dt_scan)
    VER["sensitivity"] = sens

    conv = json.loads((res / "q1_convergence.json").read_text(encoding="utf-8"))
    VER["error_analysis"] = error_budget(conv)

    write_artifacts(res, VER, sens, cmpAB, VER["compare_2D"], cmpDC, cmpINT)
    write_figure_inputs(res, conv, sens, joint, env, cmpAB, i_out)

    print(f"总耗时 {time.time() - t_start:.1f} s。已写 results/q1_verification.json 等")

if __name__ == "__main__":
```

```
    sys.exit(main())
import json
import os
import sys
import openpyxl as xl
import numpy as np

WORK = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
RESULTS = os.path.join(WORK, "results")

_DATA_DIR = os.environ.get("MM_DATA_DIR", WORK)
f1 = os.path.join(_DATA_DIR, "附件", "附件1.xlsx")
f3 = os.path.join(_DATA_DIR, "附件", "附件3") + os.sep


def load_attachment1():
    wb = xl.load_workbook(f1, data_only=True)
    ws = wb["Sheet1"]
    rows = list(ws.iter_rows(min_row=2, values_only=True))
    t = np.array([r[0] for r in rows], dtype=float)
    T = np.array([r[1] for r in rows], dtype=float)
    C = np.array([r[2] for r in rows], dtype=float)
    return t, T, C


def repr_list(s):
    return [repr(float(x)) for x in s]


def _attachment1_summary(t, T, C, n, dt, gaps_bad, d, T_max, C_max,
                         i1800, first_dec_t, end_T, end_C, last_10T,
                         last_10C):
```

```
 return {
        "文件": "附件1.xlsx",
        "sheet": "Sheet1",
        "表头": ["时间", "温度", "水分浓度"],
        "数据行数": n,
        "时间单位": "s",
        "温度单位": "°C",
        "水分浓度单位": "kg/kg",
        "时间（首值）": repr(float(t[0])),
        "时间（末值）": repr(float(t[-1])),
        "时间步长（唯一值）": [repr(float(x)) for x in np.
    unique(dt)],
        "是否等间隔60s": bool(np.all(dt == 60.0)),
        "非60s间隔的位置（空 = 无）": [int(i) for i in
    gaps_bad],
        "重复时间点（空 = 无）": [repr(float(x)) for x in d],
        "温度（全区间最小值）": repr(float(T.min())),
        "温度（全区间最大值）": repr(float(T.max())),
        "温度（最大值对应时间）": repr(float(t[T_max])),
        "水分浓度（全区间最小值）": repr(float(C.min())),
        "水分浓度（全区间最大值）": repr(float(C.max())),
        "水分浓度（最大值对应时间）": repr(float(t[C_max])),
        "温度（首次下降对应时间）": repr(first_dec_t),
        "t1800（温度）": repr(float(T[i1800])),
        "t1800（水分浓度）": repr(float(C[i1800])),
        "末段t>=13000s（温度最小值）": repr(float(end_T.min())
    ),
        "末段t>=13000s（温度最大值）": repr(float(end_T.max())
    ),
        "末段t>=13000s（水分浓度最小值）": repr(float(end_C.
    min())),
        "末段t>=13000s（水分浓度最大值）": repr(float(end_C.
    max())),
        "最后10点（温度最小值）": repr(float(last_10T.min())),
        "最后10点（温度最大值）": repr(float(last_10T.max())),
```

```
        "最后10点（水分浓度最小值）": repr(float(last_10C.min
    ())),
        "最后10点（水分浓度最大值）": repr(float(last_10C.max
    ())),
        "末段最后10点（温度）": repr_list(last_10T),
        "末段最后10点（水分浓度）": repr_list(last_10C),
        "全量（时间）": repr_list(t),
        "全量（温度）": repr_list(T),
        "全量（水分浓度）": repr_list(C),
    }


def check_attachment1():
    t, T, C = load_attachment1()
    n = len(t)
    dt = np.diff(t)

    gaps_bad = np.where(dt != 60.0)[0]
    u, c = np.unique(t, return_counts=True)
    d = u[c > 1]

    mask_end = t >= 13000.0
    end_T = T[mask_end]
    end_C = C[mask_end]
    last_10T = end_T[-10:]
    last_10C = end_C[-10:]

    dT = np.diff(T)
    neg = np.where(dT < 0.0)[0]
    first_dec_t = t[neg + 1] if len(neg) else None

    T_max = int(np.argmax(T))
    C_max = int(np.argmax(C))

    i1800 = int(np.where(t == 1800.0)[0][0])
```

```
629
630     return _attachment1_summary(t, T, C, n, dt, gaps_bad, d,
    T_max, C_max,
631                                 i1800, first_dec_t, end_T,
    end_C, last_10T,
632                                 last_10C)
633     return summary
634
635
636 def check_templates():
637     out = {}
638     for name in ["result1.xlsx", "result2.xlsx", "result3.xlsx
    ", "result4.xlsx"]:
639         wb = xl.load_workbook(f3 + name, data_only=True)
640         info = {"sheets": list(wb.sheetnames)}
641         sheets = {}
642         for sn in wb.sheetnames:
643             ws = wb[sn]
644             rows = [list(r) for r in ws.iter_rows(values_only=
    True)]
645             sheets[sn] = {
646                 "维度": ws.dimensions,
647                 "max_row": ws.max_row,
648                 "max_col": ws.max_column,
649                 "合并单元格数": len(ws.merged_cells.ranges),
650                 "单元格逐位": rows,
651             }
652         info["sheets_detail"] = sheets
653         out[name] = info
654     return out
655
656
657 def main():
658     os.makedirs(RESULTS, exist_ok=True)
659     att1 = check_attachment1()
```

```
    tmpl = check_templates()

    result = {
        "附件1_校验": att1,
        "附件3_模板结构": tmpl,
    }

    out_path = os.path.join(RESULTS, "data_check.json")
    with open(out_path, "w", encoding="utf-8") as f:
        json.dump(result, f, ensure_ascii=False, indent=2)

    print("附件1: 校验结论")
    print("数据行数:", att1["数据行数"], "| 时间范围:",
          att1["时间（首值）"], "~", att1["时间（末值）"], "s"
)
    print("时间步长唯一值:", att1["时间步长（唯一值）"],
          "| 等间隔60s:", att1["是否等间隔60s"],
          "| 非60s间隔位置:", att1["非60s间隔的位置（空 = 无）
"],
          "| 重复时间点:", att1["重复时间点（空 = 无）"])
    print("温度全区间:", att1["温度（全区间最小值）"], "~",
          att1["温度（全区间最大值）"], "°C",
          "| 最大值@t=", att1["温度（最大值对应时间）"], "s")
    print("水分浓度全区间:", att1["水分浓度（全区间最小值）"],
     "~",
          att1["水分浓度（全区间最大值）"], "kg/kg",
          "| 最大值@t=", att1["水分浓度（最大值对应时间）"], "
s")
    print("温度首次下降@t=", att1["温度（首次下降对应时间）"],
     "s")
    print("t=1800s:", "T=", att1["t1800（温度）"], "°C, C=",
          att1["t1800（水分浓度）"], "kg/kg")
    print("末段(t>=13000s) 温度摆动:",
          att1["末段t>=13000s（温度最小值）"], "~",
          att1["末段t>=13000s（温度最大值）"], "°C")
```

```
    print("末段(t>=13000s) 水分浓度摆动:",
          att1["末段t>=13000s（水分浓度最小值）"], "~",
          att1["末段t>=13000s（水分浓度最大值）"], "kg/kg")
    print("最后10点 温度摆动:", att1["最后10点（温度最小值）"], "~",
          att1["最后10点（温度最大值）"], "°C")
    print("最后10点 水分浓度摆动:", att1["最后10点（水分浓度最小值）"], "~",
          att1["最后10点（水分浓度最大值）"], "kg/kg")
    print("[附件3 模板结构]")
    for name, info in tmpl.items():
        print(f"{name}: sheets={info['sheets']}")
        for sn, d in info["sheets_detail"].items():
            print(f"sheet[{sn}] {d['维度']} "
                  f"row={d['max_row']} col={d['max_col']} "
                  f"合并单元格={d['合并单元格数']}")
    print("写入至:", out_path)


if __name__ == "__main__":
    sys.exit(main())
from __future__ import annotations
import json
import os
import sys
from pathlib import Path
import numpy as np
from scipy.special import j0, j1
from scipy.optimize import brentq

RHO = 820.0
CP = 2600.0
K = 0.36
H = 25.0
R = 0.02
```

```
T_INIT = 28.0
ALPHA = K / (RHO * CP)
BI = H * R / K

_BASE = Path(__file__).resolve().parents[1]
_DATA_DIR = Path(os.environ.get("MM_DATA_DIR", str(_BASE)))
ATTACH1 = _DATA_DIR / "附件" / "附件1.xlsx"
OUTJSON = _BASE / "results" / "independent_check_q1_temp.json"


def load_oven_temperature():
    import pandas as pd
    df = pd.read_excel(ATTACH1)
    t = df["时间"].to_numpy(dtype=float)
    T = df["温度"].to_numpy(dtype=float)
    return t, T


def bessel_roots(n_roots: int, bi: float) -> np.ndarray:
    def f(lam):
        return lam * j1(lam) - bi * j0(lam)

    roots = []
    lo = 1e-9
    step = 0.05
    lam = lo
    prev = f(lam)
    while len(roots) < n_roots and lam < 4000.0:
        lam_next = lam + step
        cur = f(lam_next)
        if prev == 0.0:
            roots.append(lam)
        elif prev * cur < 0.0:
            roots.append(brentq(f, lam, lam_next, xtol=1e-14,
   rtol=1e-15))
```

```
        lam, prev = lam_next, cur
        if len(roots) > 6:
            step = 0.5
    return np.array(roots[:n_roots])


def duhamel_temperature(t_query, r_query, t_air, T_air,
   n_roots=80):
    lam = bessel_roots(n_roots, BI)
    Cn = 2.0 * j1(lam) / (lam * (j0(lam) ** 2 + j1(lam) ** 2))
    beta = ALPHA * lam ** 2 / R ** 2

    slopes = np.diff(T_air) / np.diff(t_air)
    seg_starts = t_air[:-1]
    seg_ends = t_air[1:]

    t_query = np.asarray(t_query, dtype=float)
    r_query = np.asarray(r_query, dtype=float)

    I = np.zeros(n_roots)
    T_out = np.empty((t_query.size, r_query.size))
    seg = 0
    last_t = 0.0
    for i, t in enumerate(t_query):
        while seg < slopes.size and seg_ends[seg] <= t + 1e
   -12:
            dt = seg_ends[seg] - last_t
            if dt > 0:
                decay = np.exp(-beta * dt)
                I = I * decay + slopes[seg] * (1.0 - decay) /
   beta
            last_t = seg_ends[seg]
            seg += 1
        dt = t - last_t
        if dt > 1e-15 and seg < slopes.size:
```

```
            decay = np.exp(-beta * dt)
            I_t = I * decay + slopes[seg] * (1.0 - decay) / beta
        else:
            I_t = I
        T_air_t = np.interp(t, t_air, T_air)
        radial = j0(np.outer(lam, r_query) / R)
        T_out[i] = T_INIT + (T_air_t - T_INIT) - (Cn[:, None] * radial * I_t[:, None]).sum(axis=0)
    return T_out


def main():
    t_air, T_oven = load_oven_temperature()
    print(f"附件1: {t_air.size} 点, t∈[{t_air[0]:.0f},{t_air[-1]:.0f}] s")
    print(f"附件1 末行逐位: t={t_air[-1]!r}, T={T_oven[-1]!r}")
    print(f"alpha = {ALPHA!r} m^2/s,  Bi = {BI!r}")

    t_tab = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
    r_tab = np.array([0.0, 0.005, 0.01, 0.015, 0.02])

    T_tab = duhamel_temperature(t_tab, r_tab, t_air, T_oven)

    smoke = {"t": 1800.0, "r0": 33.5765, "rR": 36.7863, "T_air": 41.5130}

    print("\n解析解（表1 格式，单位℃）")
    header = "时间/s " + "".join(f"{r*100:>12.4f}" for r in r_tab)
    print(header)
    for i, t in enumerate(t_tab):
        print(f"{t:7.0f} " + "".join(f"{T_tab[i, j]:12.4f}"
```

```
    for j in range(r_tab.size)))
817
818  print("\n与 smoke test 对比")
819  a0, aR = T_tab[-1, 0], T_tab[-1, -1]
820  print(f"  解析解  中心={a0:.10f}  表面={aR:.10f}")
821  print(f"  数值解  中心={smoke['r0']:.10f}  表面={smoke['rR
    ']:.10f}")
822  print(f"  偏差    中心={abs(a0-smoke['r0']):.3e}  表面={
    abs(aR-smoke['rR']):.3e}")
823
824  T0 = duhamel_temperature(np.array([0.0]), r_tab, t_air,
    T_oven)
825  print("\n解析解自检")
826  print(f"  t=0 全场 = {T0[0]}  (应全为 {T_INIT})  -> {'PASS
    ' if np.allclose(T0[0], T_INIT, atol=1e-10) else 'FAIL'}")
827
828  out = {
829      "note": "独立解析验算，不引用solver_q1.py",
830      "params": {"rho": RHO, "cp": CP, "k": K, "h": H, "R":
    R, "T_init": T_INIT,
831                 "alpha": ALPHA, "Bi": BI},
832      "attach1_last_row": {"t": float(t_air[-1]), "T": float
    (T_oven[-1])},
833      "table1_like": {"times_s": t_tab.tolist(), "radii_cm":
     (r_tab * 100).tolist(),
834                      "T_C": T_tab.tolist()},
835      "vs_smoke_at_1800": {"analytic_center": a0, "
    analytic_surface": aR,
836                           "numeric_center_reported": smoke[
    "r0"],
837                           "numeric_surface_reported": smoke
    ["rR"],
838                           "abs_diff_center": abs(a0 - smoke
    ["r0"]),
839                           "abs_diff_surface": abs(aR -
```

```
    smoke["rR"])},
    }
    OUTJSON.parent.mkdir(parents=True, exist_ok=True)
    OUTJSON.write_text(json.dumps(out, ensure_ascii=False,
    indent=2), encoding="utf-8")
    print(f"\n已写出 {OUTJSON}")


if __name__ == "__main__":
    sys.exit(main())
from __future__ import annotations
import hashlib
import json
import os
import platform
import sys
import time
from datetime import datetime
from pathlib import Path
import numpy as np

_out = sys.stdout
if hasattr(_out, "reconfigure"):
    _out.reconfigure(encoding="utf-8", errors="replace")

_CODE = Path(os.path.realpath(__file__))
ROOT = _CODE.parents[1]

from solver_q1 import (
    solve_q1, ATT1, ATT2, PDF, R_CM, RHO_D, HM, T_air_at,
    C_air_at,
    rho_v_sat, rho_v_inf, default_params,
)
from independent_check_q1_temp import (
    load_oven_temperature, duhamel_temperature,
```

```
)

SEED = 42
DR_CM = 0.01
DT_IN = 1.0 / 64
DT_OUT = 1.0
T_END = 1800.0
R_OUT_CM = np.arange(0.0, 2.0 + 1e-9, 0.1)
T_TAB = np.array([100, 300, 600, 900, 1200, 1500, 1800], float)
R_TAB_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])

TEMPERATURE_SOURCE = ("analytic —— Bessel 级数 + Duhamel 卷积 n_roots=320")
MOISTURE_SURFACE_COLUMN = "Richardson p=2.0, N400/N800 (路线①)"

FIELD_T_STRIDE = 20
FIELD_R_STRIDE = 2

OUTPUT_FILES = ["result1.xlsx", "result1_t0_row.xlsx", "result1_caseB.xlsx", "results.json",
                "q1_table1_temperature.csv", "q1_table2_moisture_caseA.csv",
                "q1_surface_column_detail.csv",
                "q1_temperature_numeric_crosscheck.csv", "q1_temperature_analytic_series.csv",
                "q1_field_T_analytic.csv", "q1_field_C_A_numeric.csv",
                "q1_field_C_B_numeric.csv",
                "fig_data/radial_profiles.csv", "fig_data/case_AB_curves.csv"]


def sha256(path):
```

```
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def analytic_T(t_query, r_cm):
    t_air, T_oven = load_oven_temperature()
    return duhamel_temperature(np.asarray(t_query, float),
                               np.asarray(r_cm, float) /
   100.0, t_air, T_oven, n_roots=320)


def input_fingerprints():
    inputs = {}
    for p in (ATT1, ATT2, PDF):
        pp = Path(p)
        inputs[pp.name] = {"path": str(pp), "sha256": sha256(
   pp) if pp.exists() else None,
                           "exists": pp.exists()}
    scripts = {}
    for s in ["solver_q1.py", "run_q1.py", "convergence_q1.py"
   , "verification_q1.py"]:
        sp = ROOT / "Q1" / s
        if sp.exists():
            scripts[s] = sha256(sp)
    tool_ref = ROOT / "Q1" / "independent_check_q1_temp.py"
    tool_ref_sha = sha256(tool_ref) if tool_ref.exists() else
   None
    return inputs, scripts, tool_ref_sha


def run_cases():
    runs = {}
```

```
    for case in ("A", "B"):
        o = solve_q1(case=case, dr_cm=DR_CM, dt=DT_IN, t_end=T_END, dt_out=DT_OUT)
        runs[case] = o
        print(f"[run] 口径 {case}: Δr={DR_CM} cm (N={o['N']}), Δt={DT_IN} s, "
              f"步数 {o['n_steps']}, 耗时 {o['elapsed_s']:.1f} s, Picard {o['picard']}")
    return runs["A"], runs["B"]


def surface_sweep(dt):
    surf = {}
    for dr_cm, tag in ((0.01, "N200"), (0.005, "N400"), (0.0025, "N800")):
        o = solve_q1(case="A", dr_cm=dr_cm, dt=dt, t_end=T_END, dt_out=DT_OUT)
        surf[tag] = o["C_out"][1:, -1].copy()
        print(f"[surface] {tag}: 耗时 {o['elapsed_s']:.1f} s 末值 {surf[tag][-1]:.9f}")
    p_surf = 2.0
    surf_ex = surf["N800"] + (surf["N800"] - surf["N400"]) / (2.0 ** p_surf - 1.0)
    surf_ex2 = surf["N400"] + (surf["N400"] - surf["N200"]) / (2.0 ** p_surf - 1.0)
    unc_t = np.maximum(np.abs(surf["N800"] - surf_ex), np.abs(surf_ex2 - surf_ex))
    surf_unc = {tag: float(np.max(np.abs(surf[tag] - surf_ex))) for tag in surf}
    surf_unc["per_time_max"] = float(unc_t.max())
    surf_unc["per_time_at_1800s"] = float(unc_t[-1])
    surf_unc["per_time_max_at_t_s"] = int(np.argmax(unc_t) + 1)
    print(f"[surface] 逐时刻网格不确定度: max {unc_t.max():.3e} @t={int(np.argmax(unc_t))+1} s; "
```

```
          f"t=1800 s 时 {unc_t[-1]:.3e} kg/kg")
    return surf, surf_ex, surf_ex2, unc_t, surf_unc


def write_xlsx(path, T_rows, C_rows, times, sheet_names=("温度", "水分浓度")):
    import openpyxl
    wb = openpyxl.Workbook()
    for i, (sn, data) in enumerate(zip(sheet_names, (T_rows, C_rows))):
        ws = wb.active if i == 0 else wb.create_sheet()
        ws.title = sn
        ws.cell(row=1, column=1, value="时间\\到药材中心的距离")
        for j, r in enumerate(R_OUT_CM):
            ws.cell(row=1, column=2 + j, value=float(round(r, 1)))
        for k, t in enumerate(times):
            ws.cell(row=2 + k, column=1, value=int(round(t)))
            for j in range(R_OUT_CM.size):
                ws.cell(row=2 + k, column=2 + j, value=float(round(data[k, j], 4)))
    wb.save(path)


def result_tables(A, B, surf, surf_ex, surf_ex2, unc_t, i_out, t_out, n_sel):
    T_win_num = A["T_out"][n_sel][:, i_out]
    T_all_num = np.vstack([A["T_out"][0][i_out][None, :], T_win_num])
    T_all = analytic_T(t_out, R_OUT_CM)
    if T_all.shape != T_all_num.shape:
        raise AssertionError("解析/数值温度场形状不一致")
    T_tab = T_all[n_sel]
    T0_row = T_all[:1]
```

```
    T_err_all = np.abs(T_all - T_all_num)

    C_tab = A["C_out"][n_sel][:, i_out].copy()
    C_tab_raw_surface = C_tab[:, -1].copy()
    C_tab[:, -1] = surf_ex
    C_tab_B = B["C_out"][n_sel][:, i_out]
    C0_row = A["C_out"][0][i_out]

    np.savetxt(res_of("q1_surface_column_detail.csv"),
               np.column_stack([t_out[n_sel], surf["N200"], surf["N400"], surf["N800"],
                                surf_ex, surf_ex2, unc_t]),
               delimiter=",", comments="",
               header="t_s,C_surf_N200,C_surf_N400,C_surf_N800,C_surf_Richardson_p2,"
                      "C_surf_Richardson_from_N200N400, grid_uncertainty_conservative")

    write_xlsx(res_of("result1.xlsx"), T_tab, C_tab, t_out[n_sel])
    write_xlsx(res_of("result1_t0_row.xlsx"), T0_row, C0_row[None, :], np.array([0]))
    write_xlsx(res_of("result1_caseB.xlsx"), T_tab, C_tab_B, t_out[n_sel])
    return dict(T_all=T_all, T_all_num=T_all_num, T_tab=T_tab, T0_row=T0_row,
                T_err_all=T_err_all, C_tab=C_tab, C_tab_B=C_tab_B,
                C_tab_raw_surface=C_tab_raw_surface, C0_row=C0_row)


def res_of(name):
    return ROOT / "results" / name

```

```
1007
1008 def field_series(A, B, C0_row, surf_ex, t_out, T_all_num):
1009     i_field_t = np.arange(0, t_out.size, FIELD_T_STRIDE)
1010     i_field_r = np.arange(0, A["N"] + 1, FIELD_R_STRIDE)
1011     t_field_s = t_out[i_field_t]
1012     r_field_cm = A["r"][i_field_r] * 100.0
1013     if t_field_s.size != 91 or r_field_cm.size != 101:
1014         raise AssertionError(f"场导出网格不是 91×101（实得 {
     t_field_s.size}×{r_field_cm.size}）")
1015
1016     T_field = analytic_T(t_field_s, r_field_cm)
1017     C_A_field = A["C_out"][np.ix_(i_field_t, i_field_r)].copy
     ()
1018     C_B_field = B["C_out"][np.ix_(i_field_t, i_field_r)].copy
     ()
1019     surf_ex_1801 = np.concatenate(([C0_row[-1]], surf_ex))
1020     if surf_ex_1801.size != t_out.size:
1021         raise AssertionError("表面列 Richardson 序列长度与输出
     时刻不符")
1022     C_A_field[:, -1] = surf_ex_1801[i_field_t]
1023     T_series = analytic_T(t_out, np.array([0.0, R_CM]))
1024
1025     hdr_field = "t_s," + ",".join(f"{v:.2f}" for v in
     r_field_cm)
1026     np.savetxt(res_of("q1_field_T_analytic.csv"),
1027                np.column_stack([t_field_s, T_field]),
     delimiter=",", comments="",
1028                header=hdr_field)
1029     np.savetxt(res_of("q1_field_C_A_numeric.csv"),
1030                np.column_stack([t_field_s, C_A_field]),
     delimiter=",", comments="",
1031                header=hdr_field)
1032     np.savetxt(res_of("q1_field_C_B_numeric.csv"),
1033                np.column_stack([t_field_s, C_B_field]),
     delimiter=",", comments="",
```

```
                header=hdr_field)
    np.savetxt(res_of("q1_temperature_analytic_series.csv"),
                np.column_stack([t_out, T_series]), delimiter="
,", comments="",
                header="t_s,T_center_C,T_surface_C")
    np.savetxt(res_of("q1_temperature_numeric_crosscheck.csv")
,
                np.column_stack([t_out, T_all_num]), delimiter=
",", comments="",
                header="t_s," + ",".join(f"{rc:g}" for rc in
R_OUT_CM))
    return i_field_t, i_field_r, t_field_s, r_field_cm,
T_field


def error_crosscheck(A, t_out, T_err_all, T_field, i_field_t,
i_field_r,
                     t_field_s, r_field_cm):
    i_win = t_out >= 1.0
    err_win = T_err_all[i_win]
    k_win = np.unravel_index(int(np.argmax(err_win)), err_win.
shape)
    err_field = np.abs(T_field - A["T_out"][np.ix_(i_field_t,
i_field_r)])
    k_fld = np.unravel_index(int(np.argmax(err_field)),
err_field.shape)
    return {
        "note": "数值解温度相对解析解的逐点偏差",
        "numeric_file": "results/
q1_temperature_numeric_crosscheck.csv",
        "window": "t=1…1800 s × r=0…2.0 cm",
        "max_abs_err_window_C": float(err_win.max()),
        "max_abs_err_window_at": {"t_s": float(t_out[i_win][
k_win[0]]),
                                  "r_cm": float(R_OUT_CM[k_win
```

```
    [1]])},
1058        "rms_abs_err_window_C": float(np.sqrt(np.mean(err_win
    ** 2))),
1059        "max_abs_err_at_1800s_C": float(err_win[-1].max()),
1060        "max_abs_err_incl_t0_C": float(T_err_all.max()),
1061        "max_abs_err_field_91x101_C": float(err_field.max()),
1062        "max_abs_err_field_at": {"t_s": float(t_field_s[k_fld
    [0]]),
1063                                 "r_cm": float(r_field_cm[
    k_fld[1]])},
1064        "acceptance_C": 5.0e-05,
1065        "meets_acceptance_full_window": bool(err_win.max() <
    5.0e-05),
1066     }, err_win, k_win
1067
1068
1069 def analytic_benchmarks(A, i_out):
1070     T_an_1800 = analytic_T([T_END], R_OUT_CM)[0]
1071     T_an_tab = analytic_T(T_TAB, R_TAB_CM)
1072     i_tab = [int(round(rc / DR_CM)) for rc in R_TAB_CM]
1073     j_tab = [int(round(rc / 0.1)) for rc in R_TAB_CM]
1074     T_num_tab = np.array([[A["T_out"][int(t)][i] for i in
    i_tab] for t in T_TAB])
1075     err_tab = np.abs(T_num_tab - T_an_tab)
1076     err_1800 = np.abs(A["T_out"][int(T_END)][i_out] -
    T_an_1800)
1077     return T_an_1800, T_an_tab, i_tab, j_tab, T_num_tab,
    err_tab, err_1800
1078
1079
1080 def conservation(A, B):
1081     sc = A["scalars"]
1082     cons_rel = float(abs((sc["mass"][0] - sc["mass"][-1]) - sc
    ["cum_flux"][-1])
1083                      / (sc["mass"][0] - sc["mass"][-1]))
```

```
    scB = B["scalars"]
    cons_rel_B = float(abs((scB["mass"][0] - scB["mass"][-1]) - scB["cum_flux"][-1])
                       / max(abs(scB["mass"][0] - scB["mass"][-1]), 1e-300))
    return sc, scB, cons_rel, cons_rel_B


def _summary_meta(inputs, tool_ref_sha):
    return {
        "meta": {
            "question": "Q1 预热平衡阶段（0–1800 s）",
            "generated_at": datetime.now().isoformat(timespec="seconds"),
            "seed": SEED,
            "config": {"dr_cm": DR_CM, "dt_internal_s": DT_IN, "dt_out_s": DT_OUT,
                       "t_end_s": T_END, "picard_tol": 1e-12, "picard_max": 50,
                       "interface_D_rule": "arithmetic mean of D(C_i), D(C_i+1)",
                       "surface_column": "Richardson p=2.0, N400/N800 (路线①)",
                       "temperature_source": TEMPERATURE_SOURCE,
                       "moisture_surface_column": MOISTURE_SURFACE_COLUMN},
            "python": platform.python_version(),
            "numpy": np.__version__,
        },
        "inputs_sha256": inputs,
        "reference_tool_sha256": tool_ref_sha,
        "units": {"T": "℃", "C": "kg/kg(干基)", "r": "cm", "t": "s",
                  "mass": "kg water per kg dry solids, 归一化
```

```
    质量比"},
1109     }
1110
1111
1112 def _summary_temperature(T_all, T_tab, T_an_1800, T_an_tab,
    err_tab,
1113                          err_1800, T_num_tab, crosscheck):
1114     return {
1115         "temperature": {
1116             "source": TEMPERATURE_SOURCE,
1117             "delivered_is": "analytic（解析解）——见analytic
    块",
1118             "reported": T_tab[-1].round(6).tolist(),
1119             "analytic": {
1120                 "note": ("常数物性下该线性热传导问题的精确解，
    "
1121                          "Bessel 级数 + Duhamel 卷积 n_roots
    =320；实现对 "
1122                          "Q1/independent_check_q1_temp.py。"
1123                          "result1*.xlsx 的「温度」sheet 四位小
    数取自本块。"),
1124                 "t_s": float(T_END),
1125                 "T_at_1800s_r_cm": R_OUT_CM.tolist(),
1126                 "T_at_1800s": T_all[-1].round(9).tolist(),
1127                 "table1": T_an_tab.round(9).tolist(),
1128                 "anchors_1800s_C": {"center": float(T_all[-1,
    0]),
1129                                     "surface": float(T_all[-1,
     -1])},
1130             },
1131             "numeric_crosscheck": crosscheck,
1132             "analytic_1800s": T_an_1800.round(9).tolist(),
1133             "abs_err_1800s": err_1800.round(12).tolist(),
1134             "max_abs_err_1800s": float(err_1800.max()),
1135             "table1_numeric": T_num_tab.round(6).tolist(),
```

```
            "table1_analytic": T_an_tab.round(9).tolist(),
            "table1_max_abs_err": float(err_tab.max()),
            "error_vs_analytic_source": "Q1/
    independent_check_q1_temp.py n_roots=320",
        },
     }


def _summary_moisture(A, B, surf, surf_ex, surf_unc, C_tab,
                      C_tab_raw_surface, C_tab_B, sc, scB,
    cons_rel,
                      cons_rel_B, i_tab, j_tab, idx):
     return {
        "moisture_caseA": {
            "surface_C_1800s_gridconverged": float(surf_ex
    [-1]),
            "surface_C_1800s_raw_N200": float(
    C_tab_raw_surface[-1]),
            "surface_C_1800s_raw_N400": float(surf["N400"
    ][-1]),
            "surface_C_1800s_raw_N800": float(surf["N800"
    ][-1]),
            "surface_grid_uncertainty": surf_unc,
            "center_C_1800s": float(A["C_out"][int(T_END)][0])
    ,
            "mass_ratio_1800s": float(sc["mass"][-1] / sc["
    mass"][0]),
            "mass_ratio_lost_pct": float(100.0 * (1.0 - sc["
    mass"][-1] / sc["mass"][0])),
            "cumulative_flux": float(sc["cum_flux"][-1]),
            "conservation_rel_error": cons_rel,
            "center_value_dev_from_init": float(abs(A["C_out"
    ][int(T_END)][0] - 2.55)),
        },
        "moisture_caseB": {
```

```
            "note": "含题目外假设 B1与 B2；",
            "rho_d_kg_m3": RHO_D,
            "surface_C_1800s": float(B["C_out"][int(T_END)][-1]),
            "center_C_1800s": float(B["C_out"][int(T_END)][0]),
            "mass_ratio_1800s": float(scB["mass"][-1] / scB["mass"][0]),
            "conservation_rel_error": cons_rel_B,
            "scoping_reference": {"surface_C_quasi_steady": 2.549885,
                                  "flux_kg_m2_s": 6.534236e-09},
        },
        "table2_caseA": {"times_s": T_TAB.tolist(), "radii_cm": R_TAB_CM.tolist(),
                         "note": "与 result1.xlsx 同源",
                         "C": np.array([[C_tab[idx(t) - 1][j] for j in j_tab]
                                        for t in T_TAB]).round(6).tolist()},
        "table2_caseB": {"times_s": T_TAB.tolist(), "radii_cm": R_TAB_CM.tolist(),
                         "C": np.array([[B["C_out"][idx(t)][i] for i in i_tab]
                                        for t in T_TAB]).round(9).tolist()},
        "picard": {"caseA": A["picard"], "caseB": B["picard"]},
    }


def build_summary(A, B, t_out, surf, surf_ex, surf_unc, T_all, T_tab,
                  T_an_1800, T_an_tab, err_tab, err_1800,
```

```
    T_num_tab,
                   C_tab, C_tab_raw_surface, C_tab_B,
    crosscheck, sc, scB,
                   cons_rel, cons_rel_B, i_out, i_tab, j_tab,
    idx,
                   inputs, tool_ref_sha):
     summary = {}
     summary.update(_summary_meta(inputs, tool_ref_sha))
     summary.update(_summary_temperature(T_all, T_tab,
    T_an_1800, T_an_tab,
                                         err_tab, err_1800,
    T_num_tab,
                                         crosscheck))
     summary.update(_summary_moisture(A, B, surf, surf_ex,
    surf_unc, C_tab,
                                      C_tab_raw_surface,
    C_tab_B, sc, scB,
                                      cons_rel, cons_rel_B,
    i_tab, j_tab,
                                      idx))
     return summary


def write_table_csv(T_an_tab, C_tab, idx, j_tab):
     hdr = "时间/s," + ",".join(f"{r:g}" for r in R_TAB_CM)
     lines = [hdr]
     for i, t in enumerate(T_TAB):
         lines.append(f"{t:g}," + ",".join(f"{v:.4f}" for v in
    T_an_tab[i]))
     res_of("q1_table1_temperature.csv").write_text("\n".join(
    lines), encoding="utf-8-sig")
     lines = [hdr]
     for i, t in enumerate(T_TAB):
         lines.append(f"{t:g}," + ",".join(
             f"{C_tab[idx(t) - 1][j]:.4f}" for j in j_tab))
```

```
    res_of("q1_table2_moisture_caseA.csv").write_text("\n".join(lines), encoding="utf-8-sig")


def write_figure_data(A, B, i_out, idx):
    figd = ROOT / "results" / "fig_data"
    prof_t = T_TAB
    np.savetxt(figd / "radial_profiles.csv",
               np.column_stack([np.repeat(prof_t, len(R_OUT_CM)),
                                np.tile(R_OUT_CM, len(prof_t)),
                                np.array([A["T_out"][idx(t)][i_out] for t in prof_t]).ravel(),
                                np.array([A["C_out"][idx(t)][i_out] for t in prof_t]).ravel(),
                                np.array([B["C_out"][idx(t)][i_out] for t in prof_t]).ravel()]),
               delimiter=",", header="t_s,r_cm,T_C_CaseA,C_CaseA,C_CaseB", comments="")

    ts = A["t_out"][1:]
    w_all = np.array([C_air_at(t) for t in ts])
    Ta_all = np.array([T_air_at(t) for t in ts])
    CsA = A["C_out"][1:, -1]
    CsB = B["C_out"][1:, -1]
    TsB = B["T_out"][1:, -1]
    qA = HM * (CsA - w_all)
    qB = HM * (rho_v_sat(TsB) - rho_v_inf(w_all, Ta_all)) / RHO_D
    np.savetxt(figd / "case_AB_curves.csv",
               np.column_stack([ts, CsA, CsB, A["C_out"][1:, 0], B["C_out"][1:, 0],
                                qA * RHO_D, qB * RHO_D]),
               delimiter=",",
```

```
                header="t_s,C_surf_CaseA,C_surf_CaseB,C_center_CaseA,C_center_CaseB,"
                       "flux_CaseA_kg_m2_s,flux_CaseB_kg_m2_s", comments="")

    (figd / "README.json").write_text(json.dumps({
        "note": "作图数据入口",
        "files": {
            "radial_profiles.csv":
                "列：t_s, r_cm, T_C_CaseA, C_CaseA, C_CaseB（t=100/300/600/900/1200/1500/1800 s）",
            "case_AB_curves.csv":
                "列：t_s, C_surf_CaseA, C_surf_CaseB, C_center_CaseA, C_center_CaseB, "
                "flux_CaseA_kg_m2_s, flux_CaseB_kg_m2_s（t=1…1800 s 逐秒）",
            "error_bars.json": "网格/时间收敛偏差",
            "sensitivity.csv": "单/多参数灵敏度扫描",
            "envelope_hm.csv": "h_m 驱动势包络扫描",
            "ab_compare.json": "口径 A/B 对比关键数值",
        },
    }, ensure_ascii=False, indent=2), encoding="utf-8")


def build_uncertainty(res, surf_unc, summary):
    unc = {
        "note": "A10.1 要求的不确定度标注。表面列分时刻值的逐秒完整序列见 "
                "results/q1_surface_column_detail.csv 的 grid_uncertainty_conservative 列。",
        "surface_column_moisture_grid": {
            "note": "本部分含两个不同对象，不可混读；均为网格不确定度",
            "method": "Richardson p=2.00；保守值再与 N=200/N=400 外推取较大",
```

```
            "delivered_column_conservative": {
                "definition": "交付表面列"
                                "max(|C_N800-C_ex|, |C_{N200,
N400}外推-C_ex|)。",
                "per_time_max": surf_unc["per_time_max"],
                "per_time_max_at_t_s": surf_unc["
per_time_max_at_t_s"],
                "per_time_at_1800s": surf_unc["
per_time_at_1800s"],
                "per_time_full_series": "results/
q1_surface_column_detail.csv  grid_uncertainty_conservative"
,
            },
            "raw_node_deviation": {
                "definition": "未外推的原始节点值与收敛值之
差，",
                "max_over_window": {"N200": surf_unc["N200"],
"N400": surf_unc["N400"],
                                    "N800": surf_unc["N800"]},
                "at_1800s": {},
            },
        },
    }

    _cv = res / "q1_convergence.json"
    if _cv.exists():
        _t4 = json.loads(_cv.read_text(encoding="utf-8"))["
table4_dr_sweep_moisture_surface"]
        unc["surface_column_moisture_grid"]["
raw_node_deviation"]["at_1800s"] = dict(
            _t4["grid_uncertainty_vs_richardson"])
        unc["surface_column_moisture_grid"]["
raw_node_deviation"]["at_1800s_note"] = \
            "源 results/q1_convergence.json
table4_dr_sweep_moisture_surface.
```

```
grid_uncertainty_vs_richardson"
 _we = res / "q1_window_error.json"
 if _we.exists():
     _w = json.loads(_we.read_text(encoding="utf-8"))["
moisture_window_error"]
     unc["moisture_time_step_residual"] = {
         "delivered_dt_s": _w["delivered_dt_s"],
         "error_estimate_p1": _w["
delivered_dt_error_estimate_p1"],
         "criterion_C_analog": _w["criterion_C_analog"],
         "n_cells_over_criterion_p1": _w["
n_cells_over_criterion_p1"],
         "n_cells_total": _w["n_cells_total"],
         "note": "Δt=1/64 s 自身时间离散残差估计（p=1）；超
判据的 2 格均位于早期表面列，"
                 "该处网格不确定度（2.71e-4）比时间误差大
3.3 倍为主导项",
         "source": "results/q1_window_error.json（由 Q1/
window_error_q1.py 生成）",
     }
 else:
     unc["moisture_time_step_residual"] = {
         "source": "results/q1_window_error.json",
     }
 unc["temperature_window_error_crosscheck"] = {
     "value_C": summary["temperature"]["numeric_crosscheck"
]["max_abs_err_window_C"],
     "at": summary["temperature"]["numeric_crosscheck"]["
max_abs_err_window_at"],
     "source": "results.json  temperature.
numeric_crosscheck",
 }
 return unc
```

```
def _run_manifest(inputs, scripts, tool_ref_sha, summary):
    manifest = {
        "seed": SEED,
        "created_at": datetime.now().isoformat(timespec="seconds"),
        "inputs_sha256": inputs,
        "reference_tool_sha256": tool_ref_sha,
        "script_sha256": scripts,
        "config": summary["meta"]["config"],
        "wall_s": summary["wall_s"],
        "outputs": {},
        "uncertainty": {},
    }
    return manifest


def _print_summary(res, t_out, err_win, k_win, err_1800, surf_ex,
                   C_tab_raw_surface, surf_unc, sc, scB, cons_rel, B,
                   summary):
    _win = t_out >= 1.0
    print(f"\n[summary] 温度 = 解析解")
    print(f"[summary] 数值解 vs 解析解 全窗口最大偏差 = {err_win.max():.3e} ℃ "
          f"@t={t_out[_win][k_win[0]]:.0f} s, r={R_OUT_CM[k_win[1]]:g} cm "
          f"（验收线 5e-5 ℃，{'达标' if err_win.max() < 5e-5 else '超标'}）；"
          f"t=1800 s 处 {err_1800.max():.3e} ℃")
    print(f"[summary] 温度 t=1800 s 与解析解最大偏差 = {err_1800.max():.3e} ℃")
    print(f"[summary] 生产场落盘：91×101（t 每 20 s、r 每 0.02 cm） q1_field_T_analytic.csv / "
          f"q1_field_C_A_numeric.csv / q1_field_C_B_numeric.
```

```
    csv")
1336    print(f"[summary] 口径A 表面 C(1800) 交付值 = {surf_ex
    [-1]:.6f}（Richardson 外推）")
1337    print(f"[summary]   └ 未外推的 N=200 原始节点值 {
    C_tab_raw_surface[-1]:.6f}，"
1338          f"其与收敛值之差 {surf_unc['N200']:.2e}")
1339    print(f"[summary] 口径A 总水量比 = {sc['mass'][-1]/sc['
    mass'][0]:.9f}；守恒相对误差 {cons_rel:.3e}")
1340    print(f"[summary] 口径B 表面 C(1800) = {B['C_out'][int(
    T_END)][-1]:.9f}；"
1341          f"总水量比 {scB['mass'][-1]/scB['mass'][0]:.9f}")
1342    print(f"[summary] 总耗时 {summary['wall_s']:.1f} s；已写 {
    res/'result1.xlsx'} 等")
1343
1344
1345 def main():
1346     t0 = time.time()
1347     res = ROOT / "results"
1348     (res / "fig_data").mkdir(parents=True, exist_ok=True)
1349
1350     inputs, scripts, tool_ref_sha = input_fingerprints()
1351     A, B = run_cases()
1352     t_out = A["t_out"]
1353     n_sel = np.arange(1, t_out.size)
1354     surf, surf_ex, surf_ex2, unc_t, surf_unc = surface_sweep(
    DT_IN)
1355
1356     i_out = np.array([int(round(rc / DR_CM)) for rc in
    R_OUT_CM])
1357     tab = result_tables(A, B, surf, surf_ex, surf_ex2, unc_t,
    i_out, t_out, n_sel)
1358     T_all, T_all_num = tab["T_all"], tab["T_all_num"]
1359     T_tab, T_err_all, C_tab = tab["T_tab"], tab["T_err_all"],
    tab["C_tab"]
1360     C_tab_B = tab["C_tab_B"]
```

```
    C_tab_raw_surface, C0_row = tab["C_tab_raw_surface"], tab["C0_row"]

    i_field_t, i_field_r, t_field_s, r_field_cm, T_field = field_series(
        A, B, C0_row, surf_ex, t_out, T_all_num)
    crosscheck, err_win, k_win = error_crosscheck(
        A, t_out, T_err_all, T_field, i_field_t, i_field_r, t_field_s, r_field_cm)
    T_an_1800, T_an_tab, i_tab, j_tab, T_num_tab, err_tab, err_1800 = \
        analytic_benchmarks(A, i_out)
    sc, scB, cons_rel, cons_rel_B = conservation(A, B)

    def idx(t):
        return int(t)

    summary = build_summary(A, B, t_out, surf, surf_ex, surf_unc, T_all, T_tab,
                            T_an_1800, T_an_tab, err_tab, err_1800, T_num_tab,
                            C_tab, C_tab_raw_surface, C_tab_B, crosscheck, sc, scB,
                            cons_rel, cons_rel_B, i_out, i_tab, j_tab, idx,
                            inputs, tool_ref_sha)
    summary["wall_s"] = time.time() - t0
    (res / "results.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")

    manifest = _run_manifest(inputs, scripts, tool_ref_sha, summary)

    write_table_csv(T_an_tab, C_tab, idx, j_tab)
```

```
    write_figure_data(A, B, i_out, idx)

    for f in OUTPUT_FILES:
        p = res / f
        manifest["outputs"][f] = sha256(p) if p.exists() else None

    manifest["uncertainty"] = build_uncertainty(res, surf_unc, summary)

    (res / "run_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")

    _print_summary(res, t_out, err_win, k_win, err_1800, surf_ex,
                   C_tab_raw_surface, surf_unc, sc, scB, cons_rel, B,
                   summary)


if __name__ == "__main__":
    sys.exit(main())
from __future__ import annotations
import hashlib
import json
import os
import time
from pathlib import Path
import numpy as np
from scipy.linalg import solve_banded
import openpyxl as xl

RHO = 820.0
CP = 2600.0
```

```
K = 0.36
H = 25.0
HM = 8.0e-7
D0 = 7.0e-9
BETA = 0.89

R_CM = 2.0
LENGTH_CM = 25.0
R = R_CM / 100.0

T_INIT = 28.0
C_INIT = 2.55

P_ATM = 101325.0
R_DA = 287.05
R_V = 461.5
EPS_RATIO = 0.622
RHO_D = RHO / (1.0 + C_INIT)

ROOT = Path(os.path.realpath(__file__)).parents[1]

_DATA_DIR = os.environ.get("MM_DATA_DIR", str(ROOT))
ATT1 = str(Path(_DATA_DIR) / "附件" / "附件1.xlsx")
ATT2 = str(Path(_DATA_DIR) / "附件" / "附件2.xlsx")
PDF = str(Path(_DATA_DIR) / "A题.pdf")

_ATT1_CACHE = None


def att1():
    global _ATT1_CACHE
    if _ATT1_CACHE is None:
        wb = xl.load_workbook(ATT1, data_only=True)
        ws = wb[wb.sheetnames[0]]
        rows = list(ws.iter_rows(min_row=2, values_only=True))
```

```
        t = np.array([r[0] for r in rows], float)
        T = np.array([r[1] for r in rows], float)
        C = np.array([r[2] for r in rows], float)
        _ATT1_CACHE = (t, T, C)
    return _ATT1_CACHE


def T_air_at(t):
    t0, T0, _ = att1()
    return np.interp(t, t0, T0)


def C_air_at(t):
    t0, _, C0 = att1()
    return np.interp(t, t0, C0)


def D_of_C(C):
    return D0 * np.exp(-BETA / C)


def D_safe(C):
    return D0 * np.exp(-BETA / np.maximum(C, 1e-12))


def alpha():
    return K / (RHO * CP)


def sat_pressure_pa(T_C):
    return 611.21 * np.exp(17.502 * T_C / (240.97 + T_C))


def rho_v_sat(T_C):
    T_K = T_C + 273.15
```

```
    return sat_pressure_pa(T_C) / (R_V * T_K)


def rho_v_inf(w, T_C):
    T_K = T_C + 273.15
    p_v = w * P_ATM / (EPS_RATIO + w)
    p_da = P_ATM - p_v
    rho_da = p_da / (R_DA * T_K)
    return w * rho_da


def build_grid(dr_cm):
    dr = dr_cm / 100.0
    N = int(round(R / dr))
    if abs(N * dr - R) > 1e-12:
        raise ValueError(f"dr={dr_cm} cm 不能整除 R={R_CM} cm"
   )
    r = np.arange(N + 1) * dr
    return r, dr, N


def heat_step(T, r, dr, dt, T_air):
    N = len(r) - 1
    al = alpha()
    lam = dt * al / (dr * dr)

    d = np.zeros(N + 1)
    u = np.zeros(N)
    l = np.zeros(N + 1)
    rhs = np.zeros(N + 1)

    rp = r + dr / 2.0
    rm = r - dr / 2.0

    for i in range(1, N):
```

```
        d[i] = 1.0 + lam * (rp[i] + rm[i]) / r[i]
        l[i] = -lam * rm[i] / r[i]
        u[i] = -lam * rp[i] / r[i]
        rhs[i] = T[i]

    d[0] = 1.0 + 4.0 * lam
    u[0] = -4.0 * lam
    rhs[0] = T[0]

    m = R * dr - dr * dr / 4.0
    a_in = 2.0 * K * (R - dr / 2.0) / dr
    a_out = 2.0 * H * R
    G = RHO * CP * m / dt
    d[N] = 1.0 + (a_in + a_out) / G
    l[N] = -a_in / G
    rhs[N] = T[N] + (a_out / G) * T_air

    ab = np.zeros((3, N + 1))
    ab[0, 1:] = u
    ab[1, :] = d
    ab[2, :-1] = l[1:]
    return solve_banded((1, 1), ab, rhs)


def solve_heat_constant(dr_cm=0.005, dt=1.0, t_end=1800.0,
   record_every=1.0):
    r, dr, N = build_grid(dr_cm)
    T = np.full(N + 1, T_INIT)

    n_steps = int(round(t_end / dt))
    rec_gap = max(1, int(round(record_every / dt)))
    t_out = []
    T_hist = []

    t = 0.0
```

```
    t_out.append(t)
    T_hist.append(T.copy())
    for step in range(1, n_steps + 1):
        t = step * dt
        T_air = T_air_at(t)
        T = heat_step(T, r, dr, dt, T_air)
        if step % rec_gap == 0:
            t_out.append(t)
            T_hist.append(T.copy())
    return np.array(t_out), np.array(T_hist), r


def surface_robin_terms(dr, gam_face, h_coef):
    m_over_pi = R * dr - dr * dr / 4.0
    a_in = 2.0 * (R - dr / 2.0) * gam_face / dr
    a_out = 2.0 * R * h_coef
    return m_over_pi, a_in, a_out


def interp_driver(t, kind):
    t0, T0, C0 = att1()
    if kind == "linear":
        return float(np.interp(t, t0, T0)), float(np.interp(t,
    t0, C0))
    if kind == "step":
        i = int(np.searchsorted(t0, t, side="right")) - 1
        i = max(0, min(i, t0.size - 1))
        return float(T0[i]), float(C0[i])
    raise ValueError(f"未知插值方式 kind={kind!r}")


def _mass_step_prep(C_old, C_it, r, dr, dt, P):
    N = len(r) - 1
    if P is None:
        P = default_params()
```

```
    C = C_it.copy()
    rp = r + dr / 2.0
    rm = r - dr / 2.0
    rho_d = P["RHO"] / (1.0 + P["C_INIT"])

    geo_in = np.zeros(N + 1)
    geo_out = np.zeros(N + 1)
    with np.errstate(divide="ignore", invalid="ignore"):
        geo_in[1:N] = dt * rm[1:N] / (r[1:N] * dr * dr)
        geo_out[1:N] = dt * rp[1:N] / (r[1:N] * dr * dr)
    c0_factor = 4.0 * dt / (dr * dr)

    rhs_old = C_old.copy()

    converged = False
    n_iter = 0
    d = np.zeros(N + 1)
    l = np.zeros(N + 1)
    u = np.zeros(N)
    ab = np.zeros((3, N + 1))
    return (N, P, C, rho_d, geo_in, geo_out, c0_factor,
   rhs_old, converged,
            n_iter, d, l, u, ab)


def _picard_solve(C, N, dr, dt, P, picard_tol, picard_max, d,
   l, u, ab, geo_in, geo_out, c0_factor, rho_d, case,
   T_surf_new, T_air_new, C_air_new, rhs_old, converged, n_iter
   ):
    for k in range(1, picard_max + 1):
        n_iter = k
        Cs = np.maximum(C, 1e-12)
        if P["D_CONST"] is None:
            Dv = P["D0"] * np.exp(-P["BETA"] / Cs)
        else:
```

```
            Dv = np.full(C.shape, P["D_CONST"])
        face = P["D_FACE"]
        if face == "arith":
            dm = 0.5 * (Dv[:-1] + Dv[1:])
        elif face == "left":
            dm = Dv[:-1].copy()
        elif face == "right":
            dm = Dv[1:].copy()
        elif face == "harm":
            dm = 2.0 * Dv[:-1] * Dv[1:] / (Dv[:-1] + Dv[1:])
        elif face == "faceC":
            Cm = 0.5 * (Cs[:-1] + Cs[1:])
            if P["D_CONST"] is None:
                dm = P["D0"] * np.exp(-P["BETA"] / Cm)
            else:
                dm = np.full(N, P["D_CONST"])
        else:
            raise ValueError(f"未知界面系数规则 D_FACE={face!r}")
        d[1:N] = 1.0 + geo_in[1:N] * dm[0:N - 1] + geo_out[1:N] * dm[1:N]
        l[1:N] = -geo_in[1:N] * dm[0:N - 1]
        u[1:N] = -geo_out[1:N] * dm[1:N]
        a0 = c0_factor * dm[0]
        d[0] = 1.0 + a0
        u[0] = -a0
        m_over_pi, a_in, a_out = surface_robin_terms(dr, dm[N - 1], P["HM"])
        d[N] = m_over_pi / dt + a_in
        l[N] = -a_in
        rhs = np.empty(N + 1)
        rhs[0:N] = rhs_old[0:N]
        rhs[N] = (m_over_pi / dt) * rhs_old[N]
        if P["HM_INF"]:
            d[N] = 1.0
```

```
            l[N] = 0.0
            rhs[N] = C_air_new
        elif case == "A":
            d[N] += a_out
            rhs[N] += a_out * C_air_new
        elif case == "B":
            q_B = P["HM"] * (rho_v_sat(T_surf_new) - rho_v_inf(C_air_new, T_air_new)) / rho_d
            rhs[N] -= 2.0 * R * q_B
        else:
            raise ValueError(f"未知口径 case={case!r}（应为 'A' 或 'B'）")
        ab[0, 1:] = u
        ab[1, :] = d
        ab[2, :-1] = l[1:]
        C_new = solve_banded((1, 1), ab, rhs)
        delta = float(np.max(np.abs(C_new - C)))
        C = C_new
        if delta < picard_tol:
            converged = True
            break
    return C, n_iter, converged


def mass_step(C_old, C_it, r, dr, dt, t_new, case="A", T_surf_new=None,
              picard_tol=1e-12, picard_max=50, P=None,
              T_air_new=None, C_air_new=None):
    (N, P, C, rho_d, geo_in, geo_out, c0_factor, rhs_old, converged,
     n_iter, d, l, u, ab) = _mass_step_prep(C_old, C_it, r, dr, dt, P)

    return _picard_solve(C, N, dr, dt, P, picard_tol, picard_max, d, l, u, ab,
```

```
                          geo_in, geo_out, c0_factor, rho_d, case, T_surf_new,
                          T_air_new, C_air_new, rhs_old, converged, n_iter)


def heat_matrix(r, dr, dt, h=H, k=K, rho=RHO, cp=CP):
    N = len(r) - 1
    al = k / (rho * cp)
    lam = dt * al / (dr * dr)
    rp = r + dr / 2.0
    rm = r - dr / 2.0
    d = np.zeros(N + 1)
    l = np.zeros(N + 1)
    u = np.zeros(N)
    d[1:N] = 1.0 + lam * (rp[1:N] + rm[1:N]) / r[1:N]
    l[1:N] = -lam * rm[1:N] / r[1:N]
    u[1:N] = -lam * rp[1:N] / r[1:N]
    d[0] = 1.0 + 4.0 * lam
    u[0] = -4.0 * lam
    m_over_pi, a_in, a_out = surface_robin_terms(dr, al, h / (rho * cp))
    G = m_over_pi / dt
    d[N] = 1.0 + (a_in + a_out) / G
    l[N] = -a_in / G
    a_out_over_G = a_out / G
    ab = np.zeros((3, N + 1))
    ab[0, 1:] = u
    ab[1, :] = d
    ab[2, :-1] = l[1:]
    return ab, a_out_over_G


def default_params():
    return dict(H=H, HM=HM, K=K, RHO=RHO, CP=CP, D0=D0, BETA=
```

```
    BETA,
                C_INIT=C_INIT, T_INIT=T_INIT, INTERP="linear",
                D_CONST=None, HM_INF=False, D_FACE="arith")


def _q1_setup(P, dr_cm, dt, t_end, dt_out, record_fields):
    r, dr, N = build_grid(dr_cm)
    n_steps = int(round(t_end / dt))
    if abs(n_steps * dt - t_end) > 1e-9:
        raise ValueError(f"t_end={t_end} 不是 dt={dt} 的整数倍")
    rec_gap = int(round(dt_out / dt))
    if rec_gap < 1 or abs(rec_gap * dt - dt_out) > 1e-9:
        raise ValueError(f"dt_out={dt_out} 必须是 dt={dt} 的整数倍")

    T0 = np.empty(n_steps + 1)
    TN = np.empty(n_steps + 1)
    C0 = np.empty(n_steps + 1)
    CN = np.empty(n_steps + 1)
    MASS = np.empty(n_steps + 1)
    FLUX = np.empty(n_steps + 1)
    TVOL = np.zeros(N + 1)
    TVOL[0] = dr * dr / 4.0
    TVOL[1:N] = 2.0 * r[1:N] * dr
    TVOL[N] = R * dr - dr * dr / 4.0

    T = np.full(N + 1, P["T_INIT"])
    C = np.full(N + 1, P["C_INIT"])
    T0[0], TN[0], C0[0], CN[0] = T[0], T[N], C[0], C[N]
    MASS[0] = float(np.dot(TVOL, C))
    FLUX[0] = 0.0

    ab_heat, a_out_over_G = heat_matrix(r, dr, dt, P["H"], P["K"], P["RHO"], P["CP"])
```

```
1743
1744     if record_fields:
1745         T_out = np.empty((n_steps // rec_gap + 1, N + 1))
1746         C_out = np.empty((n_steps // rec_gap + 1, N + 1))
1747         T_out[0] = T
1748         C_out[0] = C
1749     else:
1750         T_out = C_out = None
1751
1752     picard_iters = np.zeros(n_steps + 1, dtype=int)
1753     n_not_conv = 0
1754     rhs_h = np.empty(N + 1)
1755     return (r, dr, N, n_steps, rec_gap, T0, TN, C0, CN, MASS,
    FLUX, TVOL, T,
1756             C, T_out, C_out, ab_heat, a_out_over_G,
    picard_iters, n_not_conv,
1757             rhs_h)
1758
1759
1760 def solve_q1(case="A", dr_cm=0.01, dt=1.0, t_end=1800.0,
    dt_out=1.0,
1761              picard_tol=1e-12, picard_max=50, record_fields=
    True,
1762              return_scalars=True, **overrides):
1763     t_start = time.time()
1764     P = default_params()
1765     P.update(overrides)
1766     (r, dr, N, n_steps, rec_gap, T0, TN, C0, CN, MASS, FLUX,
    TVOL, T, C,
1767       T_out, C_out, ab_heat, a_out_over_G, picard_iters,
    n_not_conv,
1768       rhs_h) = _q1_setup(P, dr_cm, dt, t_end, dt_out,
    record_fields)
1769
1770     for step in range(1, n_steps + 1):
```

```
        t_new = step * dt
        T_air_new, C_air_new = interp_driver(t_new, P["INTERP"])

        rhs_h[:] = T
        rhs_h[N] += a_out_over_G * T_air_new
        T = solve_banded((1, 1), ab_heat, rhs_h)

        C, k_it, ok = mass_step(C, C, r, dr, dt, t_new, case=case,
                                T_surf_new=T[N], picard_tol=picard_tol,
                                picard_max=picard_max, P=P,
                                T_air_new=T_air_new, C_air_new=C_air_new)
        picard_iters[step] = k_it
        if not ok:
            n_not_conv += 1

        T0[step], TN[step], C0[step], CN[step] = T[0], T[N], C[0], C[N]
        MASS[step] = float(np.dot(TVOL, C))
        if P["HM_INF"]:
            q = 0.0
        elif case == "A":
            q = P["HM"] * (CN[step] - C_air_new)
        else:
            q = (P["HM"] * (rho_v_sat(T[N]) - rho_v_inf(C_air_new, T_air_new))
                 / (P["RHO"] / (1.0 + P["C_INIT"])))
        FLUX[step] = FLUX[step - 1] + 2.0 * R * q * dt

        if record_fields and step % rec_gap == 0:
            i = step // rec_gap
            T_out[i] = T
```

```
            C_out[i] = C

    t_out = np.arange(0, n_steps + 1, rec_gap) * dt
    out = dict(
        params=P,
        r=r, dr=dr, N=N, case=case, dt=dt, dt_out=dt_out, t_end=t_end,
        t_out=t_out, T_out=T_out, C_out=C_out,
        scalars=dict(t=np.arange(n_steps + 1) * dt, T0=T0, TN=TN, C0=C0, CN=CN,
                     mass=MASS, cum_flux=FLUX, tvol=TVOL),
        picard=dict(max_iters=int(picard_iters.max()), n_not_converged=int(n_not_conv),
                    mean_iters=float(picard_iters[1:].mean()) if n_steps else 0.0),
        elapsed_s=time.time() - t_start,
        n_steps=n_steps,
    )
    return out


def smoke_test():
    print("[smoke test] 常物性纯导热，不接 D(C)")
    dr_cm = 0.005
    dt = 1.0
    t_end = 1800.0
    t_out, T_hist, r = solve_heat_constant(dr_cm=dr_cm, dt=dt, t_end=t_end)
    N = len(r) - 1
    print(f"网格：dr={dr_cm} cm  N={N} 单元，节点数 {N+1}，r∈[0,{R_CM}] cm")
    print(f"时间：dt={dt} s，共 {len(t_out)-1} 步，记录 {len(t_out)} 个时刻")
    print(f"热扩散系数 alpha = k/(rho*cp) = {alpha():.6e} m^2/s")
```

```
    for t_target in [0.0, 100.0, 300.0, 600.0, 900.0, 1200.0, 1500.0, 1800.0]:
        i = int(round(t_target / dt))
        T_center = T_hist[i, 0]
        T_surf = T_hist[i, N]
        T_air = T_air_at(t_target)
        print(f"  t={t_target:6.0f}s  中心 T={T_center:8.4f} °C  "
              f"表面 T={T_surf:8.4f} °C  烘房 T_air={T_air:8.4f} °C")
    T_final = T_hist[-1]
    ok_rise = T_final[0] > T_INIT
    ok_order = T_final[N] >= T_final[0]
    ok_bound = T_final.max() <= T_air_at(t_end) + 1e-9
    ok_phys = ok_rise and ok_order and ok_bound
    print(f"物理自检：中心升温={ok_rise}，表面≥中心={ok_order}，"
          f"不超烘房温度={ok_bound}  {'PASS' if ok_phys else 'FAIL'}")
    return ok_phys


if __name__ == "__main__":
    smoke_test()
from __future__ import annotations
import json
import math
import sys
import time
from pathlib import Path
import numpy as np
from scipy.linalg import solve_banded
from scipy.sparse import coo_matrix
from scipy.sparse.linalg import splu

```

```
_SELF_DIR = Path(__file__).resolve().parent
ROOT = _SELF_DIR.parent

from solver_q1 import (
    solve_q1, build_grid, D_of_C, alpha, H, HM, K, RHO, CP, R,
    R_CM,
    T_INIT, C_INIT, interp_driver, surface_robin_terms, RHO_D,
    rho_v_sat, rho_v_inf,
)
from independent_check_q1_temp import (
    load_oven_temperature, duhamel_temperature,
)

T_REF_CENTER = 33.575344826514
T_REF_SURF = 36.785560351561
T_END = 1800.0
DR_CM = 0.01
DT_IN = 1.0 / 64

_LINKS = None


def _mass_step_buggy(C_old, r, dr, dt, t_new, d_bug_iters=30):
    N = len(r) - 1
    C = C_old.copy()
    rp = r + dr / 2.0
    rm = r - dr / 2.0
    geo_in = np.zeros(N + 1)
    geo_out = np.zeros(N + 1)
    geo_in[1:N] = dt * rm[1:N] / (r[1:N] * dr * dr)
    geo_out[1:N] = dt * rp[1:N] / (r[1:N] * dr * dr)
    c0_factor = 4.0 * dt / (dr * dr)
    C_air_new = interp_driver(t_new, "linear")[1]
    for _ in range(d_bug_iters):
        C_it = C.copy()
```

```
        Dv = D_of_C(np.maximum(C_it, 1e-12))
        dm = 0.5 * (Dv[:-1] + Dv[1:])
        d = np.zeros(N + 1)
        l = np.zeros(N + 1)
        u = np.zeros(N)
        d[1:N] = 1.0 + geo_in[1:N] * dm[0:N - 1] + geo_out[1:N] * dm[1:N]
        l[1:N] = -geo_in[1:N] * dm[0:N - 1]
        u[1:N] = -geo_out[1:N] * dm[1:N]
        a0 = c0_factor * dm[0]
        d[0], u[0] = 1.0 + a0, -a0
        m, a_in, a_out = surface_robin_terms(dr, dm[N - 1], HM)
        d[N] = m / dt + a_in + a_out
        l[N] = -a_in
        rhs = C_it.copy()
        rhs[N] = (m / dt) * C_it[N] + a_out * C_air_new
        ab = np.zeros((3, N + 1))
        ab[0, 1:] = u
        ab[1, :] = d
        ab[2, :-1] = l[1:]
        C = solve_banded((1, 1), ab, rhs)
    return C


def counterexample_surface_c(r, dr, N):
    C_bug = np.full(N + 1, C_INIT)
    dt_bug = 1.0
    n_steps = int(round(T_END / dt_bug))
    for k in range(1, n_steps + 1):
        C_bug = _mass_step_buggy(C_bug, r, dr, dt_bug, k * dt_bug, d_bug_iters=5)
    return float(C_bug[-1])


```

```
def check_P0(quick=False):
    r, dr, N = build_grid(DR_CM)
    dt = DT_IN
    o2 = solve_q1(case="A", dr_cm=DR_CM, dt=dt, t_end=50.0, dt_out=50.0, picard_max=2)
    o50 = solve_q1(case="A", dr_cm=DR_CM, dt=dt, t_end=50.0, dt_out=50.0, picard_max=50)
    dmax = float(np.max(np.abs(o2["C_out"][-1] - o50["C_out"][-1])))
    print()
    print("P0  Picard 内迭代冻结上一时间层（单步迭代次数 2 vs 50 应不再变化）")
    print(f"  t=50 s 场：max|C(2 次迭代) - C(50 次迭代)| = {dmax:.3e} kg/kg")
    print(f"  生产运行 Picard 统计：{o50['picard']}")

    o_prod = solve_q1(case="A", dr_cm=DR_CM, dt=DT_IN, t_end=T_END, dt_out=T_END)
    C_surf_correct = float(o_prod["scalars"]["CN"][-1])
    C_surf_bug = counterexample_surface_c(r, dr, N)
    print(f"  反例复现（Δt=1 s、每步 5 次迭代）：表面 C(1800 s) 正确 = {C_surf_correct:.6f}；"
          f"把上一迭代解当上一时间层 = {C_surf_bug:.6f} "
          f"（偏差 {100*(C_surf_bug-C_surf_correct)/C_surf_correct:+.2f}%）")
    return {"max_abs_diff_2_vs_50_iters": dmax,
            "pass": bool(dmax < 1e-9),
            "criterion": "2 次 vs 50 次迭代的场差 ≤1e-9 kg/kg",
            "picard_prod": o_prod["picard"],
            "counterexample_surface_C_correct": C_surf_correct,
            "counterexample_surface_C_buggy": C_surf_bug,
            "counterexample_rel_pct": 100 * (C_surf_bug - C_surf_correct) / C_surf_correct}
```

```
def make_links_2d(dr_cm=0.05, dz_cm=0.5, L_CM=25.0):
    n_r = int(round(R_CM / dr_cm))
    dr = R / n_r
    Lh = L_CM / 2.0 / 100.0
    n_z = int(round(Lh / (dz_cm / 100.0)))
    dz = Lh / n_z
    r = np.arange(n_r + 1) * dr
    r_out = np.minimum(r + dr / 2.0, R)
    r_in = np.maximum(r - dr / 2.0, 0.0)
    ann = r_out ** 2 - r_in ** 2
    dz_cell = np.full(n_z + 1, dz)
    dz_cell[0] = dz / 2.0
    dz_cell[n_z] = dz / 2.0
    n_rn, n_zn = n_r + 1, n_z + 1
    n_cells = n_rn * n_zn

    def flat(i, j):
        return i * n_zn + j

    cell_vol = np.empty(n_cells)
    bnd_coef = np.zeros(n_cells)
    edge_p, edge_q, edge_a = [], [], []
    for i in range(n_rn):
        for j in range(n_zn):
            p = flat(i, j)
            cell_vol[p] = ann[i] * dz_cell[j]
            if i > 0:
                edge_p.append(p); edge_q.append(flat(i - 1, j)
    )
                edge_a.append(2.0 * r_in[i] * dz_cell[j] / dr)
            if i < n_r:
                edge_p.append(p); edge_q.append(flat(i + 1, j)
    )
```

```
                edge_a.append(2.0 * r_out[i] * dz_cell[j] / dr)
            else:
                bnd_coef[p] += 2.0 * R * dz_cell[j]
            if j > 0:
                edge_p.append(p); edge_q.append(flat(i, j - 1))
                edge_a.append(ann[i] / dz)
            if j < n_z:
                edge_p.append(p); edge_q.append(flat(i, j + 1))
                edge_a.append(ann[i] / dz)
            if j == 0:
                bnd_coef[p] += ann[i]
    return dict(r=r, z=np.arange(n_z + 1) * dz, dr=dr, dz=dz, Nr=n_r, Nz=n_z,
                nr=n_rn, nz=n_zn, n=n_cells, idx=flat, vol=cell_vol,
                bcoef_geom=bnd_coef, lp=np.array(edge_p), lq=np.array(edge_q),
                lAd=np.array(edge_a))


def build_2d_operator(G, vol, bcoef, dt):
    edge_g = 0.5 * (G[_LINKS["lp"]] + G[_LINKS["lq"]]) * _LINKS["lAd"]
    g_sum = np.bincount(_LINKS["lp"], weights=edge_g, minlength=_LINKS["n"])
    diag = 1.0 + (g_sum + bcoef) * dt / vol
    offd = -edge_g * dt / vol[_LINKS["lp"]]
    rows = np.concatenate([_LINKS["lp"], np.arange(_LINKS["n"])])
    cols = np.concatenate([_LINKS["lq"], np.arange(_LINKS["n"])])
    data = np.concatenate([offd, diag])
```

```
    return coo_matrix((data, (rows, cols)), shape=(_LINKS["n"], _LINKS["n"])).tocsc()


def run_2d_case(dr_cm=0.05, dz_cm=0.5, dt=1.0 / 8, t_end=T_END, dt_out=1.0,
                n_picard=2, L_CM=25.0, verbose=False):
    global _LINKS
    _LINKS = make_links_2d(dr_cm, dz_cm, L_CM)
    n_cells, cell_vol = _LINKS["n"], _LINKS["vol"]
    bnd_coef = _LINKS["bcoef_geom"]
    al = alpha()
    A_h = build_2d_operator(np.full(n_cells, al), cell_vol, bnd_coef * H / (RHO * CP), dt)
    lu_h = splu(A_h)
    bh_bnd = bnd_coef * (H / (RHO * CP)) * dt / cell_vol

    T = np.full(n_cells, T_INIT)
    C = np.full(n_cells, C_INIT)
    n_steps = int(round(t_end / dt))
    gap = int(round(dt_out / dt))
    T_last = None
    C_last = None
    for step in range(1, n_steps + 1):
        t_new = step * dt
        T_air, C_air = interp_driver(t_new, "linear")
        T = lu_h.solve(T + bh_bnd * T_air)
        C_old = C.copy()
        C_rhs = C_old + bnd_coef * HM * dt / cell_vol * C_air
        for _ in range(n_picard):
            Dv = D_of_C(np.maximum(C, 1e-12))
            A_m = build_2d_operator(Dv, cell_vol, bnd_coef * HM, dt)
            C = splu(A_m).solve(C_rhs)
        if step % gap == 0:
```

```
            T_last = T.copy()
            C_last = C.copy()
    return dict(T=T_last.reshape(_LINKS["nr"], _LINKS["nz"]),
                C=C_last.reshape(_LINKS["nr"], _LINKS["nz"]),
    geom=_LINKS)


def block_p3(A, B):
    sc, scB = A["scalars"], B["scalars"]
    consA = float(abs((sc["mass"][0] - sc["mass"][-1]) - sc["
    cum_flux"][-1])
                  / (sc["mass"][0] - sc["mass"][-1]))
    consB = float(abs((scB["mass"][0] - scB["mass"][-1]) - scB
    ["cum_flux"][-1])
                  / abs(scB["mass"][0] - scB["mass"][-1]))
    consB_abs = float(abs((scB["mass"][0] - scB["mass"][-1]) -
     scB["cum_flux"][-1]))
    consB_scaled = consB_abs / scB["mass"][0]
    print()
    print(f"P3  守恒校验：口径 A 相对误差 {consA:.3e}（判据 ≤1
    e-8） "
          f"{'PASS' if consA <= 1e-8 else 'FAIL'}")
    print(f"                口径 B 相对失水量误差 {consB:.3e}
    （失水量仅 {scB['mass'][0]-scB['mass'][-1]:.3e}，"
          f"该比值被机器舍入主导）")
    print(f"                口径 B 绝对闭合误差 {consB_abs:.3e
    }，相对初始总量 {consB_scaled:.3e}  "
          f"{'PASS' if consB_scaled <= 1e-8 else 'FAIL'}")
    return {"caseA_rel_err": consA,
            "caseB_rel_err_vs_loss": consB,
            "caseB_abs_err": consB_abs,
            "caseB_rel_err_vs_total_mass": consB_scaled,
            "criterion": "相对误差 ≤1e-8（口径 B 失水量 ~1e-9
    量级，以相对初始总量的闭合误差判定）",
            "pass": bool(consA <= 1e-8 and consB_scaled <= 1e
```

```
    -8)}
2063
2064
2065 def block_s_checks(A, B):
2066     sc = A["scalars"]
2067     print()
2068     print("S1–S10 物理自检（实测数值 + 判定）")
2069     S = {}
2070     dT0 = float(np.max(np.abs(A["T_out"][0] - T_INIT)))
2071     dC0 = float(np.max(np.abs(A["C_out"][0] - C_INIT)))
2072     S["S1_t0_equals_init"] = {"max_dev_T": dT0, "max_dev_C":
    dC0,
2073                               "pass": bool(dT0 <= 1e-12 and
    dC0 <= 1e-12)}
2074     dT = np.diff(sc["T0"])
2075     S["S2_center_T_nondecreasing"] = {"min_step_delta": float(
    dT.min()),
2076                                       "n_negative_steps": int
    ((dT < 0).sum()),
2077                                       "pass": bool(dT.min() >=
     0.0)}
2078     diff3 = sc["TN"] - sc["T0"]
2079     S["S3_surface_T_ge_center_T"] = {"min_surface_minus_center
    ": float(diff3.min()),
2080                                      "at_1800s": float(diff3
    [-1]),
2081                                      "pass": bool(diff3.min()
    >= -1e-12)}
2082     T_air_series = np.array([interp_driver(t, "linear")[0] for
     t in A["t_out"]])
2083     exc = float(np.max(A["T_out"] - T_air_series[:, None]))
2084     S["S4_below_oven_T"] = {"max_excess_C": exc, "criterion":
    1e-9,
2085                             "pass": bool(exc <= 1e-9)}
2086     dm = np.diff(sc["mass"])
```

```
 S["S5_mass_strictly_decreasing"] = {"max_step_delta": float(dm.max()),
                                    "n_positive_steps": int((dm > 0).sum()),
                                    "mass_ratio_1800": float(sc["mass"][-1] / sc["mass"][0]),
                                    "pass": bool(dm.max() < 0.0)}
 dC_prof = np.diff(A["C_out"], axis=1)
 S["S6_C_profile_radially_nonincreasing"] = {
     "max_increase": float(dC_prof.max()), "pass": bool(dC_prof.max() <= 1e-12)}
 dT_prof = np.diff(A["T_out"], axis=1)
 S["S7_T_profile_radially_nondecreasing"] = {
     "min_increase": float(dT_prof.min()), "pass": bool(dT_prof.min() >= -1e-12)}
 dev8 = float(abs(A["C_out"][int(T_END)][0] - C_INIT))
 S["S8_center_moisture_static"] = {"abs_dev_1800": dev8, "criterion": 1e-3,
                                   "pass": bool(dev8 <= 1e-3)}
 t_air, T_oven = load_oven_temperature()
 i_out = np.arange(0, A["N"] + 1, 10)
 T_an = duhamel_temperature([T_END], i_out * DR_CM / 100.0, t_air, T_oven,
                            n_roots=320)[0]
 e9 = np.abs(A["T_out"][int(T_END)][i_out] - T_an)
 S["S9_vs_analytic"] = {"max_abs_err_C": float(e9.max()), "at_center": float(e9[0]),
                        "at_surface": float(e9[-1]), "criterion": 5e-5,
                        "pass": bool(e9.max() < 5e-5)}
 CsB = float(B["C_out"][int(T_END)][-1])
 devB = float(np.max(np.abs(B["C_out"][int(T_END)] - C_INIT)))
```

```
    S["S10_caseB_no_drying"] = {"surface_C_1800": CsB, "max_field_dev": devB,
                                "scoping_surface_ref": 2.549885,
                                "pass": bool(abs(CsB - 2.549885) < 1e-3 and devB < 1e-3)}
    for k, v in S.items():
        print(f"  {k:<40} {'PASS' if v['pass'] else 'FAIL'}  "
              + ", ".join(f"{kk}={vv:.6g}" for kk, vv in v.items()
                          if kk not in ("pass",) and isinstance(vv, (int, float))))
    return S, T_air_series


def block_case_ab(A, B):
    print()
    print("对比验证 ①: 口径 A vs 口径 B（核心建模选择的替代方案数值对比）")
    qA = HM * (A["C_out"][1:, -1] - np.array([interp_driver(t, "linear")[1]
                                              for t in A["t_out"][1:]]))
    TsB = B["T_out"][1:, -1]
    w_all = np.array([interp_driver(t, "linear")[1] for t in B["t_out"][1:]])
    Ta_all = np.array([interp_driver(t, "linear")[0] for t in B["t_out"][1:]])
    qB = HM * (rho_v_sat(TsB) - rho_v_inf(w_all, Ta_all)) / RHO_D
    sc, scB = A["scalars"], B["scalars"]
    cmpAB = {
        "surface_C_1800s": {"A": float(A["C_out"][int(T_END)][-1]),
                            "B": float(B["C_out"][int(T_END)
```

```
    ][-1])},
2133        "center_C_1800s": {"A": float(A["C_out"][int(T_END)
    ][0]),
2134                           "B": float(B["C_out"][int(T_END)
    ][0])},
2135        "mass_ratio_1800s": {"A": float(sc["mass"][-1] / sc["
    mass"][0]),
2136                             "B": float(scB["mass"][-1] / scB[
    "mass"][0])},
2137        "flux_kg_m2_s_1800s": {"A": float(qA[-1] * RHO_D), "B"
    : float(qB[-1] * RHO_D)},
2138        "flux_ratio_A_over_B": float(abs(qA[-1] / qB[-1])),
2139        "rho_d_kg_m3": RHO_D,
2140     }
2141     print(f"  表面 C(1800 s): A {cmpAB['surface_C_1800s']['A
    ']:.6f} vs "
2142           f"B {cmpAB['surface_C_1800s']['B']:.9f}")
2143     print(f"  总水量比: A {cmpAB['mass_ratio_1800s']['A']:.6f}
     vs "
2144           f"B {cmpAB['mass_ratio_1800s']['B']:.9f}")
2145     print(f"  通量比 A/B = {cmpAB['flux_ratio_A_over_B']:.4e}"
    )
2146     return cmpAB
2147
2148
2149 def block_2d(A, T_air_series, quick):
2150     print("对比验证 ③: 1D vs 2D 含端面")
2151     if quick:
2152         print("  --quick: 跳过 2D")
2153         return {"skipped": True}
2154     t0 = time.time()
2155     DT_2D = 1.0 / 4
2156     o2 = run_2d_case(dr_cm=0.05, dz_cm=0.5, dt=DT_2D, t_end=
    T_END, n_picard=2)
2157     Tm = o2["T"]
```

```
    C2 = o2["C"]
    g2 = o2["geom"]
    k_mid = g2["Nz"]
    o1 = solve_q1(case="A", dr_cm=0.05, dt=DT_2D, t_end=T_END,
    dt_out=T_END)
    T_1d_c = float(o1["T_out"][-1][0])
    C_1d_c = float(o1["C_out"][-1][0])
    d1 = float(abs(Tm[0, k_mid] - T_1d_c))
    d2 = float(Tm[0, 0] - T_1d_c)
    d3 = float(abs(C2[0, k_mid] - C_1d_c))
    zrow = Tm[0, :]
    d4 = float((zrow.max() - zrow.min()) / (T_air_series[-1] -
    T_INIT))
    cmp2d = {"dr_cm": 0.05, "dz_cm": 0.5, "Nz": g2["Nz"], "
dt_s": DT_2D,
            "T_mid_center_1800": float(Tm[0, k_mid]),
            "T_end_center_1800": float(Tm[0, 0]),
            "T_1d_center_1800_same_mesh_and_dt": T_1d_c,
            "T_1d_center_1800_production": float(A["T_out"][
int(T_END)][0]),
            "delta1_mid_plane_C": d1, "delta2_end_face_C": d2
,
            "delta3_mid_plane_C_moisture": d3,
            "delta4_z_nonuniformity": d4,
            "criteria": {"delta1_le_0.5C": bool(d1 <= 0.5),
                         "delta3_le_5e-3": bool(d3 <= 5e-3)},
            "wall_s": time.time() - t0}
    print(f"  Δ1 中截面偏差 = {d1:.4f} ℃（判据 ≤0.5） {'PASS'
if d1 <= 0.5 else 'FAIL'}")
    print(f"  Δ2 端面中心偏差 = {d2:.4f} ℃（单独报告，预期偏
大）")
    print(f"  Δ3 中截面水分偏差 = {d3:.3e} kg/kg（判据 ≤5e-3）
    {'PASS' if d3 <= 5e-3 else 'FAIL'}")
    print(f"  Δ4 z 向不均匀度 = {d4:.4f}; 耗时 {cmp2d['wall_s
'']:.1f} s")
```

```
    return cmp2d


def block_D_and_interp(A, N):
    print("对比验证㊺：常数 D vs D(C)；线性 vs 阶梯插值")
    oD = solve_q1(case="A", dr_cm=DR_CM, dt=1.0 / 8, t_end=T_END, dt_out=T_END,
                  D_CONST=float(D_of_C(C_INIT)))
    oS = solve_q1(case="A", dr_cm=DR_CM, dt=1.0 / 8, t_end=T_END, dt_out=T_END,
                  INTERP="step")
    oL = solve_q1(case="A", dr_cm=DR_CM, dt=1.0 / 8, t_end=T_END, dt_out=T_END)
    cmpDC = {"D_const_value": float(D_of_C(C_INIT)),
             "surface_C_1800_variableD": float(oL["C_out"][-1][-1]),
             "surface_C_1800_constD": float(oD["C_out"][-1][-1]),
             "surface_diff": float(oD["C_out"][-1][-1] - oL["C_out"][-1][-1]),
             "center_diff": float(oD["C_out"][-1][0] - oL["C_out"][-1][0]),
             "r1cm_diff": float(oD["C_out"][-1][N // 2] - oL["C_out"][-1][N // 2]),
             "mass_ratio_variableD": float(oL["scalars"]["mass"][-1] / oL["scalars"]["mass"][0]),
             "mass_ratio_constD": float(oD["scalars"]["mass"][-1] / oD["scalars"]["mass"][0])}
    cmpINT = {"linear_surface_C_1800": float(oL["C_out"][-1][-1]),
              "step_surface_C_1800": float(oS["C_out"][-1][-1]),
              "surface_diff": float(oS["C_out"][-1][-1] - oL["C_out"][-1][-1]),
              "linear_center_C_1800": float(oL["C_out"]
```

```
    ][-1][0]),
2206                "step_center_C_1800": float(oS["C_out"][-1][0]),
2207                "center_diff": float(oS["C_out"][-1][0] - oL["
    C_out"][-1][0]),
2208                "T_center_diff_1800": float(oS["T_out"][-1][0] -
     oL["T_out"][-1][0]),
2209                "T_surface_diff_1800": float(oS["T_out"][-1][-1]
     - oL["T_out"][-1][-1])}
2210     print(f"  常数 D：表面 {cmpDC['surface_C_1800_constD']:.6f
    } vs 变系数 "
2211           f"{cmpDC['surface_C_1800_variableD']:.6f}（差 {cmpDC
    ['surface_diff']:+.6f}）")
2212     print(f"  阶梯插值：表面 {cmpINT['step_surface_C_1800']:.6
    f} vs 线性 "
2213           f"{cmpINT['linear_surface_C_1800']:.6f}（差 {cmpINT
    ['surface_diff']:+.6f}）")
2214     return cmpDC, cmpINT
2215
2216
2217 def block_envelope(i_out, dt_scan):
2218     print("对比验证 ⑥：驱动势包络 h_m × {0, 0.001, …, 1000,
    ∞}")
2219     env = []
2220     mults = [0.0, 0.001, 0.01, 0.1, 1.0, 10.0, 100.0, 1000.0]
2221     for m in mults:
2222         o = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=
    T_END, dt_out=T_END,
2223                      HM=HM * m)
2224         env.append({"hm_mult": m, "hm_value": HM * m,
2225                     "C_1800": [float(v) for v in o["C_out"
    ][-1][i_out]],
2226                     "mass_ratio": float(o["scalars"]["mass"
    ][-1] / o["scalars"]["mass"][0])})
2227     o_inf = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=
    T_END, dt_out=T_END,
```

```
                    HM_INF=True)
        env.append({"hm_mult": float("inf"), "hm_value": None,
                    "C_1800": [float(v) for v in o_inf["C_out"][-1][i_out]],
                    "mass_ratio": float(o_inf["scalars"]["mass"][-1] / o_inf["scalars"]["mass"][0])})
        for e in env:
            print(f"  h_m×{str(e['hm_mult']):<7} C(2cm,1800) = {e['C_1800'][-1]:.6f}"
                  f"   C(1cm)={e['C_1800'][10]:.6f}  mass={e['mass_ratio']:.6f}")
        mon = all(env[i]["C_1800"][k] >= env[i + 1]["C_1800"][k] - 1e-12
                  for i in range(len(env) - 1) for k in range(len(i_out)))
        return env, {
            "rows": env, "monotone_nonincreasing_in_hm": bool(mon),
            "acceptance": {"hm0_max_dev_from_init": float(np.max(np.abs(
                np.array(env[0]["C_1800"]) - C_INIT))),
                "hm_inf_surface_vs_Cair": float(abs(env[-1]["C_1800"][-1] - 0.03307))}}


def block_sensitivity(N, dt_scan):
    print()
    print("灵敏度：单参数（h、h_m、k、ρ、c□）+ 多参数联合（h×h_m）")
    scans = {
        "h": ("H", [H * 0.5, H, H * 2.0]),
        "h_m": ("HM", [HM * 0.5, HM, HM * 2.0]),
        "k": ("K", [K * 0.5, K, K * 2.0]),
        "rho": ("RHO", [RHO * 0.5, RHO, RHO * 2.0]),
        "c0": ("C_INIT", [C_INIT * 0.9, C_INIT, C_INIT * 1.1])
```

```
,
 }
 ref = {"H": H, "HM": HM, "K": K, "RHO": RHO, "C_INIT": C_INIT}
 sens = {}
 for name, (key, vals) in scans.items():
     rows = []
     for v in vals:
         o = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=T_END, dt_out=T_END,
                      **{key: v})
         rows.append({"value": float(v), "mult": float(v / ref[key]),
                      "T_center_1800": float(o["T_out"][-1][0]),
                      "T_surface_1800": float(o["T_out"][-1][-1]),
                      "C_center_1800": float(o["C_out"][-1][0]),
                      "C_r1cm_1800": float(o["C_out"][-1][N // 2]),
                      "C_surface_1800": float(o["C_out"][-1][-1]),
                      "mass_ratio": float(o["scalars"]["mass"][-1] / o["scalars"]["mass"][0])})
     sens[name] = {"param_key": key, "rows": rows}
     print(f"  {name:<5} " + " | ".join(
         f"×{row['mult']:.2f}: C_s={row['C_surface_1800']:.4f} T_c={row['T_center_1800']:.4f}"
         for row in rows))
 o_rho = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan, t_end=T_END, dt_out=T_END,
                  RHO=976.4)
 sens["alpha_pm19pct"] = {
     "note": "ρ 由 820 换成附录3 的 ρ(C=2.55)=976.4  α 降
```

```
    19.07%，仅影响温度场",
2276            "T_center_1800": float(o_rho["T_out"][-1][0]),
2277            "T_surface_1800": float(o_rho["T_out"][-1][-1]),
2278            "C_surface_1800": float(o_rho["C_out"][-1][-1]),
2279        }
2280        joint = []
2281        for hh in [H * 0.5, H, H * 2.0]:
2282            for mm in [HM * 0.5, HM, HM * 2.0]:
2283                o = solve_q1(case="A", dr_cm=DR_CM, dt=dt_scan,
    t_end=T_END, dt_out=T_END,
2284                             H=hh, HM=mm)
2285                joint.append({"h": float(hh), "hm": float(mm),
2286                              "T_center_1800": float(o["T_out"
    ][-1][0]),
2287                              "C_surface_1800": float(o["C_out"
    ][-1][-1]),
2288                              "mass_ratio": float(o["scalars"]["
    mass"][-1] / o["scalars"]["mass"][0])})
2289        sens["joint_h_x_hm"] = {"rows": joint}
2290        return sens, joint
2291
2292
2293    def block_error_analysis(conv):
2294        return {
2295            "temperature_time_discretization": {
2296                "min_combo": conv["table2_dt_sweep_temperature"]["
    min_combo_meeting_acceptance"],
2297                "order_p": conv["table2_dt_sweep_temperature"]["
    p_order_center"]},
2298            "temperature_space_discretization": {
2299                "p_order": conv["table1_dr_sweep_temperature"]["
    p_order_center"], },
2300            "moisture_surface_grid_uncertainty": {
2301                "p_order": conv["table4_dr_sweep_moisture_surface"
    ]["p_order"],
```

```
            "richardson": conv["table4_dr_sweep_moisture_surface"]["richardson_surface_C"],
            "uncertainty": conv["table4_dr_sweep_moisture_surface"]["grid_uncertainty_vs_richardson"]},
        "model_error_latent_heat": {
            "note": "能量方程未耦合蒸发潜热（题面未给 ΔH 与吸附平衡关系） 系统性高估表面温度；"
                    "潜热/显热比 ≈5.2~18.5（建模文档 §6.3）",
            "ratio_range": [5.2, 18.5]},
     }


def dump(res, VER, sens, cmpAB, cmpDC, cmpINT):
    for fn, obj in [("q1_verification.json", VER),
                    ("q1_sensitivity.json", sens),
                    ("q1_compare.json", {"caseAB": cmpAB, "twoD": VER["compare_2D"],
                                         "D_const": cmpDC, "interp": cmpINT,
                                         "envelope": VER["compare_envelope_hm"]})]:
        (res / fn).write_text(json.dumps(obj, ensure_ascii=False, indent=2), encoding="utf-8")


def dump_figure_inputs(res, conv, sens, joint, env, cmpAB, i_out):
    figd = res / "fig_data"
    t1 = conv["table1_dr_sweep_temperature"]["rows"]
    t2 = conv["table2_dt_sweep_temperature"]["rows"]
    t4 = conv["table4_dr_sweep_moisture_surface"]["rows"]
    err_bars = [{"label": "Δr 空间（温度，Δt=1 s）",
                 "x": [row["dr_cm"] for row in t1],
                 "err_center": [row["err_center"] for row in
```

```
    t1],
2328                "err_surface": [row["err_surf"] for row in t1
    ]},
2329               {"label": "Δt 时间（温度，Δr=0.01 cm）",
2330                "x": [row["dt"] for row in t2],
2331                "err_center": [row["err_center"] for row in
    t2],
2332                "err_surface": [row["err_surf"] for row in t2
    ]},
2333               {"label": "Δr 空间（水分表面列，Δt=1/32 s）",
2334                "x": [row["dr_cm"] for row in t4],
2335                "value": [row["C_surf"] for row in t4]}]
2336     (figd / "error_bars.json").write_text(json.dumps(err_bars,
     ensure_ascii=False, indent=2),
2337                                           encoding="utf-8")
2338     with open(figd / "sensitivity.csv", "w", encoding="utf-8-
    sig") as fh:
2339         fh.write("param,mult,value,T_center_1800,
    T_surface_1800,C_center_1800,"
2340                  "C_r1cm_1800,C_surface_1800,mass_ratio\n")
2341         for name, grp in sens.items():
2342             if name in ("joint_h_x_hm", "alpha_pm19pct"):
2343                 continue
2344             for row in grp["rows"]:
2345                 fh.write(",".join([name, f"{row['mult']:.4f}",
     f"{row['value']:.6g}",
2346                                    f"{row['T_center_1800']:.9f
    }", f"{row['T_surface_1800']:.9f}",
2347                                    f"{row['C_center_1800']:.9f
    }", f"{row['C_r1cm_1800']:.9f}",
2348                                    f"{row['C_surface_1800']:.9
    f}",
2349                                    f"{row['mass_ratio']:.9f}"
    ]) + "\n")
2350         for row in joint:
```

```
            fh.write(",".join(["h_x_hm_joint", f"{row['h']:.6g}x{row['hm']:.6g}", "",
                               f"{row['T_center_1800']:.9f}", "", "", "",
                               f"{row['C_surface_1800']:.9f}",
                               f"{row['mass_ratio']:.9f}"]) + "\n")
    with open(figd / "envelope_hm.csv", "w", encoding="utf-8-sig") as fh:
        fh.write("hm_mult,r_cm,"
                 + ",".join(f"C_1800_r{r:g}cm" for r in i_out * DR_CM * 100) + "\n")
        for item in env:
            fh.write(f"{item['hm_mult']}," + ",".join(f"{v:.9f}" for v in item["C_1800"]) + "\n")
    (figd / "ab_compare.json").write_text(json.dumps(cmpAB, ensure_ascii=False, indent=2),
                                          encoding="utf-8")


def main():
    quick = "--quick" in sys.argv
    t_start = time.time()
    res = ROOT / "results"
    (res / "fig_data").mkdir(parents=True, exist_ok=True)
    VER = {}

    A = solve_q1(case="A", dr_cm=DR_CM, dt=DT_IN, t_end=T_END, dt_out=1.0)
    B = solve_q1(case="B", dr_cm=DR_CM, dt=DT_IN, t_end=T_END, dt_out=1.0)
    N = A["N"]
    i_out = np.arange(0, N + 1, 10)
    dt_scan = 1.0 / 8

```

```
    VER["P0_picard_frozen_old_layer"] = check_P0()
    VER["P3_conservation"] = block_p3(A, B)
    VER["S_checks"], T_air_series = block_s_checks(A, B)

    cmpAB = block_case_ab(A, B)
    VER["compare_2D"] = block_2d(A, T_air_series, quick)

    cmpDC, cmpINT = block_D_and_interp(A, N)
    VER["compare_D_const"] = cmpDC
    VER["compare_interp"] = cmpINT

    env, VER["compare_envelope_hm"] = block_envelope(i_out, dt_scan)

    sens, joint = block_sensitivity(N, dt_scan)
    VER["sensitivity"] = sens

    conv = json.loads((res / "q1_convergence.json").read_text(encoding="utf-8"))
    VER["error_analysis"] = block_error_analysis(conv)

    dump(res, VER, sens, cmpAB, cmpDC, cmpINT)
    dump_figure_inputs(res, conv, sens, joint, env, cmpAB, i_out)

    print()
    print(f"完成，总耗时 {time.time() - t_start:.1f} s。已写 results/q1_verification.json 等")


if __name__ == "__main__":
    sys.exit(main())
from __future__ import annotations
import json
import sys
```

```
import time
from pathlib import Path
import numpy as np

ROOT = Path(__file__).resolve().parent.parent

from solver_q1 import solve_q1
from independent_check_q1_temp import (
    load_oven_temperature, duhamel_temperature,
)

T_TAB = [100, 300, 600, 900, 1200, 1500, 1800]
R_TAB_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
R_ALL_CM = list(np.arange(0.0, 2.0 + 1e-9, 0.1))
DR_CM = 0.01
T_ACC = 5.0e-5
M_ACC = 5.0e-5
T_END = 1800.0


def sweep_time_steps(i_tab, i_all, an_tab, an_1800, an_win):
    out = {"criterion_C": T_ACC, "dr_cm": DR_CM,
           "note": "验收线按窗口内逐点最大误差核对", "rows": []}
    print(f"{'Δt (s)':>12}{'表1(7×5)最大':>16}  {'1800s 21点最大':>16}{'窗口合计最大':>16} 判定")
    c_win = {}
    dt_list = [1.0, 0.5, 0.25, 0.125, 0.0625, 1.0 / 32, 1.0 / 64, 1.0 / 128]
    for dt in dt_list:
        o = solve_q1(case="A", dr_cm=DR_CM, dt=dt, t_end=T_END, dt_out=1.0)
        num_tab = np.array([[o["T_out"][int(t)][i] for i in i_tab] for t in T_TAB])
        e_tab = np.abs(num_tab - an_tab)
```

```
        it, ir = np.unravel_index(np.argmax(e_tab), e_tab.shape)
        e_1800 = float(np.max(np.abs(o["T_out"][int(T_END)][i_all] - an_1800)))
        dev_win = np.abs(o["T_out"][1:, i_all] - an_win)
        e_win_all = float(dev_win.max())
        iw_, jw_ = np.unravel_index(np.argmax(dev_win), dev_win.shape)
        e_tab_max = float(e_tab.max())
        ok = max(e_tab_max, e_1800, e_win_all) < T_ACC
        c_win[dt] = o["C_out"][1:][:, i_all].copy()
        out["rows"].append(dict(dt=dt, err_tab7x5_max=e_tab_max,
                                err_tab7x5_argmax={"t_s": T_TAB[it], "r_cm": R_TAB_CM[ir]},
                                err_1800s_21pt=e_1800, err_window_all_cells=e_win_all,
                                err_window_all_argmax={"t_s": int(iw_) + 1, "r_cm": float(R_ALL_CM[jw_])},
                                meets=T_ACC if ok else None, pass_=bool(ok),
                                wall_s=o["elapsed_s"]))
        print(f"{dt:>12.6f}{e_tab_max:>16.3e}@{T_TAB[it]:<5d}{e_1800:>16.3e}{e_win_all:>16.3e}"
              f"  {'PASS' if ok else '超线'}")
    ok_rows = [r for r in out["rows"] if r["pass_"]]
    out["min_dt_meeting_full_window"] = (max(ok_rows, key=lambda r: r["dt"])["dt"]
                                         if ok_rows else None)
    return out, c_win, dt_list


def moisture_vs_reference(c_win, dt_list, out):
    ref_dt = dt_list[-1]
    ref = c_win[ref_dt]
```

```
    t_axis = np.arange(1, 1801, dtype=float)
    m_rows = []
    for dt in dt_list:
        d = np.abs(c_win[dt] - ref)
        iw, jw = np.unravel_index(np.argmax(d), d.shape)
        m_rows.append(dict(dt=dt, max_abs_diff_vs_ref=float(d.max()),
                           argmax={"t_s": float(t_axis[iw]), "r_cm": float(R_ALL_CM[jw])}))
        out["rows"][dt_list.index(dt)]["err_window_moisture_vs_1_128"] = float(d.max())
    return ref_dt, ref, t_axis, m_rows


def moisture_error_stats(c_win, ref, ref_dt, t_axis, m_rows, rich_err, out):
    d_64 = c_win[1.0 / 64] - ref
    d_32 = c_win[1.0 / 32] - c_win[1.0 / 64]
    a_64 = np.abs(d_64)
    err64_p1, err64_p2 = rich_err(float(a_64.max()), 1.0), rich_err(float(a_64.max()), 2.0)
    per_t64 = np.abs(d_64).max(axis=1) * 2.0
    n_over = int((np.abs(d_64) * 2.0 > M_ACC).sum())
    iw, jw = np.unravel_index(np.argmax(a_64), a_64.shape)
    out["moisture_window_error"] = {
        "reference_dt_s": ref_dt,
        "criterion_C_analog": M_ACC,
        "criterion_note": "水分无解析解  以更细步长解为参考的时间步自收敛；判据取 5e-5 kg/kg"
                          "，与温度同口径",
        "scope": "仅时间离散",
        "rows": m_rows,
        "delivered_dt_s": 1.0 / 64,
        "raw_gap_1_64_vs_1_128_max": float(a_64.max()),
        "raw_gap_1_64_vs_1_128_argmax": {"t_s": float(t_axis[
```

```
    iw]), "r_cm": float(R_ALL_CM[jw])},
          "delivered_dt_error_estimate_p1": err64_p1,
          "delivered_dt_error_estimate_p2": err64_p2,
          "delivered_dt_meets_analog_criterion_p1": bool(
    err64_p1 < M_ACC),
          "delivered_dt_meets_analog_criterion_p2": bool(
    err64_p2 < M_ACC),
          "n_cells_over_criterion_p1": n_over,
          "n_cells_total": int(a_64.size),
          "per_time_max_p1": {"t_max_s": float(t_axis[int(np.
    argmax(per_t64))]),
                              "max": float(per_t64.max()),
                              "max_at_1800s": float(per_t64[-1])
    ,
                              "max_in_first_60s": float(per_t64
    [:60].max())},
          "dt_1_32_vs_1_64_max": float(np.abs(d_32).max()),
          "dt_1_32_error_estimate_p1": rich_err(float(np.abs(
    d_32).max()), 1.0),
          "order_p_note": "后向 Euler 一阶  取 p=1 为主估计；同
    时给 p=2 的乐观界",
          "implied_1_128_error_estimate_p1": float(a_64.max() /
    (2.0 ** 1.0 - 1.0)),
      }
      return a_64, d_32, err64_p1, err64_p2, n_over, iw, jw


def main():
      t0 = time.time()
      t_air, T_oven = load_oven_temperature()
      i_tab = [int(round(rc / DR_CM)) for rc in R_TAB_CM]
      i_all = [int(round(rc / DR_CM)) for rc in R_ALL_CM]
      an_tab = duhamel_temperature(T_TAB, np.array(R_TAB_CM) /
    100.0, t_air, T_oven, n_roots=320)
      an_1800 = duhamel_temperature([T_END], np.array(R_ALL_CM)
```

```
/ 100.0, t_air, T_oven, n_roots=320)[0]
 an_win = duhamel_temperature(np.arange(1, 1801, dtype=float), np.array(R_ALL_CM) / 100.0, t_air, T_oven, n_roots=320)

 out, c_win, dt_list = sweep_time_steps(i_tab, i_all, an_tab, an_1800, an_win)

 ref_dt, ref, t_axis, m_rows = moisture_vs_reference(c_win, dt_list, out)

 def rich_err(diff_max, p):
     return float(diff_max * (2.0 ** p) / (2.0 ** p - 1.0))

 a_64, d_32, err64_p1, err64_p2, n_over, iw, jw = moisture_error_stats(c_win, ref, ref_dt, t_axis, m_rows, rich_err, out)
 out["wall_s"] = time.time() - t0
 p = ROOT / "results" / "q1_window_error.json"
 p.write_text(json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
 print(f"\n结论：满足「全窗口逐点 < 5e-5 ℃」的最大 Δt = "
       f"{out['min_dt_meeting_full_window']} s（本生产取 1/64 s）")
 print(f"水分场（固定 Δr=0.01 cm，原始节点值）：1/64 vs 1/128 实测最大差 = {a_64.max():.3e} kg/kg "
       f"@t={t_axis[iw]:g}s, r={R_ALL_CM[jw]:g}cm")
 print(f"   Δt=1/64 s 自身误差估计：p=1 {err64_p1:.3e}（判据 {M_ACC:.0e}  "
       f"{'PASS' if err64_p1 < M_ACC else '超线'}）/ p=2 {err64_p2:.3e}"
       f"（{'PASS' if err64_p2 < M_ACC else '超线'}）；超判据单元 {n_over}/{a_64.size}")
 print(f"   1/32 -> 1/64 实测最大差 {np.abs(d_32).max():.3e}（其 p=1 误差估计 "
```

```
          f"{rich_err(float(np.abs(d_32).max()), 1.0):.3e}）")
    print(f"总耗时 {out['wall_s']:.1f} s；已写出 {p}")


if __name__ == "__main__":
    sys.exit(main())
```

q2.py

```
from __future__ import annotations

import json
import re
from pathlib import Path

_PROJ_ROOT = Path(__file__).resolve().parents[1]
_FACTS_PATH = _PROJ_ROOT / "数据台账" / "DATA_FACTS.json"
_REPORT_PATH = _PROJ_ROOT / "数据台账" / "MODELING_REPORT.md"
_LEDGER_PATH = _PROJ_ROOT / "数据台账" / "CROSS_PROBLEM_LEDGER
    .json"

_FACTS = json.loads(_FACTS_PATH.read_text(encoding="utf-8"))
_GIVEN = {item["name"]: item for item in _FACTS["given"]}

_NUM = re.compile(r"[-+]?\d+(?:\.\d+)?(?:[eE][-+]?\d+)?")


def const(name: str) -> float:
    item = _GIVEN[name]
    assert item["kind"] == "constant", f"{name} 不是 constant
    条目"
    return float(item["value"])


def formula_coeffs(name: str, count: int) -> list[float]:
    item = _GIVEN[name]
```

```
    assert item["kind"] == "formula", f"{name} 不是 formula 条目"
    rhs = item["form"].split("=", 1)[1]
    found = [float(x) for x in _NUM.findall(rhs)]
    assert len(found) >= count, f"{name} 解析到的系数不足 {count} 个：{found}"
    return found[:count]


LENGTH_M = const("药材几何-长")
R_FIX = const("药材几何-半径")
T_INIT_C = const("初始温度")
C_INIT = const("初始水分浓度")
H_CONV = const("对流换热系数 h")
HM_MASS = const("对流传质系数 h_m")
C_CRIT = const("问题三烘干终点阈值")

_RHO_A, _RHO_B = formula_coeffs("密度经验公式(Q2/Q3)", 2)
_CP_A, _CP_B = formula_coeffs("比热容经验公式(Q2/Q3)", 2)
_K_A, _K_B = formula_coeffs("热传导系数经验公式(Q2/Q3)", 2)
_D_PRE, _D_EC, _D_ET = formula_coeffs("水分扩散系数经验公式(Q2/Q3)", 3)

RHO_COEF = (_RHO_A, _RHO_B)
CP_COEF = (_CP_A, _CP_B)
K_COEF = (_K_A, _K_B)
D_COEF = (_D_PRE, _D_EC, _D_ET)
RHO_DRY_REF = _RHO_A

_REPORT = _REPORT_PATH.read_text(encoding="utf-8")
_ROW = re.compile(r"^\|[^|]+\|\s*(" + _NUM.pattern + r")\s*\|[^|]*\|\s*'([A-Za-z_]+)'\s*\|", re.M)
_CAL = {label: float(value) for value, label in _ROW.findall(_REPORT)}
assert _CAL, "未能从 MODELING_REPORT.md §7.3 解析参数口径表"
```

```
N_NODES = int(_CAL["N_nodes"])
DT_SOLVER = _CAL["dt_solver"]
T_AIR_HOLD = _CAL["T_air_hold"]
C_AIR_HOLD = _CAL["C_air_hold"]

PICARD_ITERS = int(re.search(r"PICARD_ITERS\s*[=:]\s*(\d+)", _REPORT).group(1))
FACE_D = "kirchhoff"
FACE_K = "arith"
AIR_EXTRAP = "hold_last"
LATENT_HEAT_COUPLING = False
SHRINKAGE = False

_UNIT_TEMP = json.loads(_LEDGER_PATH.read_text(encoding="utf-8"))["unit_ledger"]["temperature_in_properties"]
KELVIN_OFFSET = float(re.search(r"\+\s*(" + _NUM.pattern + r")", _UNIT_TEMP).group(1))

OUT_DR_CM = 0.1
OUT_R_MAX_CM = R_FIX * 100.0
OUT_N_COLS = int(round(OUT_R_MAX_CM / OUT_DR_CM)) + 1
Q2_SPAN_S = 3 * 3600
Q2_TABLE_STEP_S = 1800
Q3_TABLE_STEP_S = 6 * 3600
Q3_REC_STEP_S = 60
ROUND_DIGITS = 4
T_MAX_S = 150 * 3600
D_SAFE_FLOOR = 1e-9

assert abs(_CAL["radius"] - R_FIX) < 1e-12, "半径口径冲突：口径表 vs DATA_FACTS"
assert abs(_CAL["length"] - LENGTH_M) < 1e-12, "长度口径冲突"
assert abs(_CAL["T_init"] - T_INIT_C) < 1e-12, "初温口径冲突"
assert abs(_CAL["C_init"] - C_INIT) < 1e-12, "初始水分口径冲突
```

```
    "
assert abs(_CAL["h_conv"] - H_CONV) < 1e-12, "对流换热系数口径冲突"
assert abs(_CAL["hm_mass"] - HM_MASS) / HM_MASS < 1e-9, "对流传质系数口径冲突"
assert abs(_CAL["C_crit"] - C_CRIT) < 1e-12, "终点阈值口径冲突"
assert abs(_CAL["rho_dry_ref"] - RHO_DRY_REF) < 1e-12, "干固体参考密度口径冲突"

_DR_MM = R_FIX / (N_NODES - 1) * 1000.0
assert abs(_DR_MM - OUT_DR_CM * 10.0) < 1e-12, (
    f"网格与输出栅格不一致：dr={_DR_MM} mm 与输出步长 {OUT_DR_CM} cm 不重合，"
    "N=21 的目的就是让 r=0/0.5/1/1.5/2 cm 直接落节点（§13 S11）")
assert N_NODES == OUT_N_COLS, "主算节点数应与输出列数一致（各 0.1 cm 一列）"
assert 200 < KELVIN_OFFSET < 300, "绝对温度换算量异常"
assert LATENT_HEAT_COUPLING is False and SHRINKAGE is False, "假设 3/6 开关被改动"

if __name__ == "__main__":
    print(f"[params] R={R_FIX} m, T0={T_INIT_C} C, C0={C_INIT} kg/kg, h={H_CONV}, hm={HM_MASS}")
    print(f"[params] rho={RHO_COEF}, cp={CP_COEF}, k={K_COEF}, D={D_COEF}, K_offset={KELVIN_OFFSET}")
    print(f"[params] N={N_NODES}, dt={DT_SOLVER}, Picard={PICARD_ITERS}, C_crit={C_CRIT}")
    print(f"[params] hold-last: T_air={T_AIR_HOLD} C, C_air={C_AIR_HOLD} kg/kg")
    print("[params] 口径一致性 OK")
from __future__ import annotations

import os
```

```
import sys
from pathlib import Path

import numpy as np

_CODE_DIR = Path(os.path.realpath(__file__)).parent
if str(_CODE_DIR) not in sys.path:
    sys.path.insert(0, str(_CODE_DIR))

from params import (C_CRIT, C_INIT, H_CONV, HM_MASS, N_NODES,
   DT_SOLVER, PICARD_ITERS, FACE_D,
                    OUT_N_COLS, Q2_SPAN_S, Q2_TABLE_STEP_S,
   Q3_REC_STEP_S, R_FIX, ROUND_DIGITS,
                    T_INIT_C)
from utils import (LIB_VERSIONS, OUTPUT_DIR, FIGURES_DIR, ROOT
   , SolverConfig, diffusivity,
                   dump_json, jsonable, load_air_boundary,
   profile_at, properties, radii_cm,
                   solve_case, table_node_index, write_csv,
   write_grid_xlsx)

BASELINE_KEY = "baseline_N21_dt1"


def baseline_config() -> SolverConfig:
    return SolverConfig(fine_step=DT_SOLVER, fine_until=float(
   Q2_SPAN_S),
                        rec_step=float(Q3_REC_STEP_S),
   extra_crit=(0.12, 0.18, 0.20))


def solve_baseline(air: dict) -> dict:
    return solve_case(BASELINE_KEY, baseline_config(), air)


```

```
def series_d(T: np.ndarray, C: np.ndarray) -> np.ndarray:
    return diffusivity(C.ravel(), T.ravel())


def window_checks(name: str, t: np.ndarray, T: np.ndarray, C: np.ndarray,
                  volume: np.ndarray, air: dict) -> dict:
    out = {"window": name, "n_rows": int(t.size), "t_span_s": [float(t[0]), float(t[-1])]}
    c_air = np.interp(t, air["t"], air["C_air"], left=air["C_air"][0], right=air["C_air"][-1])

    out["C1"] = {"C_min": float(C.min()), "C_max": float(C.max())}
    assert C.min() >= -1e-9 and C.max() <= C_INIT + 1e-9, f"C1 违反@{name}: {out['C1']}"

    cap = float(air["T_air"].max()) + 0.5
    out["C2"] = {"T_min": float(T.min()), "T_max": float(T.max()), "cap": cap}
    assert T.min() >= T_INIT_C - 1e-6 and T.max() <= cap, f"C2 违反@{name}: {out['C2']}"

    for field_name, field in (("C", C), ("T", T)):
        first = float(np.abs(field[:, 1] - field[:, 0]).max())
        second = float(np.abs(field[:, 2] - field[:, 1]).max())
        out[f"C3_{field_name}"] = {"d01": first, "d12": second}
        assert first <= second + 1e-12, f"C3 违反@{name}（{field_name}）: {out[f'C3_{field_name}']}"

    w_hist = C @ volume
    out["C5"] = {"max_dW": float(np.diff(w_hist).max())}
    assert np.diff(w_hist).max() <= 1e-12, f"C5 违反@{name}: {
```

```
    out['C5']}"
162
163     margin = float((C[:, -1] - c_air).min())
164     out["C6"] = {"min_surface_margin": margin}
165     assert margin >= 0.0, f"C6 违反@{name}: 表面浓度低于空气侧
    {margin:.3e}"
166
167     out["C9"] = {"max_dC_dr": float(np.diff(C, axis=1).max())}
168     assert np.diff(C, axis=1).max() <= 1e-12, f"C9 违反@{name}
    : {out['C9']}"
169     return out
170
171
172 def validate_constraints(sol: dict, air: dict, tables: dict)
    -> dict:
173     fine = sol["fine"]
174     coarse = sol["coarse"]
175     volume = sol["grid"]["volume"]
176     report = {"windows": [
177         window_checks("result2_1s_0-3h", fine["t"], fine["T"],
    fine["C"], volume, air),
178         window_checks("full_60s_0-tstar", coarse["t"], coarse[
    "T"], coarse["C"], volume, air)]}
179     for key in ("C1", "C2", "C5", "C6", "C9"):
180         report[key] = {win["window"]: win[key] for win in
    report["windows"]}
181     report["C3"] = {win["window"]: {"C": win["C3_C"], "T": win
    ["C3_T"]} for win in report["windows"]}
182
183     d_full = series_d(coarse["T"], coarse["C"])
184     d_fine = series_d(fine["T"], fine["C"])
185     cv = float(d_full.std() / d_full.mean())
186     source = (_CODE_DIR / "utils.py").read_text(encoding="utf
    -8")
187     report["C4"] = {"cv_D_full_run": cv, "cv_D_first_3h":
```

```
    float(d_fine.std() / d_fine.mean()),
188                 "D_min": float(d_full.min()), "D_max":
    float(d_full.max()),
189                 "D_dynamic_range": float(d_full.max() /
    d_full.min()),
190                 "has_moisture_factor": "moisture_factor"
    in source,
191                 "has_absolute_temperature": "KELVIN_OFFSET
    " in source,
192                 "props_recomputed_per_picard_sweep":
    PICARD_ITERS}
193  assert cv > 0.5, f"C4 违反：全程 std/mean(D)={cv:.4f} <=
    0.5，物性疑似被常量化"
194  assert d_full.max() / d_full.min() > 10.0, "C4 违反：D 动
    态范围不足，疑似取了常量"
195
196  report["C8"] = {"eps_mass": float(sol["eps_mass"]), "W0":
    float(sol["W0"]),
197                 "W_end": float(sol["W_end"]), "Q_out":
    float(sol["Q_out"])}
198  assert abs(sol["eps_mass"]) < 1e-10, f"C8 违反：{report['
    C8']}"
199
200  cols = radii_cm()
201  report["C10"] = {"n_cols": int(cols.size), "first": float(
    cols[0]), "last": float(cols[-1]),
202                   "sheets": sorted(tables["result2"]["
    sheets"]),
203                   "digits": ROUND_DIGITS}
204  assert cols.size == OUT_N_COLS and abs(cols[-1] - 2.0) < 1
    e-12, f"C10 违反：{report['C10']}"
205  assert sorted(tables["result2"]["sheets"]) == ["水分浓度",
     "温度"], "C10 违反：result2 工作表名不符"
206  assert tables["result2"]["sheets"]["温度"] == (Q2_SPAN_S,
    OUT_N_COLS), (
```

```
        f"C10 违反：result2 形状 {tables['result2']['sheets
    ']['温度']} 应为 ({Q2_SPAN_S}, {OUT_N_COLS})")
    return report


def validate_capability(sol: dict, tables: dict) -> dict:
    source = (_CODE_DIR / "utils.py").read_text(encoding="utf
    -8")
    verdict = {}

    probe_c, probe_t = np.array([2.55, 0.15]), np.array([28.0,
     50.0])
    rho, cp, k = properties(probe_c)
    d_two = diffusivity(probe_c, probe_t)
    d_no_moisture = np.exp(-3850.0 / (probe_t + 273.15)) * 2.4
    e-3
    assert rho[0] != rho[1] and cp[0] != cp[1] and k[0] != k
    [1], "P2-C1 不成立：物性对 C 无响应"
    assert not np.allclose(d_two, d_no_moisture), "P2-C1 不成
    立：D 缺 exp(-0.45/C) 因子"
    assert "temperature_factor" in source and "KELVIN_OFFSET"
    in source, "P2-C1 不成立：D 缺绝对温度因子"
    assert "properties(C)" in source, "P2-C1 不成立：物性未在
    时间循环内按当前 C 重算"
    ratio = float(diffusivity(np.array([2.55]), np.array
    ([50.0]))[0]
                  / diffusivity(np.array([0.15]), np.array
    ([50.0]))[0])
    verdict["P2-C1"] = {"holds": True, "D_ratio_wet_over_dry":
     ratio,
                        "rho": jsonable(rho), "cp": jsonable(
    cp), "k": jsonable(k)}

    for name in ("table3", "table4"):
        info = tables[name]
```

```
        assert info["rows"] == Q2_SPAN_S // Q2_TABLE_STEP_S, f"P2-C2 不成立：{name} 行数 {info['rows']}"
    verdict["P2-C2"] = {"holds": True, "rows": tables["table3"]["rows"],
                        "radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0], "digits": ROUND_DIGITS,
                        "csv": tables["table3"]["path"]}

    shapes = tables["result2"]["sheets"]
    assert set(shapes) == {"温度", "水分浓度"}, "P2-C3 不成立：工作表名不符"
    for name, shape in shapes.items():
        assert shape == (Q2_SPAN_S, OUT_N_COLS), f"P2-C3 不成立：{name} 形状 {shape}"
    verdict["P2-C3"] = {"holds": True, "sheets": jsonable(shapes),
                        "bytes": tables["result2"]["bytes"], "sha256": tables["result2"]["sha256"]}
    return verdict


def _solve_and_tables(air):
    sol = solve_baseline(air)
    idx = table_node_index()
    cols_cm = radii_cm()
    times = np.arange(Q2_TABLE_STEP_S, Q2_SPAN_S + 1, Q2_TABLE_STEP_S, dtype=float)
    table3 = np.array([[profile_at(sol, t, "T")[j] for j in idx] for t in times])
    table4 = np.array([[profile_at(sol, t, "C")[j] for j in idx] for t in times])
    header = ["时间/s", "时间/h", *[f"r={cols_cm[j]:.1f}cm" for j in idx]]
    info = {}
    info["table3"] = write_csv(OUTPUT_DIR / "
```

```
    table3_temperature.csv", header,
254                                 [[t, t / 3600.0, *np.round(row,
     ROUND_DIGITS)]
255                                  for t, row in zip(times,
    table3)])
256     info["table4"] = write_csv(OUTPUT_DIR / "table4_moisture.
    csv", header,
257                                 [[t, t / 3600.0, *np.round(row,
     ROUND_DIGITS)]
258                                  for t, row in zip(times,
    table4)])
259     return sol, cols_cm, times, table3, table4, info
260
261
262 def _write_result_tables(sol: dict, info: dict) -> dict:
263     fine = sol["fine"]
264     assert fine["t"].size == Q2_SPAN_S and abs(fine["t"][0] -
    1.0) < 1e-12, "细记录应为 t=1..10800 s"
265     info["result2"] = write_grid_xlsx(OUTPUT_DIR / "result2.
    xlsx",
266                                       {"温度": (fine["t"],
    fine["T"]),
267                                        "水分浓度": (fine["t"],
     fine["C"])})
268     coarse = sol["coarse"]
269     info["result2_full_60s"] = write_grid_xlsx(OUTPUT_DIR / "
    result2_full_60s.xlsx",
270                                                {"温度": (
    coarse["t"], coarse["T"]),
271                                                 "水分浓度": (
    coarse["t"], coarse["C"])})
272     info["W_history"] = write_csv(OUTPUT_DIR / "W_history.csv"
    ,
273                                   ["t_s", "W", "Q_out", "
    eps_mass", "Cmax", "T_air", "C_air"],
```

```
                                    np.column_stack([coarse["t"], coarse["W"], coarse["Q_out"],
                                                     coarse["eps"], coarse["cmax"],
                                                     coarse["T_air"], coarse["C_air"]]))
    info["center_surface"] = write_csv(
        OUTPUT_DIR / "center_surface.csv",
        ["t_s", "T_center", "T_surface", "C_center", "C_surface", "T_air", "C_air"],
        np.column_stack([coarse["t"], coarse["T"][:, 0], coarse["T"][:, -1],
                         coarse["C"][:, 0], coarse["C"][:, -1], coarse["T_air"], coarse["C_air"]]))
    return coarse


def _independent_recompute(sol: dict, air: dict) -> tuple:
    check = solve_case("q2_independent_3h", SolverConfig(t_end=float(Q2_SPAN_S),

    rec_step=float(Q3_REC_STEP_S)), air)
    drift_t = float(np.abs(check["T"] - profile_at(sol, float(Q2_SPAN_S), "T")).max())
    drift_c = float(np.abs(check["C"] - profile_at(sol, float(Q2_SPAN_S), "C")).max())
    assert drift_t < 1e-12 and drift_c < 1e-12, f"独立复算漂移: dT={drift_t:.2e}, dC={drift_c:.2e}"

    at3h_T, at3h_C = profile_at(sol, 10800.0, "T"), profile_at(sol, 10800.0, "C")
    return drift_t, drift_c, at3h_T, at3h_C


def _write_results_json(air: dict, sol: dict, info: dict,
```

```
    cols_cm: np.ndarray, times: np.ndarray,
297                      table3: np.ndarray, table4: np.ndarray
    , constraints: dict,
298                      capability: dict, drift_t: float,
    drift_c: float, at3h_T: np.ndarray,
299                      at3h_C: np.ndarray, coarse: dict) ->
    None:
300  payload = {
301      "problem": 2, "title": "整个烘干过程的温度场与水分浓度
    场",
302      "method": {"scheme": "conservative radial FVM +
    backward Euler + Picard",
303                 "face_diffusivity": FACE_D, "N_nodes":
    N_NODES, "dt_s": DT_SOLVER,
304                 "picard_iters": PICARD_ITERS, "R_m": R_FIX,
     "h": H_CONV, "hm": HM_MASS,
305                 "libs": LIB_VERSIONS},
306      "grid": {"r_cm": jsonable(cols_cm), "volume_weighted":
     jsonable(sol["grid"]["volume"])},
307      "table3_temperature": {"times_s": jsonable(times), "
    radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
308                             "values": jsonable(np.round(
    table3, ROUND_DIGITS))},
309      "table4_moisture": {"times_s": jsonable(times), "
    radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
310                         "values": jsonable(np.round(table4
    , ROUND_DIGITS))},
311      "profiles_3h": {"T": jsonable(np.round(at3h_T,
    ROUND_DIGITS)),
312                      "C": jsonable(np.round(at3h_C,
    ROUND_DIGITS))},
313      "center_surface_60s": {"t_s": jsonable(coarse["t"][
    coarse["t"] <= Q2_SPAN_S]),
314                             "T_center": jsonable(coarse["T"
    ][coarse["t"] <= Q2_SPAN_S, 0]),
```

```
315                                 "T_surface": jsonable(coarse["T
    "][coarse["t"] <= Q2_SPAN_S, -1]),
316                                 "C_center": jsonable(coarse["C"
    ][coarse["t"] <= Q2_SPAN_S, 0]),
317                                 "C_surface": jsonable(coarse["C
    "][coarse["t"] <= Q2_SPAN_S, -1])},
318         "boundary_data": {"t_s": jsonable(air["t"]), "T_air":
    jsonable(air["T_air"]),
319                           "C_air": jsonable(air["C_air"]), "
    sha256": air["sha256"]},
320         "conservation": {"W0": sol["W0"], "W_end_full_run":
    sol["W_end"],
321                          "Q_out_full_run": sol["Q_out"], "
    eps_mass": sol["eps_mass"],
322                          "dehydration_full_run": sol["
    dehydration"]},
323         "diagnostics": jsonable(sol["diag"]),
324         "constraints": jsonable(constraints), "capability":
    jsonable(capability),
325         "independent_recompute_drift": {"dT_max": drift_t, "
    dC_max": drift_c},
326         "artifacts": jsonable(info),
327     }
328     dump_json(FIGURES_DIR / "problem_2_results.json", payload)
329
330
331 def _write_check_report(sol: dict, constraints: dict,
    capability: dict, drift_t: float,
332                         drift_c: float, at3h_T: np.ndarray,
    at3h_C: np.ndarray) -> None:
333     lines = ["# 问题二 自检报告", "",
334              f"- 求解口径：N={N_NODES}, dt={DT_SOLVER} s,
    Picard={PICARD_ITERS}, face_D={FACE_D}",
335              f"- 3 h 中心：T={at3h_T[0]:.4f} ℃（检查点□区间
    [49,50]），"
```

```
            f"C={at3h_C[0]:.4f} kg/kg（区间 [1.6,1.9]）",
            f"- 3 h 表面：T={at3h_T[-1]:.4f} ℃, C={at3h_C[-1]:.4f} kg/kg",
            f"- 守恒残差 |eps_mass|={abs(sol['eps_mass']):.3e} < 1e-10（C8）",
            *[f"- 窗口 {win['window']}（{win['n_rows']}行）: "
              f"C∈[{win['C1']['C_min']:.6f},{win['C1']['C_max']:.6f}]（C1）; "
              f"T∈[{win['C2']['T_min']:.4f},{win['C2']['T_max']:.4f}]≤{win['C2']['cap']:.4f}（C2）; "
              f"max dW={win['C5']['max_dW']:.3e}（C5）; "
              f"min(C_s-C_air)={win['C6']['min_surface_margin']:.3e}（C6）; "
              f"max dC/dr={win['C9']['max_dC_dr']:.3e}（C9）"
              for win in constraints["windows"]],
            f"- std(D)/mean(D)={constraints['C4']['cv_D_full_run']:.4f} > 0.5（C4 动态派生，全程窗口；"
            f"仅 0–3 h 为 {constraints['C4']['cv_D_first_3h']:.4f}，因退化层在 3 h 后才出现），"
            f"D 动态范围 {constraints['C4']['D_dynamic_range']:.1f} 倍",
            f"- 独立复算漂移 dT={drift_t:.2e}, dC={drift_c:.2e}（< 1e-12）",
            f"- 能力项：{', '.join(f'{k}=成立' for k in capability)}",
            "", "结论：C1–C6、C8–C10 与 P2-C1/C2/C3 全部通过。"]
    (ROOT / "_tmp").mkdir(parents=True, exist_ok=True)
    (ROOT / "_tmp" / "problem_2_check.md").write_text("\n".join(lines) + "\n", encoding="utf-8")


def main() -> int:
    air = load_air_boundary()
```

```
    sol, cols_cm, times, table3, table4, info = _solve_and_tables(air)

    coarse = _write_result_tables(sol, info)

    constraints = validate_constraints(sol, air, info)
    capability = validate_capability(sol, info)

    drift_t, drift_c, at3h_T, at3h_C = _independent_recompute(sol, air)

    _write_results_json(air, sol, info, cols_cm, times, table3, table4, constraints,
                        capability, drift_t, drift_c, at3h_T, at3h_C, coarse)

    _write_check_report(sol, constraints, capability, drift_t, drift_c, at3h_T, at3h_C)

    print(f"[problem2] 3h 中心 T={at3h_T[0]:.4f}℃ / C={at3h_C[0]:.4f} kg/kg；"
          f"表面 T={at3h_T[-1]:.4f} / C={at3h_C[-1]:.4f}")
    print(f"[problem2] |eps_mass|={abs(sol['eps_mass']):.3e}, std(D)/mean(D)="
          f"{constraints['C4']['cv_D_full_run']:.4f}（全程）, 独立复算漂移 {max(drift_t, drift_c):.2e}")
    print(f"[problem2] 落盘 {len(info)} 件: {', '.join(v['path'] for v in info.values())}")
    print("[problem2] SELF-CHECK PASS（C1–C6, C8–C10, P2-C1/C2/C3）")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

```python
from __future__ import annotations

import hashlib
import json
import platform
import sys
import time
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import openpyxl
import scipy
from scipy.linalg import solve_banded
from scipy.optimize import brentq
from scipy.special import exp1, j0, j1, jn_zeros

_HERE = Path(__file__).parent.resolve()
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))

from params import (
    R_FIX, T_INIT_C, C_INIT, H_CONV, HM_MASS, C_CRIT, KELVIN_OFFSET,
    RHO_COEF, CP_COEF, K_COEF, D_COEF, N_NODES, DT_SOLVER, PICARD_ITERS,
    FACE_D, FACE_K, AIR_EXTRAP, T_AIR_HOLD, C_AIR_HOLD, T_MAX_S, D_SAFE_FLOOR,
    Q3_REC_STEP_S, OUT_N_COLS, OUT_DR_CM, ROUND_DIGITS,
)

ROOT = _HERE
OUTPUT_DIR = ROOT / "output"
FIGURES_DIR = ROOT / "figures"
AIR_FILE = ROOT.parent / "附件" / "附件1.xlsx"
```

```
AIR_SHEET = "Sheet1"

LIB_VERSIONS = {
    "python": platform.python_version(),
    "numpy": np.__version__,
    "scipy": scipy.__version__,
    "openpyxl": openpyxl.__version__,
}


def sha256_of(path: Path) -> str:
    digest = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def load_air_boundary() -> dict:
    profile = json.loads((ROOT.parent / "数据台账" / "
   DATA_PROFILE.json").read_text(encoding="utf-8"))
    entry = profile["files"].get(AIR_FILE.name, {})
    assert "total_rows" in entry, (
        f"[未建档] {AIR_FILE.name} 不在 DATA_PROFILE.json 或建
   档失败（{entry.get('error', '缺记录')}）")

    book = openpyxl.load_workbook(AIR_FILE, data_only=True,
   read_only=True)
    assert AIR_SHEET in book.sheetnames, f"附件1 缺工作表 {
   AIR_SHEET}: {book.sheetnames}"
    sheet = book[AIR_SHEET]
    rows = list(sheet.iter_rows(values_only=True))
    book.close()

    header = [str(x).strip() if x is not None else "" for x in
```

```
    rows[0]]
    idx = {name: header.index(name) for name in ("时间", "温度", "水分浓度")}
    records = [r for r in rows[1:] if r[idx["时间"]] is not None]

    t = np.array([float(r[idx["时间"]]) for r in records])
    T = np.array([float(r[idx["温度"]]) for r in records])
    C = np.array([float(r[idx["水分浓度"]]) for r in records])

    assert len(records) == entry["total_rows"] - 1 or len(records) == entry["total_rows"], (
        f"[摄入不全] 附件1 实读 {len(records)} 条数据行，建档 total_rows={entry['total_rows']}"
        f"（{entry['n_sheets']} 张 sheet）——检查是否被截断或只读了首表")
    assert np.all(np.diff(t) > 0), "附件1 时间列非严格递增"
    steps = np.unique(np.diff(t))
    assert steps.size == 1 and abs(steps[0] - 60.0) < 1e-9, f"附件1 时间步长非 60 s：{steps}"
    assert np.all(np.isfinite(T)) and np.all(np.isfinite(C)), "附件1 含非有限值"
    print(f"[data_ingest] 附件1 全量 {len(records)} 行 / {entry['n_sheets']} sheet 已核对通过；"
          f"t∈[{t.min():.0f},{t.max():.0f}] s, T_air∈[{T.min():.3f},{T.max():.3f}] ℃, "
          f"C_air∈[{C.min():.5f},{C.max():.5f}] kg/kg")
    return {"t": t, "T_air": T, "C_air": C, "sha256": sha256_of(AIR_FILE)}


PHI_TABLE_N = 131072
PHI_ERR_TOL = 1e-6


```

```python
def phi_exact(C: np.ndarray) -> np.ndarray:
    C = np.maximum(np.asarray(C, dtype=float), 1e-300)
    a = -D_COEF[1]
    return C * np.exp(-a / C) - a * exp1(a / C)


def build_kirchhoff_table(n_points: int = PHI_TABLE_N) -> tuple[np.ndarray, np.ndarray]:
    grid = np.concatenate(([0.0], np.logspace(-4.0, np.log10(3.0), n_points)))
    integrand = np.zeros_like(grid)
    positive = grid > 0
    integrand[positive] = np.exp(D_COEF[1] / grid[positive])
    phi = np.concatenate(([0.0], np.cumsum(np.diff(grid) * 0.5 * (integrand[1:] + integrand[:-1]))))
    assert np.all(np.diff(phi) >= 0.0), "Kirchhoff 势必须单调不减"
    return grid, phi


_PHI_GRID, _PHI_VALS = build_kirchhoff_table()
_PHI_PROBE = np.linspace(0.02, 3.0, 2000)
PHI_TABLE_ERR = float(np.max(np.abs(np.interp(_PHI_PROBE, _PHI_GRID, _PHI_VALS)
                                    - phi_exact(_PHI_PROBE)) / phi_exact(_PHI_PROBE)))
assert PHI_TABLE_ERR < PHI_ERR_TOL, (
    f"Kirchhoff 查表精度 {PHI_TABLE_ERR:.3e} 未达 §7.2 口径 {PHI_ERR_TOL}")


def phi_of(C: np.ndarray) -> np.ndarray:
    return np.interp(np.asarray(C, dtype=float), _PHI_GRID, _PHI_VALS)

```

```
def moisture_factor(C: np.ndarray) -> np.ndarray:
    return np.exp(D_COEF[1] / np.maximum(np.asarray(C, dtype=float), D_SAFE_FLOOR))


def temperature_factor(T_c: np.ndarray) -> np.ndarray:
    return np.exp(D_COEF[2] / (np.asarray(T_c, dtype=float) + KELVIN_OFFSET))


def diffusivity(C: np.ndarray, T_c: np.ndarray) -> np.ndarray:
    return D_COEF[0] * moisture_factor(C) * temperature_factor(T_c)


def properties(C: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    C = np.asarray(C, dtype=float)
    ratio = C / (C + 1.0)
    rho = RHO_COEF[0] + RHO_COEF[1] * C
    cp = CP_COEF[0] + CP_COEF[1] * ratio
    k = K_COEF[0] + K_COEF[1] * ratio
    return rho, cp, k


def face_diffusivity(C: np.ndarray, T_c: np.ndarray, mode: str = FACE_D) -> np.ndarray:
    T_face = 0.5 * (T_c[:-1] + T_c[1:])
    arr_t = temperature_factor(T_face)
    if mode == "kirchhoff":
        dC = C[1:] - C[:-1]
        small = np.abs(dC) < 1e-12
        C_bar = 0.5 * (C[:-1] + C[1:])
        slope = np.where(small, moisture_factor(C_bar),
```

```
                          (phi_of(C[1:]) - phi_of(C[:-1])) / np
    .where(small, 1.0, dC))
        return D_COEF[0] * arr_t * slope
    if mode == "midpoint":
        return D_COEF[0] * arr_t * moisture_factor(0.5 * (C
    [:-1] + C[1:]))
    node_d = diffusivity(C, T_c)
    if mode == "arith":
        return 0.5 * (node_d[:-1] + node_d[1:])
    if mode == "harmonic":
        lo, hi = node_d[:-1], node_d[1:]
        return np.where(lo + hi > 0.0, 2.0 * lo * hi / np.
    where(lo + hi > 0.0, lo + hi, 1.0), 0.0)
    raise ValueError(f"未知界面格式：{mode}")


def face_conductivity(k_node: np.ndarray, mode: str = FACE_K)
    -> np.ndarray:
    if mode == "arith":
        return 0.5 * (k_node[:-1] + k_node[1:])
    if mode == "harmonic":
        lo, hi = k_node[:-1], k_node[1:]
        return 2.0 * lo * hi / (lo + hi)
    raise ValueError(f"未知界面格式：{mode}")


def radial_grid(n_nodes: int, radius: float = R_FIX, stretch:
    float = 1.0) -> dict:
    assert n_nodes >= 3, "至少 3 个节点"
    assert stretch >= 1.0, "stretch<1 会把网格挤向中心，与退化
    层加密的目的相反"
    if stretch == 1.0:
        r = np.linspace(0.0, radius, n_nodes)
    else:
        widths = stretch ** np.arange(n_nodes - 1)
```

```
        r = np.concatenate(([0.0], np.cumsum(widths) / widths.sum() * radius))
    r[-1] = radius
    r_face = 0.5 * (r[:-1] + r[1:])
    edges = np.concatenate(([0.0], r_face, [radius]))
    volume = 0.5 * (edges[1:] ** 2 - edges[:-1] ** 2)
    assert abs(volume.sum() - 0.5 * radius ** 2) < 1e-15 * radius ** 2, "加权体积未闭合到 R^2/2"
    return {"dr": float(r[1] - r[0]), "r": r, "volume": volume, "r_face": r_face,
            "dr_face": np.diff(r), "radius": radius, "stretch": stretch}


def solve_tridiag(volume: np.ndarray, dt: float, time_coef: np.ndarray,
                  conductance: np.ndarray, robin: float, robin_source: float,
                  old: np.ndarray) -> tuple[np.ndarray, float]:
    n = volume.size
    ab = np.zeros((3, n))
    rhs = time_coef * old
    diag = time_coef.copy()
    diag[:-1] += conductance
    diag[1:] += conductance
    diag[-1] += robin
    rhs[-1] += robin_source
    ab[0, 1:] = -conductance
    ab[1, :] = diag
    ab[2, :-1] = -conductance
    off_sum = np.zeros(n)
    off_sum[:-1] += conductance
    off_sum[1:] += conductance
    margin = float(np.min(diag - off_sum))
```

```
    return solve_banded((1, 1), ab, rhs), margin


@dataclass
class SolverConfig:
    n_nodes: int = N_NODES
    dt: float = DT_SOLVER
    picard_iters: int = PICARD_ITERS
    face_d: str = FACE_D
    face_k: str = FACE_K
    air_extrap: str = AIR_EXTRAP
    radius: float = R_FIX
    stretch: float = 1.0
    t_init: float = T_INIT_C
    c_init: float = C_INIT
    s_d: float = 1.0
    s_hm: float = 1.0
    s_h: float = 1.0
    t_air_shift: float = 0.0
    const_d: float | None = None
    freeze_props: bool = False
    air_const: tuple | None = None
    c_crit: float = C_CRIT
    extra_crit: tuple = ()
    t_end: float | None = None
    t_max: float = T_MAX_S
    rec_step: float = Q3_REC_STEP_S
    fine_step: float | None = None
    fine_until: float = 0.0


def air_series(air: dict, cfg: SolverConfig):
    if cfg.air_const is not None:
        t_const, c_const = float(cfg.air_const[0]), float(cfg.
   air_const[1])
```

```
        return (lambda _t: t_const), (lambda _t: c_const)
    grid = air["t"]
    temp = air["T_air"] + cfg.t_air_shift
    conc = air["C_air"]
    if cfg.air_extrap == "hold_last":
        right_t, right_c = temp[-1], conc[-1]
        assert abs(right_t - cfg.t_air_shift - T_AIR_HOLD) < 5e-4, "hold-last 末值温度与口径表不符"
        assert abs(right_c - C_AIR_HOLD) < 5e-6, "hold-last 末值浓度与口径表不符"
    elif cfg.air_extrap == "mean_tail":
        tail = grid >= grid[-1] - 3600.0
        right_t, right_c = float(temp[tail].mean()), float(conc[tail].mean())
    else:
        raise ValueError(f"未知外推口径：{cfg.air_extrap}")
    return (lambda tt: float(np.interp(tt, grid, temp, left=temp[0], right=right_t)),
            lambda tt: float(np.interp(tt, grid, conc, left=conc[0], right=right_c)))


def _advance_in_time(cfg: SolverConfig, T: np.ndarray, C: np.ndarray, q_out, w_prev, w0,
                     cmax_prev, tail_target, diag: dict, t_hard_stop, dr, volume, r_face,
                     radius: float, dt: float, robin_m, robin_h, hm_eff, n: int, t_air_of,
                     c_air_of, props_frozen, rec_every: int, fine_every: int, rec: dict,
                     fine: dict, thresholds: dict):
    while True:
        if t_hard_stop is not None and diag["steps"] * dt >= t_hard_stop - 1e-9:
            break
```

```
        if tail_target is not None and diag["steps"] * dt >= tail_target - 1e-9:
            break
        step = diag["steps"] + 1
        t = step * dt
        t_air, c_air = t_air_of(t), c_air_of(t)
        T_old, C_old = T, C
        for _ in range(cfg.picard_iters):
            rho, cp, k_node = props_frozen if props_frozen is not None else properties(C)
            k_face = face_conductivity(k_node, cfg.face_k)
            T, margin_h = solve_tridiag(volume, dt, volume * rho * cp / dt,
                                        r_face * k_face / dr, robin_h, robin_h * t_air, T_old)
            if cfg.const_d is not None:
                d_face = np.full(n - 1, float(cfg.const_d))
            else:
                d_face = face_diffusivity(C, T, cfg.face_d)
            d_face = cfg.s_d * d_face
            C, margin_m = solve_tridiag(volume, dt, volume / dt,
                                        r_face * d_face / dr, robin_m, robin_m * c_air, C_old)
            diag["margin"] = min(diag["margin"], margin_h, margin_m)

        q_out += hm_eff * (C[-1] - c_air) * radius * dt
        w_now = float(C @ volume)
        diag["dw_max"] = max(diag["dw_max"], w_now - w_prev)
        w_prev = w_now
        diag["d_min"] = min(diag["d_min"], float(d_face.min()))
        diag["d_max"] = max(diag["d_max"], float(d_face.max()))
```

```
        diag["t_min"] = min(diag["t_min"], float(T.min()))
        diag["t_max"] = max(diag["t_max"], float(T.max()))
        diag["c_min"] = min(diag["c_min"], float(C.min()))
        diag["c_max"] = max(diag["c_max"], float(C.max()))
        diag["surf_margin"] = min(diag["surf_margin"], float(C[-1] - c_air))
        diag["steps"] = step

        cmax = float(C.max())
        for level in thresholds:
            if thresholds[level] is None and cmax < level <= cmax_prev:
                thresholds[level] = t - dt + (cmax_prev - level) / (cmax_prev - cmax) * dt
        cmax_prev = cmax
        if tail_target is None and thresholds[cfg.c_crit] is not None and cfg.t_end is None:
            tail_target = np.ceil(thresholds[cfg.c_crit] / cfg.rec_step) * cfg.rec_step

        if fine_every and step % fine_every == 0 and t <= cfg.fine_until + 1e-9:
            fine["t"].append(t); fine["T"].append(T.copy()); fine["C"].append(C.copy())
        if step % rec_every == 0:
            rec["t"].append(t); rec["T"].append(T.copy()); rec["C"].append(C.copy())
            rec["W"].append(w_now); rec["Q_out"].append(q_out)
            rec["eps"].append((w0 - w_now - q_out) / w0)
            rec["cmax"].append(cmax); rec["T_air"].append(t_air); rec["C_air"].append(c_air)
     return T, C, q_out, w_prev


def run_solver(air: dict, cfg: SolverConfig | None = None) ->
```

```
    dict:
694     cfg = cfg or SolverConfig()
695     grid = radial_grid(cfg.n_nodes, cfg.radius, cfg.stretch)
696     dr, volume, r_face = grid["dr_face"], grid["volume"], grid
    ["r_face"]
697     radius, dt = grid["radius"], cfg.dt
698     t_air_of, c_air_of = air_series(air, cfg)
699     hm_eff, h_eff = HM_MASS * cfg.s_hm, H_CONV * cfg.s_h
700     robin_m, robin_h = radius * hm_eff, radius * h_eff
701 
702     n = cfg.n_nodes
703     T = np.full(n, cfg.t_init, dtype=float)
704     C = np.full(n, cfg.c_init, dtype=float)
705     props_frozen = properties(np.full(n, cfg.c_init)) if cfg.
    freeze_props else None
706 
707     rec_every = int(round(cfg.rec_step / dt))
708     fine_every = int(round(cfg.fine_step / dt)) if cfg.
    fine_step else 0
709     assert abs(rec_every * dt - cfg.rec_step) < 1e-12, "记录粒
    度必须是 dt 的整数倍"
710 
711     rec = {"t": [0.0], "T": [T.copy()], "C": [C.copy()], "W":
    [], "Q_out": [],
712             "eps": [], "cmax": [float(C.max())], "T_air": [
    t_air_of(0.0)], "C_air": [c_air_of(0.0)]}
713     fine = {"t": [], "T": [], "C": []}
714     w0 = float(C @ volume)
715     rec["W"].append(w0); rec["Q_out"].append(0.0); rec["eps"].
    append(0.0)
716 
717     q_out = 0.0
718     cmax_prev = float(C.max())
719     thresholds = dict.fromkeys((cfg.c_crit, *cfg.extra_crit))
720     tail_target = None
```

```
 diag = {"margin": np.inf, "d_min": np.inf, "d_max": -np.inf, "surf_margin": np.inf,
         "dw_max": -np.inf, "t_min": np.inf, "t_max": -np.inf,
         "c_min": np.inf, "c_max": -np.inf, "steps": 0}
 w_prev = w0
 t_hard_stop = cfg.t_end if cfg.t_end is not None else cfg.t_max
 wall0 = time.perf_counter()

 T, C, q_out, w_prev = _advance_in_time(cfg, T, C, q_out, w_prev, w0, cmax_prev,
                                        tail_target, diag, t_hard_stop, dr, volume,
                                        r_face, radius, dt, robin_m, robin_h, hm_eff, n,
                                        t_air_of, c_air_of, props_frozen, rec_every,
                                        fine_every, rec, fine, thresholds)

 t_final = diag["steps"] * dt
 eps_mass = (w0 - w_prev - q_out) / w0
 t_star = thresholds[cfg.c_crit]
 return {"grid": grid, "cfg": cfg, "T": T, "C": C, "t_final": t_final,
         "coarse": {key: (np.array(val) if key != "t" else np.array(val)) for key, val in rec.items()},
         "fine": {key: np.array(val) for key, val in fine.items()},
         "W0": w0, "W_end": w_prev, "Q_out": q_out, "eps_mass": eps_mass,
         "t_star": t_star, "thresholds": dict(thresholds), "diag": diag,
         "dehydration": 1.0 - w_prev / w0, "wall_s": time.
```

```
    perf_counter() - wall0,
            "hit": t_star is not None}


def solve_case(key: str, cfg: SolverConfig, air: dict, *,
    force: bool = False) -> dict:
     print(f"[solve] {key}: N={cfg.n_nodes}, dt={cfg.dt},
    face_d={cfg.face_d} ...", flush=True)
     sol = run_solver(air, cfg)
     print(f"[solve] {key} done in {sol['wall_s']:.1f} s: t*={
    sol['t_star']}, "
           f"eps={sol['eps_mass']:.3e}, steps={sol['diag']['
    steps']:.0f}", flush=True)
     return sol


def robin_cylinder_series(r: np.ndarray, t: float, radius:
    float, alpha: float,
                          biot: float, n_terms: int = 60) ->
    np.ndarray:
     zeros = np.concatenate(([0.0], jn_zeros(0, n_terms + 1)))
     lam = np.array([brentq(lambda x: x * j1(x) - biot * j0(x),
                            zeros[i] + 1e-12, zeros[i + 1] - 1e
    -12, xtol=1e-14, rtol=1e-15)
                     for i in range(n_terms)])
     coef = 2.0 * j1(lam) / (lam * (j0(lam) ** 2 + j1(lam) **
    2))
     shape = j0(np.outer(np.asarray(r, dtype=float) / radius,
    lam))
     decay = np.exp(-lam ** 2 * alpha * t / radius ** 2)
     return shape @ (coef * decay)


def radii_cm() -> np.ndarray:
     return np.round(np.arange(OUT_N_COLS) * OUT_DR_CM, 1)
```

```
def table_node_index() -> list[int]:
    step = int(round(0.5 / OUT_DR_CM))
    return list(range(0, OUT_N_COLS, step))


def profile_at(sol: dict, t: float, field: str = "C") -> np.ndarray:
    grid_t = sol["coarse"]["t"]
    data = sol["coarse"][field]
    idx = int(np.searchsorted(grid_t, t))
    if idx < grid_t.size and abs(grid_t[idx] - t) < 1e-9:
        return data[idx]
    assert 0 < idx < grid_t.size, f"t={t} 超出记录区间 [{grid_t[0]}, {grid_t[-1]}]"
    weight = (t - grid_t[idx - 1]) / (grid_t[idx] - grid_t[idx - 1])
    return (1.0 - weight) * data[idx - 1] + weight * data[idx]


def write_grid_xlsx(path: Path, sheets: dict) -> dict:
    book = openpyxl.Workbook(write_only=True)
    header = ["时间\\到药材中心的距离", *[float(x) for x in radii_cm()]]
    shapes = {}
    for name, (times, field) in sheets.items():
        sheet = book.create_sheet(title=name)
        sheet.append(header)
        for row_t, row in zip(times, field):
            sheet.append([float(row_t), *[round(float(v), ROUND_DIGITS) for v in row]])
        shapes[name] = (len(times), field.shape[1])
    path.parent.mkdir(parents=True, exist_ok=True)
    book.save(path)
```

```
    book.close()
    return {"path": str(path.relative_to(ROOT).as_posix()), "sheets": shapes,
            "sha256": sha256_of(path), "bytes": path.stat().st_size}


def write_csv(path: Path, header: list[str], rows) -> dict:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [",".join(header)]
    lines += [",".join(f"{float(v):.10g}" for v in row) for row in rows]
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")
    return {"path": str(path.relative_to(ROOT).as_posix()), "rows": len(lines) - 1,
            "sha256": sha256_of(path)}


def dump_json(path: Path, payload: dict) -> dict:
    path.parent.mkdir(parents=True, exist_ok=True)
    text = json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False, sort_keys=False)
    path.write_text(text, encoding="utf-8")
    return {"path": str(path.relative_to(ROOT).as_posix()), "bytes": len(text.encode("utf-8"))}


def jsonable(value):
    if isinstance(value, (np.floating, np.integer)):
        return value.item()
    if isinstance(value, np.ndarray):
        return [jsonable(v) for v in value.tolist()]
    if isinstance(value, dict):
        return {str(k): jsonable(v) for k, v in value.items()}
    if isinstance(value, (list, tuple)):
```

```
        return [jsonable(v) for v in value]
    return value
```

q3.py

```python
from __future__ import annotations

import json
import re
from pathlib import Path

_PROJ_ROOT = Path(__file__).resolve().parents[1]
_FACTS_PATH = _PROJ_ROOT / "数据台账" / "DATA_FACTS.json"
_REPORT_PATH = _PROJ_ROOT / "数据台账" / "MODELING_REPORT.md"
_LEDGER_PATH = _PROJ_ROOT / "数据台账" / "CROSS_PROBLEM_LEDGER.json"

_FACTS = json.loads(_FACTS_PATH.read_text(encoding="utf-8"))
_GIVEN = {item["name"]: item for item in _FACTS["given"]}

_NUM = re.compile(r"[-+]?\d+(?:\.\d+)?(?:[eE][-+]?\d+)?")


def const(name: str) -> float:
    item = _GIVEN[name]
    assert item["kind"] == "constant", f"{name} 不是 constant 条目"
    return float(item["value"])


def formula_coeffs(name: str, count: int) -> list[float]:
    item = _GIVEN[name]
    assert item["kind"] == "formula", f"{name} 不是 formula 条目"
    rhs = item["form"].split("=", 1)[1]
    found = [float(x) for x in _NUM.findall(rhs)]
```

```
    assert len(found) >= count, f"{name} 解析到的系数不足 {count} 个：{found}"
    return found[:count]


LENGTH_M = const("药材几何-长")
R_FIX = const("药材几何-半径")
T_INIT_C = const("初始温度")
C_INIT = const("初始水分浓度")
H_CONV = const("对流换热系数 h")
HM_MASS = const("对流传质系数 h_m")
C_CRIT = const("问题三烘干终点阈值")

_RHO_A, _RHO_B = formula_coeffs("密度经验公式(Q2/Q3)", 2)
_CP_A, _CP_B = formula_coeffs("比热容经验公式(Q2/Q3)", 2)
_K_A, _K_B = formula_coeffs("热传导系数经验公式(Q2/Q3)", 2)
_D_PRE, _D_EC, _D_ET = formula_coeffs("水分扩散系数经验公式(Q2/Q3)", 3)

RHO_COEF = (_RHO_A, _RHO_B)
CP_COEF = (_CP_A, _CP_B)
K_COEF = (_K_A, _K_B)
D_COEF = (_D_PRE, _D_EC, _D_ET)
RHO_DRY_REF = _RHO_A

_REPORT = _REPORT_PATH.read_text(encoding="utf-8")
_ROW = re.compile(r"^\|[^|]+\|\s*(" + _NUM.pattern + r")\s*\|[^|]*\|\s*'([A-Za-z_]+)'\s*\|", re.M)
_CAL = {label: float(value) for value, label in _ROW.findall(_REPORT)}
assert _CAL, "未能从 MODELING_REPORT.md §7.3 解析参数口径表"

N_NODES = int(_CAL["N_nodes"])
DT_SOLVER = _CAL["dt_solver"]
T_AIR_HOLD = _CAL["T_air_hold"]
```

```
C_AIR_HOLD = _CAL["C_air_hold"]

PICARD_ITERS = int(re.search(r"PICARD_ITERS\s*[=:]\s*(\d+)", _REPORT).group(1))
FACE_D = "kirchhoff"
FACE_K = "arith"
AIR_EXTRAP = "hold_last"
LATENT_HEAT_COUPLING = False
SHRINKAGE = False

_UNIT_TEMP = json.loads(_LEDGER_PATH.read_text(encoding="utf-8"))["unit_ledger"]["temperature_in_properties"]
KELVIN_OFFSET = float(re.search(r"\+\s*(" + _NUM.pattern + r")", _UNIT_TEMP).group(1))

OUT_DR_CM = 0.1
OUT_R_MAX_CM = R_FIX * 100.0
OUT_N_COLS = int(round(OUT_R_MAX_CM / OUT_DR_CM)) + 1
Q2_SPAN_S = 3 * 3600
Q2_TABLE_STEP_S = 1800
Q3_TABLE_STEP_S = 6 * 3600
Q3_REC_STEP_S = 60
ROUND_DIGITS = 4
T_MAX_S = 150 * 3600
D_SAFE_FLOOR = 1e-9

assert abs(_CAL["radius"] - R_FIX) < 1e-12, "半径口径冲突：口径表 vs DATA_FACTS"
assert abs(_CAL["length"] - LENGTH_M) < 1e-12, "长度口径冲突"
assert abs(_CAL["T_init"] - T_INIT_C) < 1e-12, "初温口径冲突"
assert abs(_CAL["C_init"] - C_INIT) < 1e-12, "初始水分口径冲突"
assert abs(_CAL["h_conv"] - H_CONV) < 1e-12, "对流换热系数口径冲突"
assert abs(_CAL["hm_mass"] - HM_MASS) / HM_MASS < 1e-9, "对流
```

```
    传质系数口径冲突"
assert abs(_CAL["C_crit"] - C_CRIT) < 1e-12, "终点阈值口径冲突
    "
assert abs(_CAL["rho_dry_ref"] - RHO_DRY_REF) < 1e-12, "干固体
    参考密度口径冲突"

_DR_MM = R_FIX / (N_NODES - 1) * 1000.0
assert abs(_DR_MM - OUT_DR_CM * 10.0) < 1e-12, (
    f"网格与输出栅格不一致：dr={_DR_MM} mm 与输出步长 {
    OUT_DR_CM} cm 不重合，"
    "N=21 的目的就是让 r=0/0.5/1/1.5/2 cm 直接落节点（§13 S11
    ）")
assert N_NODES == OUT_N_COLS, "主算节点数应与输出列数一致（各
    0.1 cm 一列）"
assert 200 < KELVIN_OFFSET < 300, "绝对温度换算量异常"
assert LATENT_HEAT_COUPLING is False and SHRINKAGE is False, "
    假设 3/6 开关被改动"

if __name__ == "__main__":
    print(f"[params] R={R_FIX} m, T0={T_INIT_C} C, C0={C_INIT}
    kg/kg, h={H_CONV}, hm={HM_MASS}")
    print(f"[params] rho={RHO_COEF}, cp={CP_COEF}, k={K_COEF},
    D={D_COEF}, K_offset={KELVIN_OFFSET}")
    print(f"[params] N={N_NODES}, dt={DT_SOLVER}, Picard={
    PICARD_ITERS}, C_crit={C_CRIT}")
    print(f"[params] hold-last: T_air={T_AIR_HOLD} C, C_air={
    C_AIR_HOLD} kg/kg")
    print("[params] 口径一致性 OK")
from __future__ import annotations

import os
import sys
from pathlib import Path

import numpy as np
```

```
113
114 _CODE_DIR = Path(os.path.realpath(__file__)).parent
115 if str(_CODE_DIR) not in sys.path:
116     sys.path.insert(0, str(_CODE_DIR))
117
118 from params import (C_CRIT, C_INIT, H_CONV, HM_MASS, N_NODES,
    DT_SOLVER, PICARD_ITERS, FACE_D,
119                     OUT_N_COLS, Q2_SPAN_S, Q2_TABLE_STEP_S,
    Q3_REC_STEP_S, R_FIX, ROUND_DIGITS,
120                     T_INIT_C)
121 from utils import (LIB_VERSIONS, OUTPUT_DIR, FIGURES_DIR, ROOT
    , SolverConfig, diffusivity,
122                    dump_json, jsonable, load_air_boundary,
    profile_at, properties, radii_cm,
123                    solve_case, table_node_index, write_csv,
    write_grid_xlsx)
124
125 BASELINE_KEY = "baseline_N21_dt1"
126
127
128 def baseline_config() -> SolverConfig:
129     return SolverConfig(fine_step=DT_SOLVER, fine_until=float(
    Q2_SPAN_S),
130                         rec_step=float(Q3_REC_STEP_S),
    extra_crit=(0.12, 0.18, 0.20))
131
132
133 def solve_baseline(air: dict) -> dict:
134     return solve_case(BASELINE_KEY, baseline_config(), air)
135
136
137 def series_d(T: np.ndarray, C: np.ndarray) -> np.ndarray:
138     return diffusivity(C.ravel(), T.ravel())
139
140
```

```python
def window_checks(name: str, t: np.ndarray, T: np.ndarray, C: np.ndarray,
                  volume: np.ndarray, air: dict) -> dict:
    out = {"window": name, "n_rows": int(t.size), "t_span_s": [float(t[0]), float(t[-1])]}
    c_air = np.interp(t, air["t"], air["C_air"], left=air["C_air"][0], right=air["C_air"][-1])

    out["C1"] = {"C_min": float(C.min()), "C_max": float(C.max())}
    assert C.min() >= -1e-9 and C.max() <= C_INIT + 1e-9, f"C1 违反@{name}: {out['C1']}"

    cap = float(air["T_air"].max()) + 0.5
    out["C2"] = {"T_min": float(T.min()), "T_max": float(T.max()), "cap": cap}
    assert T.min() >= T_INIT_C - 1e-6 and T.max() <= cap, f"C2 违反@{name}: {out['C2']}"

    for field_name, field in (("C", C), ("T", T)):
        first = float(np.abs(field[:, 1] - field[:, 0]).max())
        second = float(np.abs(field[:, 2] - field[:, 1]).max())
        out[f"C3_{field_name}"] = {"d01": first, "d12": second}
        assert first <= second + 1e-12, f"C3 违反@{name}（{field_name}）: {out[f'C3_{field_name}']}"

    w_hist = C @ volume
    out["C5"] = {"max_dW": float(np.diff(w_hist).max())}
    assert np.diff(w_hist).max() <= 1e-12, f"C5 违反@{name}: {out['C5']}"

    margin = float((C[:, -1] - c_air).min())
    out["C6"] = {"min_surface_margin": margin}
```

```
    assert margin >= 0.0, f"C6 违反@{name}：表面浓度低于空气侧 {margin:.3e}"

    out["C9"] = {"max_dC_dr": float(np.diff(C, axis=1).max())}
    assert np.diff(C, axis=1).max() <= 1e-12, f"C9 违反@{name}: {out['C9']}"
    return out


def validate_constraints(sol: dict, air: dict, tables: dict) -> dict:
    fine = sol["fine"]
    coarse = sol["coarse"]
    volume = sol["grid"]["volume"]
    report = {"windows": [
        window_checks("result2_1s_0-3h", fine["t"], fine["T"], fine["C"], volume, air),
        window_checks("full_60s_0-tstar", coarse["t"], coarse["T"], coarse["C"], volume, air)]}
    for key in ("C1", "C2", "C5", "C6", "C9"):
        report[key] = {win["window"]: win[key] for win in report["windows"]}
    report["C3"] = {win["window"]: {"C": win["C3_C"], "T": win["C3_T"]} for win in report["windows"]}

    d_full = series_d(coarse["T"], coarse["C"])
    d_fine = series_d(fine["T"], fine["C"])
    cv = float(d_full.std() / d_full.mean())
    source = (_CODE_DIR / "utils.py").read_text(encoding="utf-8")
    report["C4"] = {"cv_D_full_run": cv, "cv_D_first_3h": float(d_fine.std() / d_fine.mean()),
                    "D_min": float(d_full.min()), "D_max": float(d_full.max()),
                    "D_dynamic_range": float(d_full.max() /
```

```
    d_full.min()),
190                 "has_moisture_factor": "moisture_factor"
    in source,
191                 "has_absolute_temperature": "KELVIN_OFFSET
    " in source,
192                 "props_recomputed_per_picard_sweep":
    PICARD_ITERS}
193     assert cv > 0.5, f"C4 违反：全程 std/mean(D)={cv:.4f} <=
    0.5，物性疑似被常量化"
194     assert d_full.max() / d_full.min() > 10.0, "C4 违反：D 动
    态范围不足，疑似取了常量"
195
196     report["C8"] = {"eps_mass": float(sol["eps_mass"]), "W0":
    float(sol["W0"]),
197                     "W_end": float(sol["W_end"]), "Q_out":
    float(sol["Q_out"])}
198     assert abs(sol["eps_mass"]) < 1e-10, f"C8 违反：{report['
    C8']}"
199
200     cols = radii_cm()
201     report["C10"] = {"n_cols": int(cols.size), "first": float(
    cols[0]), "last": float(cols[-1]),
202                      "sheets": sorted(tables["result2"]["
    sheets"]),
203                      "digits": ROUND_DIGITS}
204     assert cols.size == OUT_N_COLS and abs(cols[-1] - 2.0) < 1
    e-12, f"C10 违反：{report['C10']}"
205     assert sorted(tables["result2"]["sheets"]) == ["水分浓度",
     "温度"], "C10 违反：result2 工作表名不符"
206     assert tables["result2"]["sheets"]["温度"] == (Q2_SPAN_S,
    OUT_N_COLS), (
207         f"C10 违反：result2 形状 {tables['result2']['sheets
    ']['温度']} 应为 ({Q2_SPAN_S}, {OUT_N_COLS})")
208     return report
209
```

```
210
211 def validate_capability(sol: dict, tables: dict) -> dict:
212     source = (_CODE_DIR / "utils.py").read_text(encoding="utf
    -8")
213     verdict = {}
214
215     probe_c, probe_t = np.array([2.55, 0.15]), np.array([28.0,
     50.0])
216     rho, cp, k = properties(probe_c)
217     d_two = diffusivity(probe_c, probe_t)
218     d_no_moisture = np.exp(-3850.0 / (probe_t + 273.15)) * 2.4
    e-3
219     assert rho[0] != rho[1] and cp[0] != cp[1] and k[0] != k
    [1], "P2-C1 不成立：物性对 C 无响应"
220     assert not np.allclose(d_two, d_no_moisture), "P2-C1 不成
    立：D 缺 exp(-0.45/C) 因子"
221     assert "temperature_factor" in source and "KELVIN_OFFSET"
    in source, "P2-C1 不成立：D 缺绝对温度因子"
222     assert "properties(C)" in source, "P2-C1 不成立：物性未在
    时间循环内按当前 C 重算"
223     ratio = float(diffusivity(np.array([2.55]), np.array
    ([50.0]))[0]
224                   / diffusivity(np.array([0.15]), np.array
    ([50.0]))[0])
225     verdict["P2-C1"] = {"holds": True, "D_ratio_wet_over_dry":
     ratio,
226                         "rho": jsonable(rho), "cp": jsonable(
    cp), "k": jsonable(k)}
227
228     for name in ("table3", "table4"):
229         info = tables[name]
230         assert info["rows"] == Q2_SPAN_S // Q2_TABLE_STEP_S, f
    "P2-C2 不成立：{name} 行数 {info['rows']}"
231     verdict["P2-C2"] = {"holds": True, "rows": tables["table3"
    ]["rows"],
```

```
                        "radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
    "digits": ROUND_DIGITS,
                        "csv": tables["table3"]["path"]}

    shapes = tables["result2"]["sheets"]
    assert set(shapes) == {"温度", "水分浓度"}, "P2-C3 不成立：工作表名不符"
    for name, shape in shapes.items():
        assert shape == (Q2_SPAN_S, OUT_N_COLS), f"P2-C3 不成立：{name} 形状 {shape}"
    verdict["P2-C3"] = {"holds": True, "sheets": jsonable(shapes),
                        "bytes": tables["result2"]["bytes"], "sha256": tables["result2"]["sha256"]}
    return verdict


def _solve_and_tables(air: dict) -> tuple:
    sol = solve_baseline(air)
    idx = table_node_index()
    cols_cm = radii_cm()
    times = np.arange(Q2_TABLE_STEP_S, Q2_SPAN_S + 1, Q2_TABLE_STEP_S, dtype=float)

    table3 = np.array([[profile_at(sol, t, "T")[j] for j in idx] for t in times])
    table4 = np.array([[profile_at(sol, t, "C")[j] for j in idx] for t in times])
    header = ["时间/s", "时间/h", *[f"r={cols_cm[j]:.1f}cm" for j in idx]]
    info = {}
    info["table3"] = write_csv(OUTPUT_DIR / "table3_temperature.csv", header,
                               [[t, t / 3600.0, *np.round(row, ROUND_DIGITS)]
```

```
                                  for t, row in zip(times,
    table3)])
     info["table4"] = write_csv(OUTPUT_DIR / "table4_moisture.
    csv", header,
                                [[t, t / 3600.0, *np.round(row,
     ROUND_DIGITS)]
                                  for t, row in zip(times,
    table4)])
     return sol, cols_cm, times, table3, table4, info


def _write_result_tables(sol: dict, info: dict) -> dict:
     fine = sol["fine"]
     assert fine["t"].size == Q2_SPAN_S and abs(fine["t"][0] -
    1.0) < 1e-12, "细记录应为 t=1..10800 s"
     info["result2"] = write_grid_xlsx(OUTPUT_DIR / "result2.
    xlsx",
                                       {"温度": (fine["t"],
    fine["T"]),
                                        "水分浓度": (fine["t"],
     fine["C"])})
     coarse = sol["coarse"]
     info["result2_full_60s"] = write_grid_xlsx(OUTPUT_DIR / "
    result2_full_60s.xlsx",
                                                {"温度": (
    coarse["t"], coarse["T"]),
                                                 "水分浓度": (
    coarse["t"], coarse["C"])})
     info["W_history"] = write_csv(OUTPUT_DIR / "W_history.csv"
    ,
                                   ["t_s", "W", "Q_out", "
    eps_mass", "Cmax", "T_air", "C_air"],
                                   np.column_stack([coarse["t"
    ], coarse["W"], coarse["Q_out"],
                                                    coarse["eps
```

```
    "], coarse["cmax"],
277                                                    coarse["
    T_air"], coarse["C_air"]]))
278     info["center_surface"] = write_csv(
279         OUTPUT_DIR / "center_surface.csv",
280         ["t_s", "T_center", "T_surface", "C_center", "
    C_surface", "T_air", "C_air"],
281         np.column_stack([coarse["t"], coarse["T"][:, 0],
    coarse["T"][:, -1],
282                          coarse["C"][:, 0], coarse["C"][:,
    -1], coarse["T_air"], coarse["C_air"]]))
283     return coarse
284
285
286 def _independent_recompute(sol: dict, air: dict) -> tuple:
287     check = solve_case("q2_independent_3h", SolverConfig(t_end
    =float(Q2_SPAN_S),
288
    rec_step=float(Q3_REC_STEP_S)), air)
289     drift_t = float(np.abs(check["T"] - profile_at(sol, float(
    Q2_SPAN_S), "T")).max())
290     drift_c = float(np.abs(check["C"] - profile_at(sol, float(
    Q2_SPAN_S), "C")).max())
291     assert drift_t < 1e-12 and drift_c < 1e-12, f"独立复算漂
    移: dT={drift_t:.2e}, dC={drift_c:.2e}"
292
293     at3h_T, at3h_C = profile_at(sol, 10800.0, "T"), profile_at
    (sol, 10800.0, "C")
294     return drift_t, drift_c, at3h_T, at3h_C
295
296
297 def _write_results_json(air: dict, sol: dict, info: dict,
    cols_cm: np.ndarray, times: np.ndarray,
298                         table3: np.ndarray, table4: np.ndarray
    , constraints: dict,
```

```
                      capability: dict, drift_t: float,
drift_c: float, at3h_T: np.ndarray,
                      at3h_C: np.ndarray, coarse: dict) ->
None:
 payload = {
     "problem": 2, "title": "整个烘干过程的温度场与水分浓度
场",
     "method": {"scheme": "conservative radial FVM +
backward Euler + Picard",
                "face_diffusivity": FACE_D, "N_nodes":
N_NODES, "dt_s": DT_SOLVER,
                "picard_iters": PICARD_ITERS, "R_m": R_FIX,
 "h": H_CONV, "hm": HM_MASS,
                "libs": LIB_VERSIONS},
     "grid": {"r_cm": jsonable(cols_cm), "volume_weighted":
 jsonable(sol["grid"]["volume"])},
     "table3_temperature": {"times_s": jsonable(times), "
radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
                            "values": jsonable(np.round(
table3, ROUND_DIGITS))},
     "table4_moisture": {"times_s": jsonable(times), "
radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
                         "values": jsonable(np.round(table4
, ROUND_DIGITS))},
     "profiles_3h": {"T": jsonable(np.round(at3h_T,
ROUND_DIGITS)),
                     "C": jsonable(np.round(at3h_C,
ROUND_DIGITS))},
     "center_surface_60s": {"t_s": jsonable(coarse["t"][
coarse["t"] <= Q2_SPAN_S]),
                            "T_center": jsonable(coarse["T"
][coarse["t"] <= Q2_SPAN_S, 0]),
                            "T_surface": jsonable(coarse["T
"][coarse["t"] <= Q2_SPAN_S, -1]),
                            "C_center": jsonable(coarse["C"
```

```
    ][coarse["t"] <= Q2_SPAN_S, 0]),
318                                 "C_surface": jsonable(coarse["C
    "][coarse["t"] <= Q2_SPAN_S, -1])},
319         "boundary_data": {"t_s": jsonable(air["t"]), "T_air":
    jsonable(air["T_air"]),
320                           "C_air": jsonable(air["C_air"]), "
    sha256": air["sha256"]},
321         "conservation": {"W0": sol["W0"], "W_end_full_run":
    sol["W_end"],
322                          "Q_out_full_run": sol["Q_out"], "
    eps_mass": sol["eps_mass"],
323                          "dehydration_full_run": sol["
    dehydration"]},
324         "diagnostics": jsonable(sol["diag"]),
325         "constraints": jsonable(constraints), "capability":
    jsonable(capability),
326         "independent_recompute_drift": {"dT_max": drift_t, "
    dC_max": drift_c},
327         "artifacts": jsonable(info),
328      }
329     dump_json(FIGURES_DIR / "problem_2_results.json", payload)
330
331
332 def _write_check_report(sol: dict, constraints: dict,
    capability: dict, drift_t: float,
333                         drift_c: float, at3h_T: np.ndarray,
    at3h_C: np.ndarray) -> None:
334     lines = ["# 问题二 自检报告", "",
335              f"- 求解口径：N={N_NODES}, dt={DT_SOLVER} s,
    Picard={PICARD_ITERS}, face_D={FACE_D}",
336              f"- 3 h 中心：T={at3h_T[0]:.4f} ℃（检查点□区间
    [49,50]）, "
337              f"C={at3h_C[0]:.4f} kg/kg（区间 [1.6,1.9]）",
338              f"- 3 h 表面：T={at3h_T[-1]:.4f} ℃, C={at3h_C
    [-1]:.4f} kg/kg",
```

```
            f"- 守恒残差 |eps_mass|={abs(sol['eps_mass']):.3e} < 1e-10（C8）",
            *[f"- 窗口 {win['window']}（{win['n_rows']} 行）: "
              f"C∈[{win['C1']['C_min']:.6f},{win['C1']['C_max']:.6f}]（C1）; "
              f"T∈[{win['C2']['T_min']:.4f},{win['C2']['T_max']:.4f}]≤{win['C2']['cap']:.4f}（C2）; "
              f"max dW={win['C5']['max_dW']:.3e}（C5）; "
              f"min(C_s-C_air)={win['C6']['min_surface_margin']:.3e}（C6）; "
              f"max dC/dr={win['C9']['max_dC_dr']:.3e}（C9）"
              for win in constraints["windows"]],
            f"- std(D)/mean(D)={constraints['C4']['cv_D_full_run']:.4f} > 0.5（C4 动态派生，全程窗口; "
            f"仅 0–3 h 为 {constraints['C4']['cv_D_first_3h']:.4f}，因退化层在 3 h 后才出现），"
            f"D 动态范围 {constraints['C4']['D_dynamic_range']:.1f} 倍",
            f"- 独立复算漂移 dT={drift_t:.2e}, dC={drift_c:.2e}（< 1e-12）",
            f"- 能力项: {', '.join(f'{k}=成立' for k in capability)}",
            "", "结论: C1–C6、C8–C10 与 P2-C1/C2/C3 全部通过。"]
    (ROOT / "_tmp").mkdir(parents=True, exist_ok=True)
    (ROOT / "_tmp" / "problem_2_check.md").write_text("\n".join(lines) + "\n", encoding="utf-8")


def main() -> int:
    air = load_air_boundary()
    sol, cols_cm, times, table3, table4, info = _solve_and_tables(air)

```

```
    coarse = _write_result_tables(sol, info)

    constraints = validate_constraints(sol, air, info)
    capability = validate_capability(sol, info)

    drift_t, drift_c, at3h_T, at3h_C = _independent_recompute(sol, air)

    _write_results_json(air, sol, info, cols_cm, times, table3, table4, constraints,
                        capability, drift_t, drift_c, at3h_T, at3h_C, coarse)

    _write_check_report(sol, constraints, capability, drift_t, drift_c, at3h_T, at3h_C)

    print(f"[problem2] 3h 中心 T={at3h_T[0]:.4f}℃ / C={at3h_C[0]:.4f} kg/kg；"
          f"表面 T={at3h_T[-1]:.4f} / C={at3h_C[-1]:.4f}")
    print(f"[problem2] |eps_mass|={abs(sol['eps_mass']):.3e}, std(D)/mean(D)="
          f"{constraints['C4']['cv_D_full_run']:.4f}（全程），独立复算漂移 {max(drift_t, drift_c):.2e}")
    print(f"[problem2] 落盘 {len(info)} 件：{', '.join(v['path'] for v in info.values())}")
    print("[problem2] SELF-CHECK PASS（C1–C6, C8–C10, P2-C1/C2/C3）")
    return 0


if __name__ == "__main__":
    sys.exit(main())
from __future__ import annotations

import sys
```

```
from pathlib import Path

import numpy as np

_APP_DIR = str(Path(__file__).resolve().parents[0])
if _APP_DIR not in sys.path:
    sys.path.insert(0, _APP_DIR)

from params import (C_CRIT, C_INIT, DT_SOLVER, FACE_D, N_NODES, OUT_N_COLS, PICARD_ITERS,
                    Q2_SPAN_S, Q3_REC_STEP_S, Q3_TABLE_STEP_S, ROUND_DIGITS, T_MAX_S)
from problem2 import BASELINE_KEY, baseline_config, solve_baseline
from utils import (FIGURES_DIR, LIB_VERSIONS, OUTPUT_DIR, ROOT, SolverConfig, dump_json, jsonable,
                    load_air_boundary, profile_at, radii_cm, solve_case, table_node_index,
                    write_csv, write_grid_xlsx)

PROTOTYPE_T_STAR_S = 205957.6284
UNREACHABLE = "在 t_max 内未达标"


def endpoint_from_records(coarse: dict, c_crit: float, dt: float) -> dict:
    cmax = coarse["C"].max(axis=1)
    hits = np.flatnonzero(cmax < c_crit)
    if hits.size == 0:
        return {"hit": False, "reason": UNREACHABLE}
    first = int(hits[0])
    gap = cmax - coarse["C"][:, 0]
    return {"hit": True, "first_record_idx": first,
            "t_first_record_s": float(coarse["t"][first]),
            "cmax_at_first_record": float(cmax[first]),
```

```
            "t_prev_record_s": float(coarse["t"][first - 1]),
            "cmax_at_prev_record": float(cmax[first - 1]),
            "record_step_s": float(coarse["t"][first] - coarse["t"][first - 1]),
            "argmax_is_center": bool(gap.max() <= 1e-12),
            "max_gap_center_vs_max": float(gap.max()),
            "n_rows_strict_argmax_off_center": int(np.count_nonzero(coarse["C"].argmax(axis=1) != 0)),
            "n_rows": int(coarse["t"].size)}


def validate_constraints(sol: dict, air: dict, tables: dict, recheck: dict) -> dict:
    coarse = sol["coarse"]
    t_star = sol["t_star"]
    report = {}

    assert t_star is not None, "C7 违反：基准算例未定位到首达时刻"
    lo, hi = recheck["t_prev_record_s"], recheck["t_first_record_s"]
    on_grid = abs(t_star / Q3_REC_STEP_S - round(t_star / Q3_REC_STEP_S)) < 1e-9
    report["C7"] = {"t_star_s": float(t_star), "t_star_h": float(t_star) / 3600.0,
                    "bracket_s": [lo, hi], "cmax_before": recheck["cmax_at_prev_record"],
                    "cmax_after": recheck["cmax_at_first_record"], "c_crit": C_CRIT,
                    "t_star_on_60s_grid": on_grid, "dt_used_for_interp_s": DT_SOLVER,
                    "argmax_r_is_center": recheck["argmax_is_center"],
                    "max_gap_center_vs_max": recheck["max_gap_center_vs_max"],
```

```
                    "n_rows_strict_argmax_off_center": recheck
["n_rows_strict_argmax_off_center"]}
    assert lo <= t_star <= hi, f"C7 违反：t*={t_star} 不在记录
括区 [{lo},{hi}] 内"
    assert recheck["cmax_at_prev_record"] >= C_CRIT > recheck[
"cmax_at_first_record"], (
        f"C7 违反：括区两端未夹住阈值 {report['C7']}")
    assert not on_grid, "C7/⑩ 违反：t* 恰落在 60 s 记录栅格
上，判据疑似写在记录分支内（预案 A7）"
    assert recheck["argmax_is_center"], (
        f"C7 违反：存在真实的内部极大值，中心值比全场最大值低
"
        f"{recheck['max_gap_center_vs_max']:.3e}（>1e-12 浮点
容差）")

    report["C9"] = {"max_dC_dr": float(np.diff(coarse["C"],
axis=1).max())}
    assert np.diff(coarse["C"], axis=1).max() <= 1e-12, f"C9
违反：{report['C9']}"

    cols = radii_cm()
    shapes = tables["result3"]["sheets"]
    expected_rows = int(np.ceil(t_star / Q3_REC_STEP_S)) + 1
    report["C10"] = {"n_cols": int(cols.size), "sheets":
sorted(shapes),
                     "rows": shapes["Sheet1"][0], "
expected_rows": expected_rows,
                     "digits": ROUND_DIGITS}
    assert sorted(shapes) == ["Sheet1"], f"C10 违反：result3
工作表名 {sorted(shapes)} 应为 ['Sheet1']"
    assert shapes["Sheet1"] == (expected_rows, OUT_N_COLS), f"
C10 违反：{report['C10']}"
    return report


```

```
def validate_capability(sol: dict, air: dict, tables: dict, recheck: dict) -> dict:
    coarse = sol["coarse"]
    volume = sol["grid"]["volume"]
    t_star = float(sol["t_star"])
    source = (Path(_APP_DIR) / "utils.py").read_text(encoding="utf-8")
    assert "C.max()" in source, "P3-C1 不成立：源码未按 max_r C 取判据"

    def first_cross(series: np.ndarray) -> float | None:
        hits = np.flatnonzero(series < C_CRIT)
        if hits.size == 0:
            return None
        i = int(hits[0])
        if i == 0:
            return 0.0
        span = coarse["t"][i] - coarse["t"][i - 1]
        return float(coarse["t"][i - 1] + (series[i - 1] - C_CRIT)
                     / (series[i - 1] - series[i]) * span)

    surface = first_cross(coarse["C"][:, -1])
    mean_w = first_cross((coarse["C"] @ volume) / volume.sum())
    assert surface is not None and surface < 0.5 * t_star, (
        "P3-C1 不成立：表面值口径未显著早于 max_r 口径，判据无从证伪")
    assert mean_w is not None and mean_w < t_star, "P3-C1 不成立：体均口径未早于 max_r 口径"
    verdict = {"P3-C1": {"holds": True, "t_star_h": t_star / 3600.0,
                         "wrong_surface_criterion_h": surface / 3600.0,
                         "wrong_volume_mean_criterion_h":
```

```
    mean_w / 3600.0,
                            "surface_underestimates_by": 1.0 -
    surface / t_star,
                            "criterion": "t* = min{t : max_r C(r,
    t) < 0.15}",
                            "evaluated_every_step": True, "dt_s":
     DT_SOLVER}}

     shapes = tables["result3"]["sheets"]
     assert tables["table5"]["rows"] == int(t_star //
    Q3_TABLE_STEP_S) + 1, (
          f"P3-C2 不成立：表5 行数 {tables['table5']['rows']} 与
     6 h 栅格+t* 行不符")
     verdict["P3-C2"] = {"holds": True, "table5_rows": tables["
    table5"]["rows"],
                         "table5_path": tables["table5"]["path"
    ],
                         "result3_sheets": jsonable(shapes),
                         "result3_bytes": tables["result3"]["
    bytes"],
                         "result3_sha256": tables["result3"]["
    sha256"]}
     return verdict


def boundary_cases(air: dict, t_star_baseline: float) -> dict:
     b6 = solve_case("B6_crit_at_initial", SolverConfig(c_crit=
    C_INIT, dt=1.0, rec_step=1.0,
                                                        t_max
    =600.0), air)
     assert b6["t_star"] is not None, "B6 不成立：阈值取初值时
    应在过程最开端即穿越"
     ratio = float(b6["t_star"]) / t_star_baseline
     assert ratio < 0.01, f"B6 不成立：t*={b6['t_star']} 占基准
     t* 的 {ratio:.2%}，未趋于 0+"
```

```
    cmax_b6 = b6["coarse"]["C"].max(axis=1)
    ulp = float(np.spacing(C_INIT))
    detect = int(np.flatnonzero(cmax_b6 < C_INIT)[0])

    b7 = solve_case("B7_crit_unreachable", SolverConfig(c_crit=0.01, dt=60.0, rec_step=3600.0,
                                                        t_max=float(T_MAX_S)), air)
    assert b7["t_star"] is None and b7["hit"] is False, "B7 不成立：不可达阈值竟返回了 t*"
    label = UNREACHABLE if b7["t_star"] is None else f"{b7['t_star']:.4f} s"
    assert isinstance(label, str), "B7 不成立：未命中分支未给出显式字符串返回值"
    return {"B6": {"c_crit": C_INIT, "t_star_s": float(b6["t_star"]),
                   "fraction_of_baseline_t_star": ratio,
                   "detect_step_s": float(b6["coarse"]["t"][detect]), "ulp_of_threshold": ulp,
                   "note": "双精度下 C_max 的下降量小于阈值的 ulp 时不可分辨，故检出延迟 ~3e2 s；"
                           "精确算术意义上首步即穿越",
                   "expected": "t*->0+（过程最开端穿越）", "holds": True},
            "B7": {"c_crit": 0.01, "t_max_s": float(T_MAX_S), "t_star": None, "label": label,
                   "c_air_hold": float(air["C_air"][-1]),
                   "cmin_reached": float(b7["diag"]["c_min"]),
                   "expected": "永不达标，显式未命中分支而非 None 参与算术", "holds": True}}


def _build_table5(sol: dict, t_star: float) -> tuple:
    idx = table_node_index()
    cols_cm = radii_cm()
```

```
    times = np.arange(Q3_TABLE_STEP_S, t_star, Q3_TABLE_STEP_S, dtype=float)
    rows, labels = [], []
    for t in times:
        rows.append([profile_at(sol, t, "C")[j] for j in idx])
        labels.append((f"{t:.0f}", f"{t / 3600.0:.4f}"))
    rows.append([profile_at(sol, t_star, "C")[j] for j in idx])
    labels.append((f"{t_star:.4f}", f"{t_star / 3600.0:.4f}"))
    table5 = np.array(rows)
    return idx, cols_cm, labels, table5


def _write_q3_tables(sol: dict, coarse: dict, cols_cm: np.ndarray, idx: list,
                     labels: list, table5: np.ndarray) -> tuple:
    info = {}
    info["table5"] = write_csv(OUTPUT_DIR / "table5_moisture_6h.csv",
                               ["时间/s", "时间/h", *[f"r={cols_cm[j]:.1f}cm" for j in idx]],
                               [[t_s, t_h, *np.round(row, ROUND_DIGITS)]
                                for (t_s, t_h), row in zip(labels, table5)])
    info["result3"] = write_grid_xlsx(OUTPUT_DIR / "result3.xlsx",
                                      {"Sheet1": (coarse["t"], coarse["C"])})

    thresholds = {f"{level:g}": (None if val is None else float(val))
                  for level, val in sorted(sol["thresholds"].items())}
```

```
    info["threshold_sensitivity"] = dump_json(
        OUTPUT_DIR / "threshold_sensitivity.json",
        {"note": "§10.2 实验3：同一次积分内每步监视多个 C_crit，避免重复求解带来的口径漂移",
         "c_crit_to_t_star_s": thresholds,
         "c_crit_to_t_star_h": {key: (None if val is None else val / 3600.0)
                                for key, val in thresholds.items()}})
    return info, thresholds


def _cross_check_q2(sol: dict, air: dict) -> tuple:
    q2 = solve_case("q2_independent_3h", SolverConfig(t_end=float(Q2_SPAN_S),
                                                      rec_step=float(Q3_REC_STEP_S)), air)
    at3h_T, at3h_C = profile_at(sol, float(Q2_SPAN_S), "T"), profile_at(sol, float(Q2_SPAN_S), "C")
    rel_t = float(np.abs(q2["T"] - at3h_T).max() / np.abs(at3h_T).max())
    rel_c = float(np.abs(q2["C"] - at3h_C).max() / np.abs(at3h_C).max())
    assert rel_t < 1e-12 and rel_c < 1e-12, f"Q2/Q3 字段不一致：dT={rel_t:.2e}, dC={rel_c:.2e}"
    return rel_t, rel_c


def _write_q3_results_json(sol: dict, info: dict, coarse: dict, cols_cm: np.ndarray, labels: list,
                           table5: np.ndarray, thresholds: dict, constraints: dict,
                           capability: dict, bounds: dict, recheck: dict, t_star: float,
                           deviation: float, rel_t: float,
```

```
    rel_c: float) -> None:
578  cmax = coarse["C"].max(axis=1)
579  payload = {
580      "problem": 3, "title": "烘干所需时间（各处水分浓度首次全部低于 0.15 kg/kg 的时刻）",
581      "answer": {"t_star_s": t_star, "t_star_h": round(t_star / 3600.0, ROUND_DIGITS),
582                 "t_star_days": round(t_star / 86400.0, ROUND_DIGITS),
583                 "criterion": "min{t : max_r C(r,t) < 0.15 kg/kg}",
584                 "bound_kind": "lower_bound",
585                 "bound_reason": "忽略汽化潜热 T 偏高 D 偏高 t* 为真实烘干时间的下界",
586                 "prototype_reference_s": PROTOTYPE_T_STAR_S,
587                 "deviation_vs_prototype": deviation},
588      "method": {"same_solver_as_problem2": True, "checkpoint_key": BASELINE_KEY,
589                 "N_nodes": N_NODES, "dt_s": DT_SOLVER, "picard_iters": PICARD_ITERS,
590                 "face_diffusivity": FACE_D, "air_extrap": baseline_config().air_extrap,
591                 "new_in_Q3": ["C_crit=0.15", "式(15)(16) 每步评估", "式(17) 线性插值定位",
592                               "输出粒度 60 s"], "libs": LIB_VERSIONS},
593      "table5_moisture": {"times_s": [float(a) for a, _ in labels],
594                          "times_h": [float(b) for _, b in labels],
595                          "radii_cm": [0.0, 0.5, 1.0, 1.5, 2.0],
596                          "values": jsonable(np.round(table5, ROUND_DIGITS)),
```

```
                            "last_row_is_t_star": True},
     "cmax_history_60s": {"t_s": jsonable(coarse["t"]), "
cmax": jsonable(cmax),
                             "c_surface": jsonable(coarse["C"
][:, -1]), "c_crit": C_CRIT},
     "radial_snapshots": {"times_h": [0, 6, 12, 18, 24, 30,
 36, 42, 48, 54,
                                      round(t_star /
3600.0, 4)],
                             "r_cm": jsonable(cols_cm),
                             "C": jsonable([np.round(
profile_at(sol, min(h * 3600.0, t_star), "C"),

ROUND_DIGITS)
                                            for h in (0, 6,
12, 18, 24, 30, 36, 42, 48, 54)]
                                           + [np.round(
profile_at(sol, t_star, "C"), ROUND_DIGITS)])},
     "drying_rate_60s": {"t_s": jsonable(coarse["t"][1:]),
                         "minus_dW_dt": jsonable(-np.diff(
coarse["W"]) / np.diff(coarse["t"])),
                         "W": jsonable(coarse["W"])},
     "threshold_sensitivity": {"c_crit_to_t_star_h":
                               {key: (None if val is None
else val / 3600.0)
                                for key, val in thresholds.
items()}},
     "endpoint_recheck_from_records": jsonable(recheck),
     "conservation": {"eps_mass": sol["eps_mass"], "W0":
sol["W0"], "W_end": sol["W_end"],
                      "Q_out": sol["Q_out"], "dehydration":
 sol["dehydration"]},
     "cross_problem": {"t_common_s": float(Q2_SPAN_S), "
rel_diff_T": rel_t, "rel_diff_C": rel_c,
                       "tolerance": 1e-12},
```

```
618         "boundary_cases": jsonable(bounds),
619         "constraints": jsonable(constraints), "capability":
    jsonable(capability),
620         "diagnostics": jsonable(sol["diag"]), "artifacts":
    jsonable(info),
621     }
622     dump_json(FIGURES_DIR / "problem_3_results.json", payload)
623
624
625 def _write_q3_check_report(sol: dict, info: dict, constraints:
    dict, capability: dict,
626                            bounds: dict, recheck: dict,
    thresholds: dict, t_star: float,
627                            deviation: float, rel_t: float,
    rel_c: float) -> None:
628     lines = ["# 问题三 自检报告", "",
629              f"- t\\* = {t_star:.4f} s = {t_star / 3600.0:.4f}
    h = {t_star / 86400.0:.4f} d"
630              f"（判据 max_r C < {C_CRIT}，每步评估 + 式(17) 插
    值）",
631              f"- 与建模原型 {PROTOTYPE_T_STAR_S:.4f} s 的偏差
    {deviation:.4%} < 2%（预案 A5 未触发）",
632              f"- 括区：{constraints['C7']['bracket_s'][0]:.0f}
    s（Cmax="
633              f"{constraints['C7']['cmax_before']:.6f}） {
    constraints['C7']['bracket_s'][1]:.0f} s"
634              f"（Cmax={constraints['C7']['cmax_after']:.6f}
    ），t* 未落在 60 s 栅格上（C7/⑩）",
635              f"- max_r C 全程位于中心：中心值与全场最大值之差
    ≤ "
636              f"{recheck['max_gap_center_vs_max']:.3e}（浮点容
    差 1e-12 内）；严格 argmax≠0 的记录点 "
637              f"{recheck['n_rows_strict_argmax_off_center']}/{
    recheck['n_rows']} 个，均为 §5.2 声明的"
638              f"内部平台等值点（差 ~1e-15）。C9 单调 max dC={
```

```
    constraints['C9']['max_dC_dr']:.3e}",
639             f"- 误用表面值判据会得 {capability['P3-C1']['
    wrong_surface_criterion_h']:.4f} h"
640             f"（低估 {capability['P3-C1']['
    surface_underestimates_by']:.1%}）；"
641             f"误用体均得 {capability['P3-C1']['
    wrong_volume_mean_criterion_h']:.4f} h（P3-C1 可证伪）",
642             f"- result3.xlsx：{constraints['C10']['rows']} 行
     × {OUT_N_COLS} 列，Sheet1（C10）",
643             f"- 表5：{info['table5']['rows']} 行（每 6 h + t*
     行）",
644             f"- 阈值敏感性 C_crit -> t*/h：" + ", ".join(
645                 f"{k}{'未达标' if v is None else f'{v /
    3600.0:.4f}'}"
646                 for k, v in thresholds.items()),
647             f"- 边界情形 B6：C_crit={C_INIT} 时 t*={bounds['
    B6']['t_star_s']:.4f} s，"
648             f"占基准 t* 的 {bounds['B6']['
    fraction_of_baseline_t_star']:.3%}（0+）；"
649             f"检出延迟源于 ulp({C_INIT})={bounds['B6']['
    ulp_of_threshold']:.2e} 的浮点分辨率，"
650             f"非首达逻辑有误",
651             f"- 边界情形 B7：C_crit=0.01 < C_air 末值 {bounds
    ['B7']['c_air_hold']:.5f} 时"
652             f"{bounds['B7']['label']}（{T_MAX_S / 3600:.0f} h
     内 C_min 只到 "
653             f"{bounds['B7']['cmin_reached']:.6f}），显式未命
    中分支，无 None 参与算术",
654             f"- Q2/Q3 字段在 t=10800 s 一致：相对差 dT={rel_t
    :.2e}, dC={rel_c:.2e} < 1e-12",
655             f"- 守恒残差 |eps_mass|={abs(sol['eps_mass']):.3e
    }，累计脱水率 "
656             f"{sol['dehydration']:.4%}",
657             "", "结论：C7、C9、C10 与 P3-C1/P3-C2 全部通过，
    B6/B7 边界情形按预期行为返回。",
```

```
            "", f"结论口吻：t\\*={t_star / 3600.0:.4f} h 是在题面给定参数体系与假设 1–6 下的",
            "计算结果，且因忽略汽化潜热而为真实烘干时间的下界（§9 假设问责表）。"]
    (ROOT / "_tmp").mkdir(parents=True, exist_ok=True)
    (ROOT / "_tmp" / "problem_3_check.md").write_text("\n".join(lines) + "\n", encoding="utf-8")


def main() -> int:
    air = load_air_boundary()
    sol = solve_baseline(air)
    t_star = float(sol["t_star"])
    coarse = sol["coarse"]
    recheck = endpoint_from_records(coarse, C_CRIT, DT_SOLVER)
    assert recheck["hit"], "基准算例应在 t_max 内达标"

    idx, cols_cm, labels, table5 = _build_table5(sol, t_star)

    info, thresholds = _write_q3_tables(sol, coarse, cols_cm, idx, labels, table5)

    constraints = validate_constraints(sol, air, info, recheck)
    capability = validate_capability(sol, air, info, recheck)
    bounds = boundary_cases(air, t_star)

    rel_t, rel_c = _cross_check_q2(sol, air)

    deviation = abs(t_star - PROTOTYPE_T_STAR_S) / PROTOTYPE_T_STAR_S
    assert deviation < 0.02, f"与建模原型偏差 {deviation:.4%} > 2%，按预案 A5 处理"
    assert 40.0 <= t_star / 3600.0 <= 80.0, f"□ 违反：t*={t_star / 3600.0:.4f} h 不在 [40,80]"
```

```
685
686     _write_q3_results_json(sol, info, coarse, cols_cm, labels,
        table5, thresholds,
687                            constraints, capability, bounds,
    recheck, t_star, deviation,
688                            rel_t, rel_c)
689
690     _write_q3_check_report(sol, info, constraints, capability,
        bounds, recheck, thresholds,
691                            t_star, deviation, rel_t, rel_c)
692
693     print(f"[problem3] t* = {t_star:.4f} s = {t_star /
    3600.0:.4f} h "
694           f"= {t_star / 86400.0:.4f} d（偏差 {deviation:.4%}
    vs 原型）")
695     print(f"[problem3] 误用表面值 {capability['P3-C1']['
    wrong_surface_criterion_h']:.4f} h / "
696           f"误用体均 {capability['P3-C1']['
    wrong_volume_mean_criterion_h']:.4f} h  判据不可替代")
697     print(f"[problem3] B6 t*={bounds['B6']['t_star_s']:.4f} s,
        B7 {bounds['B7']['label']}")
698     print(f"[problem3] 落盘 {len(info)} 件: {', '.join(v['path
    '] for v in info.values())}")
699     print("[problem3] SELF-CHECK PASS（C7, C9, C10, P3-C1/P3-
    C2, B6/B7）")
700     return 0
701
702
703 if __name__ == "__main__":
704     sys.exit(main())
705 from __future__ import annotations
706
707 import hashlib
708 import json
709 import platform
```

```
import sys
import time
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import openpyxl
import scipy
from scipy.linalg import solve_banded
from scipy.optimize import brentq
from scipy.special import exp1, j0, j1, jn_zeros

_HERE = Path(__file__).parent.resolve()
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))

from params import (
    R_FIX, T_INIT_C, C_INIT, H_CONV, HM_MASS, C_CRIT, KELVIN_OFFSET,
    RHO_COEF, CP_COEF, K_COEF, D_COEF, N_NODES, DT_SOLVER, PICARD_ITERS,
    FACE_D, FACE_K, AIR_EXTRAP, T_AIR_HOLD, C_AIR_HOLD, T_MAX_S, D_SAFE_FLOOR,
    Q3_REC_STEP_S, OUT_N_COLS, OUT_DR_CM, ROUND_DIGITS,
)

ROOT = _HERE
OUTPUT_DIR = ROOT / "output"
FIGURES_DIR = ROOT / "figures"
AIR_FILE = ROOT.parent / "附件" / "附件1.xlsx"
AIR_SHEET = "Sheet1"

LIB_VERSIONS = {
    "python": platform.python_version(),
    "numpy": np.__version__,
```

```
    "scipy": scipy.__version__,
    "openpyxl": openpyxl.__version__,
}


def sha256_of(path: Path) -> str:
    digest = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def load_air_boundary() -> dict:
    profile = json.loads((ROOT.parent / "数据台账" / "DATA_PROFILE.json").read_text(encoding="utf-8"))
    entry = profile["files"].get(AIR_FILE.name, {})
    assert "total_rows" in entry, (
        f"[未建档] {AIR_FILE.name} 不在 DATA_PROFILE.json 或建档失败（{entry.get('error', '缺记录')}）")

    book = openpyxl.load_workbook(AIR_FILE, data_only=True, read_only=True)
    assert AIR_SHEET in book.sheetnames, f"附件1 缺工作表 {AIR_SHEET}: {book.sheetnames}"
    sheet = book[AIR_SHEET]
    rows = list(sheet.iter_rows(values_only=True))
    book.close()

    header = [str(x).strip() if x is not None else "" for x in rows[0]]
    idx = {name: header.index(name) for name in ("时间", "温度", "水分浓度")}
    records = [r for r in rows[1:] if r[idx["时间"]] is not None]
```

```
    t = np.array([float(r[idx["时间"]]) for r in records])
    T = np.array([float(r[idx["温度"]]) for r in records])
    C = np.array([float(r[idx["水分浓度"]]) for r in records])

    assert len(records) == entry["total_rows"] - 1 or len(records) == entry["total_rows"], (
        f"[摄入不全] 附件1 实读 {len(records)} 条数据行，建档 total_rows={entry['total_rows']}"
        f"（{entry['n_sheets']} 张 sheet）——检查是否被截断或只读了首表")
    assert np.all(np.diff(t) > 0), "附件1 时间列非严格递增"
    steps = np.unique(np.diff(t))
    assert steps.size == 1 and abs(steps[0] - 60.0) < 1e-9, f"附件1 时间步长非 60 s：{steps}"
    assert np.all(np.isfinite(T)) and np.all(np.isfinite(C)), "附件1 含非有限值"
    print(f"[data_ingest] 附件1 全量 {len(records)} 行 / {entry['n_sheets']} sheet 已核对通过；"
          f"t∈[{t.min():.0f},{t.max():.0f}] s, T_air∈[{T.min():.3f},{T.max():.3f}] ℃, "
          f"C_air∈[{C.min():.5f},{C.max():.5f}] kg/kg")
    return {"t": t, "T_air": T, "C_air": C, "sha256": sha256_of(AIR_FILE)}


PHI_TABLE_N = 131072
PHI_ERR_TOL = 1e-6


def phi_exact(C: np.ndarray) -> np.ndarray:
    C = np.maximum(np.asarray(C, dtype=float), 1e-300)
    a = -D_COEF[1]
    return C * np.exp(-a / C) - a * exp1(a / C)

```

```
797
798 def build_kirchhoff_table(n_points: int = PHI_TABLE_N) ->
    tuple[np.ndarray, np.ndarray]:
799     grid = np.concatenate(([0.0], np.logspace(-4.0, np.log10
    (3.0), n_points)))
800     integrand = np.zeros_like(grid)
801     positive = grid > 0
802     integrand[positive] = np.exp(D_COEF[1] / grid[positive])
803     phi = np.concatenate(([0.0], np.cumsum(np.diff(grid) * 0.5
     * (integrand[1:] + integrand[:-1]))))
804     assert np.all(np.diff(phi) >= 0.0), "Kirchhoff 势必须单调
    不减"
805     return grid, phi
806
807
808 _PHI_GRID, _PHI_VALS = build_kirchhoff_table()
809 _PHI_PROBE = np.linspace(0.02, 3.0, 2000)
810 PHI_TABLE_ERR = float(np.max(np.abs(np.interp(_PHI_PROBE,
    _PHI_GRID, _PHI_VALS)
811                                     - phi_exact(_PHI_PROBE)) /
     phi_exact(_PHI_PROBE)))
812 assert PHI_TABLE_ERR < PHI_ERR_TOL, (
813     f"Kirchhoff 查表精度 {PHI_TABLE_ERR:.3e} 未达 §7.2 口径 {
    PHI_ERR_TOL}")
814
815
816 def phi_of(C: np.ndarray) -> np.ndarray:
817     return np.interp(np.asarray(C, dtype=float), _PHI_GRID,
    _PHI_VALS)
818
819
820 def moisture_factor(C: np.ndarray) -> np.ndarray:
821     return np.exp(D_COEF[1] / np.maximum(np.asarray(C, dtype=
    float), D_SAFE_FLOOR))
822
```

```
823
824 def temperature_factor(T_c: np.ndarray) -> np.ndarray:
825     return np.exp(D_COEF[2] / (np.asarray(T_c, dtype=float) +
    KELVIN_OFFSET))
826
827
828 def diffusivity(C: np.ndarray, T_c: np.ndarray) -> np.ndarray:
829     return D_COEF[0] * moisture_factor(C) * temperature_factor
    (T_c)
830
831
832 def properties(C: np.ndarray) -> tuple[np.ndarray, np.ndarray,
     np.ndarray]:
833     C = np.asarray(C, dtype=float)
834     ratio = C / (C + 1.0)
835     rho = RHO_COEF[0] + RHO_COEF[1] * C
836     cp = CP_COEF[0] + CP_COEF[1] * ratio
837     k = K_COEF[0] + K_COEF[1] * ratio
838     return rho, cp, k
839
840
841 def face_diffusivity(C: np.ndarray, T_c: np.ndarray, mode: str
     = FACE_D) -> np.ndarray:
842     T_face = 0.5 * (T_c[:-1] + T_c[1:])
843     arr_t = temperature_factor(T_face)
844     if mode == "kirchhoff":
845         dC = C[1:] - C[:-1]
846         small = np.abs(dC) < 1e-12
847         C_bar = 0.5 * (C[:-1] + C[1:])
848         slope = np.where(small, moisture_factor(C_bar),
849                          (phi_of(C[1:]) - phi_of(C[:-1])) / np
    .where(small, 1.0, dC))
850         return D_COEF[0] * arr_t * slope
851     if mode == "midpoint":
852         return D_COEF[0] * arr_t * moisture_factor(0.5 * (C
```

```
    [:-1] + C[1:]))
    node_d = diffusivity(C, T_c)
    if mode == "arith":
        return 0.5 * (node_d[:-1] + node_d[1:])
    if mode == "harmonic":
        lo, hi = node_d[:-1], node_d[1:]
        return np.where(lo + hi > 0.0, 2.0 * lo * hi / np.where(lo + hi > 0.0, lo + hi, 1.0), 0.0)
    raise ValueError(f"未知界面格式：{mode}")


def face_conductivity(k_node: np.ndarray, mode: str = FACE_K) -> np.ndarray:
    if mode == "arith":
        return 0.5 * (k_node[:-1] + k_node[1:])
    if mode == "harmonic":
        lo, hi = k_node[:-1], k_node[1:]
        return 2.0 * lo * hi / (lo + hi)
    raise ValueError(f"未知界面格式：{mode}")


def radial_grid(n_nodes: int, radius: float = R_FIX, stretch: float = 1.0) -> dict:
    assert n_nodes >= 3, "至少 3 个节点"
    assert stretch >= 1.0, "stretch<1 会把网格挤向中心，与退化层加密的目的相反"
    if stretch == 1.0:
        r = np.linspace(0.0, radius, n_nodes)
    else:
        widths = stretch ** np.arange(n_nodes - 1)
        r = np.concatenate(([0.0], np.cumsum(widths) / widths.sum() * radius))
    r[-1] = radius
    r_face = 0.5 * (r[:-1] + r[1:])
    edges = np.concatenate(([0.0], r_face, [radius]))
```

```
    volume = 0.5 * (edges[1:] ** 2 - edges[:-1] ** 2)
    assert abs(volume.sum() - 0.5 * radius ** 2) < 1e-15 * radius ** 2, "加权体积未闭合到 R^2/2"
    return {"dr": float(r[1] - r[0]), "r": r, "volume": volume, "r_face": r_face,
            "dr_face": np.diff(r), "radius": radius, "stretch": stretch}


def solve_tridiag(volume: np.ndarray, dt: float, time_coef: np.ndarray,
                  conductance: np.ndarray, robin: float, robin_source: float,
                  old: np.ndarray) -> tuple[np.ndarray, float]:
    n = volume.size
    ab = np.zeros((3, n))
    rhs = time_coef * old
    diag = time_coef.copy()
    diag[:-1] += conductance
    diag[1:] += conductance
    diag[-1] += robin
    rhs[-1] += robin_source
    ab[0, 1:] = -conductance
    ab[1, :] = diag
    ab[2, :-1] = -conductance
    off_sum = np.zeros(n)
    off_sum[:-1] += conductance
    off_sum[1:] += conductance
    margin = float(np.min(diag - off_sum))
    return solve_banded((1, 1), ab, rhs), margin


@dataclass
class SolverConfig:
```

```
    n_nodes: int = N_NODES
    dt: float = DT_SOLVER
    picard_iters: int = PICARD_ITERS
    face_d: str = FACE_D
    face_k: str = FACE_K
    air_extrap: str = AIR_EXTRAP
    radius: float = R_FIX
    stretch: float = 1.0
    t_init: float = T_INIT_C
    c_init: float = C_INIT
    s_d: float = 1.0
    s_hm: float = 1.0
    s_h: float = 1.0
    t_air_shift: float = 0.0
    const_d: float | None = None
    freeze_props: bool = False
    air_const: tuple | None = None
    c_crit: float = C_CRIT
    extra_crit: tuple = ()
    t_end: float | None = None
    t_max: float = T_MAX_S
    rec_step: float = Q3_REC_STEP_S
    fine_step: float | None = None
    fine_until: float = 0.0


def air_series(air: dict, cfg: SolverConfig):
    if cfg.air_const is not None:
        t_const, c_const = float(cfg.air_const[0]), float(cfg.
   air_const[1])
        return (lambda _t: t_const), (lambda _t: c_const)
    grid = air["t"]
    temp = air["T_air"] + cfg.t_air_shift
    conc = air["C_air"]
    if cfg.air_extrap == "hold_last":
```

```
945             right_t, right_c = temp[-1], conc[-1]
946             assert abs(right_t - cfg.t_air_shift - T_AIR_HOLD) < 5
    e-4, "hold-last 末值温度与口径表不符"
947             assert abs(right_c - C_AIR_HOLD) < 5e-6, "hold-last 末
    值浓度与口径表不符"
948         elif cfg.air_extrap == "mean_tail":
949             tail = grid >= grid[-1] - 3600.0
950             right_t, right_c = float(temp[tail].mean()), float(
    conc[tail].mean())
951         else:
952             raise ValueError(f"未知外推口径：{cfg.air_extrap}")
953         return (lambda tt: float(np.interp(tt, grid, temp, left=
    temp[0], right=right_t)),
954                 lambda tt: float(np.interp(tt, grid, conc, left=
    conc[0], right=right_c)))
955
956
957 def _advance_in_time(cfg: SolverConfig, T: np.ndarray, C: np.
    ndarray, q_out, w_prev, w0,
958                      cmax_prev, tail_target, diag: dict,
    t_hard_stop, dr, volume, r_face,
959                      radius: float, dt: float, robin_m,
    robin_h, hm_eff, n: int, t_air_of,
960                      c_air_of, props_frozen, rec_every: int,
    fine_every: int, rec: dict,
961                      fine: dict, thresholds: dict):
962     while True:
963         if t_hard_stop is not None and diag["steps"] * dt >=
    t_hard_stop - 1e-9:
964             break
965         if tail_target is not None and diag["steps"] * dt >=
    tail_target - 1e-9:
966             break
967         step = diag["steps"] + 1
968         t = step * dt
```

```
        t_air, c_air = t_air_of(t), c_air_of(t)
        T_old, C_old = T, C
        for _ in range(cfg.picard_iters):
            rho, cp, k_node = props_frozen if props_frozen is
    not None else properties(C)
            k_face = face_conductivity(k_node, cfg.face_k)
            T, margin_h = solve_tridiag(volume, dt, volume *
    rho * cp / dt,
                                        r_face * k_face / dr,
     robin_h, robin_h * t_air, T_old)
            if cfg.const_d is not None:
                d_face = np.full(n - 1, float(cfg.const_d))
            else:
                d_face = face_diffusivity(C, T, cfg.face_d)
            d_face = cfg.s_d * d_face
            C, margin_m = solve_tridiag(volume, dt, volume /
    dt,
                                        r_face * d_face / dr,
     robin_m, robin_m * c_air, C_old)
            diag["margin"] = min(diag["margin"], margin_h,
    margin_m)

        q_out += hm_eff * (C[-1] - c_air) * radius * dt
        w_now = float(C @ volume)
        diag["dw_max"] = max(diag["dw_max"], w_now - w_prev)
        w_prev = w_now
        diag["d_min"] = min(diag["d_min"], float(d_face.min())
    )
        diag["d_max"] = max(diag["d_max"], float(d_face.max())
    )
        diag["t_min"] = min(diag["t_min"], float(T.min()))
        diag["t_max"] = max(diag["t_max"], float(T.max()))
        diag["c_min"] = min(diag["c_min"], float(C.min()))
        diag["c_max"] = max(diag["c_max"], float(C.max()))
        diag["surf_margin"] = min(diag["surf_margin"], float(C
```

```
    [-1] - c_air))
996         diag["steps"] = step
997 
998         cmax = float(C.max())
999         for level in thresholds:
1000             if thresholds[level] is None and cmax < level <=
    cmax_prev:
1001                 thresholds[level] = t - dt + (cmax_prev -
    level) / (cmax_prev - cmax) * dt
1002         cmax_prev = cmax
1003         if tail_target is None and thresholds[cfg.c_crit] is
    not None and cfg.t_end is None:
1004             tail_target = np.ceil(thresholds[cfg.c_crit] / cfg
    .rec_step) * cfg.rec_step
1005 
1006         if fine_every and step % fine_every == 0 and t <= cfg.
    fine_until + 1e-9:
1007             fine["t"].append(t); fine["T"].append(T.copy());
    fine["C"].append(C.copy())
1008         if step % rec_every == 0:
1009             rec["t"].append(t); rec["T"].append(T.copy()); rec
    ["C"].append(C.copy())
1010             rec["W"].append(w_now); rec["Q_out"].append(q_out)
1011             rec["eps"].append((w0 - w_now - q_out) / w0)
1012             rec["cmax"].append(cmax); rec["T_air"].append(
    t_air); rec["C_air"].append(c_air)
1013     return T, C, q_out, w_prev
1014 
1015 
1016 def run_solver(air: dict, cfg: SolverConfig | None = None) ->
    dict:
1017     cfg = cfg or SolverConfig()
1018     grid = radial_grid(cfg.n_nodes, cfg.radius, cfg.stretch)
1019     dr, volume, r_face = grid["dr_face"], grid["volume"], grid
    ["r_face"]
```

```
    radius, dt = grid["radius"], cfg.dt
    t_air_of, c_air_of = air_series(air, cfg)
    hm_eff, h_eff = HM_MASS * cfg.s_hm, H_CONV * cfg.s_h
    robin_m, robin_h = radius * hm_eff, radius * h_eff

    n = cfg.n_nodes
    T = np.full(n, cfg.t_init, dtype=float)
    C = np.full(n, cfg.c_init, dtype=float)
    props_frozen = properties(np.full(n, cfg.c_init)) if cfg.freeze_props else None

    rec_every = int(round(cfg.rec_step / dt))
    fine_every = int(round(cfg.fine_step / dt)) if cfg.fine_step else 0
    assert abs(rec_every * dt - cfg.rec_step) < 1e-12, "记录粒度必须是 dt 的整数倍"

    rec = {"t": [0.0], "T": [T.copy()], "C": [C.copy()], "W": [], "Q_out": [],
           "eps": [], "cmax": [float(C.max())], "T_air": [t_air_of(0.0)], "C_air": [c_air_of(0.0)]}
    fine = {"t": [], "T": [], "C": []}
    w0 = float(C @ volume)
    rec["W"].append(w0); rec["Q_out"].append(0.0); rec["eps"].append(0.0)

    q_out = 0.0
    cmax_prev = float(C.max())
    thresholds = dict.fromkeys((cfg.c_crit, *cfg.extra_crit))
    tail_target = None
    diag = {"margin": np.inf, "d_min": np.inf, "d_max": -np.inf, "surf_margin": np.inf,
            "dw_max": -np.inf, "t_min": np.inf, "t_max": -np.inf,
            "c_min": np.inf, "c_max": -np.inf, "steps": 0}
```

```
    w_prev = w0
    t_hard_stop = cfg.t_end if cfg.t_end is not None else cfg.t_max
    wall0 = time.perf_counter()

    T, C, q_out, w_prev = _advance_in_time(cfg, T, C, q_out, w_prev, w0, cmax_prev,
                                           tail_target, diag, t_hard_stop, dr, volume,
                                           r_face, radius, dt, robin_m, robin_h, hm_eff, n,
                                           t_air_of, c_air_of, props_frozen, rec_every,
                                           fine_every, rec, fine, thresholds)

    t_final = diag["steps"] * dt
    eps_mass = (w0 - w_prev - q_out) / w0
    t_star = thresholds[cfg.c_crit]
    return {"grid": grid, "cfg": cfg, "T": T, "C": C, "t_final": t_final,
            "coarse": {key: (np.array(val) if key != "t" else np.array(val)) for key, val in rec.items()},
            "fine": {key: np.array(val) for key, val in fine.items()},
            "W0": w0, "W_end": w_prev, "Q_out": q_out, "eps_mass": eps_mass,
            "t_star": t_star, "thresholds": dict(thresholds), "diag": diag,
            "dehydration": 1.0 - w_prev / w0, "wall_s": time.perf_counter() - wall0,
            "hit": t_star is not None}


def solve_case(key: str, cfg: SolverConfig, air: dict, *,
```

```
    force: bool = False) -> dict:
     print(f"[solve] {key}: N={cfg.n_nodes}, dt={cfg.dt},
    face_d={cfg.face_d} ...", flush=True)
     sol = run_solver(air, cfg)
     print(f"[solve] {key} done in {sol['wall_s']:.1f} s: t*={
    sol['t_star']}, "
           f"eps={sol['eps_mass']:.3e}, steps={sol['diag']['
    steps']:.0f}", flush=True)
     return sol


def robin_cylinder_series(r: np.ndarray, t: float, radius:
    float, alpha: float,
                          biot: float, n_terms: int = 60) ->
    np.ndarray:
     zeros = np.concatenate(([0.0], jn_zeros(0, n_terms + 1)))
     lam = np.array([brentq(lambda x: x * j1(x) - biot * j0(x),
                            zeros[i] + 1e-12, zeros[i + 1] - 1e
    -12, xtol=1e-14, rtol=1e-15)
                     for i in range(n_terms)])
     coef = 2.0 * j1(lam) / (lam * (j0(lam) ** 2 + j1(lam) **
    2))
     shape = j0(np.outer(np.asarray(r, dtype=float) / radius,
    lam))
     decay = np.exp(-lam ** 2 * alpha * t / radius ** 2)
     return shape @ (coef * decay)


def radii_cm() -> np.ndarray:
     return np.round(np.arange(OUT_N_COLS) * OUT_DR_CM, 1)


def table_node_index() -> list[int]:
     step = int(round(0.5 / OUT_DR_CM))
     return list(range(0, OUT_N_COLS, step))
```

```
def profile_at(sol: dict, t: float, field: str = "C") -> np.ndarray:
    grid_t = sol["coarse"]["t"]
    data = sol["coarse"][field]
    idx = int(np.searchsorted(grid_t, t))
    if idx < grid_t.size and abs(grid_t[idx] - t) < 1e-9:
        return data[idx]
    assert 0 < idx < grid_t.size, f"t={t} 超出记录区间 [{grid_t[0]}, {grid_t[-1]}]"
    weight = (t - grid_t[idx - 1]) / (grid_t[idx] - grid_t[idx - 1])
    return (1.0 - weight) * data[idx - 1] + weight * data[idx]


def write_grid_xlsx(path: Path, sheets: dict) -> dict:
    book = openpyxl.Workbook(write_only=True)
    header = ["时间\\到药材中心的距离", *[float(x) for x in radii_cm()]]
    shapes = {}
    for name, (times, field) in sheets.items():
        sheet = book.create_sheet(title=name)
        sheet.append(header)
        for row_t, row in zip(times, field):
            sheet.append([float(row_t), *[round(float(v), ROUND_DIGITS) for v in row]])
        shapes[name] = (len(times), field.shape[1])
    path.parent.mkdir(parents=True, exist_ok=True)
    book.save(path)
    book.close()
    return {"path": str(path.relative_to(ROOT).as_posix()), "sheets": shapes,
            "sha256": sha256_of(path), "bytes": path.stat().st_size}
```

```


def write_csv(path: Path, header: list[str], rows) -> dict:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [",".join(header)]
    lines += [",".join(f"{float(v):.10g}" for v in row) for row in rows]
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")
    return {"path": str(path.relative_to(ROOT).as_posix()), "rows": len(lines) - 1,
            "sha256": sha256_of(path)}


def dump_json(path: Path, payload: dict) -> dict:
    path.parent.mkdir(parents=True, exist_ok=True)
    text = json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False, sort_keys=False)
    path.write_text(text, encoding="utf-8")
    return {"path": str(path.relative_to(ROOT).as_posix()), "bytes": len(text.encode("utf-8"))}


def jsonable(value):
    if isinstance(value, (np.floating, np.integer)):
        return value.item()
    if isinstance(value, np.ndarray):
        return [jsonable(v) for v in value.tolist()]
    if isinstance(value, dict):
        return {str(k): jsonable(v) for k, v in value.items()}
    if isinstance(value, (list, tuple)):
        return [jsonable(v) for v in value]
    return value
```

q4.py

```
from __future__ import annotations
```

```
2
3  import hashlib
4  import json
5  import math
6  import os
7  import platform
8  import sys
9  import time
10 from datetime import datetime
11 from pathlib import Path
12
13 import numpy as np
14
15 _ENC = getattr(sys.stdout, "encoding", "").lower()
16 _STDOUT = sys.stdout
17 if _ENC != "utf-8" and hasattr(_STDOUT, "reconfigure"):
18     _STDOUT.reconfigure(encoding="utf-8", errors="replace")
19
20 _SELF = Path(__file__).resolve()
21 _PROJ = _SELF.parents[1]
22 sys.path.insert(0, os.path.dirname(os.path.realpath(__file__))
      )
23
24 from solver_q4 import (
25     solve_q4, radius_at, driver_at, radius_check, att1, att2,
26     R0, R_MIN, R_MIN_CM, R_FREEZE_T, C_INIT, T_INIT_C, C_DRY,
      MARGIN_S,
27     T_CAP_S, T_FREEZE_S, TA_FREEZE_C, CA_FREEZE, D_FLOOR,
      D_AP4_2P55_323K,
28     ATT1, ATT2, PDF, RESULT4_TEMPLATE,
29 )
30
31 SEED = 42
32 N_MAIN = 400
33 DT_I = 1.25
```

```
DT_II = 5.0
DT_OUT = 60.0
R_OUT_CM = np.round(np.arange(0.0, 1.1 + 1e-9, 0.1), 10)
R_OUT_CM_WIDE = np.round(np.arange(0.0, 1.9 + 1e-9, 0.1), 10)
R_TAB6_CM = np.array([0.0, 0.5, 1.0])
T_TAB6_H = np.arange(6.0, 73.0, 6.0)


def sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def pchip_eval(x, y, xq):
    from scipy.interpolate import PchipInterpolator
    return PchipInterpolator(x, y, extrapolate=False)(xq)


def linear_eval(x, y, xq):
    return np.interp(xq, x, y)


def fv_hermite_eval(x, y, xq):
    n = x.size
    h = np.diff(x)
    d = np.diff(y) / h
    m = np.zeros(n)
    if n == 2:
        m[:] = d[0]
    else:
        w1 = 2 * h[1:] + h[:-1]
        w2 = h[1:] + 2 * h[:-1]
```

```
        dd0, dd1 = d[:-1], d[1:]
        same = dd0 * dd1 > 0
        den = np.where(same, w1 / np.where(dd0 == 0, 1.0, dd0)
                       + w2 / np.where(dd1 == 0, 1.0, dd1),
    1.0)
        m[1:-1] = np.where(same, (w1 + w2) / den, 0.0)
        for k, (h0, h1, mm0, mm1) in ((0, (h[0], h[1], d[0], d
    [1])),
                                      (n - 1, (h[-1], h[-2], d
    [-1], d[-2]))):
            dv = ((2 * h0 + h1) * mm0 - h0 * mm1) / (h0 + h1)
            if np.sign(dv) != np.sign(mm0):
                dv = 0.0
            elif (np.sign(mm0) != np.sign(mm1)) and abs(dv) >
    3.0 * abs(mm0):
                dv = 3.0 * mm0
            m[k] = dv
     i = np.clip(np.searchsorted(x, xq, side="right") - 1, 0, n
     - 2)
     s = (xq - x[i]) / h[i]
     hi = h[i]
     h00 = (1 + 2 * s) * (1 - s) ** 2
     h10 = s * (1 - s) ** 2
     h01 = s * s * (3 - 2 * s)
     h11 = s * s * (s - 1)
     return h00 * y[i] + h10 * hi * m[i] + h01 * y[i + 1] + h11
     * hi * m[i + 1]


def map_output_full(xi_nodes, C_nodes, R_now, r_out_cm, method
    ="pchip"):
     vals = map_output(xi_nodes, C_nodes, R_now, r_out_cm,
    method)
     return np.concatenate([vals, [float(C_nodes[-1])]])

```

```
96
97  def map_output(xi_nodes, C_nodes, R_now, r_out_cm, method="
       pchip"):
98      r_out_m = np.asarray(r_out_cm, float) / 100.0
99      Rm = float(R_now)
100     vals = np.full(r_out_m.shape, np.nan)
101     inside = r_out_m <= Rm + 1e-15
102     if not np.any(inside):
103         return vals
104     xq = r_out_m[inside] / Rm
105     if method == "pchip":
106         vals[inside] = pchip_eval(xi_nodes, C_nodes, xq)
107     elif method == "linear":
108         vals[inside] = linear_eval(xi_nodes, C_nodes, xq)
109     elif method == "fvhermite":
110         vals[inside] = fv_hermite_eval(xi_nodes, C_nodes, xq)
111     else:
112         raise ValueError(method)
113     j = np.flatnonzero(inside)
114     for k in j:
115         if r_out_m[k] == 0.0:
116             vals[k] = C_nodes[0]
117         elif abs(r_out_m[k] - Rm) <= 1e-15:
118             vals[k] = C_nodes[-1]
119     return vals
120
121
122 def locate_crossing(tt, cm, level):
123     idx = np.flatnonzero(cm <= level)
124     if idx.size == 0:
125         return None
126     n = int(idx[0])
127     out = {"n_star": n, "level": float(level), "t_step_s":
       float(tt[n]),
128             "t_after_s": float(tt[n]), "cmax_after": float(cm[n
```

```
    ])}
     if n == 0:
         out.update(t_before=None, cmax_before=None, t_linear=
    float(tt[0]),
                    t_hermite=None, localization_error=None,
    slope=0.0,
                    slope_at_cross=0.0)
         return out
     t0, t1 = float(tt[n - 1]), float(tt[n])
     c0, c1 = float(cm[n - 1]), float(cm[n])
     out.update(t_before=t0, cmax_before=c0)
     if c1 == c0:
         out.update(t_linear=t1, t_hermite=None,
    localization_error=None,
                    slope=0.0, slope_at_cross=0.0)
         return out
     w = (level - c0) / (c1 - c0)
     t_lin = t0 + w * (t1 - t0)

     def slope_at(i):
         if i <= 0:
             return (cm[1] - cm[0]) / (tt[1] - tt[0])
         if i >= cm.size - 1:
             return (cm[i] - cm[i - 1]) / (tt[i] - tt[i - 1])
         return (cm[i + 1] - cm[i - 1]) / (tt[i + 1] - tt[i -
    1])
     m0, m1 = slope_at(n - 1), slope_at(n)
     h = t1 - t0
     lo, hi = 0.0, 1.0
     for _ in range(80):
         s = 0.5 * (lo + hi)
         s2, s3 = s * s, s * s * s
         val = ((2 * s3 - 3 * s2 + 1) * c0 + (s3 - 2 * s2 + s)
    * h * m0
                + (-2 * s3 + 3 * s2) * c1 + (s3 - s2) * h * m1
```

```
   - level)
        if val > 0:
            lo = s
        else:
            hi = s
    t_herm = t0 + 0.5 * (lo + hi) * h
    out.update(t_linear=float(t_lin), t_hermite=float(t_herm),
               slope=float((c1 - c0) / h),
               slope_at_cross=float(0.5 * (m0 + m1)),
               localization_error=float(abs(t_herm - t_lin)))
    return out


def write_xlsx(path, times_s, rows, header, decimals=4, sheet=
   "Sheet1"):
    import openpyxl
    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = sheet
    ws.append(header)
    for k, t in enumerate(times_s):
        vals = []
        for v in rows[k]:
            vals.append(None if (v is None or (isinstance(v,
   float) and math.isnan(v)))
                        else float(round(float(v), decimals)))
        ws.append([int(round(t))] + vals)
    wb.save(path)


T_DRY_MAIN_COMPONENT = "grid_contribution_s"
_T_DRY_CV = "q4_convergence.json"
_T_DRY_EB = "q4_error_budget.json"
_ASSUMED_FIELD_ERROR_KGKG = 1.0e-5

```

```
def fwd(pth):
    return str(pth).replace("\\", "/")


def jget(d, *keys):
    for k in keys:
        if not isinstance(d, dict) or k not in d:
            return None
        d = d[k]
    return d


def load_json_soft(path):
    path = Path(path)
    if not path.is_file():
        return None, "缺 %s" % fwd(path)
    try:
        return json.loads(path.read_text(encoding="utf-8")),
   None
    except Exception as e:
        return None, "%s 解析失败: %r" % (fwd(path), e)


def parse_h_interval(stmt):
    if not isinstance(stmt, str):
        return None
    i, j = stmt.find("["), stmt.find("] h")
    if i < 0 or j <= i:
        return None
    try:
        lo, hi = stmt[i + 1:j].split(",")
        return [float(lo.strip()), float(hi.strip())]
    except ValueError:
        return None
```

```
224
225
226 def grid_keys():
227     return str(N_MAIN), str(2 * N_MAIN), str(N_MAIN // 2)
228
229
230 def grid_component(lin, step_seq, cfg, t_nom_h):
231     if lin is None:
232         return None, None, None, None
233     k_prod, k_fine, k_coarse = grid_keys()
234     cfg_ok = (isinstance(cfg, dict) and cfg.get("dt1") == DT_I
235               and cfg.get("dt2") == DT_II and cfg.get("
    t_switch") == T_FREEZE_S)
236     nom_ok = (lin.get(k_prod) is not None
237               and abs(float(lin[k_prod]) - t_nom_h) <= 1e-9 *
    max(1.0, abs(t_nom_h)))
238     have = all(k in lin for k in (k_coarse, k_prod, k_fine))
239     if not cfg_ok:
240         return None, None, None, (
241             "results/%s  A_grid_independence.config = %r 与本
    轮生产配置"
242             "（dt1=%r、dt2=%r、t_switch=%r）不一致  该加密档与
    生产不同源、不可比"
243             % (_T_DRY_CV, cfg, DT_I, DT_II, T_FREEZE_S))
244     if not nom_ok:
245         return None, None, None, (
246             "results/%s  A_grid_independence.t_dry_lin_h[%s] =
     %r 与本轮生产档 "
247             "t_dry.t_nominal_linear_h = %r 不一致  档位与生产
    解不同源"
248             % (_T_DRY_CV, k_prod, lin.get(k_prod), t_nom_h))
249     if not have:
250         return None, None, None, (
251             "results/%s  A_grid_independence.t_dry_lin_h 缺 %s
     / %s / %s 档"
```

```
            % (_T_DRY_CV, k_coarse, k_prod, k_fine))
    grid = abs(float(lin[k_fine]) - float(lin[k_prod])) * 3600.0
    grid_up = abs(float(lin[k_prod]) - float(lin[k_coarse])) * 3600.0
    step_diff = None
    if step_seq and all(k in step_seq for k in (k_prod, k_fine)):
        step_diff = abs(float(step_seq[k_fine]) - float(step_seq[k_prod])) * 3600.0
    return grid, grid_up, step_diff, None


def build_t_dry_uncertainty(res, t_dry_step, t_dry_nom, t_dry_herm, loc_err,
                            cond_s_per_unit, dt_ii):
    base_h, cfg, cv, e1_rel, e2_pct, fe, interval, lin, order_p, res, sens, spread_h, step_seq, t_nom_h, t_step_h, why = load_uncertainty_inputs(
        cond_s_per_unit,
        res,
        t_dry_nom,
        t_dry_step,
    )

    grid, grid_up, step_diff, why_bad = grid_component(lin, step_seq, cfg, t_nom_h)
    if why_bad:
        why = why_bad

    rec = ("t_dry = %s s = %s h（首个 C_max ≤ 0.15 的内部步；线性/三次 Hermite 未舍入值"
           "另见 t_dry.t_nominal_linear_s = %s / t_dry.t_nominal_hermite_s = %s）。"
           % (float(t_dry_step), t_step_h, float(t_dry_nom),
```

```
    t_dry_herm))
277  dep_why, doc_note, fe_note, grid_note, rec, up_note =
    fill_grid_components(
278      cfg,
279      dt_ii,
280      fe,
281      grid,
282      grid_up,
283      interval,
284      lin,
285      loc_err,
286      order_p,
287      rec,
288      sens,
289      spread_h,
290      step_diff,
291      why,
292  )
293
294  mdata_note = ("来源 results/%s  interval_statement（逐位照
    抄该字符串中的 [%s, %s] h）: "
295                "E1（缺潜热  t_dry 偏短 %.1f %%，上端 =
    base×(1+E1)，E1_relative = %s）与"
296                "E2（1D 无限长  偏长 ≤ +%.1f %%，下端 =
    base×(1−E2)，E2.upper_bound_pct = %s）"
297                "的方向性上界，非统计置信区间；base =
    baseline_t_dry_h = %s。"
298                " 与 Q2 的 model_data_uncertainty_interval_h
     口径不同（Q2 的是驱动平台"
299                "情景之差），不要混用。"
300                % (_T_DRY_EB,
301                   interval[0] if interval else None,
    interval[1] if interval else None,
302                   e1_rel * 100.0 if e1_rel is not None else
     float("nan"),
```

```
                    e1_rel, e2_pct if e2_pct is not None else
    float("nan"),
                    e2_pct, base_h)) if interval is not None
   else (
                "待回填：%s（E1/E2 方向性上界，须读 results
   /%s  interval_statement）"
                % (why, _T_DRY_EB))

    return uncertainty_record(
        cv,
        dep_why,
        doc_note,
        dt_ii,
        fe,
        fe_note,
        grid,
        grid_note,
        grid_up,
        interval,
        loc_err,
        mdata_note,
        rec,
        sens,
        spread_h,
        up_note,
        why,
    )

def load_uncertainty_inputs(cond_s_per_unit, res, t_dry_nom,
   t_dry_step):
    res = Path(res)
    sens = cond_s_per_unit
    fe = _ASSUMED_FIELD_ERROR_KGKG * sens
    t_step_h = float(t_dry_step) / 3600.0
    t_nom_h = float(t_dry_nom) / 3600.0
```

```
    cv, why_cv = load_json_soft(res / _T_DRY_CV)
    eb, why_eb = load_json_soft(res / _T_DRY_EB)
    lin = jget(cv, "A_grid_independence", "t_dry_lin_h")
    step_seq = jget(cv, "A_grid_independence", "t_dry_h")
    cfg = jget(cv, "A_grid_independence", "config")
    spread_h = jget(cv, "A_grid_independence", "t_dry_spread_h
")
    order_p = jget(cv, "A_grid_independence", "
observed_order_p")
    interval = parse_h_interval(jget(eb, "interval_statement")
)
    e1_rel = jget(eb, "E1_latent_heat", "E1_relative")
    e2_pct = jget(eb, "E2_end_face", "upper_bound_pct")
    base_h = jget(eb, "baseline_t_dry_h")
    why = why_cv or why_eb or "未知"
    return (
        base_h,
        cfg,
        cv,
        e1_rel,
        e2_pct,
        fe,
        interval,
        lin,
        order_p,
        res,
        sens,
        spread_h,
        step_seq,
        t_nom_h,
        t_step_h,
        why,
    )

def fill_grid_components(
```

```
    cfg,
    dt_ii,
    fe,
    grid,
    grid_up,
    interval,
    lin,
    loc_err,
    order_p,
    rec,
    sens,
    spread_h,
    step_diff,
    why,
):
    if grid is not None:
        rec += ("数值不确定度主报 ±%s s（≈ ±9.1×10□ s = %s h，Q4 自己的实测网格项："
                "生产档 N=%s vs 更细一档 N=%s，与 Q2 的「生产 vs 更细一档」口径严格同构）；"
                "±%s s 为灵敏度说明（= 1/|dCmax_dt_at_crossing| × 假定场值误差 "
                "1e-5 kg/kg），不得单独当作 t_dry 的不确定度（本项是网格项的 %s 倍，"
                "比网格项大  单用它会高估）；"
                % (grid, float(grid) / 3600.0, N_MAIN, 2 * N_MAIN, fe, fe / float(grid)))
        if interval is not None:
            rec += ("模型/数据不确定度另以区间 [%s, %s] h（E1/E2 方向性上界，"
                    "非统计置信区间）表述。" % (interval[0], interval[1]))
        if grid_up is not None:
            rec += ("另登记网格敏感度上界（粗一档 N=%s vs 生产 N=%s）= %s s"
```

```
                    "（见 grid_sensitivity_upper_bound_s；为网格项的 %s 倍）。"
                    % (N_MAIN // 2, N_MAIN, grid_up, grid_up / float(grid)))
        rec += "不得声称 4 位小数有效（docs/Q4_建模文档.md §9.2）。"
    else:
        rec += ("数值不确定度主报项 = 实测网格项（生产档 N=%d vs 更细一档）待回填"
                "（%s）—— 本脚本单次运行算不出该值，须先跑 'src/verify_q4_convergence.py' "
                "产出 'results/%s' 之后重跑本脚本；回填前不得报任何 t_dry 不确定度数。"
                "不得声称 4 位小数有效（docs/Q4_建模文档.md §9.2）。"
                % (N_MAIN, why, _T_DRY_CV))

    if grid is not None:
        k_prod, k_fine, k_coarse = grid_keys()

        why_lin = ("为何用 t_dry_lin_h 而非 t_dry_h：Q2 网格项取的就是线性插值定位"
                   "（results/q2_error_budget.json … space_grid_tail_measured."
                   "t_dry_dr_0_01_cm 在 src/verify_q2_error_budget.py 中读的是 q2_parts 的 "
                   "t_dry_linear），同口径才可比较；"
                   + ("换用 t_dry_h（首个满足步）序列时同一对的差为 %s s。" % step_diff
                      if step_diff is not None else "（t_dry_h 序列该对值不可用。）"))
        grid_note = (
            "Q4 有自己的网格收敛研究，本键由它独立复算得出（不是从 Q2 继承、不是硬编码）。"
            "复算过程：读 results/%s  A_grid_independence.
```

```
    t_dry_lin_h（该档 config: dt1 = %s、"
415             "dt2 = %s、t_switch = %s，只改 N，与 Q2 网格项的「只改 Δr」口径相同），取 "
416             "N=%s  %s h（生产档，逐位等于 results4.json  t_dry.t_nominal_linear_h）、"
417             "N=%s  %s h（更细一档）；|%s − %s| × 3600 = %s s。"
418             "口径为什么是 N=%s□N=%s（生产 vs 更细一档）：Q2 的生产网格是 N=200，其网格项 "
419             "= |t_dry(N=200) − t_dry(N=400)| 即「生产 vs 更细一档」；Q4 的生产网格是 N=%s"
420             "（results4.json  meta.config.N），严格同构的量为 |t_dry(N=%s) − t_dry(N=%s)| "
421             "本键取此。%s旁证：同档 observed_order_p = %s（p > 2，加密仍在收敛）；"
422             "t_dry_spread_h = %s（四档 step 序列极差 = %.1f s，同值见 "
423             "results/q4_error_budget.json  E4_discrete.t_dry_grid_spread_h）。"
424             % (_T_DRY_CV, cfg.get("dt1"), cfg.get("dt2"), cfg.get("t_switch"),
425                k_prod, lin[k_prod], k_fine, lin[k_fine], lin[k_fine], lin[k_prod], grid,
426                k_prod, k_fine, N_MAIN, N_MAIN, 2 * N_MAIN, why_lin,
427                order_p, spread_h, float(spread_h) * 3600.0))
428         up_note = (
429             "= |t_dry_lin_h(N=%s) − t_dry_lin_h(N=%s)| × 3600 = |%s − %s| × 3600 = %s s"
430             "（同档 config: dt1 = %s、dt2 = %s、t_switch = %s，只改 N）。"
431             "这是「粗一档 vs 生产」的网格敏感度上界，不是主报项 —— 它含更粗档 N=%s 自身"
432             "的误差，不能当作生产解 N=%s 的网格不确定度；主报项 = 生产 vs 更细一档 = "
```

```
            "grid_contribution_s = %s s（本键为它的 %s 倍）。主报改用与 Q2 严格同构的 "
            "N=%s□N=%s，本键作为独立键保留此值以便溯源与保守对照。"
          % (k_coarse, k_prod, lin[k_prod], lin[k_coarse], grid_up,
             cfg.get("dt1"), cfg.get("dt2"), cfg.get("t_switch"),
             k_coarse, k_prod, grid, grid_up / grid, k_prod, k_fine)
          + ("旁证（step 序列同一对）= %s s。" % step_diff
             if step_diff is not None else ""))
    else:
        grid_note = ("待回填：%s。主报网格项（|t_dry_lin_h(N=%s) − t_dry_lin_h(N=%s)| × "
                     "3600）须读 results/%s 现算 —— 不填数、不估算。回填："
                     "python src/verify_q4_convergence.py，再重跑本脚本。"
                     % (why, N_MAIN, 2 * N_MAIN, _T_DRY_CV))
        up_note = "待回填：%s" % why

    if grid is not None:
        fe_note = ("= field_error_sensitivity_s_per_kgkg × 1e-5 kg/kg（假定场值误差，"
                   "不是实测误差）—— 仅灵敏度说明，非主报项；本项是主报网格项的 %s 倍"
                   "（比网格项大 单用它当 t_dry 的不确定度会高估）。"
                   "本键承载的正是原 t_dry.uncertainty.field_error_1e-5_kgkg_to_time_s"
                   "（旧主报值），已降格；同名旧键仍在顶层保留（见 legacy_compat_note）。"
                   % (fe / float(grid),))
        dep_why = ("原 uncertainty 块把灵敏度换算（%s s = ∂
```

```
    t_dry/∂C_max × 假定场值误差"
455                 " 1e-5 kg/kg）与定位括住宽（%.1f s）、定位
    误差（%r s）平列，并以 statement "
456                 "宣称「t_dry 不得声称 4 位小数有效：场值误
    差 1e-5 kg/kg  %.1f s」—— "
457                 "那 %s s 是灵敏度换算、不是 t_dry 的数值不
    确定度（它是「假定场值误差 "
458                 "1e-5 kg/kg」下换算出来的，不是本问实测的离
    散误差）；Q4 自己的实测网格项"
459                 "（主报项，生产档 vs 更细一档）= %s s。2026
    -09-11 跨问口径预检确认该根因"
460                 "已在 Q2/Q3/Q4 复制 3 次；本块按 Q2 的分层
    结构改写，原键值"
461                 "全部保留（顶层同名 + 本 deprecated 块），
    未删除。"
462                 % (fe, dt_ii, loc_err, fe, fe, grid))
463         doc_note = (" 待裁决 "
464                     "§9.2 的条件数与 ±15.2 s、§9.4 余量表的
    ±15 s 均为设计期值（k_lump 由"
465                     "预研原型两个诊断点推出，文档自述属③层冒烟
    判据）；实测条件数为 %s s per "
466                     "(kg/kg)、场值误差 1e-5 kg/kg 的时间当量为
     %s s（即设计期 15.2 s 的 "
467                     "%.3f 倍）。本块已按实测落盘（数值不确定度
    主报项 = 实测网格项 %s s， "
468                     "非灵敏度项）；文档侧的更正见本轮跨问口径
    一批改的文档改动。"
469                     % (sens, fe, fe / 15.2, grid))
470     else:
471         fe_note = ("仅灵敏度说明，非主报项：= 1/|
    dCmax_dt_at_crossing| × 假定场值误差 "
472                    "1e-5 kg/kg（本脚本自算）。主报项（实测网格
    项）待回填前，不得用本项"
473                    "代替 t_dry 的不确定度。")
474         dep_why = "待回填：%s" % why
```

```
    return dep_why, doc_note, fe_note, grid_note, rec, up_note

def uncertainty_record(
    cv,
    dep_why,
    doc_note,
    dt_ii,
    fe,
    fe_note,
    grid,
    grid_note,
    grid_up,
    interval,
    loc_err,
    mdata_note,
    rec,
    sens,
    spread_h,
    up_note,
    why,
):
    return {
        "main_reported_component": T_DRY_MAIN_COMPONENT,
        "recommended_statement": rec,
        "grid_contribution_s": grid,
        "grid_contribution_note": grid_note,
        "grid_sensitivity_upper_bound_s": grid_up,
        "grid_sensitivity_upper_bound_note": up_note,
        "time_step_contribution_s": None,
        "time_step_contribution_note": (
            "置 null：Q4 没有生产网格上的 Δtt_dry 落盘项，故不移植、不新算"
            "（本项目纪律：不得自算无落盘依据的数）。落盘事实三条：① results/%s  "
            "B_time_convergence.config.N = 200（不是生产网格 N
```

```
    =%s），其 t_dry_lin_h 六档"
508             "序列 I20_II60…I0.625_II2.5 的极差 = %s h = %.1f
    s（含 I20_II60 等粗档，"
509             "口径是 spread 不是 contribution；逐位等于 results
    /%s  E4_discrete."
510             "t_dry_dt_spread_h）；② A_grid_independence.
    t_dry_spread_h = %s h（= %.1f s，"
511             "四档网格 spread）；③ Q2 的对应键
    time_step_contribution_s = 2.9718922144239177 s "
512             "出自「Δt_II 1/8 vs 1/16 的 C_max 轨迹最大偏差 ×
    实测放大系数」，Q4 无同口径落盘量。"
513             "建议：由编程手在 N=%s 上补一对 Δt 加密（或明确以
    B_time_convergence 的 "
514             "N=200 口径为准）后回填本键。"
515             % (_T_DRY_CV, N_MAIN, dt_spread_h(cv), dt_spread_h
    (cv) * 3600.0,
516                _T_DRY_EB, spread_h, float(spread_h) * 3600.0,
    N_MAIN)
517             if (dt_spread_h(cv) is not None and spread_h is
    not None) else
518             "待回填：%s（Δt 加密研究的实测值；口径为 spread 不
    是 contribution，"
519             "不能直接当不确定度）" % why),
520         "localization_only_s": loc_err,
521         "localization_only_note": ("Q4 自己的线性 vs 三次
    Hermite 定位差（= 本文件 t_dry."
522                                    "localization_error_s）；比
    网格项小 6 个量级。"),
523         "field_error_sensitivity_s_per_kgkg": sens,
524         "field_error_sensitivity_note": (
525             "= 1/|t_dry.dCmax_dt_at_crossing|（实测跨越点斜率
    %r kg/(kg·s)）= 灵敏度换算系数，"
526             "不是不确定度本身；逐位等于本文件 t_dry.
    condition_number_s_per_kgkg（Q4 的规范"
527             "位置在 t_dry 顶层，本键仅为与 Q2 键名对齐而设）。
```

```
    " % (1.0 / sens)),
        "field_error_at_1e-5_kgkg_s": fe,
        "field_error_at_1e-5_kgkg_note": fe_note,
        "model_data_uncertainty_interval_h": interval,
        "model_data_uncertainty_note": mdata_note,
        "doc_consistency_note": doc_note,
        "field_to_time_factor_s_per_kgkg": sens,
        "field_to_time_factor_s_per_kgkg_status": (
            "本键为对齐 Q2 键名而设；Q4 的规范位置是 t_dry.condition_number_s_per_kgkg"
            "（逐位同值）。 不要与 Q2 的同名键 953600.0 混淆 —— 那是 Q2 的设计期名义值"
            "（已废弃）。本键值是实测跨越点斜率倒数，可用。"),
        "deprecated": {
            "field_error_1e-5_kgkg_to_time_s": fe,
            "field_error_1e-4_kgkg_to_time_s": 1e-4 * sens,
            "bracket_width_s": dt_ii,
            "localization_s": loc_err,
            "statement": ("t_dry 不得声称 4 位小数有效：场值误差 1e-5 kg/kg  %.1f s；"
                          "定位括住宽 %.0f s；缩小 Δt 不改善 t_dry（它由场值误差决定，"
                          "§9.2 推论 2）" % (fe, dt_ii)),
            "why": dep_why,
        },
        "legacy_compat_note": (
            "顶层 bracket_width_s / localization_s / field_error_1e-5_kgkg_to_time_s 三个键"
            "按原名原值保留，唯一原因是 src/build_answer_bundle.py 直接按名读取"
            "（_unc[\"field_error_1e-5_kgkg_to_time_s\"]、_unc[\"localization_s\"]、"
            "_unc[\"bracket_width_s\"]），移走或改名会使该脚本当场 KeyError。它们不是 "
            "t_dry 的数值不确定度：主报项见
```

```
    grid_contribution_s。"),
554         "bracket_width_s": dt_ii,
555         "localization_s": loc_err,
556         "field_error_1e-5_kgkg_to_time_s": fe,
557         "field_error_1e-4_kgkg_to_time_s": 1e-4 * sens,
558     }
559
560
561 def dt_spread_h(cv):
562     return jget(cv, "B_time_convergence", "t_dry_spread_h")
563
564
565 def _assert_status(o):
566     if o["status"] == "not_dry_by_cap":
567         raise SystemExit("[FATAL] cap 14 d reached, C_max ≤ %g
    unmet" % C_DRY)
568     if o["status"] == "extremum_violation":
569         for v in o["violations"][:10]:
570             print("  [violation]", v)
571         raise SystemExit("[FATAL] extremum principle violated
    (Q4-27)")
572     if o["status"] not in ("ok", "fixed_end"):
573         raise SystemExit("[FATAL] unknown status=%r" % o["
    status"])
574     if o["floor"]["count"] > 0:
575         for e in o["floor"]["first_events"][:10]:
576             print("   ", e)
577         raise SystemExit("[FATAL] D_floor active %d times" % o
    ["floor"]["count"])
578     if o["picard"]["n_not_converged"] > 0:
579         raise SystemExit("[FATAL] Picard unconverged steps %d"
     % o["picard"]["n_not_converged"])
580     if o["mass_audit"]["max_rel"] > 1e-8:
581         raise SystemExit("[FATAL] mass audit residual %.3e > 1
    e-8" % o["mass_audit"]["max_rel"])
```

```
def _locate_and_build(o):
    tt, cm = o["cmax"]["t"], o["cmax"]["cmax"]
    cross = locate_crossing(tt, cm, C_DRY)
    if cross is None:
        raise SystemExit("[FATAL] no inner step with C_max ≤ %g" % C_DRY)
    t_dry_step = cross["t_step_s"]
    t_dry_nom = cross["t_linear"]
    slope_cross = cross["slope_at_cross"]
    dCmax_dt = abs(slope_cross)
    cond_s_per_unit = 1.0 / dCmax_dt if dCmax_dt > 0 else float("inf")
    snap_t, snap_R, snap_C, xi = o["snap_t"], o["snap_R"], o["snap_C"], o["xi"]
    t_last_out = math.floor(t_dry_step / DT_OUT + 1e-9) * DT_OUT
    i_last = int(np.argmin(np.abs(snap_t - t_last_out)))
    if abs(snap_t[i_last] - t_last_out) > 1e-9:
        raise SystemExit("[FATAL] no output snapshot at t=%g s" % t_last_out)
    n_row = i_last
    if n_row < 1:
        raise SystemExit("[FATAL] zero delivery rows, t_dry=%s" % t_dry_step)
    times_out = snap_t[1:n_row + 1]
    R_out_t = snap_R[1:n_row + 1]
    Cpk = np.empty((n_row, R_OUT_CM.size + 1))
    Cln = np.empty_like(Cpk)
    Chm = np.empty_like(Cpk)
    assert_blank = 0
    for k in range(n_row):
        Ck = snap_C[k + 1]
        Rk = snap_R[k + 1]
```

```
611         Cpk[k] = map_output_full(xi, Ck, Rk, R_OUT_CM, "pchip"
    )
612         Cln[k] = map_output_full(xi, Ck, Rk, R_OUT_CM, "linear
    ")
613         Chm[k] = map_output_full(xi, Ck, Rk, R_OUT_CM, "
    fvhermite")
614         assert_blank += int(np.count_nonzero(np.isnan(Cpk[k
    ][:-1])))
615     if assert_blank != 0:
616         raise SystemExit("[FATAL] %d cells outside r > R(t) in
     main column" % assert_blank)
617     return (tt, cm, cross, t_dry_step, t_dry_nom, slope_cross,
     dCmax_dt, cond_s_per_unit,
618             snap_t, snap_R, snap_C, xi, n_row, times_out,
    R_out_t, Cpk, Cln, Chm,
619             t_last_out, assert_blank)
620
621
622 def _prepare_and_solve():
623     dry = os.environ.get("MM_Q4_DRY") == "1"
624     global N_MAIN, DT_I, DT_II
625     t0 = time.time()
626     if dry:
627         N_MAIN, DT_I, DT_II = 40, 20.0, 60.0
628         res = _PROJ / "results" / "_dryrun_q4"
629         print("[dryrun] MM_Q4_DRY=1: 粗网格/粗步长，产物只落 %
    s" % res)
630     else:
631         res = _PROJ / "results"
632     res.mkdir(parents=True, exist_ok=True)
633
634     inputs = {}
635     for p in (ATT1, ATT2, PDF, RESULT4_TEMPLATE):
636         pp = Path(p)
637         inputs[pp.name] = {"path": str(pp),
```

```
                          "sha256": sha256(pp) if pp.exists()
 else None,
                          "exists": pp.exists()}
 scripts = {}
 for s in ("solver_q4.py", "run_q4.py"):
     sp = _PROJ / "Q4" / s
     scripts[s] = sha256(sp) if sp.exists() else None

 rchk = radius_check("shrink")
 if not (rchk["monotone_ok"] and rchk["mono_nonincreasing"
]):
     raise SystemExit("[FATAL] radius violates monotone
rule, %s" % rchk)

 print("[run_q4] 生产运行：N=%d（Δξ=%g）、Δt_I=%g s、Δt_II
=%g s、输出 %g s"
       % (N_MAIN, 1.0 / N_MAIN, DT_I, DT_II, DT_OUT))
 o = solve_q4(N=N_MAIN, dt1=DT_I, dt2=DT_II, t_switch=
T_FREEZE_S,
              R_mode="shrink", geo_layer="nplus1",
face_rule="arith",
              props="app4", bc="robin", T_mode="prescribed"
,
              stop_on_dry=True, snap_every_s=DT_OUT,
              cmax_track=True, audit=True, audit_series=
True,
              verbose=True, progress_every=2000)
 print("[run_q4] status=%s  t_final=%.6f s (%.4f h)  t_dry
=%s  steps=%d  墙钟 %.1f s"
       % (o["status"], o["t_final"], o["t_final"] / 3600.0,
          ("%.6f s" % o["t_dry"]) if o["t_dry"] else "None"
,
          o["n_steps"], o["elapsed_s"]), flush=True)
 return dry, res, inputs, scripts, t0, o, rchk

```

```
663
664 def _write_cmax_traces(res, tt, cm, o, t_dry_nom):
665     np.savez_compressed(res / "q4_cmax_trajectory.npz",
666                         t_s=tt, C_max=cm, argmax_i=o["cmax"]["
    argmax"],
667                         C_center=o["cmax"]["c0"], C_surface=o[
    "cmax"]["cR"],
668                         C_min=o["cmax"]["cmin"], R_m=radius_at
    (tt, "shrink"))
669     m = (tt >= t_dry_nom - 600.0) & (tt <= t_dry_nom + 600.0)
670     np.savetxt(res / "q4_cmax_crossing_window.csv",
671                np.column_stack([tt[m], cm[m], o["cmax"]["c0"][
    m], o["cmax"]["cR"][m],
672                                 o["cmax"]["cmin"][m],
    radius_at(tt[m], "shrink")]),
673                delimiter=",", comments="",
674                header="t_s,C_max,C_center,C_surface,C_min,R_m"
    )
675     i60 = np.searchsorted(tt, np.arange(DT_OUT, tt[-1] + 1e-9,
     DT_OUT))
676     np.savetxt(res / "q4_cmax_60s.csv",
677                np.column_stack([tt[i60], cm[i60], o["cmax"]["
    c0"][i60],
678                                 o["cmax"]["cR"][i60],
    radius_at(tt[i60], "shrink")]),
679                delimiter=",", comments="",
680                header="t_s,C_max,C_center,C_surface,R_m")
681     ms = o["mass_audit_series"]
682     np.savetxt(res / "q4_mass_audit.csv",
683                np.column_stack([ms["t"], ms["lhs"], ms["rhs"],
     ms["rel"], ms["cmax"]]),
684                delimiter=",", comments="", header="t_s,lhs,rhs
    ,rel_resid,C_max")
685
686
```

```
def _build_table6(res, tt, cross, snap_t, snap_C, snap_R, xi, t_dry_step):
    i_dry = cross["n_star"]
    tab6_t_h = T_TAB6_H[T_TAB6_H * 3600.0 <= t_dry_step + 1e-9]
    rows6, times6 = [], []
    surf_vals = []
    for th in tab6_t_h:
        ts = th * 3600.0
        i = int(round(ts / DT_OUT))
        if abs(snap_t[i] - ts) > 1e-9:
            raise SystemExit("[FATAL] table 6 t=%g h misses snapshot" % th)
        vals = map_output_full(xi, snap_C[i], snap_R[i], R_TAB6_CM, "pchip")
        vals[0] = snap_C[i][0]
        rows6.append(vals); times6.append(th)
        surf_vals.append((float(th), float(snap_C[i][-1]), float(snap_R[i])))
    i_near = int(np.argmin(np.abs(snap_t - tt[i_dry])))
    C_end_field = snap_C[i_near]
    R_end_field = snap_R[i_near]
    vals_end = map_output_full(xi, C_end_field, R_end_field, R_TAB6_CM, "pchip")
    vals_end[0] = C_end_field[0]
    table6 = dict(
        header=["时间/h", "0", "0.5", "1.0", "药材表面"],
        unit="C / (kg/kg)；时间 / h",
        rows_h=[float(v) for v in times6] + ["烘干结束时间"],
        values=[[float(v) for v in r] for r in rows6]
        + [[float(v) for v in vals_end.flatten()]],
        t_dry_h_rounded_3=float(round(t_dry_step / 3600.0, 3)),
        t_end_row_field_snapshot_s=float(snap_t[i_near])
    )
```

```
    with open(res / "q4_table6.csv", "w", encoding="utf-8-sig") as f:
        f.write(",".join(table6["header"]) + "\n")
        for i, th in enumerate(times6):
            f.write("%g," % th + ",".join("%.4f" % v for v in rows6[i]) + "\n")
        f.write("烘干结束时间,%s\n" % ",".join("%.4f" % v for v in vals_end))
    return table6, times6, vals_end, rows6, surf_vals, i_near


def _write_result4_outputs(res, times_out, snap_C, snap_R, xi, n_row, Cpk, Cln, Chm, R_out_t):
    Cwide = np.full((n_row, R_OUT_CM_WIDE.size + 1), np.nan)
    for k in range(n_row):
        Cwide[k] = map_output_full(xi, snap_C[k + 1], snap_R[k + 1], R_OUT_CM_WIDE, "pchip")
    write_xlsx(res / "result4_wide.xlsx", times_out, Cwide,
               ["时间\\到药材中心的距离"]
               + [float(round(rc, 1)) for rc in R_OUT_CM_WIDE] + ["药材表面"],
               decimals=4, sheet="Sheet1")
    np.savez_compressed(res / "result4_full.npz", t_s=times_out, r_cm=R_OUT_CM,
                        C_pchip=Cpk, C_linear=Cln, C_fvhermite=Chm, R_m=R_out_t)
    with open(res / "result4_full.csv", "w", encoding="utf-8") as f:
        f.write("t_s," + ",".join("%.1f" % rc for rc in R_OUT_CM) + ",药材表面,R_m\n")
        for k in range(n_row):
            f.write("%d," % int(round(times_out[k]))
                    + ",".join("%.10f" % v for v in Cpk[k])
                    + ",%.10f\n" % R_out_t[k])
    dmap = np.abs(Cpk[:, :-1] - Cln[:, :-1])
```

```
    dmap2 = np.abs(Cpk[:, :-1] - Chm[:, :-1])
    np.savetxt(res / "q4_mapping_uncertainty.csv",
               np.column_stack([times_out, dmap, dmap2]),
               delimiter=",", comments="",
               header=("t_s," + ",".join("dPL_%.1f" % rc for rc in R_OUT_CM)
                       + "," + ",".join("dPH_%.1f" % rc for rc in R_OUT_CM)))
    return dmap, dmap2


def main():
    dry, res, inputs, scripts, t0, o, rchk = _prepare_and_solve()

    _assert_status(o)

    tt, cm, cross, t_dry_step, t_dry_nom, slope_cross, dCmax_dt, cond_s_per_unit, snap_t, snap_R, snap_C, xi, n_row, times_out, R_out_t, Cpk, Cln, Chm, t_last_out, assert_blank = _locate_and_build(o)
    if not np.array_equal(times_out, np.arange(DT_OUT, t_last_out + 1e-9, DT_OUT)):
        raise SystemExit("[FATAL] delivery axis is not 60 s")

    header4 = (["时间\\到药材中心的距离"]
               + [float(round(rc, 1)) for rc in R_OUT_CM] + ["药材表面"])
    write_xlsx(res / "result4.xlsx", times_out, Cpk, header4, decimals=4, sheet="Sheet1")

    dmap, dmap2 = _write_result4_outputs(res, times_out, snap_C, snap_R, xi, n_row,
                                         Cpk, Cln, Chm, R_out_t)
```

```
764
765     table6, times6, vals_end, rows6, surf_vals, i_near =
    _build_table6(
766         res, tt, cross, snap_t, snap_C, snap_R, xi, t_dry_step
    )
767
768     _write_cmax_traces(res, tt, cm, o, t_dry_nom)
769     et = o["audit_energy_terms"]["values"]
770     audit_summary = build_audit_summary(et, o)
771     (res / "q4_audit_summary.json").write_text(
772         json.dumps(audit_summary, ensure_ascii=False, indent
    =2), encoding="utf-8")
773
774     j, k, prof = write_xi_profile(R_out_t, n_row, res, snap_C,
     times_out, xi)
775
776     chk = monotonic_check(assert_blank, cm, o, prof,
    t_dry_step)
777
778     wall = time.time() - t0
779     g5_tiering = {"trigger": "G5（§15.1）口径降级", "
    registered_at": "2026-09-11",
780                   "note": "缺 results/q4_convergence.json  分
    层声明未登记（须先跑 verify_q4_convergence.py）"}
781     _cvp = res / "q4_convergence.json"
782     if _cvp.exists():
783         _cvj = json.loads(_cvp.read_text(encoding="utf-8"))
784         _A = _cvj["A_grid_independence"]
785         _d4 = _A["diff_vs_N800"]["400"]
786         _dt = _cvj["B_time_convergence"]["diff_vs_finest"]["I1
    .25_II5"]
787         g5_tiering = {
788             "trigger": "G5（§15.1）口径降级",
789             "registered_at": "2026-09-11",
790             "source": "results/q4_convergence.json
```

```
A_grid_independence.g5_disclosure",
791             "rule": ("交付表（result4.xlsx / 表 6）的有效位数
按实际离散不确定度分层："
792                      "t ≥ 1 h 支撑 4 位小数；t < 1 h（前 59
行）不支撑第 4 位。"),
793             "evidence": {
794                 "t_ge_3600s_grid_N400_vs_N800": _d4["
max_abs_t_ge_3600s"],
795                 "t_ge_3600s_dt_prod_vs_finest": _dt["
max_abs_t_ge_3600s"],
796                 "t_lt_3600s_max": _d4["max_abs"],
797                 "t_lt_3600s_max_at": "t=60 s、药材表面列",
798                 "layer2_tol_kgkg": _A["layer2_tol_kgkg"],
799             },
800             "reason": ("t<1 h 的最大差出现在 t=60 s 表面列（
%.3e）；δ(60 s)=2.507×10□² cm "
801                        "≈ 5.01Δr@N=400  初始边界层已被本生产配
置覆盖（不是「不可分辨」）；"
802                        "加密序列在该窗口仍以 p≈2.35 收敛、N
=800 全程仅 %.1f s  继续加密可望"
803                        "达标（按 p 外推 N=1600 ≈ 2.4×10□⁵），
本轮未做。"
804                        % (_d4["max_abs"], _A["wall_s"]["800"])
),
805             "statement": ("论文与 result4.xlsx 须并列声明：t ≥
 1 h 的 4 位小数由 ② 层线"
806                           "（5×10□⁵）支撑；t < 1 h 不支撑第 4
位。"
807                           "详见 results/q4_convergence.json
g5_disclosure。"),
808         }
809     summary = build_summary(
810         Cpk,
811         audit_summary,
812         chk,
```

```
        cond_s_per_unit,
        cross,
        dmap,
        dmap2,
        g5_tiering,
        inputs,
        j,
        k,
        o,
        rchk,
        res,
        scripts,
        slope_cross,
        surf_vals,
        t_dry_nom,
        t_dry_step,
        table6,
        times_out,
        tt,
        wall,
    )
    (res / "results4.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")

    write_manifest(inputs, k, res, scripts, summary, wall)

    print_summary(
        Cpk,
        chk,
        cond_s_per_unit,
        cross,
        dmap,
        dmap2,
        k,
```

```
        o,
        res,
        rows6,
        slope_cross,
        t_dry_nom,
        t_dry_step,
        table6,
        times6,
        times_out,
        vals_end,
        wall,
    )
    if not all(chk.values()):
        raise SystemExit("[FATAL] physical self-check failed")
    return 0

def build_audit_summary(et, o):
    audit_summary = dict(
        moisture=(dict(max_rel_resid=o["mass_audit"]["max_rel"],
                       argmax_t_s=o["mass_audit"]["worst_t"],
                       n_violations=o["mass_audit"]["n_ge_1e8"],
                       first_step=o["mass_audit"]["first"],
                       criterion="相对残差 ≤ 1e-8（> 1e-10 即停）",
                       identity="Σ_i W_i^ξ(C_i^{n+1}-C_i^n) = Δt·h_m(C_a^{n+1}-C_N^{n+1})/R^{n+1}")),
        moisture_closure=dict(m0=o["mass"]["m0"], m1=o["mass"]["m1"],
                              ratio=o["mass"]["ratio"],
                              cum_flux=o["mass"]["cum_flux"],
                              rel_err=o["mass"]["rel_err"]),
        energy=dict(form=o["audit"]["form"], applicable=o["audit"]["applicable"],
```

```
                    n_steps=o["audit"]["n"],
                    max_rel_self=o["audit"]["max_rel_global"],
                    max_rel_envelope=o["audit"]["max_rel_env"
],
                    E_scale_max=o["audit"]["E_scale_max"],
                    max_abs=o["audit"]["max_abs"], n_ge_1e8=o[
"audit"]["n_ge_1e8"],
                    n_transient_steps=o["audit"]["n_transient"
],
                    max_rel_transient=o["audit"]["
max_rel_transient"],
                    max_rel_envelope_transient=o["audit"]["
max_rel_env_transient"],
                    max_abs_transient=o["audit"]["
max_abs_transient"],
                    samples=o["audit"]["samples"],
                    note=o["audit"]["note"]),
        energy_terms_B=dict(header=o["audit_energy_terms"]["
header"],
                            n=o["audit_energy_terms"]["n"],
                            max_abs_dH=float(np.abs(et[:, 1]).
max()) if et.size else 0.0,
                            max_abs_cap_account=float(np.abs(
et[:, 2]).max()) if et.size else 0.0,
                            max_abs_geo_term=float(np.abs(et
[:, 3]).max()) if et.size else 0.0,
                            max_abs_boundary_flux=float(np.abs
(et[:, 4]).max()) if et.size else 0.0,
                            note=("写法 B 三分项（§6.7）：总焓
变 / 热容变化账 / 几何项 / "
                                  "边界导热净流入；几何项只在
此处出现，绝不进右端通量。")),
        T_ref_invariance=dict(
            note=("§6.7 末的代码级断言：对同一 (T,C) 用 'T' 与
 'T+273.15' 各算一次"
```

```
                    "(Q4-E-A) 左端，两者之差 < 1e-12×|左端|；该断言能一次性抓住「误用乘积"
                    "形式」这一类错误。本文件在 verify_q4_audit.py 中实现并落盘。")),
     )
     return audit_summary

def write_xi_profile(R_out_t, n_row, res, snap_C, times_out, xi):
     xi_cols = np.round(np.arange(0.0, 1.0 + 1e-12, 0.005), 10)
     if abs(round(0.005 / (1.0 / N_MAIN)) * (1.0 / N_MAIN) - 0.005) < 1e-12:
          j = np.rint(xi_cols / (1.0 / N_MAIN)).astype(int)
          if not np.allclose(xi[j], xi_cols, atol=1e-12):
               raise SystemExit("[FATAL] mother column xi is not a node subset")
          prof_cols = j
     else:
          raise SystemExit("[FATAL] N=%d dxi does not divide 0.005" % N_MAIN)
     prof = snap_C[1:n_row + 1][:, prof_cols]
     np.savez_compressed(res / "q4_xi_profile.npz", t_s=times_out, xi=xi_cols,
                         C_kgkg=prof, R_m=R_out_t)
     with open(res / "q4_xi_profile.csv", "w", encoding="utf-8") as f:
          f.write("t_s,R_m," + ",".join("xi=%.3f" % v for v in xi_cols) + "\n")
          for k in range(prof.shape[0]):
               f.write("%d,%.10f," % (int(round(times_out[k])), R_out_t[k])
                       + ",".join("%.10f" % v for v in prof[k]) + "\n")
     return j, k, prof

```

```
def monotonic_check(assert_blank, cm, o, prof, t_dry_step):
    MONO_TOL = 1e-12
    dcm = np.diff(cm)
    n_rise = int(np.count_nonzero(dcm > MONO_TOL))
    chk = {
        "no_floor_activation": bool(o["floor"]["count"] == 0),
        "picard_all_converged": bool(o["picard"]["
  n_not_converged"] == 0),
        "no_extremum_violation": bool(len(o["violations"]) ==
  0),
        "mass_audit_le_1e-8": bool(o["mass_audit"]["max_rel"]
  <= 1e-8),
        "mass_closure_le_1e-8": bool(o["mass"]["rel_err"] <= 1
  e-8),
        "R_monotone_within_bounds": bool(o["R_min_seen"] >=
  R_MIN - 1e-12),
        "t_dry_above_mixing_lower_bound": bool(t_dry_step >
  30083.5),
        "C_max_monotone_nonincreasing": bool(n_rise == 0),
        "C_radial_nonincreasing_all_window": bool(
            np.all(np.diff(prof, axis=1) <= 1e-9)),
        "delivered_columns_inside_domain": bool(assert_blank
  == 0),
        "t_dry_within_R_data_window": bool(t_dry_step <=
  R_FREEZE_T),
        "no_r_extrapolation": bool(t_dry_step <= R_FREEZE_T),
    }
    return chk

def build_summary(
    Cpk,
    audit_summary,
    chk,
    cond_s_per_unit,
    cross,
```

```
    dmap,
    dmap2,
    g5_tiering,
    inputs,
    j,
    k,
    o,
    rchk,
    res,
    scripts,
    slope_cross,
    surf_vals,
    t_dry_nom,
    t_dry_step,
    table6,
    times_out,
    tt,
    wall,
):
    summary = {
        "meta": {
            "question": "2026 国赛 A 题 问题 4（收缩域，材料坐标 ξ = r/R(t)）",
            "generated_at": datetime.now().isoformat(timespec="seconds"),
            "seed": SEED, "python": platform.python_version(), "numpy": np.__version__,
            "wall_s": wall, "solver_wall_s": o["elapsed_s"],
            "config": {
                "N": N_MAIN, "dxi": 1.0 / N_MAIN, "dt_I_s": DT_I, "dt_II_s": DT_II,
                "t_switch_s": T_FREEZE_S, "dt_out_s": DT_OUT,
                "R_mode": "shrink（附件 2 分段线性，路线 (a)）",
                "geo_layer": "nplus1（1/R^{n+1,2}，步内常数、
```

```
    不进 Picard）",
978                 "face_rule": "arith（界面系数 = 相邻节点算术平
    均）",
979                 "props": "附录 4", "bc": "robin（口径 A）",
980                 "T_mode": "prescribed（T := T_a(t)，主口径）",
981                 "picard_tol": 1e-12, "picard_max": 100,
982                 "t_dry_criterion": "C_max(t) ≤ %g kg/kg（首个
    满足的内部步）" % C_DRY,
983                 "margin_s": MARGIN_S, "t_cap_s": T_CAP_S,
984                 "n_steps": o["n_steps"], "t_final_s": o["
    t_final"],
985                 "status": o["status"],
986             },
987         },
988         "inputs_sha256": inputs,
989         "script_sha256": scripts,
990         "model_choices": {
991             "equations": "∂C/∂t|_ξ = (1/(R²ξ))∂/∂ξ(ξD∂C/∂ξ)；"
992                          "ρ(C)c_p(C)∂T/∂t|_ξ = (1/(R²ξ))∂/∂ξ(
    ξk∂T/∂ξ)（Q4-9 / Q4-12b）",
993             "props_appendix4": ("ρ=760+90C；c_p=1850+2150C/(C
    +1)；k=0.12+0.20C/(C+1)；"
994                                 "D=4.2e-4·e^(-0.30/C)·e
    ^(-3850/T)"),
995             "D_T_unit_declaration": (" D 的指数项 e^(-3850/T)
    中 T 必须用热力学温度 K"
996                                      "（输入 28 ℃ 用 301.15 K
    ）；输出阶段才转 ℃"),
997             "h_hm_source": ("附录 4 未给  沿用附录 2 的 h=25 W
    /(m²·K)、h_m=8e-7 m/s"
998                             "（唯一可溯来源的推断，Q4-A4，非题
    给）"),
999             "R_source": "附件 2 的 145 行半径序列，分段线性插
    值；t>259200 s 冻结 1.198 cm",
1000            "driver": "附件 1 分段线性插值 [0,14400] s；t
```

```
    >14400 s 冻结 T_a=50.165 ℃、"
1001                    "C_a=0.04986 kg/kg",
1002            "initial": "T=28 ℃、C=2.55 kg/kg 均匀，t=0 起算
    （不承接 Q3 终态）",
1003            "D_eval": ("对数空间 log D = log(4.2e-4) − 0.30/C
    − 3850/T(K)，再 exp；"
1004                    "防卫下界 D_floor=1e-25 m²/s（非干扰：
    低于双精度相对分辨率）"),
1005            "D_floor": D_FLOOR,
1006            "picard": ("单列 Picard（预设 T 口径）；累积项右端
    恒取 n 层（R3 条款）、"
1007                    "边界驱动取 n+1 层、几何因子取 n+1 层且
    不进 Picard"),
1008            "mapping": "ξ 向 PCHIP（主）+ 线性（对照）+ FV-
    Hermite（互校）；表面/中心列不插值",
1009            "termination": "首个 C_max ≤ %g 的内部步 + %g s 余
    量" % (C_DRY, MARGIN_S),
1010        },
1011        "radius_check": rchk,
1012        "attachment2_key_values": {
1013            "R0_cm": 2.0, "R_final_cm": R_MIN_CM, "t_final_s":
     R_FREEZE_T,
1014            "R_min_m": float(radius_at(np.array([R_FREEZE_T]),
     "shrink")[0]),
1015            "note": "逐位照抄附件 2；不做四舍五入",
1016        },
1017        "attachment1_key_values": {
1018            "T_a_min_C": float(att1()[1].min()), "T_a_max_C":
    float(att1()[1].max()),
1019            "C_a_min": float(att1()[2].min()), "C_a_max":
    float(att1()[2].max()),
1020            "freeze_values": {"T_a_C": TA_FREEZE_C, "C_a":
    CA_FREEZE},
1021        },
1022        "t_dry": {
```

```
            "t_step_s": float(t_dry_step),
            "t_step_h": float(t_dry_step / 3600.0),
            "t_step_d": float(t_dry_step / 86400.0),
            "t_nominal_linear_s": float(t_dry_nom),
            "t_nominal_linear_h": float(t_dry_nom / 3600.0),
            "t_nominal_hermite_s": cross["t_hermite"],
            "bracket": {"t_before_s": cross["t_before"], "
C_max_before": cross["cmax_before"],
                        "t_after_s": cross["t_after_s"], "
C_max_after": cross["cmax_after"],
                        "width_s": (cross["t_after_s"] - cross
["t_before"])
                        if cross["t_before"] is not None else
None},
            "localization_error_s": cross["localization_error"
],
            "dCmax_dt_at_crossing": slope_cross,
            "condition_number_s_per_kgkg": cond_s_per_unit,
            "uncertainty": build_t_dry_uncertainty(
                res, t_dry_step, t_dry_nom, cross["t_hermite"
],
                cross["localization_error"], cond_s_per_unit,
DT_II),
            "t_end_s": o["t_end"], "margin_s": MARGIN_S, "
status": o["status"],
            "n_steps_to_dry": cross["n_star"],
            "mixing_lower_bound_s": 30083.5,
            "mixing_lower_bound_check": bool(t_dry_step >
30083.5),
        },
        "cross_validation_attachment2": {
            "note": ("§9.3：模型量 't_dry' 与数据量「半径停止
变化」的四个平台首现时刻"
                     "并列报告，两者口径不同、不构成等式。"),
            "T_R_first_appearance_s": {"1.201": 106200.0, "
```

```
    1.200": 126000.0,
1048                                        "1.199": 208800.0, "
    1.198": 241200.0},
1049            "T_R_first_appearance_h": {"1.201": 29.5, "1.200":
     35.0,
1050                                        "1.199": 58.0, "1.198":
     67.0},
1051            "consistency_rule": "t_dry ∈ [35.0 h, 67.0 h]  一
    致；□ [29.5 h, 72.0 h]  红旗",
1052            "verdict": ("一致（落在尺寸基本定型的窗口内）"
1053                        if 35.0 <= t_dry_step / 3600.0 <= 67.0
     else
1054                        ("边缘/红旗：t_dry = %.3f h" % (
    t_dry_step / 3600.0))),
1055            "t_dry_h": float(t_dry_step / 3600.0),
1056        },
1057        "result4": {
1058            "path": "results/result4.xlsx", "sheet": "Sheet1",
1059            "shape": [int(Cpk.shape[0]), int(Cpk.shape[1])],
1060            "t_range_s": [float(times_out[0]), float(times_out
    [-1])],
1061            "t_step_s": DT_OUT, "r_cm": R_OUT_CM.tolist(),
1062            "columns_total": int(Cpk.shape[1] + 1),
1063            "decimals": 4,
1064            "unrounded_npz": "results/result4_full.npz",
1065            "unrounded_csv": "results/result4_full.csv",
1066            "wide_backup": "results/result4_wide.xlsx",
1067            "template_cells": {
1068                "A1": "时间\\到药材中心的距离",
1069                "A2": 60, "A3": 120,
1070                "B1": 0.0, "C1": 0.1, "D1": 0.2, "last": "药材
    表面",
1071                "sheet_name": "Sheet1",
1072            },
1073        },
```

```
        "tables": {"table6": table6},
        "tables6_surface_column_detail": [
            {"t_h": th, "t_s": th * 3600.0, "C_surface": cs, "R_m": rm,
             "R_cm": rm * 100.0}
            for (th, cs, rm) in surf_vals],
        "cmax_trajectory": {
            "npz": "results/q4_cmax_trajectory.npz", "n_internal_steps": int(tt.size),
            "columns": ["t_s", "C_max", "argmax_i", "C_center", "C_surface", "C_min",
                        "R_m"],
            "dt_I_s": DT_I, "dt_II_s": DT_II, "bracket_width_s": DT_II,
            "note": "逐内部步（§9.2）；60 s 采样仅供参考、不用于定位",
        },
        "picard": {k: v for k, v in o["picard"].items()},
        "residual_inf": o["residual"],
        "picard_step_dmax": o["picard_step_dmax"],
        "floor_monitor": o["floor"],
        "extremum_principle": {"n_violations": len(o["violations"]),
                               "violations": o["violations"][:10],
                               "bounds": o["extremum_bounds"],
                               "declared": {"T_C": [28.0, 50.246],
                                            "C_kgkg": [0.01963, 2.55]}},
        "audit": audit_summary,
        "physical_checks": chk,
        "precision_tiering": g5_tiering,
        "mapping_uncertainty": {
            "csv": "results/q4_mapping_uncertainty.csv",
```

```
            "max_abs_diff_pchip_linear": float(np.nanmax(dmap)),
            "max_abs_diff_pchip_fvhermite": float(np.nanmax(dmap2)),
            "per_column_max": {("%.1f" % rc): float(np.nanmax(dmap[:, j]))
                               for j, rc in enumerate(R_OUT_CM)},
            "note": ("PCHIP（主）与线性（对照）之差即映射不确定度（§8.4 第 2 条）；"
                     "FV-Hermite 为节点导数取自 FV 通量差商的自实现，用于与 PCHIP 互校。"),
        },
        "delivery_form_percent": {
            "R_min_over_R0_sq": float((R_MIN / R0) ** 2),
            "note": "收缩使体积（单位长度）缩到 (R_min/R_0)² = 0.359 倍",
        },
    }
    return summary

def write_manifest(inputs, k, res, scripts, summary, wall):
    manifest = {
        "seed": SEED,
        "created_at": datetime.now().isoformat(timespec="seconds"),
        "question": "2026 国赛 A 题 问题 4",
        "inputs_sha256": inputs,
        "script_sha256": scripts,
        "config": summary["meta"]["config"],
        "model_choices": summary["model_choices"],
        "wall_s": wall,
        "outputs": {},
        "uncertainty": {
            "t_dry": summary["t_dry"]["uncertainty"],
```

```
            "mapping": summary["mapping_uncertainty"],
            "grid_and_time": ("见 results/q4_convergence.json（Δξ）与 "
                              "results/q4_dt_convergence.json（Δt）"),
            "model_error": ("见 results/q4_error_budget.json（E1–E6）；"
                            "E1 的 0-D 集总对照为必做项"),
            "note": "本块只登记口径；数值在验证脚本产出后回填",
        },
        "reproduce": "python src/run_q4.py（从项目根以相对路径可跑；seed=42 固定）",
    }
    OUTPUTS = ["result4.xlsx", "result4_wide.xlsx", "result4_full.npz",
               "result4_full.csv", "q4_table6.csv", "q4_xi_profile.npz",
               "q4_xi_profile.csv", "q4_cmax_trajectory.npz",
               "q4_cmax_crossing_window.csv", "q4_cmax_60s.csv",
               "q4_mass_audit.csv", "q4_audit_summary.json",
               "q4_mapping_uncertainty.csv", "results4.json"]
    for f in OUTPUTS:
        p = res / f
        manifest["outputs"][f] = sha256(p) if p.exists() else None
    def _load(fn):
        p = res / fn
        return json.loads(p.read_text(encoding="utf-8")) if p.exists() else None
    cv, sen, eb, cmpj, bl = (_load("q4_convergence.json"), _load("q4_sensitivity.json"),
                             _load("q4_error_budget.json"), _load("q4_compare.json"),
```

```
                                _load("q4_baseline.json"))
    if cv:
        g = cv["A_grid_independence"]["diff_vs_N800"]["400"]
        d = cv["B_time_convergence"]["diff_vs_finest"]["I1.25
    _II5"]
        manifest["uncertainty"]["grid_and_time"] = dict(
            grid_N400_vs_N800_t_ge_3600s_kgkg=g["
    max_abs_t_ge_3600s"],
            grid_N400_vs_N800_fullwindow_kgkg=g["max_abs"],
            grid_fullwindow_dominated_by=("t=60 s 的初始窗口（
    δ(60 s)=2.507×10□² cm ≈ "
                                          "5.01Δr@N=400；本生
    产配置 N=400 在该窗口未达 ② 层线）"),
            dt_prod_vs_finest_t_ge_3600s_kgkg=d["
    max_abs_t_ge_3600s"],
            dt_prod_vs_finest_fullwindow_kgkg=d["max_abs"],
            layer2_tol_kgkg=5e-5,
            verdict=("t ≥ 1 h：Δξ 与 Δt 均 < 5×10□⁵（② 层
    线）；"
                     "t < 1 h 不支撑第 4 位（G5 口径降级，已在
     results4.json 分层声明）"),
            t_dry_grid_spread_h=cv["A_grid_independence"]["
    t_dry_spread_h"],
            t_dry_dt_spread_h=cv["B_time_convergence"]["
    t_dry_spread_h"])
    if sen:
        manifest["uncertainty"]["sensitivity"] = dict(
            h_hm_3x3=sen["h_hm_2d"]["grid"],
            h_hm_3x3_spread_pct=sen["h_hm_2d"]["spread_pct"],
            h_degeneracy=sen["h_hm_2d"].get("degeneracy",
                "在预设 T 口径下 'h' 恒无影响（能量方程不求
    解） 3×3 的 9 组退化为 3 个不同值；"
                "本模型对 'h' 的灵敏度低于真实情况（§11.4）。"
    ),
            driver_scenarios=sen["driver_scenarios"])
```

```
    if eb:
        manifest["uncertainty"]["model_error"] = {
            k: eb[k] for k in ("E1_latent_heat", "E2_end_face", "E3_h_hm",
                               "E4_discrete", "E5_R_interp", "E6_driver",
                               "interval_statement", "reporting_rule")}
    if cmpj:
        manifest["uncertainty"]["comparison"] = {
            k: cmpj[k] for k in cmpj if k.startswith("group")}
    if bl:
        manifest["uncertainty"]["baseline_groups"] = {
            "group1_verdict": bl["group1_degenerate_regression"].get("group1_verdict"),
            "group2": bl["group2_well_mixed_lower_bound"],
            "group3b": {k: v for k, v in bl["group3b_bessel_dirichlet"].items()
                        if k in ("D", "analytic", "targets", "pass")}}
    manifest["uncertainty"]["backfilled_at"] = datetime.now().isoformat(timespec="seconds")
    (res / "run_manifest_q4.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")

def print_summary(
    Cpk,
    chk,
    cond_s_per_unit,
    cross,
    dmap,
    dmap2,
    k,
    o,
```

```
    res,
    rows6,
    slope_cross,
    t_dry_nom,
    t_dry_step,
    table6,
    times6,
    times_out,
    vals_end,
    wall,
):
    print()
    print("[summary] t_dry（操作口径，首个内部步）= %.6f s = %.4f h = %.4f d"
          % (t_dry_step, t_dry_step / 3600.0, t_dry_step / 86400.0))
    print("[summary] 标称（步内线性插值）= %.6f s = %.4f h；Hermite = %s；定位差 %.3e s"
          % (t_dry_nom, t_dry_nom / 3600.0,
             ("%.6f s" % cross["t_hermite"]) if cross["t_hermite"] else "None",
             cross["localization_error"] or 0.0))
    print("[summary] 括住区间 [%s, %s] s（宽 %s s）；跨越点 dC_max/dt = %.6e kg/(kg·s)；"
          "条件数 %.6e s/(kg/kg)"
          % (cross["t_before"], cross["t_after_s"], DT_II,
             slope_cross, cond_s_per_unit))
    print("[summary] t_end = %s s；t_final = %.6f s；步数 %d"
          % (o["t_end"], o["t_final"],
             o["n_steps"]))
    print("[summary] 表 6（kg/kg）：")
    print("          " + "  ".join("%10s" % h for h in table6["header"]))
    for i, th in enumerate(times6):
```

```
        print("   %6g h " % th + "  ".join("%10.4f" % v for v in rows6[i]))
    print("   %6s   " % "结束" + "  ".join("%10.4f" % v for v in vals_end))
    print("[summary] result4.xlsx: %d 行 × %d 列（含时间列），t ∈ [%g, %g] s，r ∈ [0, 1.1] cm + 药材表面"
          % (Cpk.shape[0], Cpk.shape[1] + 1, times_out[0], times_out[-1]))
    print("[summary] Picard 均值 %.3f 中位 %.1f 最大 %d 未收敛 %d；直方 %s"
          % (o["picard"]["mean_iters"], o["picard"]["median_iters"],
             o["picard"]["max_iters"], o["picard"]["n_not_converged"],
             dict(zip(o["picard"]["hist_iters"], o["picard"]["hist_counts"]))))
    print("[summary] 水分审计 max 相对残差 %.3e（worst t=%.1f s）；闭合相对误差 %.3e"
          % (o["mass_audit"]["max_rel"], o["mass_audit"]["worst_t"] or -1.0,
             o["mass"]["rel_err"]))
    print("[summary] 能量审计（写法 A）：预设定 T 口径不适用审计判据；"
          "自尺度 max %.3e（退化，仅参考）；包络尺度 max %.3e（判据位）；"
          "升温段包络尺度 max %.3e（= 预设口径的热失衡诊断量）"
          % (o["audit"]["max_rel_global"], o["audit"]["max_rel_env"],
             o["audit"]["max_rel_env_transient"]))
    print("[summary] 映射不确定度 max：PCHIP−线性 %.3e；PCHIP−FV−Hermite %.3e"
          % (np.nanmax(dmap), np.nanmax(dmap2)))
    print("[summary] 物理自检：" + "；".join("%s=%s" % (k, "PASS" if v else "FAIL")
```

```
                                        for k, v in chk.items()))
    print("[summary] 总耗时 %.1f s（求解 %.1f s）；产物已写 %s" % (wall, o["elapsed_s"], res))


if __name__ == "__main__":
    sys.exit(main())
from __future__ import annotations

import math
import os
import sys
import time
from pathlib import Path

import numpy as np
from scipy.linalg.lapack import dgtsv

_OUT = sys.stdout
if hasattr(_OUT, "reconfigure"):
    _OUT.reconfigure(encoding="utf-8", errors="replace")

_BASE = Path(__file__).resolve().parent.parent

RHO_A4, RHO_B4 = 760.0, 90.0
CP_A4, CP_B4 = 1850.0, 2150.0
K_A4, K_B4 = 0.12, 0.20
D0_AP4, BETA_AP4, E_D_AP4 = 4.2e-4, 0.30, 3850.0

A2_RHO, A2_CP, A2_K = 820.0, 2600.0, 0.36
A2_D0, A2_BETA = 7.0e-9, 0.89
LOG_A2_D0 = math.log(A2_D0)

H_DEFAULT = 25.0
```

```
HM_DEFAULT = 8.0e-7

R_CM = 2.0
R0 = R_CM / 100.0
R_MIN_CM = 1.198
R_MIN = R_MIN_CM / 100.0
LENGTH_CM = 25.0
T_INIT_C = 28.0
C_INIT = 2.55

T_FREEZE_S = 14400.0
TA_FREEZE_C = 50.165
CA_FREEZE = 0.04986
TA_MAX_C = 50.246
CA_MIN = 0.01963

R_FREEZE_T = 259200.0
R_FREEZE = R_MIN

C_DRY = 0.15
MARGIN_S = 10800.0
T_CAP_S = 14.0 * 86400.0

D_FLOOR = 1.0e-25
LOG_D_FLOOR = math.log(D_FLOOR)
C_UNDERFLOW = 0.30 / 745.1332
LOG_D0_AP4 = math.log(D0_AP4)

D_AP4_2P55_323K = 2.5002509122378996e-9

def find_data_dir() -> Path:
    if os.environ.get("MM_DATA_DIR"):
        return Path(os.environ["MM_DATA_DIR"])
    for _c in (_BASE, _BASE / "A题", _BASE / "附件", _BASE.
   parent / "A题",
```

```
                Path.cwd() / "A题", Path.cwd()):
        if (_c / "附件" / "附件1.xlsx").is_file():
            return _c
    raise SystemExit(
        "未找到 A 题数据目录（须含 附件/附件1.xlsx）：请设置环境变量 MM_DATA_DIR 指向该目录，"
        "或把「A题」目录放到项目根/项目同级。"
    )


_DATA_DIR = str(find_data_dir())
ATT1 = str(Path(_DATA_DIR) / "附件" / "附件1.xlsx")
ATT2 = str(Path(_DATA_DIR) / "附件" / "附件2.xlsx")
ATT3_DIR = Path(_DATA_DIR) / "附件" / "附件3"
RESULT4_TEMPLATE = str(ATT3_DIR / "result4.xlsx")
PDF = str(Path(_DATA_DIR) / "A题.pdf")

_ATT1_CACHE = None
_ATT2_CACHE = None
_DRIVER_CACHE = {}


def att1():
    global _ATT1_CACHE
    if _ATT1_CACHE is None:
        import openpyxl
        wb = openpyxl.load_workbook(ATT1, data_only=True)
        ws = wb[wb.sheetnames[0]]
        rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[0] is not None]
        t = np.array([float(r[0]) for r in rows])
        T = np.array([float(r[1]) for r in rows])
        C = np.array([float(r[2]) for r in rows])
        _ATT1_CACHE = (t, T, C)
    return _ATT1_CACHE
```

```


def att2():
    global _ATT2_CACHE
    if _ATT2_CACHE is None:
        import openpyxl
        wb = openpyxl.load_workbook(ATT2, data_only=True)
        ws = wb[wb.sheetnames[0]]
        rows = [r for r in ws.iter_rows(min_row=2, values_only
   =True) if r[0] is not None]
        t = np.array([float(r[0]) for r in rows])
        R_cm = np.array([float(r[1]) for r in rows])
        _ATT2_CACHE = (t, R_cm / 100.0)
    return _ATT2_CACHE


def driver_table():
    global _DRIVER_CACHE
    if not _DRIVER_CACHE:
        t, T, C = att1()
        dt_s = float(t[1] - t[0])
        nseg = int(round(t[-1] / dt_s))
        if abs(t[0]) > 1e-12 or abs(t[-1] - T_FREEZE_S) > 1e
   -9:
            raise ValueError("附件 1 时间轴不是 0…14400 s（步
   长 %g）" % dt_s)
        _DRIVER_CACHE = dict(dt_s=dt_s, nseg=nseg, T=T.copy(),
    C=C.copy(),
                             inv_dt=1.0 / dt_s, Ta_max=float(T
   .max()),
                             Ca_min=float(C.min()), Ca_max=
   float(C.max()))
    return _DRIVER_CACHE


```

```
def driver_at(t, interp="linear"):
    if t >= T_FREEZE_S:
        return TA_FREEZE_C, CA_FREEZE
    d = driver_table()
    x = t * d["inv_dt"]
    i = int(x)
    if i >= d["nseg"]:
        i = d["nseg"] - 1
    if interp == "step":
        return float(d["T"][i]), float(d["C"][i])
    f = x - i
    T = float(d["T"][i] + (d["T"][i + 1] - d["T"][i]) * f)
    C = float(d["C"][i] + (d["C"][i + 1] - d["C"][i]) * f)
    return T, C


def radius_at(t, mode="shrink"):
    arr = np.asarray(t, float)
    if mode == "const":
        out = np.full(arr.shape, R0)
    elif mode == "pchip":
        from scipy.interpolate import PchipInterpolator
        tt, RR = att2()
        out = PchipInterpolator(tt, RR, extrapolate=False)(np.
   clip(arr, 0.0, R_FREEZE_T))
        out = np.where(arr <= 0.0, R0, out)
        out = np.where(arr >= R_FREEZE_T, R_FREEZE, out)
    else:
        tt, RR = att2()
        out = np.interp(arr, tt, RR)
        out = np.where(arr <= 0.0, R0, out)
        out = np.where(arr >= R_FREEZE_T, R_FREEZE, out)
    return out if np.ndim(t) else float(out)


```

```
def radius_check(mode="shrink", n=20001):
    tt = np.linspace(0.0, R_FREEZE_T * 1.5, n)
    RR = radius_at(tt, mode)
    return dict(R_min=float(RR.min()), R_max=float(RR.max()),
                mono_nonincreasing=bool(np.all(np.diff(RR) <= 1e-15)),
                monotone_ok=bool(RR.min() >= R_MIN - 1e-12 and RR.max() <= R0 + 1e-12),
                n_samples=int(n))


def default_params():
    return dict(
        RHO_A=RHO_A4, RHO_B=RHO_B4, CP_A=CP_A4, CP_B=CP_B4, K_A=K_A4, K_B=K_B4,
        D0=D0_AP4, BETA=BETA_AP4, E_D=E_D_AP4, LOG_D0=LOG_D0_AP4,
        A2_RHO=A2_RHO, A2_CP=A2_CP, A2_K=A2_K, A2_D0=A2_D0, A2_BETA=A2_BETA,
        LOG_A2_D0=LOG_A2_D0,
        H=H_DEFAULT, HM=HM_DEFAULT,
        props="app4",
        D_CONST=None,
        RHO_SCALE=1.0, CP_SCALE=1.0,
        T_INIT=T_INIT_C, C_INIT=C_INIT,
    )


def rho_of_C(C, P):
    if P["props"] == "app2":
        return np.full_like(C, P["A2_RHO"])
    return P["RHO_A"] + P["RHO_B"] * C


def cp_of_C(C, P):
```

```
    if P["props"] == "app2":
        return np.full_like(C, P["A2_CP"])
    return P["CP_A"] + P["CP_B"] * C / (C + 1.0)


def omega_of_C(C, P):
    sc = P.get("RHO_SCALE", 1.0) * P.get("CP_SCALE", 1.0)
    base = rho_of_C(C, P) * cp_of_C(C, P)
    return base if sc == 1.0 else base * sc


def k_of_C(C, P):
    if P["props"] == "app2":
        return np.full_like(C, P["A2_K"])
    return P["K_A"] + P["K_B"] * C / (C + 1.0)


def logD_of_CT(C, T_C, P):
    if P.get("D_CONST") is not None:
        return np.full_like(np.asarray(C, float), math.log(P["
   D_CONST"]))
    if P["props"] == "app2":
        return P["LOG_A2_D0"] - P["A2_BETA"] / C
    T_K = T_C + 273.15
    return P["LOG_D0"] - P["BETA"] / C - P["E_D"] / T_K


class FloorMonitor:

    def __init__(self):
        self.count = 0
        self.events = []
        self.max_events = 20

    def note(self, mask, logD, C, T_C, t):
```

```
        n = int(np.count_nonzero(mask))
        if n == 0:
            return
        self.count += n
        Tc = np.asarray(T_C, float)
        for i in np.flatnonzero(mask):
            if len(self.events) >= self.max_events:
                break
            self.events.append(dict(t_s=float(t), i=int(i), C=float(C[i]),
                                    T_C=float(Tc[i]), logD=float(logD[i]),
                                    logD_floor=LOG_D_FLOOR))

    def to_dict(self):
        return dict(count=int(self.count), log_D_floor=LOG_D_FLOOR, D_floor=D_FLOOR,
                    c_underflow_threshold=C_UNDERFLOW,
                    note=("激活次数必须为 0（§5.3：极值原理给 C ≥ min(C_a)=%r，"
                          "是下溢点 %r 的 48.8 倍）。> 0 表示前提被推翻  停止并回报。"
                          % (CA_MIN, C_UNDERFLOW)),
                    first_events=self.events)


def eval_D(logD, monitor, C, T_C, t):
    if float(logD.min()) < LOG_D_FLOOR:
        monitor.note(logD < LOG_D_FLOOR, logD, C, T_C, t)
        logD = np.maximum(logD, LOG_D_FLOOR)
    return np.exp(logD)


def interface_gam(node, rule):
    if rule == "arith":
```

```
        return 0.5 * (node[:-1] + node[1:])
    if rule == "harm":
        a, b = node[:-1], node[1:]
        return 2.0 * a * b / (a + b)
    raise ValueError("未知界面系数规则 face_rule=%r" % (rule,))


def face_gam(kind, node, C, T_C, rule, P):
    if rule == "faceCT":
        Cf = 0.5 * (C[:-1] + C[1:])
        Tf = 0.5 * (T_C[:-1] + T_C[1:])
        if kind == "k":
            return k_of_C(Cf, P)
        return np.exp(np.maximum(logD_of_CT(Cf, Tf, P), LOG_D_FLOOR))
    return interface_gam(node, rule)


def build_grid(N):
    dxi = 1.0 / N
    xi = np.arange(N + 1) * dxi
    W = np.empty(N + 1)
    W[0] = dxi * dxi / 8.0
    W[1:N] = xi[1:N] * dxi
    W[N] = dxi / 2.0 - dxi * dxi / 8.0
    return xi, dxi, N, W


def make_geo(N, dxi, W):
    ii = np.arange(1, N)
    xip = ii * dxi + dxi / 2.0
    xim = ii * dxi - dxi / 2.0
    return dict(N=N, dxi=dxi, W=W,
                G1=xip / (dxi * W[1:N]),
```

```
                G2=xim / (dxi * W[1:N]),
                AXC=0.5 / W[0],
                SFC=(1.0 - dxi / 2.0) / (dxi * W[N]),
                SBND=1.0 / W[N])


def tdsolve(d, l, u, rhs):
    n = rhs.size
    x = dgtsv(l[1:], d, u[:-1], rhs.reshape(n, 1), overwrite_b
   =1)[3]
    return x[:, 0]


def resid_of(m, phi):
    d, l, u, rhs = m
    return banded_residual(d, l, u, phi, rhs)


def banded_residual(d, l, u, phi, rhs):
    res = d * phi
    res[:-1] += u[:-1] * phi[1:]
    res[1:] += l[1:] * phi[:-1]
    res -= rhs
    return float(np.abs(res).max())


def assemble(g, geo, gam_face, cap, om_extra_h, R_new, bc,
   rhs_old, driver_new):
    N = geo["N"]
    d = np.empty(N + 1); l = np.zeros(N + 1); u = np.zeros(N +
    1)
    a1 = g * geo["G1"] * gam_face[1:N] / cap[1:N]
    a2 = g * geo["G2"] * gam_face[0:N - 1] / cap[1:N]
    d[1:N] = 1.0 + a1 + a2; u[1:N] = -a1; l[1:N] = -a2
    a0 = g * geo["AXC"] * gam_face[0] / cap[0]
```

```
    d[0] = 1.0 + a0; u[0] = -a0
    alpha = g * geo["SFC"] * gam_face[N - 1] / cap[N]
    beta = g * geo["SBND"] * R_new * om_extra_h / cap[N]
    d[N] = 1.0 + alpha + beta; l[N] = -alpha
    rhs = rhs_old.copy()
    rhs[N] += beta * driver_new
    if bc == "dirichlet":
        d[N] = 1.0; l[N] = 0.0; rhs[N] = driver_new
    rhs_keep = rhs.copy()
    phi = tdsolve(d, l, u, rhs)
    return phi, (d, l, u, rhs_keep)


class Grow:

    def __init__(self, cap=4096):
        self.a = np.empty(cap)
        self.n = 0

    def append(self, v):
        if self.n == self.a.size:
            self.a = np.resize(self.a, self.a.size * 2)
        self.a[self.n] = v
        self.n += 1

    def array(self):
        return self.a[:self.n].copy()


def scenario_driver(scen, samples, const):
    s = np.asarray(samples, float)
    if scen == "freeze_last":
        return s.copy()
    if scen == "const":
        return np.full_like(s, float(const))
```

```
    tail = s[-60:]
    if scen == "tail_mean":
        return np.concatenate([s[:-60], np.full(60, float(tail.mean()))])
    if scen == "row_min":
        return np.concatenate([s[:-60], np.full(60, float(tail.min()))])
    if scen == "row_max":
        return np.concatenate([s[:-60], np.full(60, float(tail.max()))])
    raise ValueError("未知情景 %r" % (scen,))


def solve_q4(N=200, dt1=5.0, dt2=20.0, t_switch=T_FREEZE_S,
             R_mode="shrink", geo_layer="nplus1", face_rule="arith",
             props="app4", bc="robin", T_mode="prescribed", block_order="TC",
             d_const=None, t_cap=T_CAP_S, margin_s=MARGIN_S, c_dry=C_DRY,
             stop_on_dry=True, t_end_override=None, t_start_state=None,
             picard_tol=1e-12, picard_max=100,
             snap_every_s=60.0, snap_until_s=None, snap_end=None, snap_nodes=None,
             profile_times=(), cmax_track=True, checkpoints=(), audit=True,
             audit_series=False,
             driver_interp="linear", ta_scenario="freeze_last",
             ca_scenario="freeze_last", const_ta=TA_FREEZE_C, const_ca=CA_FREEZE,
             verbose=True, progress_every=0, **overrides):
    Ca_scen, N, P, Ta_scen, W, _inv_dt, _nseg, dxi, geo, t_wall0, use_frozen_tbl, xi = setup_params(N, R_mode, T_mode
```

```
    , bc, ca_scenario, const_ca, const_ta, d_const, dt1, dt2,
    geo_layer, overrides, props, snap_every_s, ta_scenario)
1627
1628     c_hi, c_lo, drv_at, t_hi, t_lo = prepare_drivers(Ca_scen,
    P, Ta_scen, _inv_dt, _nseg, driver_interp, use_frozen_tbl)
1629
1630     t = 0.0
1631     step = 0
1632     cum_flux = 0.0
1633     if t_start_state is not None:
1634         t = float(t_start_state["t"])
1635         step = int(t_start_state.get("n_steps", 0))
1636         T = np.array(t_start_state["T"], float)
1637         C = np.array(t_start_state["C"], float)
1638         cum_flux = float(t_start_state.get("cum_flux", 0.0))
1639         if T.size != N + 1:
1640             raise ValueError("检查点网格与 N=%d 不符" % N)
1641     else:
1642         T = np.full(N + 1, float(P["T_INIT"]))
1643         C = np.full(N + 1, float(P["C_INIT"]))
1644
1645     R_now = float(radius_at(t, R_mode))
1646     ta_old = float(drv_at(t)[0])
1647     floor_mon = FloorMonitor()
1648     viol = []
1649
1650     if snap_until_s is None and snap_end is None:
1651         snap_stop = float("inf")
1652     else:
1653         snap_stop = snap_until_s if snap_end is None else
    snap_end
1654     sn = np.arange(N + 1) if snap_nodes is None else np.
    asarray(snap_nodes, int)
1655     snap_t, snap_T, snap_C, snap_R = [], [], [], []
1656     if snap_every_s > 0:
```

```
        snap_next = (math.floor(t / snap_every_s + 1e-12) + 1)
 * snap_every_s
        if t == 0.0 and snap_stop >= 0.0:
            snap_t.append(0.0); snap_T.append(T[sn].copy())
            snap_C.append(C[sn].copy()); snap_R.append(R_now)
    else:
        snap_next = None

    tr = {k: (Grow() if cmax_track else None)
          for k in ("t", "cmax", "arg", "c0", "cR", "cmin")}
    ckpt, profiles = {}, {}
    prof_left = sorted(float(v) for v in profile_times)
    picard_iters = Grow()
    n_not_conv = 0
    res_max_T = res_max_C = 0.0
    dT_seen = dC_seen = 0.0

    aud = dict(n=0, max_abs=0.0, max_rel_global=None,
max_rel_perstep=0.0,
               max_rel_env=0.0, sum_abs=0.0,
               samples=[], peak_dE=0.0, peak_flux=0.0,
n_ge_1e8=0,
               max_rel_transient=0.0, n_transient=0,
max_abs_transient=0.0,
               max_rel_env_transient=0.0, E_scale_max=0.0)
    om_n = omega_of_C(C, P)
    energy_terms = []
    mass_aud = dict(max_rel=0.0, n_ge_1e8=0, max_abs=0.0,
worst_t=None, first=None)
    aud_s = (dict(t=Grow(), lhs=Grow(), rhs=Grow(), rel=Grow()
, cmax=Grow())
             if audit_series else None)
    m_prev = float(np.dot(W, C))
    flux_cum_audit = 0.0
    flux_cum_phys = 0.0
```

```
1686
1687     dt_now = dt1
1688     n_switch = 0
1689     t_end = None
1690     t_dry = None
1691     t_dry_prev = None
1692     cmax_prev = None
1693     status = "unknown"
1694     n_steps_total = step
1695     mass = m_prev
1696     R_min_seen, R_max_seen = R_now, R_now
1697     g_geo_prev = None
1698
1699     while True:
1700         if stop_on_dry:
1701             if t_dry is not None and t >= t_end - 1e-9:
1702                 status = "ok"
1703                 break
1704         else:
1705             if t_end_override is not None and t >=
    t_end_override - 1e-9:
1706                 status = "fixed_end"
1707                 break
1708         if t >= t_cap - 1e-9:
1709             status = "not_dry_by_cap"
1710             break
1711         if n_switch == 0 and dt2 != dt1 and t >= t_switch - 1e
    -12:
1712             dt_now = dt2
1713             n_switch = 1
1714             if verbose:
1715                 print("[stage] 段切换: t = %.6f s  Δt = %g s"
    % (t, dt_now), flush=True)
1716
1717         t_new = t + dt_now
```

```
        ta_new, ca_new = drv_at(t_new)
        R_new = float(radius_at(t_new, R_mode))
        R_old = R_now
        if R_mode in ("shrink", "pchip"):
            if R_new > R_old + 1e-15:
                raise AssertionError("R(t) 非单调不增（t=%.6f: %r  %r）"
                                     % (t_new, R_old, R_new))
            if not (R_MIN - 1e-12 <= R_new <= R0 + 1e-12):
                raise AssertionError("R(t) 越界（t=%.6f: R=%r）" % (t_new, R_new))
        R_min_seen = min(R_min_seen, R_new)
        R_max_seen = max(R_max_seen, R_new)

        R_geo = R_new if geo_layer == "nplus1" else R_old
        g = dt_now / (R_geo * R_geo)

        k_it = 0
        converged = False
        C_it = C
        T_it = np.full(N + 1, ta_new) if T_mode == "prescribed" else T
        C_prev, T_prev = C_it, T_it
        mT = None
        mC = None
        for m in range(1, picard_max + 1):
            k_it = m
            om = omega_of_C(C_it, P)
            kv = k_of_C(C_it, P)
            Dv = eval_D(logD_of_CT(C_it, T_it, P), floor_mon, C_it, T_it, t_new)

            if T_mode == "coupled" and block_order == "TC":
                kf = face_gam("k", kv, C_it, T_it, face_rule, P)
```

```
            T_it, mT = assemble(g, geo, kf, om, P["H"], R_new, bc, T, ta_new)
            Dv = eval_D(logD_of_CT(C_it, T_it, P), floor_mon, C_it, T_it, t_new)

        Df = face_gam("D", Dv, C_it, T_it, face_rule, P)
        C_it, mC = assemble(g, geo, Df, np.ones(N + 1), P["HM"], R_new, bc, C, ca_new)

        if T_mode == "coupled" and block_order == "CT":
            om = omega_of_C(C_it, P)
            kv = k_of_C(C_it, P)
            kf = face_gam("k", kv, C_it, T_it, face_rule, P)
            T_it, mT = assemble(g, geo, kf, om, P["H"], R_new, bc, T, ta_new)

        dT = float(np.abs(T_it - T_prev).max())
        dC = float(np.abs(C_it - C_prev).max())
        T_prev, C_prev = T_it, C_it
        dT_seen = max(dT_seen, dT)
        dC_seen = max(dC_seen, dC)
        if T_mode == "prescribed":
            if dC < picard_tol:
                converged = True
                break
        elif dT < picard_tol and dC < picard_tol:
            converged = True
            break

    if T_mode == "coupled":
        res_max_T = max(res_max_T, resid_of(mT, T_it))
    res_max_C = max(res_max_C, resid_of(mC, C_it))

    picard_iters.append(k_it)
```

```
        if not converged:
            n_not_conv += 1

        T_new, C_new = T_it, C_it
        cmax_now = float(C_new.max())
        cmin_now = float(C_new.min())
        tmin_now = float(T_new.min())
        tmax_now = float(T_new.max())

        if cmin_now < c_lo - 1e-12 or cmax_now > c_hi + 1e-12:
            viol.append(dict(t_s=float(t_new), kind="C", min=
    cmin_now, max=cmax_now,
                             bound=[c_lo, c_hi]))
        if tmin_now < t_lo - 1e-9 or tmax_now > t_hi + 1e-9:
            viol.append(dict(t_s=float(t_new), kind="T", min=
    tmin_now, max=tmax_now,
                             bound=[t_lo, t_hi]))
        if viol:
            status = "extremum_violation"
            t, n_steps_total = t_new, step + 1
            break

        if audit and bc == "robin":
            lhs_m = float(np.dot(W, C_new - C))
            rhs_m = dt_now * P["HM"] * (ca_new - float(C_new[N
    ])) / R_new
            sc_m = max(abs(lhs_m), abs(rhs_m))
            rel_m = abs(lhs_m - rhs_m) / sc_m if sc_m > 0 else
     0.0
            if rel_m > mass_aud["max_rel"]:
                mass_aud["max_rel"] = rel_m
                mass_aud["max_abs"] = abs(lhs_m - rhs_m)
                mass_aud["worst_t"] = float(t_new)
            if mass_aud["first"] is None:
                mass_aud["first"] = dict(t_s=float(t_new), lhs
```

```
=lhs_m, rhs=rhs_m, rel=rel_m)
            if rel_m > 1e-8:
                mass_aud["n_ge_1e8"] += 1
            if aud_s is not None:
                aud_s["t"].append(float(t_new)); aud_s["lhs"].
append(lhs_m)
                aud_s["rhs"].append(rhs_m); aud_s["rel"].
append(rel_m)
                aud_s["cmax"].append(cmax_now)
            if rel_m > 1e-10:
                raise AssertionError(
                    "水分守恒审计越界（§6.6）：t=%.6f s 相对残
差 %.3e > 1e-10 —— "
                    "先查累积项右端取的是哪一层（R3 条款），再
查边界符号、再查系数层。"
                    % (t_new, rel_m))
            flux_cum_audit += dt_now * P["HM"] * (ca_new -
float(C_new[N])) / R_new
            flux_cum_phys += dt_now * R_new * P["HM"] * (float
(C_new[N]) - ca_new)

        om_new = omega_of_C(C_new, P)
        Tn = T if T_mode == "coupled" else np.full(N + 1,
ta_old)
        Tp = T_new
        if audit:
            lhsA = float(np.dot(W, om_new * Tp) - np.dot(W,
om_n * Tn)
                         - np.dot(W, Tn * (om_new - om_n)))
            rhsA = dt_now * P["H"] * (ta_new - float(Tp[N])) /
 R_new
            residA = lhsA - rhsA
            scA = max(abs(lhsA), abs(rhsA))
            relA = abs(residA) / scA if scA > 0 else 0.0
            E_scale = float(np.dot(W, np.abs(om_new * Tp)))
```

```
            aud["E_scale_max"] = max(aud["E_scale_max"],
    E_scale)
            relEnv = abs(residA) / E_scale if E_scale > 0 else
     0.0
            aud["n"] += 1
            aud["max_abs"] = max(aud["max_abs"], abs(residA))
            aud["max_rel_perstep"] = max(aud["max_rel_perstep"
    ], relA)
            aud["max_rel_env"] = max(aud["max_rel_env"],
    relEnv)
            aud["peak_dE"] = max(aud["peak_dE"], abs(lhsA))
            aud["peak_flux"] = max(aud["peak_flux"], abs(rhsA)
    )
            aud["sum_abs"] += abs(residA)
            ref = relEnv
            if ref > 1e-8:
                aud["n_ge_1e8"] += 1
            if t_new <= T_FREEZE_S:
                aud["n_transient"] += 1
                aud["max_rel_transient"] = max(aud["
    max_rel_transient"], ref)
                aud["max_rel_env_transient"] = max(aud["
    max_rel_env_transient"], relEnv)
                aud["max_abs_transient"] = max(aud["
    max_abs_transient"], abs(residA))
            if len(aud["samples"]) < 5:
                aud["samples"].append(dict(t_s=float(t_new),
    lhs_A=lhsA, rhs_A=rhsA,
                                           residual=residA,
    rel_self=relA,
                                           rel_envelope=relEnv
    ))
            if T_mode == "coupled" and relEnv > 1e-8:
                raise AssertionError(
                    "能量审计越界（§6.7 写法 A）：t=%.6f s 包
```

```
    络相对残差 %.3e > 1e-8 "
1858                    "（绝对 %.3e；自尺度 %.3e）—— 先查累积项冻结层（R3）、再查边界项符号、"
1859                    "再查 ω 取的是哪一层。" % (t_new, relEnv, abs(residA), relA))
1860          if audit and len(energy_terms) < 200000:
1861              dH_phys = (R_new ** 2) * float(np.dot(W, om_new * Tp)) - \
1862                        (R_old ** 2) * float(np.dot(W, om_n * Tn))
1863              cap_acct = (R_new ** 2) * float(np.dot(W, Tn * (om_new - om_n)))
1864              geo_term = (R_new ** 2 - R_old ** 2) * float(np.dot(W, om_n * Tn))
1865              bnd_flux = dt_now * R_new * P["H"] * (ta_new - float(Tp[N]))
1866              energy_terms.append((float(t_new), dH_phys, cap_acct, geo_term, bnd_flux))
1867          om_n = om_new
1868          ta_old = ta_new
1869
1870          T, C = T_new, C_new
1871          R_now = R_new
1872          mass = float(np.dot(W, C_new))
1873          t, step = t_new, step + 1
1874          n_steps_total = step
1875
1876          if cmax_track:
1877              tr["t"].append(t); tr["cmax"].append(cmax_now)
1878              tr["arg"].append(float(np.argmax(C_new)))
1879              tr["c0"].append(float(C_new[0])); tr["cR"].append(float(C_new[-1]))
1880              tr["cmin"].append(cmin_now)
1881
1882          if stop_on_dry and t_dry is None and cmax_now <= c_dry
```

```
:
1883            t_dry = t
1884            t_dry_prev = (t - dt_now, cmax_prev)
1885            t_end = t + margin_s
1886            n_end = math.ceil(t_end / dt_now - 1e-9)
1887            t_end = n_end * dt_now
1888            if verbose:
1889                print("[dry] 首达 C_max ≤ %g: t = %.6f s (%.4f
     h, %.4f d); C_max=%.9f; "
1890                      " t_end = %.4f s" % (c_dry, t, t /
    3600.0, t / 86400.0,
1891                                          cmax_now, t_end),
    flush=True)
1892
1893        if snap_next is not None:
1894            while t >= snap_next - 1e-9 and snap_next <=
    snap_stop + 1e-9:
1895                snap_t.append(t); snap_T.append(T[sn].copy())
1896                snap_C.append(C[sn].copy()); snap_R.append(
    R_now)
1897                snap_next += snap_every_s
1898
1899        while prof_left and t >= prof_left[0] - 1e-9:
1900            profiles[prof_left.pop(0)] = (T.copy(), C.copy())
1901
1902        for tc in checkpoints:
1903            if tc not in ckpt and t >= tc - 1e-9:
1904                ckpt[tc] = dict(t=t, T=T.copy(), C=C.copy(),
    cum_flux=cum_flux,
1905                                mass=mass, n_steps=step,
    floor_count=floor_mon.count,
1906                                N=N, R_mode=R_mode)
1907
1908        if verbose and progress_every and step %
    progress_every == 0:
```

```
            nn = min(2000, picard_iters.n)
            print("[prog] t=%10.3f s (%.3f h) n=%8d C_max=%.9f
 C(0)=%.9f R=%.6f cm "
                  "it=%.2f floor=%d wall=%.1f s"
                  % (t, t / 3600.0, step, cmax_now, float(
C_new[0]), R_now * 100.0,
                     float(picard_iters.a[picard_iters.n - nn:
picard_iters.n].mean()),
                     floor_mon.count, time.time() - t_wall0),
flush=True)
        cmax_prev = cmax_now

    iters = picard_iters.array()
    uniq, cnt = (np.unique(iters, return_counts=True) if iters
.size else
                 (np.array([]), np.array([])))
    et = np.array(energy_terms) if energy_terms else np.zeros
((0, 5))
    return dict(
        status=status, params=P,
        xi=xi, dxi=dxi, N=N, W=W, R_mode=R_mode, geo_layer=
geo_layer,
        face_rule=face_rule, props=props, bc=bc, T_mode=T_mode
, block_order=block_order,
        d_const=d_const,
        dt1=dt1, dt2=dt2, t_switch=t_switch, dt_final=dt_now,
n_switch=n_switch,
        n_steps=n_steps_total, t_final=t, t_dry=t_dry,
t_dry_prev=t_dry_prev,
        t_end=t_end, c_dry=c_dry, R_min_seen=R_min_seen,
R_max_seen=R_max_seen,
        snap_t=np.array(snap_t), snap_T=np.array(snap_T),
snap_C=np.array(snap_C),
        snap_R=np.array(snap_R), snap_nodes=sn,
        T_final=T.copy(), C_final=C.copy(), R_final=R_now,
```

```
        picard=dict(max_iters=int(iters.max()) if iters.size
    else 0,
                    mean_iters=float(iters.mean()) if iters.
    size else 0.0,
                    median_iters=float(np.median(iters)) if
    iters.size else 0.0,
                    n_not_converged=int(n_not_conv),
                    hist_iters=[int(v) for v in uniq],
                    hist_counts=[int(v) for v in cnt]),
        residual=dict(max_T=res_max_T, max_C=res_max_C),
        picard_step_dmax=dict(T=dT_seen, C=dC_seen),
        floor=floor_mon.to_dict(),
        violations=viol,
        extremum_bounds=dict(T_C=[t_lo, t_hi], C_kgkg=[c_lo,
    c_hi]),
        audit=dict(n=aud["n"], max_abs=aud["max_abs"],
                   max_rel_global=(aud["max_abs"]
                                   / max(aud["peak_dE"], aud["
    peak_flux"], 1e-300)),
                   max_rel_perstep=aud["max_rel_perstep"],
                   max_rel_env=aud["max_rel_env"],
                   E_scale_max=aud["E_scale_max"],
                   n_ge_1e8=aud["n_ge_1e8"],
                   max_rel_transient=aud["max_rel_transient"],
                   max_rel_env_transient=aud["
    max_rel_env_transient"],
                   max_abs_transient=aud["max_abs_transient"],
                   n_transient=aud["n_transient"],
                   peak_dE=aud["peak_dE"], peak_flux=aud["
    peak_flux"],
                   mean_abs=(aud["sum_abs"] / aud["n"]) if aud
    ["n"] else 0.0,
                   samples=aud["samples"],
                   form="写法 A（Q4-E-A，ξ 测度、无几何项、与
    Q2-25 形式同构）",
```

```
                applicable=(T_mode == "coupled" and bc == "robin"),
                criterion_note=("判据 = 'max_rel_global'（= 全程 max|残差| / "
                                "全程 max(|ΔH|,|边界通量|)，与 Q2 的 §8.7 同构，"
                                "≤1e-8）；逐走判据用 'max_rel_env'（除以焓包络 "
                                "Σwω|T|，≤1e-8），因小步上自尺度会退化。"),
                note=("T_mode='coupled' 时 (Q4-E-A) 对本离散格式是代数恒等式"
                      "（判据 'max_rel_global' ≤1e-8，与 Q2 §8.7 同构）；"
                      "T_mode='prescribed'（主口径）下不求解能量方程，该式在恒温段"
                      "（t>14400 s，T_a 常数）两端同为零、残差应为舍入零，在升温段的"
                      "残差是「预设 T 口径的热失衡诊断量」。预设口径下自尺度退化"
                      "（两端可同时为零） 用 'max_rel_env'（除以焓包络 "
                      "'Σwω|T|'）作为普适量度，不作为审计判据。")),
        audit_energy_terms=dict(
            note="写法 B（物理焓）三分项逐内部步（§6.7 必落盘）；几何项绝不进右端通量",
            header=["t_s", "dH_phys", "cap_account_R2", "geo_term", "boundary_flux"],
            n=int(et.shape[0]), values=et),
        mass=dict(m0=m_prev, m1=mass,
                  ratio=mass / m_prev if m_prev else float("nan"),
                  cum_flux=flux_cum_audit, cum_flux_phys=flux_cum_phys,
```

```
                    rel_err=(abs((mass - m_prev) - flux_cum_audit) / abs(mass - m_prev)
                             if m_prev != mass else 0.0),
                    note="P3（§6.6）：Δ(ΣW_i^ξ C_i) 应等于 Σ Δt·h_m(C_a-C_N)/R；"
                         "跨界面通量逐对相消，严格成立"),
          mass_audit=mass_aud,
          mass_audit_series=(None if aud_s is None else
                             dict(t=aud_s["t"].array(), lhs=aud_s["lhs"].array(),
                                  rhs=aud_s["rhs"].array(), rel=aud_s["rel"].array(),
                                  cmax=aud_s["cmax"].array())),
          cmax=dict(t=tr["t"].array() if cmax_track else None,
                    cmax=tr["cmax"].array() if cmax_track else None,
                    argmax=tr["arg"].array() if cmax_track else None,
                    c0=tr["c0"].array() if cmax_track else None,
                    cR=tr["cR"].array() if cmax_track else None,
                    cmin=tr["cmin"].array() if cmax_track else None),
          checkpoints=ckpt, profiles=profiles,
          elapsed_s=time.time() - t_wall0,
      )

def setup_params(N, R_mode, T_mode, bc, ca_scenario, const_ca, const_ta, d_const, dt1, dt2, geo_layer, overrides, props, snap_every_s, ta_scenario):
    t_wall0 = time.time()
    P = default_params()
    P.update(overrides)
    P["props"] = props
    P["D_CONST"] = d_const
    P["LOG_D0"] = math.log(P["D0"])
```

```
    P["LOG_A2_D0"] = math.log(P["A2_D0"])
    if R_mode not in ("shrink", "const", "pchip"):
        raise ValueError("R_mode 必须为 'shrink' / 'const' / 'pchip'")
    if geo_layer not in ("nplus1", "n"):
        raise ValueError("geo_layer 必须为 'nplus1' 或 'n'")
    if T_mode not in ("prescribed", "coupled"):
        raise ValueError("T_mode 必须为 'prescribed' 或 'coupled'")
    if bc not in ("robin", "dirichlet"):
        raise ValueError("bc 必须为 'robin' 或 'dirichlet'")
    for nm, dtc in (("dt1", dt1), ("dt2", dt2)):
        if dtc <= 0 or abs(round(60.0 / dtc) * dtc - 60.0) > 1e-9:
            raise ValueError("%s=%g 不能整除 60 s（§7.1 C4：输出必须落在内部步上）"
                             % (nm, dtc))
    for nm, sv in (("snap_every_s", snap_every_s),):
        if sv > 0:
            for dtc in (dt1, dt2):
                if abs(round(sv / dtc) * dtc - sv) > 1e-9:
                    raise ValueError("%s=%g 不是 Δt=%g 的整数倍" % (nm, sv, dtc))

    xi, dxi, N, W = build_grid(N)
    geo = make_geo(N, dxi, W)

    drv = driver_table()
    Ta_scen = scenario_driver(ta_scenario, drv["T"], const_ta)
    Ca_scen = scenario_driver(ca_scenario, drv["C"], const_ca)
    use_frozen_tbl = (ta_scenario == "freeze_last" and ca_scenario == "freeze_last")
    _inv_dt, _nseg = drv["inv_dt"], drv["nseg"]
    return Ca_scen, N, P, Ta_scen, W, _inv_dt, _nseg, dxi, geo, t_wall0, use_frozen_tbl, xi
```

```
2031
2032 def prepare_drivers(Ca_scen, P, Ta_scen, _inv_dt, _nseg,
     driver_interp, use_frozen_tbl):
2033     def drv_at(t):
2034         if use_frozen_tbl:
2035             return driver_at(t, driver_interp)
2036         if t >= T_FREEZE_S:
2037             return float(Ta_scen[-1]), float(Ca_scen[-1])
2038         x = t * _inv_dt
2039         i = int(x)
2040         if i >= _nseg:
2041             i = _nseg - 1
2042         f = x - i
2043         return (float(Ta_scen[i] + (Ta_scen[i + 1] - Ta_scen[i
     ]) * f),
2044                 float(Ca_scen[i] + (Ca_scen[i + 1] - Ca_scen[i
     ]) * f))
2045
2046     ta_bound = max(float(np.max(Ta_scen)), float(Ta_scen[-1]))
2047     ca_bound = min(float(np.min(Ca_scen)), float(Ca_scen[-1]))
2048     c_lo = min(float(P["C_INIT"]), ca_bound)
2049     c_hi = float(P["C_INIT"])
2050     t_lo = min(float(P["T_INIT"]), float(np.min(Ta_scen)))
2051     t_hi = max(float(P["T_INIT"]), ta_bound)
2052     return c_hi, c_lo, drv_at, t_hi, t_lo
2053
2054
2055 def smoke_test(N=200, dt=20.0, t_end=21600.0, verbose=True):
2056     o = solve_q4(N=N, dt1=dt, dt2=dt, t_switch=1e18,
     stop_on_dry=False,
2057                  t_end_override=t_end, snap_every_s=60.0,
     snap_until_s=t_end,
2058                  cmax_track=True, verbose=False)
2059     C = o["C_final"]
2060     checks = {
```

```
        "C_max_decreasing": bool(C.max() < C_INIT),
        "C_radial_nonincreasing": bool(np.all(np.diff(C) <= 1e
    -12)),
        "C_below_init": bool(C.max() <= C_INIT + 1e-12),
        "C_above_ca": bool(C.min() >= CA_MIN - 1e-12),
        "no_floor_activation": bool(o["floor"]["count"] == 0),
        "picard_all_converged": bool(o["picard"]["
    n_not_converged"] == 0),
        "no_extremum_violation": bool(len(o["violations"]) ==
    0),
        "mass_audit_le_1e-8": bool(o["mass_audit"]["max_rel"]
    <= 1e-8),
        "R_within_bounds": bool(o["R_min_seen"] >= R_MIN - 1e
    -12),
     }
     if verbose:
        print("耗时 %.2f s；步数 %d；Picard 均值 %.3f 最大 %d
    未收敛 %d"
              % (o["elapsed_s"], o["n_steps"], o["picard"]["
    mean_iters"],
                 o["picard"]["max_iters"], o["picard"]["
    n_not_converged"]))
        print("C: 中心 %.9f  表面 %.9f  C_max %.9f  C_min %.9f
    "
              % (C[0], C[-1], C.max(), C.min()))
        print("水分审计 max 相对残差 %.3e（worst t=%s）；
    D_floor 激活 %d 次"
              % (o["mass_audit"]["max_rel"], o["mass_audit"]["
    worst_t"],
                 o["floor"]["count"]))
        for k, v in checks.items():
            print("  %-24s %s" % (k, "PASS" if v else "FAIL"))
     return o, checks


```

```
if __name__ == "__main__":
    print("[solver_q4] smoke test  N=200, dt=20 s, t_end=21600 s")
    o, ck = smoke_test()
    print("全部自检通过" if all(ck.values()) else "存在失败项")
from __future__ import annotations

import json
import math
import sys
import time
from datetime import datetime
from pathlib import Path

import numpy as np

_OUT_ENC = getattr(sys.stdout, "encoding", None)
_STREAM = sys.stdout
if _OUT_ENC != "utf-8" and callable(getattr(_STREAM, "reconfigure", None)):
    _STREAM.reconfigure(encoding="utf-8", errors="replace")

_TOP = Path(__file__).resolve().parents[1]
_SRC_DIR = None
for _cand in (_TOP / "Q1",):
    if (_cand / "solver_q1.py").is_file():
        _SRC_DIR = str(_cand)
        break
if _SRC_DIR is None:
    _SRC_DIR = str(_TOP / "src")
sys.path.insert(0, _SRC_DIR)

from solver_q4 import (
    solve_q4, build_grid, tdsolve, radius_at, driver_at, att2,
```

```
    R0, R_MIN, C_INIT, T_INIT_C, CA_MIN, HM_DEFAULT, H_DEFAULT,
    A2_D0, A2_BETA, D_AP4_2P55_323K,
)

RES = _TOP / "results"
MACH_TOL = 1e-12
ACCEPT_TOL = 5e-5

Q1_ANCHOR = dict(surface_C_1800s_raw_N200=1.5102532607720367,
                 center_C_1800s=2.54999231711862,
                 mass_ratio_1800s=0.8994286105470196)

T_DRY_MIXING_S = 30083.5
T_DRY_MIXING_H = 8.356527777777778

TARGET_C0_1800 = 2.5499999988706192
TARGET_C0_10800 = 2.431266543861803


def bessel_j0_zeros(n):
    from scipy.special import j0, j1
    zs = np.empty(n)
    for k in range(n):
        x = (k + 0.75) * math.pi
        for _ in range(80):
            f = j0(x)
            df = -j1(x)
            dx = f / df
            x -= dx
            if abs(dx) < 1e-15:
                break
        zs[k] = x
    return zs

```

```
2151
2152 def bessel_analytic_C0(t, D, R, n_roots=400):
2153     from scipy.special import j1
2154     zs = bessel_j0_zeros(n_roots)
2155     An = 2.0 / (zs * j1(zs))
2156     tv = np.atleast_1d(np.asarray(t, float))
2157     out = np.zeros_like(tv)
2158     for z, A in zip(zs, An):
2159         out += A * np.exp(-D * z * z * tv / (R * R))
2160     return out if np.ndim(t) else float(out[0])
2161
2162
2163 def _g1_stage1(q1, out, dr_cm, dt, t_end):
2164     N = int(round(R0 / (dr_cm / 100.0)))
2165     t0 = time.time()
2166     oq1 = q1.solve_q1(case="A", dr_cm=dr_cm, dt=dt, t_end=
    t_end, dt_out=1.0,
2167                       picard_tol=1e-12, picard_max=50,
    record_fields=True)
2168     q1_wall = time.time() - t0
2169     t0 = time.time()
2170     oq4 = solve_q4(N=N, dt1=dt, dt2=dt, t_switch=1e18, R_mode=
    "const",
2171                    props="app2", bc="robin", T_mode="
    prescribed",
2172                    stop_on_dry=False, t_end_override=t_end,
2173                    snap_every_s=1.0, snap_until_s=t_end,
    cmax_track=False,
2174                    verbose=False)
2175     q4_wall = time.time() - t0
2176
2177     i1800 = int(round(t_end))
2178     C1_1800 = oq1["C_out"][i1800]
2179     C4_1800 = oq4["snap_C"][-1]
2180     d1a = float(np.abs(C1_1800 - C4_1800).max())
```

```
 m1a_q1 = float(np.dot(oq1["scalars"]["tvol"], C1_1800))
 m1a_q4 = float(np.dot(oq4["W"], C4_1800))
 m0_q4 = float(np.dot(oq4["W"], np.full(N + 1, C_INIT)))
 ratio_q4 = m1a_q4 / m0_q4
 out["group1a_DC_appendix2"] = dict(
     q1_wall_s=q1_wall, q4_wall_s=q4_wall,
     max_abs_field_diff_vs_q1=float(np.abs(C1_1800 - C4_1800).max()),
     max_rel_field_diff_vs_q1=float(
         (np.abs(C1_1800 - C4_1800) / np.maximum(C1_1800, 1e-30)).max()),
     machine_tol=MACH_TOL, pass_machine=bool(np.abs(C1_1800 - C4_1800).max() <= MACH_TOL),
     anchor=dict(Q1_surface=Q1_ANCHOR["surface_C_1800s_raw_N200"],
                 Q4_surface=float(C4_1800[-1]),
                 Q1_center=Q1_ANCHOR["center_C_1800s"], Q4_center=float(C4_1800[0]),
                 Q1_mass_ratio=Q1_ANCHOR["mass_ratio_1800s"], Q4_mass_ratio=ratio_q4),
     anchor_diff=dict(surface=abs(float(C4_1800[-1]) - Q1_ANCHOR["surface_C_1800s_raw_N200"]),
                      center=abs(float(C4_1800[0]) - Q1_ANCHOR["center_C_1800s"]),
                      mass_ratio=abs(ratio_q4 - Q1_ANCHOR["mass_ratio_1800s"])),
     anchor_tol=ACCEPT_TOL,
     pass_anchor=bool(
         abs(float(C4_1800[-1]) - Q1_ANCHOR["surface_C_1800s_raw_N200"]) < ACCEPT_TOL
         and abs(float(C4_1800[0]) - Q1_ANCHOR["center_C_1800s"]) < ACCEPT_TOL
         and abs(ratio_q4 - Q1_ANCHOR["mass_ratio_1800s"]) < ACCEPT_TOL),
     mass_audit_max_rel=oq4["mass_audit"]["max_rel"],
```

```
    )
    return N, i1800


def _g1_stage2(q1, out, dr_cm, dt, t_end, N, i1800, d_const_val):
    t0 = time.time()
    oq1b = q1.solve_q1(case="A", dr_cm=dr_cm, dt=dt, t_end=t_end, dt_out=1.0,
                       picard_tol=1e-12, picard_max=50, record_fields=True,
                       D_CONST=d_const_val)
    q1b_wall = time.time() - t0
    t0 = time.time()
    oq4b = solve_q4(N=N, dt1=dt, dt2=dt, t_switch=1e18, R_mode="const",
                    props="app2", bc="robin", T_mode="prescribed", d_const=d_const_val,
                    stop_on_dry=False, t_end_override=t_end,
                    snap_every_s=1.0, snap_until_s=t_end, cmax_track=False,
                    verbose=False)
    q4b_wall = time.time() - t0
    C1b_1800 = oq1b["C_out"][i1800]
    C4b_1800 = oq4b["snap_C"][-1]
    out["group1b_D_const"] = dict(
        D_const_value=d_const_val,
        D_const_source="附录 2 的 D(2.55) = 7×10□□·e^(-0.89/2.55)",
        q1_wall_s=q1b_wall, q4_wall_s=q4b_wall,
        max_abs_field_diff_vs_q1=float(np.abs(C1b_1800 - C4b_1800).max()),
        surface_Q1=float(C1b_1800[-1]), surface_Q4=float(C4b_1800[-1]),
        center_Q1=float(C1b_1800[0]), center_Q4=float(C4b_1800
```

```
    [0]),
2230        machine_tol=MACH_TOL,
2231        pass_machine=bool(np.abs(C1b_1800 - C4b_1800).max() <=
     MACH_TOL),
2232        mass_audit_max_rel=oq4b["mass_audit"]["max_rel"],
2233     )
2234     return C1b_1800, C4b_1800
2235
2236
2237 def _g1_stage3(q1, out, dr_cm, dt, N, d_const_val, C1b_1800,
    C4b_1800):
2238     scale = {}
2239     for te in (180.0, 900.0, 1800.0):
2240         o1 = q1.solve_q1(case="A", dr_cm=dr_cm, dt=dt, t_end=
    te, dt_out=te,
2241                          picard_tol=1e-12, picard_max=50,
    record_fields=True,
2242                          D_CONST=d_const_val)
2243         o4 = solve_q4(N=N, dt1=dt, dt2=dt, t_switch=1e18,
    R_mode="const",
2244                       props="app2", bc="robin", T_mode="
    prescribed",
2245                       d_const=d_const_val, stop_on_dry=False,
    t_end_override=te,
2246                       snap_every_s=te, snap_until_s=te,
    cmax_track=False, verbose=False)
2247         c1 = o1["C_out"][-1]
2248         c4 = o4["snap_C"][-1]
2249         scale["t_end_%ds" % int(te)] = dict(
2250             n_steps=int(round(te / dt)),
2251             max_abs_diff=float(np.abs(c1 - c4).max()),
2252             max_rel_diff=float((np.abs(c1 - c4) / np.maximum(
    np.abs(c1), 1e-30)).max()),
2253         )
2254     d_seq = [scale["t_end_%ds" % int(te)]["max_abs_diff"] for
```

```
    te in (180.0, 900.0, 1800.0)]
    n_seq = [scale["t_end_%ds" % int(te)]["n_steps"] for te in
    (180.0, 900.0, 1800.0)]
    ratio_seq = [d_seq[i] / n_seq[i] for i in range(3)]
    roundoff_like = bool(max(ratio_seq) / max(min(ratio_seq),
    1e-300) < 10.0
                         and max(d_seq) <= 1e-11)
    out["group1b_roundoff_attribution"] = dict(
        per_window=scale, diff_seq=d_seq, n_steps_seq=n_seq,
        diff_per_step=ratio_seq,
        eps64=float(np.finfo(float).eps),
        interpretation=("两份实现的代数式逐项相同（见脚注推
    导）、仅运算次序不同；"
                        "若 max|Δ| 随步数线性增长且 'max|Δ|/步
    数 ≈ 1e-16 量级', "
                        "即为浮点舍入累积，不是实现差异。"),
        roundoff_like=roundoff_like,
    )
    out["group1b_D_const"]["magnitude_of_C"] = float(np.abs(
    C1b_1800).max())
    out["group1b_D_const"]["max_rel_field_diff_vs_q1"] = float
    (
        (np.abs(C1b_1800 - C4b_1800) / np.maximum(np.abs(
    C1b_1800), 1e-30)).max())
    out["group1b_D_const"]["pass_nominal_line_1e-12"] = bool(
        np.abs(C1b_1800 - C4b_1800).max() <= MACH_TOL)
    out["group1b_D_const"]["pass_roundoff_attributed"] =
    roundoff_like


def _g1_stage4(out):
    R_test = [0.0, 1.0, 1800.0, 14400.0, 86400.0]
    rc = dict(
        R_const=[float(radius_at(t, "const")) for t in R_test
    ],
```

```
        R_shrink=[float(radius_at(t, "shrink")) for t in R_test],
        R0=R0,
        const_all_equal_R0=bool(all(abs(float(radius_at(t, "const")) - R0) < 1e-18
                                    for t in R_test)),
        shrink_at_t0_is_R0=bool(abs(float(radius_at(0.0, "shrink")) - R0) < 1e-18),
    )
    out["group1c_switch_selfconsistency"] = rc
    return rc


def _g1_stage5(out, rc):
    b1 = out["group1b_D_const"]
    out["group1_verdict"] = dict(
        a_strict_1e12=bool(out["group1a_DC_appendix2"]["pass_machine"]),
        a_anchor_5e5=bool(out["group1a_DC_appendix2"]["pass_anchor"]),
        b_nominal_1e12=bool(b1["pass_nominal_line_1e-12"]),
        b_value=float(b1["max_abs_field_diff_vs_q1"]),
        b_roundoff_attributed=bool(b1["pass_roundoff_attributed"]),
        statement=("组 1a 在 1e-12 线内通过；组 1b 实测 2.835e-12（相对 1.11e-12），"
                   "高于名义线 2.8 倍，但按步数标定（差值/步数 ≈ 0.13·ε 且不随步数"
                   "放大）判定为浮点舍入累积，非实现差异。两份实现的离散式逐项相同，"
                   "仅运算次序不同（推导见 docstring 的 W^ξ □ W^r 换算）。"
                   " 未解释项（如实登记，2026-09-11 门禁 2a 审查「低级-3」）："
                   "1a（变 D）在同一 115200 步（Δt=1/64 s、
```

```
    t_end=1800 s）下的实测差值 "
2304                "5.285e-13 比 1b（常数 D）的 2.835e-12 小
    5.4 倍（每步当量 ≈ 0.021·ε "
2305                "vs 0.11~0.13·ε），本轮未查明该倍差的来源
    （候选解释：常数 D 路径下"
2306                "两实现的算子装配与右端项运算次序差异更集中
    —— 未验证）。"
2307                "「浮点舍入」是对 1b 判据口径的解释，不是对
    1a/1b 差异的完整归因。"),
2308     )
2309     out["pass_group1"] = bool(out["group1_verdict"]["
    a_strict_1e12"]
2310                               and out["group1_verdict"]["
    a_anchor_5e5"]
2311                               and (out["group1_verdict"]["
    b_nominal_1e12"]
2312                                    or out["group1_verdict"]["
    b_roundoff_attributed"])
2313                               and rc["const_all_equal_R0"] and
     rc["shrink_at_t0_is_R0"])
2314
2315
2316 def _g1_report(out, rc, verbose):
2317     if verbose:
2318         a = out["group1a_DC_appendix2"]
2319         print("[组 1a] D(C) 非线性：max|Q1−Q4| = %.3e（判据 ≤
    %.0e） %s"
2320               % (a["max_abs_field_diff_vs_q1"], MACH_TOL,
2321                  "PASS" if a["pass_machine"] else "FAIL"))
2322         print("        锚点：表面 Q1 %.10f vs Q4 %.10f（差 %.2
    e）；"
2323               "中心 %.10f vs %.10f；水量比 %.10f vs %.10f"
2324               % (a["anchor"]["Q1_surface"], a["anchor"]["
    Q4_surface"],
2325                  a["anchor_diff"]["surface"], a["anchor"]["
```

```
    Q1_center"],
2326                a["anchor"]["Q4_center"], a["anchor"]["
    Q1_mass_ratio"],
2327                a["anchor"]["Q4_mass_ratio"]))
2328        b = out["group1b_D_const"]
2329        r = out["group1b_roundoff_attribution"]
2330        print("[组 1b] D≡%.12e：max|Q1−Q4| = %.3e（相对 %.2e）
     名义线 1e-12：%s；"
2331              "舍入归属：%s"
2332              % (b["D_const_value"], b["
    max_abs_field_diff_vs_q1"],
2333                 b["max_rel_field_diff_vs_q1"],
2334                 "PASS" if b["pass_nominal_line_1e-12"] else "
    略高",
2335                 "成立" if b["pass_roundoff_attributed"] else
    "不成立"))
2336        print("        舍入标定 max|Δ|/步数：%s（步数 %s）"
2337              % (["%.2e" % v for v in r["diff_per_step"]], r["
    n_steps_seq"]))
2338        print("[组 1c] 单开关自洽：R_const 恒等于 R_0 = %s；
    R_shrink(0)=R_0 = %s"
2339              % (rc["const_all_equal_R0"], rc["
    shrink_at_t0_is_R0"]))
2340
2341
2342 def group1(verbose=True):
2343     import solver_q1 as q1
2344
2345     dr_cm, dt, t_end = 0.01, 1.0 / 64.0, 1800.0
2346     out = {"config": dict(dr_cm=dr_cm, dt_s=dt, t_end_s=t_end,
2347                           R_const_m=R0, props="附录 2", case="
    A（口径 A）"),
2348            "note": ("两个实现必须在同一代码路径下、只用一个开
    关切换；"
2349                     "本脚本的固定域参照为 'src/solver_q1.py'
```

```
    （只读 import、不改）——"
                    "Q4 侧的 'R_mode='const'' 开关使 ξ 坐标逐字退化为固定域。")}

    N, i1800 = _g1_stage1(q1, out, dr_cm, dt, t_end)
    d_const_val = A2_D0 * math.exp(-A2_BETA / C_INIT)
    C1b_1800, C4b_1800 = _g1_stage2(q1, out, dr_cm, dt, t_end, N, i1800, d_const_val)
    _g1_stage3(q1, out, dr_cm, dt, N, d_const_val, C1b_1800, C4b_1800)
    rc = _g1_stage4(out)
    _g1_stage5(out, rc)
    _g1_report(out, rc, verbose)
    return out


def group2(verbose=True):
    def run(dt, t_max=200000.0):
        C = C_INIT
        t = 0.0
        n = 0
        while t < t_max:
            t0, C0 = t, C
            tm = t + 0.5 * dt
            Rm = float(radius_at(tm, "shrink"))
            Ca = float(driver_at(tm)[1])
            rate = 2.0 * HM_DEFAULT * dt / Rm
            C = Ca + (C - Ca) * math.exp(-rate)
            t = (n + 1) * dt
            n += 1
            if C <= 0.15:
                return t, C, (t0, C0, t, C)
        return None, C, (t, C, t, C)

    res = {}
```

```
    for dt in (0.5, 0.25, 0.125, 0.0625):
        t_cross, C_end, br = run(dt)
        t0, C0, t1, C1 = br
        w = (0.15 - C0) / (C1 - C0) if C1 != C0 else 0.0
        res["dt_%g" % dt] = dict(t_cross_s=t_cross, t_cross_h=t_cross / 3600.0,
                                 C_end=C_end,
                                 t_interp_s=t0 + w * (t1 - t0),
                                 bracket=[t0, t1], C_bracket=[C0, C1])
    t05 = res["dt_0.5"]["t_cross_s"]
    t025 = res["dt_0.25"]["t_cross_s"]
    t5i = res["dt_0.5"]["t_interp_s"]
    target = T_DRY_MIXING_S
    out = dict(
        method=("ODE dC/dt = −2•h_m•(C − C_a(t))/R(t)，步内指数精确"
                "（R、C_a 步内取中点；C_a 为分段线性、R 为分段线性  该步进为二阶）"),
        h_m=HM_DEFAULT, R_source="附件 2 分段线性（与主模型同源）",
        C_a_source="附件 1 分段线性 + t>14400 s 冻结 0.04986 kg/kg",
        results=res,
        target_s=target, target_h=T_DRY_MIXING_H,
        t_cross_dt05_s=t05, t_interp_dt05_s=t5i,
        diff_step_dt05_s=abs(t05 - target),
        diff_interp_dt05_s=abs(t5i - target),
        dt_refinement_delta_s=abs(t05 - t025),
        converged=bool(abs(t05 - t025) < 0.5),
        note=("靶值 30083.5 s 由建模文档以 Δt=0.5 s 给出（落在其步网格上）；"
              "本实现在同一 Δt 上得到同一格，差 1 步 = 0.5 s；加密到 Δt=0.0625 s 后"
```

```
2407                "首达步时刻稳定在 %s s。" % res["dt_0.0625"]["
    t_cross_s"]),
2408      )
2409      out["pass"] = bool(abs(t05 - target) < 1.0)
2410      if verbose:
2411          print("[组 2] 充分混合下界：Δt=0.5 s  首达步 %.4f s
    （步内插值 %.4f s）= %.6f h"
2412                "（靶 %.6f h，差 %.4f s）" % (t05, t5i, t05 /
    3600.0, T_DRY_MIXING_H,
2413                                             abs(t05 - target)))
2414          print("        Δt 序列：%s；加密后首达步 %s s；判据 ≤1
     s  %s"
2415                % (["%g%.4f s" % (d, res["dt_%g" % d]["t_cross_s
    "])
2416                    for d in (0.5, 0.25, 0.125, 0.0625)],
2417                    res["dt_0.0625"]["t_cross_s"],
2418                    "PASS" if out["pass"] else "FAIL"))
2419      return out
2420
2421
2422  def _g3b_runs(out, D):
2423      ok_all = True
2424      for N, dt in ((200, 1.0), (200, 0.5), (200, 0.25), (200,
    0.125),
2425                    (100, 0.25), (400, 1.0), (400, 0.5), (400,
    0.25), (800, 1.0),
2426                    (800, 0.25)):
2427          o = solve_q4(N=N, dt1=dt, dt2=dt, t_switch=1e18,
    R_mode="const",
2428                       props="app4", bc="dirichlet", T_mode="
    prescribed",
2429                       d_const=D, ca_scenario="const", const_ca
    =0.0,
2430                       stop_on_dry=False, t_end_override
    =10800.0,
```

```
                    snap_every_s=1800.0, snap_until_s
    =10800.0,
                    cmax_track=False, verbose=False)
        st = o["snap_t"]
        i1 = int(np.argmin(np.abs(st - 1800.0)))
        i2 = int(np.argmin(np.abs(st - 10800.0)))
        c1 = float(o["snap_C"][i1][0]); c2 = float(o["snap_C"
    ][i2][0])
        e1 = abs(c1 - TARGET_C0_1800); e2 = abs(c2 -
    TARGET_C0_10800)
        ok = (e1 < ACCEPT_TOL) and (e2 < ACCEPT_TOL)
        ok_all = ok_all and ok
        out["runs"]["N%d_dt%g" % (N, dt)] = dict(
            N=N, dt_s=dt, t1_s=float(st[i1]), t2_s=float(st[i2
    ]),
            C0_1800s=c1, C0_10800s=c2, err_1800=e1, err_10800=
    e2,
            tol=ACCEPT_TOL, pass_=bool(ok),
            mass_audit_max_rel=o["mass_audit"]["max_rel"],
    wall_s=o["elapsed_s"])


def _g3b_order(out):
    main_line = [k for k in out["runs"] if k.startswith("N200_
    ") and out["runs"][k]["pass_"]]
    out["reporting_case"] = "N200_dt0.25"
    def _ord(keys, tag):
        seq = [out["runs"][k]["err_10800"] for k in keys]
        return dict(tag=tag, keys=keys, err_seq=seq,
                    ratio_seq=[seq[i] / seq[i + 1] for i in
    range(len(seq) - 1)],
                    p_seq=[math.log2(seq[i] / seq[i + 1]) for
    i in range(len(seq) - 1)])
    out["observed_order"] = dict(
        spatial_at_dt025=_ord(["N100_dt0.25", "N200_dt0.25", "
```

```
    N400_dt0.25",
                                  "N800_dt0.25"], "Δt=0.25 s 固定，N 加倍"),
        temporal_at_N200=_ord(["N200_dt1", "N200_dt0.5", "N200_dt0.25",
                                  "N200_dt0.125"], "N=200 固定，Δt 减半"),
        note=("Dirichlet 极限的初始数据在不透面上有跳跃（C(•,0)=2.55  C(R,0)=0），"
              "该奇性使向后 Euler 的时间收敛退化为次线性（实测 p≈0.1~0.4）；"
              "这是 Dirichlet 理想化本身的代价，不代表生产 Robin 问题的收敛率"
              "（生产问题在 ξ=1 上是通量连续、无跳跃）。"))


def _g3b_notes(out):
    out["pass"] = bool(out["runs"]["N200_dt0.25"]["pass_"]
                       and out["runs"]["N400_dt0.25"]["pass_"]
                       and out["runs"]["N800_dt1"]["pass_"])
    out["line_5e-5_note"] = ("② 层线 5×10□⁵ 在 N=200 上要求 Δt ≤ 0.5 s 量级"
                             "（Dirichlet 极限的初始边界层比生产 Robin 问题更陡）；"
                             "生产口径的 Δt 由 run_q4 的 Δt 加密序列单独定案（§7.3）。")
    out["audit_note"] = ("Dirichlet 口径不计水分守恒审计（(Q4-28) 的右端是 Robin 通量，"
                         "对 Dirichlet 边界不成立） 本节 'mass_audit_max_rel=0' 表示"
                         "未做，不是通过。")


def _g3b_report(out, an, verbose):
    if verbose:
```

```
        for k, r in out["runs"].items():
            print("[组 3b] %-12s C(0,1800)=%.12f (err %.2e)  C(0,10800)=%.12f (err %.2e)"
                  "  %.2f s   %s"
                  % (k, r["C0_1800s"], r["err_1800"], r["C0_10800s"], r["err_10800"],
                     r["wall_s"], "PASS" if r["pass_"] else "FAIL"))
        print("        解析级数自校验：C(0,1800)=%.12f（靶 %.12f，差 %.2e）；"
              "C(0,10800)=%.12f（靶 %.12f，差 %.2e）"
              % (an[0], TARGET_C0_1800, abs(float(an[0]) - TARGET_C0_1800),
                 an[1], TARGET_C0_10800, abs(float(an[1]) - TARGET_C0_10800)))
        print("        报告口径 %s；② 层线 5e-5 达标的最小档见上表"
              % out["reporting_case"])


def group3b(verbose=True):
    D = D_AP4_2P55_323K
    t_targets = np.array([1800.0, 10800.0])
    an = bessel_analytic_C0(t_targets, D, R0, n_roots=400) * C_INIT
    out = dict(D=D, D_source="附录 4 在 C=2.55、T=323.15 K 处求值（§12.2 组 3b 逐位）",
               R_m=R0, bc="Dirichlet C(R,t)=0", C_init=C_INIT,
               analytic=dict(C0_1800s=float(an[0]), C0_10800s=float(an[1])),
               targets=dict(C0_1800s=TARGET_C0_1800, C0_10800s=TARGET_C0_10800),
               target_match=dict(d1800=abs(float(an[0]) - TARGET_C0_1800),
                                 d10800=abs(float(an[1]) -
```

```
    TARGET_C0_10800)),
2503                runs={})
2504     _g3b_runs(out, D)
2505     _g3b_order(out)
2506     _g3b_notes(out)
2507     _g3b_report(out, an, verbose)
2508     return out
2509
2510
2511 def main():
2512     RES.mkdir(parents=True, exist_ok=True)
2513     t0 = time.time()
2514     print("[verify_q4_baseline] 第一优先级三组地基基准（建模文
    档 §12.2 组 1 / 组 2 / 组 3b）")
2515     out = {"generated_at": datetime.now().isoformat(timespec="
    seconds"),
2516            "note": ("三组都不经过温度场（§16.1 第 6 条）。组
    1/3b 用到求解器；"
2517                     "组 2 为独立 ODE 积分。")}
2518     out["group1_degenerate_regression"] = group1()
2519     out["group2_well_mixed_lower_bound"] = group2()
2520     out["group3b_bessel_dirichlet"] = group3b()
2521     out["wall_s"] = time.time() - t0
2522     out["all_pass"] = bool(out["group1_degenerate_regression"
    ]["pass_group1"]
2523                            and out["
    group2_well_mixed_lower_bound"]["pass"]
2524                            and out["group3b_bessel_dirichlet"
    ]["pass"])
2525     (RES / "q4_baseline.json").write_text(
2526         json.dumps(out, ensure_ascii=False, indent=2),
    encoding="utf-8")
2527     print("三组总判定：%s（产物 results/q4_baseline.json；总耗
    时 %.1f s）"
2528           % ("全部 PASS" if out["all_pass"] else "存在 FAIL 项
```

```
    ", out["wall_s"]))
    return 0 if out["all_pass"] else 1


if __name__ == "__main__":
    sys.exit(main())
from __future__ import annotations

import json
import math
import sys
import time
from datetime import datetime
from pathlib import Path

import numpy as np

_chunk_enc = (sys.stdout.encoding or "").lower()
_S = sys.stdout
if _chunk_enc != "utf-8" and hasattr(_S, "reconfigure"):
    _S.reconfigure(encoding="utf-8", errors="replace")

_SRC_FILE = Path(__file__).resolve()
_ROOTDIR = _SRC_FILE.parent.parent
sys.path.insert(0, str(_ROOTDIR / "Q4"))

from solver_q4 import (
    solve_q4, radius_at, att2, C_DRY, T_FREEZE_S, R0, R_MIN,
    H_DEFAULT, HM_DEFAULT, A2_RHO, A2_CP, A2_K, A2_D0, A2_BETA,
    RHO_A4, RHO_B4, CP_A4, CP_B4, K_A4, K_B4, D0_AP4, BETA_AP4, E_D_AP4,
)

RES = _ROOTDIR / "results"
```

```
N_CMP = 400
DT1, DT2 = 1.25, 5.0
TOL_TEMP = 5e-5
Q1_T_ANCHOR = dict(center_1800s_C=33.5753448265136,
   surface_1800s_C=36.78556035156142)
AP3 = dict(RHO_A=650.0, RHO_B=128.0, CP_A=1450.0, CP_B=2736.0,
           K_A=0.21, K_B=0.38, D0=2.4e-3, BETA=0.45, E_D
   =3850.0)


def t_dry_of(**kw):
    kw.setdefault("N", N_CMP)
    kw.setdefault("dt1", DT1)
    kw.setdefault("dt2", DT2)
    kw.setdefault("t_switch", T_FREEZE_S)
    kw.setdefault("T_mode", "prescribed")
    kw.setdefault("props", "app4")
    kw.setdefault("audit", True)
    o = solve_q4(stop_on_dry=True, snap_every_s=60.0,
   cmax_track=True,
                 verbose=False, **kw)
    if o["status"] != "ok":
        return None, o
    tt, cm = o["cmax"]["t"], o["cmax"]["cmax"]
    i = int(np.flatnonzero(cm <= C_DRY)[0])
    if i == 0:
        return float(tt[0]), o
    t0, t1, c0, c1 = tt[i - 1], tt[i], cm[i - 1], cm[i]
    return float(t0 + (C_DRY - c0) / (c1 - c0) * (t1 - t0)), o


def _group3_anchor_appendix2():
    print("[组 3] 附录 2 常数物性 + 固定 R + 耦合 T，[0,1800]
   s  Q1 解析锚点")
    o3 = solve_q4(N=200, dt1=1.0 / 64.0, dt2=1.0 / 64.0,
```

```
    t_switch=1e18,
2592                R_mode="const", props="app2", bc="robin",
    T_mode="coupled",
2593                stop_on_dry=False, t_end_override=1800.0,
2594                snap_every_s=1.0, snap_until_s=1800.0,
    cmax_track=False,
2595                audit=True, verbose=False)
2596     T1800 = o3["T_final"]
2597     e_c = abs(float(T1800[0]) - Q1_T_ANCHOR["center_1800s_C"])
2598     e_s = abs(float(T1800[-1]) - Q1_T_ANCHOR["surface_1800s_C"
    ])
2599     g3 = dict(config="N=200（Δr=0.01 cm）、Δt=1/64 s、附录 2
    常数物性、R≡0.02 m、耦合 T",
2600               numeric_center_C=float(T1800[0]),
    numeric_surface_C=float(T1800[-1]),
2601               anchor_center_C=Q1_T_ANCHOR["center_1800s_C"],
2602               anchor_surface_C=Q1_T_ANCHOR["surface_1800s_C"],
2603               err_center_C=e_c, err_surface_C=e_s, tol_C=
    TOL_TEMP,
2604               pass_=bool(max(e_c, e_s) < TOL_TEMP),
2605               energy_audit_max_rel=o3["audit"]["max_rel_global
    "],
2606               energy_audit_max_rel_env=o3["audit"]["
    max_rel_env"],
2607               picard_notconv=o3["picard"]["n_not_converged"],
2608               wall_s=o3["elapsed_s"])
2609     print("    中心 %.10f ℃（锚 %.10f，差 %.3e）；表面 %.10f ℃
    （锚 %.10f，差 %.3e） %s"
2610           % (g3["numeric_center_C"], g3["anchor_center_C"],
    e_c,
2611              g3["numeric_surface_C"], g3["anchor_surface_C"],
    e_s,
2612               "PASS" if g3["pass_"] else "FAIL"), flush=True)
2613     print("    能量审计（写法 A，coupled 口径下是判据）
    max_rel_global %.3e；"
```

```
          "包络 %.3e；未收敛步 %d"
          % (g3["energy_audit_max_rel"], g3["
    energy_audit_max_rel_env"],
             g3["picard_notconv"]), flush=True)
     return g3


def _group4_shrinkage_isolation(base):
     print("[组 4] 收缩效应隔离：①附录3+固定R ②附录4+固定R ③附
    录4+R(t)")
     c1, _ = t_dry_of(R_mode="const", props="app4", **AP3)
     c2, _ = t_dry_of(R_mode="const", props="app4")
     c3, _ = base, None
     g4 = dict(case1=dict(label="① 附录 3 物性 + 固定 R0（≈Q3
    口径）",
                          t_dry_h=c1 / 3600.0 if c1 else None),
              case2=dict(label="② 附录 4 物性 + 固定 R0（Q4 基
    线）",
                          t_dry_h=c2 / 3600.0 if c2 else None),
              case3=dict(label="③ 附录 4 物性 + R(t)（主模型）
    ",
                          t_dry_h=base / 3600.0),
              property_effect_1to2_pct=100.0 * (c2 - c1) / c1
    if (c1 and c2) else None,
              shrinkage_effect_2to3_pct=100.0 * (c3 - c2) / c2
     if c2 else None,
              total_1to3_pct=100.0 * (c3 - c1) / c1 if c1 else
     None,
              shrinkage_ratio=c2 / c3 if c3 else None,
              note=("§12.2 组 4 / Q1 §5.4：①−③ 的总差不能全部
    归因于收缩；"
                    "论文只允许用 ②−③ 说收缩效应、①−② 说物性效
    应。"))
     print("   ① %.4f h  ② %.4f h（物性效应 %+.2f %%） ③ %.4f h
    （收缩效应 %+.2f %%，"
```

```
2638            "缩短 %.2f 倍）；总差 %+.2f %%"
2639            % (g4["case1"]["t_dry_h"], g4["case2"]["t_dry_h"],
2640               g4["property_effect_1to2_pct"], g4["case3"]["
    t_dry_h"],
2641               g4["shrinkage_effect_2to3_pct"], g4["
    shrinkage_ratio"],
2642               g4["total_1to3_pct"]), flush=True)
2643     return g4
2644
2645
2646 def _group5_radius_interp(base):
2647     print("[组 5] R(t) 插值口径：分段线性（主）vs 保单调 PCHIP
    （对照）")
2648     tc = np.linspace(0.0, 259200.0, 4001)
2649     Rl = radius_at(tc, "shrink")
2650     Rp = radius_at(tc, "pchip")
2651     pchk = dict(monotone=bool(np.all(np.diff(Rp) <= 1e-15)),
2652                 range_ok=bool(Rp.min() >= R_MIN - 1e-12 and Rp
    .max() <= R0 + 1e-12),
2653                 max_abs_diff_m=float(np.abs(Rl - Rp).max()))
2654     c5, o5 = t_dry_of(R_mode="pchip")
2655     g5 = dict(pchip_checks=pchk,
2656               t_dry_linear_h=base / 3600.0,
2657               t_dry_pchip_h=(c5 / 3600.0) if c5 else None,
2658               delta_h=((c5 - base) / 3600.0) if c5 else None,
2659               delta_pct=(100.0 * (c5 - base) / base) if c5
    else None,
2660               quantization_cm=0.001,
2661               rule=("PCHIP 必须通过 '0.01198 ≤ R ≤ 0.02' 且 '□
    ≤ 0' 的逐点校验，"
2662                     "否则判 PCHIP 不合格（不是相反，§12.2 组 5
    ）"))
2663     print("   PCHIP 校验：单调 %s、取值域 %s、max|ΔR| %.3e m；
    t_dry 差 %s h（%s %%）"
2664           % (pchk["monotone"], pchk["range_ok"], pchk["
```

```
    max_abs_diff_m"],
2665                ("%.4f" % g5["delta_h"]) if g5["delta_h"] is not
    None else "−",
2666                ("%+.3f" % g5["delta_pct"]) if g5["delta_pct"] is
     not None else "−"),
2667              flush=True)
2668     return g5
2669
2670
2671 def _group7_geo_layer():
2672     print("[组 7] 几何因子层位：1/R^{n+1,2}（主）vs 1/R^{n,2}
    （对照）")
2673     g7 = {"pairs": {}}
2674     for (d1, d2) in ((5.0, 20.0), (1.25, 5.0)):
2675         ta, _ = t_dry_of(geo_layer="nplus1", dt1=d1, dt2=d2)
2676         audit_ev = None
2677         try:
2678             t_dry_of(geo_layer="n", dt1=d1, dt2=d2)
2679         except AssertionError as exc:
2680             audit_ev = str(exc)
2681         tb, ob = t_dry_of(geo_layer="n", dt1=d1, dt2=d2, audit
    =False)
2682         g7.setdefault("audit_violation_evidence", {})["dt_%g_%
    g" % (d1, d2)] = audit_ev
2683         g7["pairs"]["dt_%g_%g" % (d1, d2)] = dict(
2684             t_dry_nplus1_h=ta / 3600.0 if ta else None,
2685             t_dry_n_h=tb / 3600.0 if tb else None,
2686             delta_s=(tb − ta) if (ta and tb) else None,
2687             delta_pct=(100.0 * (tb − ta) / ta) if (ta and tb)
    else None,
2688             mass_audit_n=(ob["mass_audit"]["max_rel"] if ob["
    mass_audit_series"] is None
2689                           else None),
2690             audit_violation_evidence=audit_ev)
2691         print("   Δt=(%g,%g)：n+1 %.6f h vs n %.6f h，差 %+.4f
```

```
    s（%+.4f %%）"
2692                % (d1, d2, ta / 3600.0, tb / 3600.0, tb - ta,
    100.0 * (tb - ta) / ta),
2693                flush=True)
2694     p1 = g7["pairs"]["dt_5_20"]["delta_s"]
2695     p2 = g7["pairs"]["dt_1.25_5"]["delta_s"]
2696     g7["ratio_20s_over_5s"] = (p1 / p2) if (p1 and p2) else
    None
2697     g7["expectation"] = ("层位差应为 O(Δt)：Δt 缩小 4 倍  差值
    应随之缩小（比值 ≈ 4）；"
2698                          "若与 Δt 无关则说明实现有误（§12.2 组
    7）")
2699     g7["pass_"] = bool(p1 and p2 and abs(p1) > abs(p2))
2700     print("   证据：'geo_layer='n'' 在 t=5 s 即触发水分守恒审
    计越界（§6.6 恒等式由 "
2701           "'1/R^{n+1,2}' 推出）  主口径取 n+1 的理由被数值证据
    支持")
2702     print("   比值（Δt=20/5 的层位差 ÷ Δt=5/1.25 的层位差）= %
    s（期望 > 1，理想 ≈ 4）"
2703           % (("%.2f" % g7["ratio_20s_over_5s"]) if g7["
    ratio_20s_over_5s"] else "-"),
2704           flush=True)
2705     return g7
2706
2707
2708 def _group8_T_mode(base):
2709     print("[组 8] T 口径：预设 T = T_a(t)（主）vs 耦合 T 场（
    §10.3 有界时间盒）")
2710     t0 = time.time()
2711     try:
2712         c8, o8 = t_dry_of(T_mode="coupled")
2713         g8 = dict(status="跑通", t_dry_coupled_h=(c8 / 3600.0)
    if c8 else None,
2714                   t_dry_prescribed_h=base / 3600.0,
2715                   delta_s=(c8 - base) if c8 else None,
```

```
                delta_pct=(100.0 * (c8 - base) / base) if c8
 else None,
                n_not_converged=o8["picard"]["
n_not_converged"],
                energy_audit_max_rel_global=o8["audit"]["
max_rel_global"],
                energy_audit_max_rel_env=o8["audit"]["
max_rel_env"],
                residual_inf_T=o8["residual"]["max_T"],
                residual_inf_C=o8["residual"]["max_C"],
                T_final_min_C=float(o8["T_final"].min()),
                T_final_max_C=float(o8["T_final"].max()),
                wall_s=time.time() - t0,
                acceptance_step4=dict(
                    fixed_radius_degenerate=False,
                    t_dry_and_table6_diff_recorded=True,
                    energy_audit_le_1e8=bool(o8["audit"]["
max_rel_global"] <= 1e-8),
                    extremum_ok=bool(len(o8["violations"])
== 0),
                    n_not_converged_zero=bool(o8["picard"]["
n_not_converged"] == 0)),
                note=("§10.5 决策规则：即使耦合跑通，主口径
仍为预设 T（不换主口径），"
                      "本行作为「T 口径对照」行报告；若未跑
通则按 G3 退回预设口径。"))
    print("   耦合跑通：t_dry %.4f h vs 预设 %.4f h，Δ
%+.4f h（%+.3f %%）；"
          "能量审计 max_rel_global %.3e；未收敛步 %d；耗时
 %.0f s"
          % (c8 / 3600.0, base / 3600.0, (c8 - base) /
3600.0,
             100.0 * (c8 - base) / base, o8["audit"]["
max_rel_global"],
             o8["picard"]["n_not_converged"], o8["
```

```
    elapsed_s"]), flush=True)
     except AssertionError as exc:
         g8 = dict(status="未跑通（G3）", error=str(exc),
                   rule="§10.3 / G3：退回预设 T = T_a(t) 主口径，不追加时间；如实声明"
                        "「耦合温度场在本轮未获验证的收敛」。",
                   wall_s=time.time() - t0)
         print("    未跑通  执行 G3：%s" % exc, flush=True)
     return g8


def main():
     t_all = time.time()
     out = {"generated_at": datetime.now().isoformat(timespec="seconds"),
            "note": ("组 1 / 2 / 3b 见 results/q4_baseline.json（本文件不重复）；"
                     "本文件补齐组 3 / 4 / 5 / 7 / 8。")}
     base, o_base = t_dry_of()
     print("[基线] t_dry = %.6f h（N=%d、Δt=%.3g/%.3g）" % (base / 3600.0, N_CMP, DT1, DT2),
           flush=True)
     out["group3_appendix2_robin_anchor"] = _group3_anchor_appendix2()
     out["group4_shrinkage_isolation"] = _group4_shrinkage_isolation(base)
     out["group5_radius_interp"] = _group5_radius_interp(base)
     out["group7_geo_layer"] = _group7_geo_layer()
     out["group8_T_mode"] = _group8_T_mode(base)
     out["group10_eulerian_variant"] = dict(
         status="N/A",
         reason=("需要另写一套物理坐标 r 的 Eulerian 求解器（含 GCL 与伪源风险），"
                 "建模文档 §12.2 将其登记为「诊断（可选）」，且 §3.5 Step 2 已用"
```

```
                "解析论证给出该变体的失效模式（∝ Γ 的表观热源，量级 ω·Γ·2ln(R0/R_min)"
                "≈ ω·Γ·1.025 Γ 显热吸收）。本轮时间预算内不做，如实标注。"))

    out["wall_s"] = time.time() - t_all
    (RES / "q4_compare.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    print("[out] results/q4_compare.json；总耗时 %.1f s" % out["wall_s"])
    return 0


if __name__ == "__main__":
    sys.exit(main())
from __future__ import annotations

import json
import math
import sys
import time
from datetime import datetime
from pathlib import Path

import numpy as np

_LOG_ENCODING = "utf-8"
if getattr(sys.stdout, "reconfigure", None):
    sys.stdout.reconfigure(encoding=_LOG_ENCODING, errors="replace")

_HERE = Path(__file__).resolve()
_PROJ = _HERE.parent.parent
_SRC_ENTRY = _PROJ / "Q4"
```

```
sys.path.insert(0, _SRC_ENTRY.as_posix())

from solver_q4 import solve_q4, driver_at, C_DRY
from run_q4 import map_output_full, locate_crossing, R_OUT_CM,
    DT_OUT

RES = _PROJ / "results"
T_COMMON = 194400.0
PLANE = np.arange(DT_OUT, T_COMMON + 1e-9, DT_OUT)
TOL_LAYER2 = 5e-5
R_OUT_ALL = np.concatenate([R_OUT_CM, [np.nan]])


def run_case(**kw):
    t0 = time.time()
    o = solve_q4(stop_on_dry=False, t_end_override=T_COMMON,
                 snap_every_s=DT_OUT, snap_until_s=T_COMMON,
                 cmax_track=True, audit=True, verbose=False,
   **kw)
    xi, snap_t, snap_C, snap_R = o["xi"], o["snap_t"], o["
   snap_C"], o["snap_R"]
    n = int(round(T_COMMON / DT_OUT))
    if snap_t.size != n + 1:
        raise SystemExit("[FATAL] snapshot count %d != %d"
                         % (snap_t.size, n + 1))
    tab = np.empty((n, R_OUT_CM.size + 1))
    for k in range(n):
        tab[k] = map_output_full(xi, snap_C[k + 1], snap_R[k +
    1], R_OUT_CM, "pchip")
    cross = locate_crossing(o["cmax"]["t"], o["cmax"]["cmax"],
    C_DRY)
    out = dict(tab=tab, t=snap_t[1:], wall=time.time() - t0,
               t_dry_step=cross["t_step_s"] if cross else None
   ,
               t_dry_lin=cross["t_linear"] if cross else None,
```

```
                N=o["N"], dt1=kw.get("dt1"), dt2=kw.get("dt2"),
                mass_aud=o["mass_audit"]["max_rel"],
                picard_max=o["picard"]["max_iters"],
                picard_notconv=o["picard"]["n_not_converged"])
    del o
    return out


def rc_p(order_seq, h_seq):
    return [math.log(order_seq[i] / order_seq[i + 1]) / math.log(h_seq[i] / h_seq[i + 1])
            for i in range(len(order_seq) - 1)]


def per_col_max(d):
    return [float(np.max(d[:, j])) for j in range(d.shape[1])]


def _grid_independence():
    print("[A] 网格无关性：N ∈ {100,200,400,800}，Δt_I=1.25 s、Δt_II=5 s（生产 Δt）")
    seqN = [100, 200, 400, 800]
    ref_key = None
    runsN = {}
    for N in seqN:
        r = run_case(N=N, dt1=1.25, dt2=5.0, t_switch=14400.0, R_mode="shrink",
                     T_mode="prescribed", props="app4", face_rule="arith")
        runsN[N] = r
        print("   N=%3d  墙钟 %5.1f s  t_dry(步) %s h  Picard max %d 未收敛 %d  质量审计 %.1e"
              % (N, r["wall"],
                 ("%.4f" % (r["t_dry_step"] / 3600.0)) if r["t_dry_step"] else "None",
```

```
                r["picard_max"], r["picard_notconv"], r["
mass_aud"]), flush=True)
 ref = runsN[800]["tab"]
 grid = dict(config=dict(dt1=1.25, dt2=5.0, t_switch
=14400.0, t_window_s=T_COMMON,
                        n_out=int(round(T_COMMON / DT_OUT)
),
                        columns=["0…1.1 cm + 药材表面"]),
            t_dry_h={str(N): (runsN[N]["t_dry_step"] /
3600.0 if runsN[N]["t_dry_step"]
                              else None) for N in seqN},
            t_dry_lin_h={str(N): (runsN[N]["t_dry_lin"] /
3600.0
                                  if runsN[N]["t_dry_lin"]
 else None) for N in seqN},
            wall_s={str(N): runsN[N]["wall"] for N in seqN
},
            diff_vs_N800={}, )
 err_seq = []
 tt = runsN[800]["t"]
 for N in seqN:
     d = np.abs(runsN[N]["tab"] - ref)
     grid["diff_vs_N800"][str(N)] = dict(
         max_abs=float(d.max()), at_col=int(np.argmax(d.max
(axis=0))),
         at_t_s=float(tt[int(np.argmax(d.max(axis=1)))]),
         per_column_max=per_col_max(d),
         surface_col_max=float(d[:, -1].max()),
         fixed_col_max=float(d[:, :-1].max()),
         max_abs_t_ge_3600s=float(d[tt >= 3600.0].max()),
         max_abs_t_ge_21600s=float(d[tt >= 21600.0].max()),
         max_abs_t_ge_3600s_fixed=float(d[tt >= 3600.0][:,
:-1].max()),
         max_abs_t_ge_3600s_surface=float(d[tt >=
3600.0][:, -1].max()),
```

```
            per_time_max_fixed=[float(x) for x in d[:, :-1].max(axis=1)],
            per_time_max_surface=[float(x) for x in d[:, -1]])
        if N < 800:
            err_seq.append(float(d.max()))
    h_seq = [1.0 / N for N in seqN[:3]]
    grid["observed_order_p"] = rc_p(np.array(err_seq) + 1e-300, h_seq)
    grid["richardson_extrapolated_err"] = (
        err_seq[-1] / (2 ** max(grid["observed_order_p"][-1], 0.1) - 1)
        if len(err_seq) >= 2 else None)
    tdy = [grid["t_dry_h"][str(N)] for N in seqN if grid["t_dry_h"][str(N)]]
    grid["t_dry_spread_h"] = (max(tdy) - min(tdy)) if tdy else None
    grid["t_dry_spread_rel"] = ((max(tdy) - min(tdy)) / min(tdy)) if tdy else None
    grid["layer2_tol_kgkg"] = TOL_LAYER2
    grid["layer2_pass"] = bool(grid["diff_vs_N800"]["200"]["max_abs"] < TOL_LAYER2)
    return seqN, grid, runsN, tt


def _time_convergence():
    print()
    print("[B] 数值收敛：Δt_I ∈ {20,10,5,2.5,1.25,0.625} s、Δt_II ∈ {60,30,20,10,5,2.5} s（N=200）")
    dt_pairs = [(20.0, 60.0), (10.0, 30.0), (5.0, 20.0), (2.5, 10.0),
                (1.25, 5.0), (0.625, 2.5)]
    runsDT = {}
    for d1, d2 in dt_pairs:
        key = "I%g_II%g" % (d1, d2)
        r = run_case(N=200, dt1=d1, dt2=d2, t_switch=14400.0,
```

```
    R_mode="shrink",
                      T_mode="prescribed", props="app4",
face_rule="arith")
        runsDT[key] = r
        print("   Δt_I=%-5g Δt_II=%-5g 墙钟 %5.1f s  t_dry(步)
 %s h  Picard max %d 未收敛 %d"
              % (d1, d2, r["wall"],
                 ("%.4f" % (r["t_dry_step"] / 3600.0)) if r["
t_dry_step"] else "None",
                 r["picard_max"], r["picard_notconv"]), flush=
True)
    refd = runsDT["I0.625_II2.5"]["tab"]
    dtd = dict(config=dict(N=200, t_switch=14400.0, t_window_s
=T_COMMON,
                           n_out=int(round(T_COMMON / DT_OUT))
),
               t_dry_h={k: (v["t_dry_step"] / 3600.0 if v["
t_dry_step"] else None)
                        for k, v in runsDT.items()},
               t_dry_lin_h={k: (v["t_dry_lin"] / 3600.0 if v["
t_dry_lin"] else None)
                            for k, v in runsDT.items()},
               wall_s={k: v["wall"] for k, v in runsDT.items()
}, diff_vs_finest={})
    ttd = runsDT["I0.625_II2.5"]["t"]
    for k, v in runsDT.items():
        d = np.abs(v["tab"] - refd)
        dtd["diff_vs_finest"][k] = dict(
            max_abs=float(d.max()), at_t_s=float(ttd[int(np.
argmax(d.max(axis=1)))]),
            per_column_max=per_col_max(d),
            surface_col_max=float(d[:, -1].max()),
            fixed_col_max=float(d[:, :-1].max()),
            max_abs_t_ge_3600s=float(d[ttd >= 3600.0].max()),
            max_abs_t_ge_21600s=float(d[ttd >= 21600.0].max())
```

```
    ,
2927            max_abs_t_ge_3600s_fixed=float(d[ttd >= 3600.0][:,
     :-1].max()),
2928            max_abs_t_ge_3600s_surface=float(d[ttd >=
    3600.0][:, -1].max()),
2929            per_time_max_fixed=[float(x) for x in d[:, :-1].
    max(axis=1)],
2930            per_time_max_surface=[float(x) for x in d[:, -1]])
2931     tdy_dt = [dtd["t_dry_h"][k] for k in dtd["t_dry_h"] if dtd
    ["t_dry_h"][k]]
2932     dtd["t_dry_spread_h"] = (max(tdy_dt) - min(tdy_dt)) if
    tdy_dt else None
2933     dtd["t_dry_spread_rel"] = ((max(tdy_dt) - min(tdy_dt)) /
    min(tdy_dt)) if tdy_dt else None
2934     dtd["layer2_tol_kgkg"] = TOL_LAYER2
2935     dtd["layer2_pass_fullwindow_vs_finest"] = bool(
2936         dtd["diff_vs_finest"]["I1.25_II5"]["max_abs"] <
    TOL_LAYER2)
2937     dtd["layer2_pass_t_ge_3600s_vs_finest"] = bool(
2938         dtd["diff_vs_finest"]["I1.25_II5"]["max_abs_t_ge_3600s
    "] < TOL_LAYER2)
2939     return dtd
2940
2941
2942 def _convergence_verdict(grid, dtd):
2943     dtd["production_choice"] = dict(
2944         dt1=1.25, dt2=5.0,
2945         reason=("两段式起点（§7.3）经 'diff_vs_finest' 定案:
    生产档 vs 最细档"
2946                 "（Δt_I=0.625 s / Δt_II=2.5 s）在 t≥3600 s 的
    逐点最大差 "
2947                 "%.3e < %.0e（② 层线）; t<3600 s 窗口在本生产
    配置下未达 ② 层线"
2948                 "（最大差出现在 t=60 s 表面列; 见 G5 口径降级
    与 §7.2 注）。"
```

```
                % (dtd["diff_vs_finest"]["I1.25_II5"]["max_abs_t_ge_3600s"],
                   TOL_LAYER2)),
        )
    grid["layer2_pass_fullwindow_vs_N800"] = bool(
        grid["diff_vs_N800"]["400"]["max_abs"] < TOL_LAYER2)
    grid["layer2_pass_t_ge_3600s_vs_N800"] = bool(
        grid["diff_vs_N800"]["400"]["max_abs_t_ge_3600s"] < TOL_LAYER2)
    grid["production_choice"] = dict(
        N=400,
        reason=("网格由 'diff_vs_N800' 定案：N=400 vs N=800 在 t≥3600 s 的逐点最大差 "
                "%.3e < %.0e（② 层线）；t=60 s 的 %.3e 属 t<1 h 窗口，"
                "本生产配置（N=400）在该窗口未达 ② 层线"
                "（δ(60 s)=2.507×10⁻² cm ≈ 5.01Δr@N=400  边界层已被覆盖、加密仍在收敛；"
                "见 'g5_disclosure'）。"
                % (grid["diff_vs_N800"]["400"]["max_abs_t_ge_3600s"], TOL_LAYER2,
                   grid["diff_vs_N800"]["400"]["max_abs"])))
    grid["g5_disclosure"] = dict(
        triggered=bool(not grid["layer2_pass_fullwindow_vs_N800"]
                       or not dtd["layer2_pass_fullwindow_vs_finest"]),
        statement=("G5（§15.1）：全窗口（含 t<3600 s）逐点最大差达不到 5×10⁻⁵。"
                   "不做无限加密；交付表的有效位数按实际不确定度分层声明："
                   "t ≥ 1 h 由 ② 层线支撑（4 位小数）；t < 1 h 的列不支撑第 4 位"
                   "—— 本生产配置（N=400）在该窗口实测 N400-vs-N800 最大差 %.3e"
```

```
                    "（t=60 s 表面列），而 δ(60 s)=2.507×10□² cm ≈ 5.01Δr@N=400 表明"
                    "初始边界层已被覆盖（不是「不可分辨」）；加密序列在该窗口仍以 "
                    "p≈2.35 收敛、N=800 全程仅 %.1f s  继续加密可望达标"
                    "（按 p 外推 N=1600 ≈ 2.4×10□⁵，本轮未做）。"
                    "该分层在 results4.json 与论文章节中如实写明。"
                    % (grid["diff_vs_N800"]["400"]["max_abs"], grid["wall_s"]["800"])))


def _crossing_check():
    print()
    print("[C] 两段式跨接检查：两段式 vs 单段 Δt=1.25 s（跨接窗口 t=14400/18000/21600/25200 s，N=200）")
    single = run_case(N=200, dt1=1.25, dt2=1.25, t_switch=1e18, R_mode="shrink",
                      T_mode="prescribed", props="app4", face_rule="arith")
    two = run_case(N=200, dt1=1.25, dt2=5.0, t_switch=14400.0, R_mode="shrink",
                   T_mode="prescribed", props="app4", face_rule="arith")
    jump = {}
    for ts in (14400.0, 18000.0, 21600.0, 25200.0):
        i = int(round(ts / DT_OUT)) - 1
        d = np.abs(two["tab"][i] - single["tab"][i])
        jump["t_%ds" % int(ts)] = dict(max_abs=float(d.max()),
                                       per_column=[float(x) for x in d])
    bref = {}
    for ts in (14400.0,):
```

```
        i0 = int(round(ts / DT_OUT)) - 1
        for off in (-1, 0, 1):
            j = i0 + off
            d = np.abs(two["tab"][j] - single["tab"][j])
            bref["t_%ds" % int(two["t"][j])] = float(d.max())
    cross_chk = dict(
        config="两段式 Δt_I=1.25 s  Δt_II=5 s（切换 t=14400 s）vs 单段 Δt=1.25 s 全程（N=200）",
        window_points=jump, near_switch=bref,
        max_abs_diff_switch_window=float(max(v["max_abs"] for v in jump.values())),
        max_abs_diff_near_switch=float(max(bref.values())),
        tol=TOL_LAYER2,
        two_stage_ok=bool(max(v["max_abs"] for v in jump.values()) < TOL_LAYER2
                          and max(bref.values()) < TOL_LAYER2),
        single_stage_wall_s=single["wall"], two_stage_wall_s=two["wall"])
    print("  跨接窗口最大差 %.3e；跨接点邻域最大差 %.3e；判据 %.0e  %s"
          % (cross_chk["max_abs_diff_switch_window"],
             cross_chk["max_abs_diff_near_switch"], TOL_LAYER2,
             "PASS" if cross_chk["two_stage_ok"] else "FAIL"), flush=True)
    return cross_chk


def _mapping_precision(runsN, tt):
    print()
    print("[D] 映射精度：N=800 的解在同一 r_out 上与 N=200 的 PCHIP 值比较")
    dmap = np.abs(runsN[400]["tab"] - runsN[800]["tab"])
    mapping = dict(
```

```
        grid_plus_mapping_uncertainty=dict(
            N_pair="400 vs 800",
            max_abs_vs_N800=float(dmap.max()),
            max_abs_t_ge_3600s=float(dmap[tt >= 3600.0].max())
,
            max_abs_t_ge_21600s=float(dmap[tt >= 21600.0].max
()),
            surface_col=float(dmap[:, -1].max()),
            fixed_cols=float(dmap[:, :-1].max()),
            per_column_max=per_col_max(dmap)),
        pchip_vs_linear_source="results/q4_mapping_uncertainty
.csv（生产运行产出）",
        tol_layer2=TOL_LAYER2,
        pass_=bool(dmap[:, :-1].max() < TOL_LAYER2),
        note=("该差为映射 + 网格的联合不确定度（§8.4 第 5
条）；末尾 '药材表面' 列"
              "为两档网格的节点值之差（不插值），性质是网格不
确定度。"),
    )
    return dmap, mapping


def _event_convergence(grid, dtd):
    _sp_dt_h = dtd["t_dry_spread_h"]
    _sp_grid_h = grid["t_dry_spread_h"]
    _pc = dtd["production_choice"]
    event = dict(
        grid_sequence=grid["t_dry_h"], dt_sequence=dtd["
t_dry_h"],
        note=("§9.2 推论 2（修订版）：'t_dry' 在生产档 '(%.2f,
 %g) s' 附近继续加密收益"
              "远小于网格项；但粗档区间内 Δt 仍是显著误差源
—— 本序列极差 "
              "'%r h'（= '%.1f s'，step）大于四档网格极差 "
              "'%r h'（= '%.1f s'），故『序列极差应远小于 ±15
```

```
    .2 s』"
                "的说法被本序列自身否证。原句保留作留痕。"
               % (_pc["dt1"], _pc["dt2"], _sp_dt_h, _sp_dt_h * 3600.0,
                  _sp_grid_h, _sp_grid_h * 3600.0)),
        note_legacy=("§9.2 推论 2：'t_dry' 由场值误差决定，缩小 Δt 不改善；"
                     "故序列的极差应远小于场值误差换算的 ±15.2 s 量级。"),
     )
     return event


def _write_and_summary(grid, dtd, cross_chk, mapping, event, seqN, dmap, t_all):
     out = {"generated_at": datetime.now().isoformat(timespec="seconds"),
            "A_grid_independence": grid, "B_time_convergence": dtd,
            "C_two_stage_crossing": cross_chk, "D_mapping_precision": mapping,
            "E_event_convergence": event,
            "wall_s": time.time() - t_all}
     (RES / "q4_convergence.json").write_text(
         json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
     print()
     g200, gp = grid["diff_vs_N800"]["400"], dtd["diff_vs_finest"]["I1.25_II5"]
     print("[summary] Δξ：N=400 vs N=800 全窗口逐点最大差 %.3e（判据 %.0e） %s；"
           "观测 p = %s"
           % (g200["max_abs"], TOL_LAYER2,
              "PASS" if grid["layer2_pass"] else "FAIL",
              ["%.2f" % p for p in grid["observed_order_p"]]))
```

```
3072     print("        分层读数：t≥3600 s %.3e（固定列 %.3e / 表面列 %.3e）；"
3073           "t≥21600 s %.3e；最大差出现于 t=%s s（列 %d）"
3074           % (g200["max_abs_t_ge_3600s"], g200["max_abs_t_ge_3600s_fixed"],
3075              g200["max_abs_t_ge_3600s_surface"], g200["max_abs_t_ge_21600s"],
3076              g200["at_t_s"], g200["at_col"]))
3077     print("[summary] Δt：生产档 vs 最细档 全窗口逐点最大差 %.3e；分层 t≥3600 s %.3e"
3078           "（固定列 %.3e / 表面列 %.3e）；最大差出现于 t=%s s"
3079           % (gp["max_abs"], gp["max_abs_t_ge_3600s"], gp["max_abs_t_ge_3600s_fixed"],
3080              gp["max_abs_t_ge_3600s_surface"], gp["at_t_s"]))
3081     print("[summary] t_dry 序列：Δξ %s h（极差 %s h）；Δt %s h（极差 %s h）"
3082           % (["%s" % grid["t_dry_h"][str(N)] for N in seqN],
3083              grid["t_dry_spread_h"], ["%s" % v for v in dtd["t_dry_h"].values()],
3084              dtd["t_dry_spread_h"]))
3085     print("[summary] 两段式跨接：%s；映射（vs N=800）%.3e；总耗时 %.1f s"
3086           % ("PASS" if cross_chk["two_stage_ok"] else "FAIL", dmap.max(),
3087              out["wall_s"]))
3088     print("[out] results/q4_convergence.json")
3089
3090
3091 def main():
3092     RES.mkdir(parents=True, exist_ok=True)
3093     t_all = time.time()
3094     results = {}
3095
3096     seqN, grid, runsN, tt = _grid_independence()
3097     dtd = _time_convergence()
```

```
    _convergence_verdict(grid, dtd)
    cross_chk = _crossing_check()
    dmap, mapping = _mapping_precision(runsN, tt)
    event = _event_convergence(grid, dtd)
    _write_and_summary(grid, dtd, cross_chk, mapping, event, seqN, dmap, t_all)
    return 0


if __name__ == "__main__":
    rc = main()
    sys.exit(rc)
from __future__ import annotations

import json
import math
import sys
import time
from datetime import datetime
from pathlib import Path

import numpy as np


def utf8_stdout():
    _SCREEN = sys.stdout
    if getattr(_SCREEN, "reconfigure", None):
        _SCREEN.reconfigure(encoding="utf-8", errors="replace")


utf8_stdout()

_SELF_FILE = Path(__file__).resolve()
_SUBMIT = _SELF_FILE.parents[1]
```

```
sys.path.insert(0, "%s/Q4" % _SUBMIT.as_posix())

from solver_q4 import (
    solve_q4, driver_at, radius_at, att2, C_DRY, HM_DEFAULT, H_DEFAULT,
    D0_AP4, BETA_AP4, E_D_AP4, R0, R_MIN, LENGTH_CM, C_INIT, CA_FREEZE,
    T_FREEZE_S, TA_FREEZE_C,
)

RES = _SUBMIT / "results"
N_SENS = 400
DT1, DT2 = 1.25, 5.0
DH_VAP = 2.26e6
RHO_D_BORROW = (760.0 + 90.0 * C_INIT) / (1.0 + C_INIT)


def t_dry_of(**kw):
    o = solve_q4(N=N_SENS, dt1=DT1, dt2=DT2, t_switch=T_FREEZE_S, R_mode="shrink",
                 T_mode="prescribed", props="app4", bc="robin", face_rule="arith",
                 stop_on_dry=True, snap_every_s=0.0, cmax_track=True, audit=True,
                 verbose=False, **kw)
    if o["status"] != "ok":
        return None, o
    tt, cm = o["cmax"]["t"], o["cmax"]["cmax"]
    i = int(np.flatnonzero(cm <= C_DRY)[0])
    if i == 0:
        return float(tt[0]), o
    t0, t1, c0, c1 = tt[i - 1], tt[i], cm[i - 1], cm[i]
    return float(t0 + (C_DRY - c0) / (c1 - c0) * (t1 - t0)), o


```

```
3161 def _scan_single_and_2d(sens, base, rows):
3162     def scan(tag, **kw):
3163         t0 = time.time()
3164         td, _ = t_dry_of(**kw)
3165         rows[tag] = dict(t_dry_s=td, t_dry_h=(td / 3600.0) if
     td else None,
3166                          delta_h=((td - base) / 3600.0) if td
     else None,
3167                          delta_pct=(100.0 * (td - base) / base
     ) if td else None,
3168                          wall_s=time.time() - t0)
3169         print("   %-26s t_dry %s h  Δ %+8.4f h (%+6.2f %%)
     %.1f s"
3170               % (tag, ("%.4f" % (td / 3600.0)) if td else "
     None",
3171                  (td - base) / 3600.0 if td else 0.0,
3172                  100.0 * (td - base) / base if td else 0.0,
     rows[tag]["wall_s"]),
3173               flush=True)
3174     print("[1] 单参数扫描（h、h_m、D□、β、E_D、ρc_p）")
3175     for m in (0.5, 2.0):
3176         scan("h_x%g" % m, H=H_DEFAULT * m)
3177     for m in (0.5, 2.0):
3178         scan("hm_x%g" % m, HM=HM_DEFAULT * m)
3179     for m in (0.5, 2.0):
3180         scan("D0_x%g" % m, D0=D0_AP4 * m)
3181     for v in (0.27, 0.33):
3182         scan("beta_%g" % v, BETA=v)
3183     for v in (3657.5, 4042.5):
3184         scan("E_D_%g" % v, E_D=v)
3185     for m in (0.9, 1.1):
3186         scan("rho_scale_%g" % m, RHO_SCALE=m)
3187     for m in (0.9, 1.1):
3188         scan("cp_scale_%g" % m, CP_SCALE=m)
3189     sens["single_parameter"] = rows
```

```
3190
3191      print("[2] 二维组合 h_m × h（3×3 = 9 组，§11.4）")
3192      grid2 = {}
3193      for mh in (0.5, 1.0, 2.0):
3194          for mm in (0.5, 1.0, 2.0):
3195              tag = "h_x%g_hm_x%g" % (mh, mm)
3196              td, _ = t_dry_of(H=H_DEFAULT * mh, HM=HM_DEFAULT *
      mm)
3197              grid2[tag] = dict(t_dry_s=td, t_dry_h=(td /
    3600.0) if td else None,
3198                                delta_pct=(100.0 * (td - base) /
      base) if td else None)
3199              print("   %-16s t_dry %8.4f h  Δ %+6.2f %%"
3200                    % (tag, td / 3600.0, 100.0 * (td - base) /
    base), flush=True)
3201      v2 = [v["t_dry_h"] for v in grid2.values() if v["t_dry_h"
    ]]
3202      sens["h_hm_2d"] = dict(
3203          grid=grid2, min_h=min(v2), max_h=max(v2),
3204          spread_h=max(v2) - min(v2),
3205          spread_pct=100.0 * (max(v2) - min(v2)) / min(v2),
3206          degeneracy=(" 在预设 T 口径下 'h' 恒无影响（能量方程不
    求解） 3×3 的 9 组"
3207                      "退化为 3 个不同值（每列相同）。这是「预设
      T 口径」的直接证据，"
3208                      "必须如实报告：本模型对 'h' 的灵敏度低于真
    实情况（§11.4）。"),
3209          direction=("h_m   表面更快  t_dry （实测 hm×0.5 +6.57
    %、hm×2 −2.00 %，"
3210                     "次线性，因 h_m 同时抬高表面浓度梯度的驱动
    压差）；"
3211                     "h 在本口径下影响恒为 0（实测 h×0.5 / h×2
    的 'delta_h' 逐位为 "
3212                     "0.0；预设 T 口径下能量方程不求解、'h' 无处
    进入 —— 属该口径的必然，"
```

```
                    "不是「影响被弱化」）。"))
    return sens, rows, grid2


def _scan_scenarios(sens, base):
    print("[3] 驱动外推情景 E0/E1/E2（T_a 与 C_a 平台，§12.2 组 6）")
    scen = {}
    for ts in ("freeze_last", "tail_mean", "row_min", "row_max"):
        for cs in ("freeze_last", "tail_mean", "row_min", "row_max"):
            tag = "Ta_%s__Ca_%s" % (ts, cs)
            if ts == "freeze_last" and cs == "freeze_last":
                continue
            td, _ = t_dry_of(ta_scenario=ts, ca_scenario=cs)
            scen[tag] = dict(t_dry_h=(td / 3600.0) if td else None,
                             delta_pct=(100.0 * (td - base) / base) if td else None)
            print("   %-34s t_dry %s h  Δ %s %%"
                  % (tag, ("%.4f" % (td / 3600.0)) if td else "None",
                     ("%+.2f" % (100.0 * (td - base) / base)) if td else "-"),
                  flush=True)
    v6 = [v["t_dry_h"] for v in scen.values() if v["t_dry_h"]]
    sens["driver_scenarios"] = dict(rows=scen, min_h=min(v6) if v6 else None,
                                    max_h=max(v6) if v6 else None,
                                    spread_h=(max(v6) - min(v6)) if v6 else None,
                                    note="E6 情景区间（§12.2 组 6）；论文以区间表述")
```

```
    return sens, scen, v6


def _e1_zero_d(base):
    print("[4] E1 潜热亏缺：0-D 集总对照（借用 ΔH_vap，只进误差分析节）")
    o_base = solve_q4(N=N_SENS, dt1=DT1, dt2=DT2, t_switch=T_FREEZE_S,
                      R_mode="shrink", T_mode="prescribed", props="app4",
                      stop_on_dry=True, snap_every_s=60.0, cmax_track=True,
                      audit=False, verbose=False)
    snap_t, snap_R, snap_C = o_base["snap_t"], o_base["snap_R"], o_base["snap_C"]
    tt, cm = o_base["cmax"]["t"], o_base["cmax"]["cmax"]
    Cs_of_t = np.interp(snap_t, tt, o_base["cmax"]["cR"])
    Cmax_of_t = np.interp(snap_t, tt, cm)
    t_end_e1 = float(o_base["t_end"])
    dtE = 60.0
    tE = np.arange(0.0, t_end_e1 + 1e-9, dtE)
    RE = radius_at(tE, "shrink")
    CsE = np.interp(tE, snap_t, Cs_of_t)
    TaE = np.array([driver_at(t)[0] for t in tE])
    CaE = np.array([driver_at(t)[1] for t in tE])
    Cmid = np.interp(tE, snap_t, Cmax_of_t)
    c_eff = (1850.0 + 2150.0 * Cmid / (Cmid + 1.0))
    Tm = np.empty(tE.size)
    Tm[0] = 28.0
    for k in range(tE.size - 1):
        R_ = RE[k]
        cap = RHO_D_BORROW * R_ * R_ / 2.0 * c_eff[k]
        q_conv = R_ * H_DEFAULT * (TaE[k] - Tm[k])
        q_evap = RHO_D_BORROW * R_ * HM_DEFAULT * (CsE[k] - CaE[k]) * DH_VAP
```

```
        Tm[k + 1] = Tm[k] + dtE * (q_conv - q_evap) / cap
    dT = TaE - Tm
    dT_qs = (RHO_D_BORROW * HM_DEFAULT * DH_VAP / H_DEFAULT) *
    (CsE - CaE)
    P_evap = 2.0 * np.pi * RE * RHO_D_BORROW * HM_DEFAULT * (
   CsE - CaE) * DH_VAP
    i_dry = int(np.argmin(np.abs(tE - base)))
    P_avg = float(np.trapezoid(P_evap[:i_dry + 1], tE[:i_dry +
    1]) / base)
    dT_avg = P_avg / (H_DEFAULT * 2.0 * np.pi * R0)
    t_eff_series = np.column_stack([tE, TaE - dT, TaE - dT_qs
   ])
    return tE, TaE, dT_qs, P_avg, dT_avg


def _e1_adopt(tE, TaE, dT_qs, dT_avg, base):
    def t_dry_with_Teff(T_eff_series, t_end_ov):
        import solver_q4 as S
        t0, T0, C0 = S.att1()
        t_new = np.arange(0.0, T_FREEZE_S + 1e-9, 60.0)
        T_new = np.interp(t_new, tE, T_eff_series)
        C_new = np.interp(t_new, t0, C0)
        shift = float(TaE[0] - T_eff_series[0])
        freeze_orig = S.TA_FREEZE_C
        S._ATT1_CACHE = (t_new, T_new, C_new)
        S.TA_FREEZE_C = freeze_orig - shift
        S._DRIVER_CACHE.clear()
        try:
            oo = solve_q4(N=N_SENS, dt1=DT1, dt2=DT2, t_switch
   =T_FREEZE_S,
                          R_mode="shrink", T_mode="prescribed"
   , props="app4",
                          stop_on_dry=False, t_end_override=
   t_end_ov,
                          snap_every_s=0.0, cmax_track=True,
```

```
    audit=False, verbose=False)
        finally:
            S._ATT1_CACHE = (t0, T0, C0)
            S.TA_FREEZE_C = freeze_orig
            S._DRIVER_CACHE.clear()
        cm2 = oo["cmax"]["cmax"]
        ii = np.flatnonzero(cm2 <= C_DRY)
        if ii.size == 0:
            return None, float(oo["t_final"]), float(cm2.min()
    )
        j = int(ii[0])
        if j == 0:
            return float(oo["cmax"]["t"][0]), float(oo["
    t_final"]), float(cm2.min())
        ta_, tb_ = oo["cmax"]["t"][j - 1], oo["cmax"]["t"][j]
        ca_, cb_ = cm2[j - 1], cm2[j]
        return (float(ta_ + (C_DRY - ca_) / (cb_ - ca_) * (tb_
     - ta_)),
                float(tE[-1]), float(cm2.min()))

     T_eff_inst = TaE - dT_qs
     T_CAP_E1 = 259200.0
     t_a, tf_a, cmin_a = t_dry_with_Teff(T_eff_inst, T_CAP_E1)
     T_eff_avg = TaE - dT_avg
     t_b, tf_b, cmin_b = t_dry_with_Teff(T_eff_avg, T_CAP_E1)
     E1 = (t_b - base) / base if t_b else None
     return t_a, cmin_a, t_b, cmin_b, E1, T_CAP_E1


def _e1_report(tE, dT_qs, P_avg, dT_avg, T_CAP_E1, t_a, cmin_a
    , t_b, cmin_b, E1):
     print("   δT_qs 峰值 %.2f K（t=%.0f s）；平均 ΔT = %.4f K
    （P_avg = %.4f W/m）"
           % (dT_qs.max(), tE[int(np.argmax(dT_qs))], dT_avg,
    P_avg), flush=True)
```

```
 print("   读法(a) 瞬时 δT(t)：72 h 窗口内 %s（C_max 终值 %.5f）"
       % ("达标 t_dry=%s h" % ("%.4f" % (t_a / 3600.0)) if t_a else "未达标",
          cmin_a), flush=True)
 print("   读法(b) 平均 ΔT = %.4f K：t_dry^corr %s h；E1 = %s %%"
       % (dT_avg, ("%.4f" % (t_b / 3600.0)) if t_b else "未达标",
          ("%+.2f" % (100 * E1)) if E1 is not None else "-"), flush=True)
 e1 = dict(method=("0-D 集总对照（§11.2 (Q4-34)）；'C_s(t)' 与 'R(t)' 取自主模型，"
                   "不含新的水分模型。两种读法：(a) 瞬时 δT(t)（准稳态表面平衡）；"
                   "(b) 平均 ΔT（先把蒸发功率在干燥窗内做时间平均，再取常数偏移）。"),
           borrowed_parameters=dict(dH_vap_J_per_kg=DH_VAP, rho_d_kg_m3=RHO_D_BORROW,
                                    note="题目未给，借自文献 / 附录 4 折算式；"
                                         "只作量级估计，绝不进交付数值"),
           delta_T_qs_max_K=float(dT_qs.max()),
           delta_T_qs_max_at_s=float(tE[int(np.argmax(dT_qs))]),
           delta_T_qs_final_K=float(dT_qs[-1]),
           P_evap_avg_W_per_m=P_avg,
           delta_T_avg_K=float(dT_avg),
           reading_a_instantaneous=dict(
               t_dry_h=(t_a / 3600.0) if t_a else None,
               reached_within_s=T_CAP_E1, C_max_final=cmin_a,
               verdict=("在 72 h 窗口内未达标" if not t_a else "达标"),
```

```
                    note=("瞬时读法在早段把表面蒸发通量按 'C_s(0) = 2.55' 全额计入，"
                          "δT 峰值 %.1f K，材料温度被压到远低于初值 —— 这是 0-D 集总"
                          "的失效模式（真实药材有内部导热供热，表面不会瞬间掉到湿球温度）。"
                          "该读法只作定性说明，不作为 E1 数值。"
                          % dT_qs.max())),
                reading_b_averaged=dict(
                    t_dry_h=(t_b / 3600.0) if t_b else None,
                    reached_within_s=T_CAP_E1, C_max_final=cmin_b),
                E1_relative=E1,
                E1_direction="t_dry 偏短（潜热缺失 → 温度偏高 → D 偏高）",
                E1_adopted="读法 (b)（平均 ΔT）；读法 (a) 的结果如实登记但不采用",
                not_main_result=True,
                note="E1 的修正结果不得作为主结果（§11.6 纪律 3）")
    return e1


def _e2_e3_blocks(grid2):
    e2 = dict(R_over_L=R0 / (LENGTH_CM / 100.0),
              area_ratio=1.0 + R0 / (LENGTH_CM / 100.0),
              upper_bound_pct=100.0 * R0 / (LENGTH_CM / 100.0),
              direction="t_dry 偏长（1D 无限长漏掉端面传质）",
              note=("与 E1/E3 方向相反、部分相消（§11.3）；题目未给轴向数据，"
                    "各向同性收缩下 R/L 不变。"))

    v3 = [v["t_dry_h"] for v in grid2.values() if v["t_dry_h"]]
```

```
    e3 = dict(h_hm_3x3_spread_h=max(v3) - min(v3),
              h_hm_3x3_spread_pct=100.0 * (max(v3) - min(v3))
    / min(v3),
              direction="双向",
              borrowed_params=False)
    return v3, e2, e3


def _e4_e5_e6_blocks(RES, v6):
    e4 = dict(source="results/q4_convergence.json", direction=
    "双向")
    pconv = RES / "q4_convergence.json"
    if pconv.exists():
        cj = json.loads(pconv.read_text(encoding="utf-8"))
        e4["grid_N400_vs_N800_t_ge_3600s"] = cj["
    A_grid_independence"][
            "diff_vs_N800"]["400"]["max_abs_t_ge_3600s"]
        e4["grid_N400_vs_N800_fullwindow"] = cj["
    A_grid_independence"][
            "diff_vs_N800"]["400"]["max_abs"]
        e4["dt_prod_vs_finest_t_ge_3600s"] = cj["
    B_time_convergence"][
            "diff_vs_finest"]["I1.25_II5"]["max_abs_t_ge_3600s
    "]
        e4["dt_prod_vs_finest_fullwindow"] = cj["
    B_time_convergence"][
            "diff_vs_finest"]["I1.25_II5"]["max_abs"]
        e4["t_dry_grid_spread_h"] = cj["A_grid_independence"][
    "t_dry_spread_h"]
        e4["t_dry_dt_spread_h"] = cj["B_time_convergence"]["
    t_dry_spread_h"]
        e4["statement"] = ("t ≥ 1 h：Δξ 与 Δt 的联合离散不确定
    度均 < 5×10□⁵（② 层线）；"
                           "t < 1 h：由初始边界层主导，最大差
    %.2e（表面列 @ t=60 s）"
```

```
                          % cj["A_grid_independence"]["diff_vs_N800"]["400"]["max_abs"])
    tt2, RR2 = att2()
    from scipy.interpolate import PchipInterpolator
    tc = np.linspace(0.0, min(259200.0, 200000.0), 4001)
    lin = np.interp(tc, tt2, RR2)
    pchp = PchipInterpolator(tt2, RR2)(tc)
    dR = np.abs(lin - pchp)
    quant = 0.001 / 100.0
    tt_int = np.arange(0.0, 259200.0 + 1e-9, 1800.0)
    e5 = dict(R_quantization_m=quant,
              pchip_monotone_ok=bool(np.all(np.diff(pchp) <= 1e-15)),
              pchip_range_ok=bool(pchp.min() >= R_MIN - 1e-12 and pchp.max() <= R0 + 1e-12),
              max_abs_diff_linear_vs_pchip_m=float(dR.max()),
              max_rel_diff=float((dR / np.maximum(lin, 1e-30)).max()),
              quant_equivalent_pct=100.0 * quant / R_MIN,
              direction="双向，小",
              note=("量化台阶 0.001 cm = %.4f %% of R_min；PCHIP 通过 '0.01198 ≤ R ≤ 0.02'"
                    "且 'Δ ≤ 0' 逐点校验（不合格则判 PCHIP 不合格，§12.2 组 5）。"
                    "主口径用分段线性（§4.3），PCHIP 仅作对照。"
                    % (100.0 * quant / R_MIN)))
    e6 = dict(ta_scenario_spread_h=(max(v6) - min(v6)) if v6 else None,
              ca_scenario_min_h=min(v6) if v6 else None,
              ca_scenario_max_h=max(v6) if v6 else None,
              direction="偏长为主（冻结压低后期驱动力）",
              borrowed_params=False)
    return e4, e5, e6, dR, quant

```

```
3418
3419 def _write_outputs(sens, base, e1, e2, e3, e4, e5, e6, E1,
     t_all):
3420     budget = {"generated_at": datetime.now().isoformat(
     timespec="seconds"),
3421               "baseline_t_dry_h": base / 3600.0, "
     baseline_t_dry_s": base,
3422               "E1_latent_heat": e1, "E2_end_face": e2, "
     E3_h_hm": e3,
3423               "E4_discrete": e4, "E5_R_interp": e5, "E6_driver
     ": e6,
3424               "reporting_rule": ("E1 – E4 分别报告，不做「总误
     差 = 各项相加」"
3425                                  "（方向不同、量级不同、互不独
     立）；论文以区间 + 方向"
3426                                  "表述（§11.1）；E1 的借用参数
     结果只进误差分析节。"),
3427               "interval_statement": (
3428                   "主结果 t_dry = %.4f h（E0 主情景、N=%d、Δt
     =%.3g/%.3g s）；"
3429                   "考虑 E1（本模型缺潜热  t_dry 偏短 %s  真值
     变长，上端 = base×(1+E1)）"
3430                   "与 E2（本模型 1D 无限长  t_dry 偏长 ≤ +%.1f
      %%  真值变短，"
3431                   "下端 = base×(1−E2)）的方向性偏差后，合理区
     间约为 [%.2f, %.2f] h"
3432                   "（方向性上界，非统计置信区间）。"
3433                   % (base / 3600.0, N_SENS, DT1, DT2,
3434                      ("%.1f %%" % (100 * abs(E1))) if E1 is
     not None else "待定",
3435                      e2["upper_bound_pct"],
3436                      base / 3600.0 * (1.0 - e2["
     upper_bound_pct"] / 100.0),
3437                      base / 3600.0 * (1.0 + (E1 if E1 else
     0.0)))))}
```

```
    (RES / "q4_error_budget.json").write_text(
        json.dumps(budget, ensure_ascii=False, indent=2),
    encoding="utf-8")
    sens["wall_s"] = time.time() - t_all
    (RES / "q4_sensitivity.json").write_text(
        json.dumps(sens, ensure_ascii=False, indent=2),
    encoding="utf-8")


def _print_summary(grid2, v3, v6, E1, dR, quant, sens):
    print("[summary] h_m×h 3×3: t_dry ∈ [%.4f, %.4f] h（极差
    %.4f h = %.2f %%）; %s"
          % (min(v3), max(v3), max(v3) - min(v3),
             100.0 * (max(v3) - min(v3)) / min(v3), grid2 and
    "方向解释见 JSON"))
    print("[summary] 驱动情景: t_dry ∈ [%.4f, %.4f] h（极差
    %.4f h）"
          % (min(v6), max(v6), max(v6) - min(v6)) if v6 else "
    ")
    print("[summary] E1 = %s %%（借用 ΔH_vap/ρ_d，只进误差分析
    节）; E2 = ≤ +8.0 %%"
          % (("%+.2f" % (100 * E1)) if E1 is not None else "-"
    ))
    print("[summary] E5: PCHIP vs 分段线性 max |ΔR| = %.3e m;
    量化 0.001 cm = %.3f %% of R_min"
          % (dR.max(), 100.0 * quant / R_MIN))
    print("[out] results/q4_sensitivity.json、results/
    q4_error_budget.json; 总耗时 %.1f s"
          % sens["wall_s"])


def main():
    import os
    skip = os.environ.get("MM_Q4_SENS_SKIP") == "1"
    RES.mkdir(parents=True, exist_ok=True)
```

```
 t_all = time.time()
 if skip:
     prev = json.loads((RES / "q4_sensitivity.json").
read_text(encoding="utf-8"))
     sens = prev
     base = prev["baseline"]["t_dry_s"]
     rows = prev["single_parameter"]
     grid2 = prev["h_hm_2d"]["grid"]
     scen = prev["driver_scenarios"]["rows"]
     v3 = [v["t_dry_h"] for v in grid2.values() if v["
t_dry_h"]]
     v6 = [v["t_dry_h"] for v in scen.values() if v["
t_dry_h"]]
     sens["baseline"]["note"] = ("MM_Q4_SENS_SKIP=1：复用上
次的单参数/二维/情景扫描结果，"
                                 "仅重算 E1 与误差预算（
2026-09-11 的 E1 读法修正）")
     print("[skip] 复用上次扫描结果（基线 %.6f h），仅重算
E1 与误差预算"
           % (base / 3600.0), flush=True)
 else:
     base, _ = t_dry_of()
     print("[基线] t_dry = %.6f s = %.6f h（N=%d、Δt=%.3g
/%.3g s）"
           % (base, base / 3600.0, N_SENS, DT1, DT2), flush
=True)
     sens = {"baseline": dict(t_dry_s=base, t_dry_h=base /
3600.0,
                              N=N_SENS, dt1=DT1, dt2=DT2)}
     rows = {}

 if not skip:
     sens, rows, grid2 = _scan_single_and_2d(sens, base,
rows)
     sens, scen, v6 = _scan_scenarios(sens, base)
```

```
3488
3489     tE, TaE, dT_qs, P_avg, dT_avg = _e1_zero_d(base)
3490     t_a, cmin_a, t_b, cmin_b, E1, T_CAP_E1 = _e1_adopt(tE, TaE
    , dT_qs, dT_avg, base)
3491     e1 = _e1_report(tE, dT_qs, P_avg, dT_avg, T_CAP_E1, t_a,
    cmin_a, t_b, cmin_b, E1)
3492     v3, e2, e3 = _e2_e3_blocks(grid2)
3493     e4, e5, e6, dR, quant = _e4_e5_e6_blocks(RES, v6)
3494     _write_outputs(sens, base, e1, e2, e3, e4, e5, e6, E1,
    t_all)
3495     _print_summary(grid2, v3, v6, E1, dR, quant, sens)
3496     return 0
3497
3498
3499 if __name__ == "__main__":
3500     sys.exit(main())
```